

A WORLD TOUR OF



TRUE
BASIC
FOR PROGRAMMING WINDOWS

John R Arscott

A World Tour of True BASIC for Windows

By John R. Arscott

**Computer programming made easy for
beginners and beyond**

ACKNOWLEDGEMENTS

**I wish to thank the following for their comments,
suggestions, proofreading and general support in the
preparation of this book:**

**Prof. Thomas E. Kurtz
Prof. John C. Baird
Prof. Rick Tarara
John A. Lutz
Christopher L. Sweeney
Bob Brannock
Anne Taggart**

Cover photo by NASA 0124 0510 2520 3058

ISBN-10: 1539313379
ISBN-13: 978-1539313373

Copyright © 2016 John R. Arscott
All rights reserved.

PREFACE

This book is not intended to be a replacement for the True BASIC Reference Manual. It is intended to be used in conjunction with the Manual, which tells you what True BASIC can do, whereas this book attempts to tell you how to do it.

INTRODUCTION

This book assumes that you are a complete novice with respect to programming computers. If you are already familiar with the concepts of programming and you are learning True BASIC as an alternative to some other language that you already know, then you will still find this book useful as an insight into the logic and simplicity of True BASIC compared to other more complicated languages.

We humans communicate with each other through the written and spoken word. As a general rule we are relatively careless about the way we do this. We are not very exact in our use of language, and we tend to use the first words that come to mind from our personal vocabulary. Fortunately for us, this works because our brains are very flexible and we automatically try to make sense of any communication by interpreting the information against our general understanding and experience of the world.

Compared to us computers are just plain dumb, at least in terms of communication and interpretation. They have infallible memory and very fast calculating abilities but that is as far as it goes. At the lowest level computers talk to each other in binary code, in other words, ones and zeros. Each digit is known as a data “bit.” These are wrapped in parcels of eight, called “bytes.” There are 256 ways of arranging ones and zeros as groups of eight bits. This is enough to construct a full alphabet, punctuation and a set of numbers from these 256 symbols, so at least this provides us with a way to communicate with the computer.

However, “understanding” is another matter altogether. Once you start putting letters together to form words in a language, then you need to teach the computer that language. We have the amazing ability to make sense of, or understand groups of words, even if the individual words are inexact and grammatically incorrect. We do this task with ease and at great speed. Computers cannot do this because it requires intelligence. In 1950 a British mathematician, Alan Turing, published a paper entitled “Computing machinery and Intelligence,” in which he proposed a simple test to determine machine intelligence and understanding. Basically the test involves a human tester, separately interrogating another human being and a computer. If the tester cannot tell the difference then we can consider the computer has intelligence. So far no computer has ever passed this test.

Obviously, attempting to translate the whole of our language and all its subtleties of meaning is a monumental task, so computer languages such as True BASIC, Pascal, Fortran, C++ etc., concentrate on interpreting a very small set of very precise words and phrases in our own language into something that the computer can deal with. The computer doesn’t suddenly become intelligent, it just blindly obeys our instructions that have been translated by the programming language. Some programs are so sophisticated that they give the impression that the computer has intelligence, but make no mistake about it: computers are dumb.

In the case of True BASIC, there are 259 built-in functions and statements of which only 21 are necessary to get you started. It is a tribute to the skill and ingenuity of J. G. Kemeny and T. E. Kurtz (the inventors of True BASIC) that they were able to reduce our language to such a few words and phrases, and yet still manage to get the computer to do practically anything you want. You will discover that the inventors went much further

than this to make learning and using a computer language easier and more natural for you.

Some computer languages, such as C, require that you think in a way that closely resembles the way a computer works, even though this may be contrary to the way we normally think and may seem illogical at times. For example, you must define all the variables you intend to use in your program before you use them. You would be horrified if you had to define all the words you intend to use before saying anything. Imagine saying, personal pronoun, present tense verb, preposition, numeric integer, and proper noun before actually saying, “I live at 14 Main Street.” In practice, the problem is much worse than this because in C there are at least 6 different types of numeric values. At this stage you may wonder what all the fuss is about, but even a moderate sized program in C can use hundreds of variables, and making a list to define each variable could work out to be a real pain.

True BASIC, on the other hand, was designed to be closer to the way we normally think as humans. For us, all numbers are just numbers. The fact that some numbers are whole numbers (integers) and others are decimal (floating point) is of little interest to us, but it is important to a computer. Similarly, words—or to be more precise, strings—are all the same. Whether they have only one letter or many letters, or consist of lots of text, they are still strings. The only difference between them is their size. Like us, True BASIC only recognizes two types of data, strings and numbers, so you don’t have to spend hours defining all the variables you want to use up front. Selecting True BASIC as your computer language means that you can forget all these tedious tasks and concentrate on the really interesting stuff, i.e., writing programs.

The point that you need to grasp is that programming languages allow you to communicate with the computer, but the language

is very precise and is very limited in terms of the instructions that you can use. You might think that because the set of instructions have nothing like the scope of Webster's dictionary, that this might limit what you can do. This is not so. Most of us are very adept at saying the same thing in several different ways, and you will need this skill and a fair degree of imagination in order to use the limited set of instructions to get the computer to do practically anything you want. Just remember that computers are dumb, so every line in your program has to be word perfect and the punctuation also has to be correct. Any mistake, however small and insignificant, will not be understood or tolerated by the computer. Fortunately, True BASIC has an excellent error-tracing feature that will point out where all your errors are located.

How is all this done and where do I start, you ask? Again, True BASIC has made things simple for you by providing you with a program called the editor. This program is very similar to any other word or text processor. A computer program is nothing more than a list of instructions, which you type at the keyboard in plain readable text, one instruction per line. Most of you will have used MS WORD or NOTEPAD or something similar, so you will find it very easy to use the editor because it works in exactly the same way. Your programs, usually called source programs, are always written in standard text format so that anyone can read it. Your source program can be saved and reopened by any word or text processor. The advantage in using the True BASIC editor rather than a standard off-the-shelf text editor is that it has a number of built-in tools and features that you will find very useful when writing programs.

When you want to run your program, True BASIC uses a special built-in routine called the compiler that translates your list of program instructions into an intermediate code. Another built-in routine called the interpreter executes or runs the compiled instructions one line at a time. It is the compiled code that the

computer runs, not your original source text. It is during this compiling process that True BASIC will pick up any errors you have made. The list of errors will tell you exactly where the errors are, but it is up to you to make the corrections before you attempt to run your program again. This process will continue until your program is totally error free. At this point your program will run and the computer will execute each instruction in the order that your program has defined.

The story doesn't quite end there. We are not infallible, so it is still possible that your program may not work properly even if it is word perfect and grammatically correct. For example, suppose the purpose of your program is to divide any two numbers that the user enters at the keyboard. All may appear to work fine because the program always produces the correct answer, until the user enters zero for one of the numbers. At this point the program will stop working because it has come across a "runtime" error, because you cannot divide a number by zero. This is a very simple example, but errors of this type can remain buried and undiscovered in programs for years. Program testing to detect runtime errors is therefore an essential part of the development of a commercial program, although it is less important if your program is for private use.

All the example programs in this book are available from the Downloads page of **www.truebasic.com**.

CONTENTS

PART I.....	17
Getting Started	17
Chapter 1	19
The True BASIC Editor	19
The True BASIC Editor menus	27
SPECIAL EDITOR FEATURES.....	45
BLOCK COMMENTS	45
MODIFYING CODE LINES	46
RENUMBERING CODE LINES	46
RIGHT CLICK MENU	46
COMMAND LINE.....	47
PART II	50
Learning the Basics	50
Chapter 2	51
DATA	51
NUMBERS	51
STRINGS	53
PARTIAL STRINGS.....	55
ARRAYS.....	56
Chapter 3	58
OPERATORS	58
ARITHMETIC OPERATORS.....	58
RELATIONAL OPERATORS.....	59
LOGICAL OPERATORS	60
Chapter 4	62
INPUT and OUTPUT.....	62
SCREEN OUTPUT.....	62
KEYBOARD INPUT	64
GET KEY.....	66
GET MOUSE	67
Chapter 5	68
DECISIONS	68

IF....THEN....ELSE.....	68
IF KEY INPUT.....	72
SELECT CASE	73
Chapter 6.....	76
LOGIC FLOW CONTROL.....	76
FOR...NEXT....STEP	76
DO LOOP.....	79
CORE INSTRUCTIONS.....	84
PART III.....	91
Beyond the Basics	91
Chapter 7.....	91
PROCEDURES	91
FUNCTIONS	92
SUB-ROUTINES	94
STATEMENTS	97
Chapter 8.....	99
CONVERSION FUNCTIONS.....	99
Chapter 9.....	106
STRING FUNCTIONS	106
Chapter 10.....	110
STRING SEARCHING.....	110
Chapter 11.....	121
NUMERIC FUNCTIONS	121
MAXNUM.....	124
Chapter 12.....	126
TRIGONOMETRIC FUNCTIONS	126
Chapter 13.....	129
DATES & TIME.....	129
Chapter 14.....	132
More Input	132
LINE INPUT	132
READ & DATA	132
RESTORE.....	136
Chapter 15.....	137
More Output.....	137

PRINT USING	138
USING\$	146
TAB	147
DATA TABLE.....	148
Chapter 16	150
Matrices	150
MAT REDIM	151
MAT READ	154
MAT INPUT	157
MAT LINE INPUT	162
MAT PRINT	163
MAT PRINT #n	168
MAT WRITE #n	169
MAT READ #n	170
MATRIX ARITHMETIC	172
MATRIX ASSIGNMENTS.....	173
MATRIX ADDITION and SUBTRACTION	176
MATRIX MULTIPLICATION and DIVISION.....	178
Chapter 17	182
MATRIX and ARRAY functions.....	182
Chapter 18	187
File Handling	187
OPENING FILES	187
NAME	188
FILE TYPES (ORGANISATION)	189
BYTE.....	189
CREATING FILES	193
ACCESSING FILES	193
CLOSING FILES	193
WTLIB library.....	194
FIELDS and ARRAYS	201
THE PRINTER.....	209
EXECLib	214
SUB Exec_AskDir	214
SUB Exec_ClimbDir	214
SUB Exec_SetTime	215

DATABASE PROGRAMS	215
SORTING.....	220
THE BINARY SEARCH.....	223
Chapter 19.....	236
Program Units.....	236
MAIN PROGRAM	236
EXTERNAL FUNCTIONS	239
EXTERNAL SUBROUTINES	248
LIBRARIES	256
MODULES	261
INTERNAL FUNCTIONS and SUBROUTINES.....	264
CHAIN.....	264
Chapter 20.....	270
Error Handling	270
ERROR TRAPPING AND HANDLING.....	270
THE PROTECTED BLOCK of STATEMENTS.....	270
THE ERROR HANDLER BLOCK.....	271
EXTYPE and EXTEXT\$ FUNCTIONS	271
Chapter 21.....	274
Graphics	274
BMP FORMAT	274
JPG FORMAT	274
VECTOR FILES	274
THE GRAPHICS SCREEN	275
WINDOW SIZE	275
SETTING THE CURSOR POSITION.....	276
FUNDAMENTALS OF DRAWING.....	277
POINTS.....	277
LINES.....	278
AREAS.....	279
COLOR	280
TEXT	284
Drawing with BOX Statements.....	284
BOX LINES	285
BOX AREA.....	285
BOX CLEAR	285

BOX CIRCLE	285
BOX ELLIPSE	285
BOX SHOW	285
BOX KEEP	285
FLOOD	287
VERTICAL TEXT	288
RUBBER BOXES	290
PICTURE PROCEDURES	291
USING TRANSFORMS with PICTURES.....	294
PICTURE LIBRARIES	295
COMPUTER AIDED DESIGN	295
PWlib	297
INTERACTIVE COMPUTER GRAPHICS.....	299
ANIMATION	303
Chapter 22	307
Sound and Music	307
SOUND	307
MUSIC.....	307
TEXT TO SPEECH.....	308
Chapter 23	310
Program Structure.....	310
WRITING STRUCTURED PROGRAMS	310
UNDERSTAND the PROBLEM	310
PLAN FIRST, PROGRAM LATER.....	311
WRITE SIMPLE PROGRAMS	312
TEST and DEBUG CAREFULLY.....	312
PROGRAM STYLE.....	313
Chapter 24	318
Problem Solving	318
PART IV.....	327
Programming for Windows	327
Chapter 25	327
Principles	327
EVENT HANDLING	328
WINDOWS CO-ORDINATES	330

User Units	331
Pixel Units	332
IDENTITIES	334
Chapter 26.....	336
Creating Controls and Objects.....	336
WINDOWS.....	336
TARGET WINDOWS	338
ACTIVE WINDOWS.....	338
MENUS.....	339
EDIT FIELDS	340
LISTBTN	341
PUSH BUTTONS.....	343
TEXT and FONTS.....	343
STATIC TEXT	345
CHECK BOX.....	346
RADIO GROUP	347
SCROLL BARS.....	348
LIST BOX.....	350
LIST BUTTON.....	351
LIST EDIT	352
CTX LIBRARY	353
CT EVENT	355
GENERAL PURPOSE ROUTINES.....	356
STATIC TEXT	359
CHECKBOX.....	361
RADIO GROUP	363
EDIT FIELD.....	365
PUSH BUTTON	368
SCROLL BARS.....	371
LISTS.....	372
RIGHT CLICK MENU.....	380
DATA TABLE	385
GROUP BOX	391
ELEVATOR BUTTON	393
BAR SCALE.....	395
IMAGES.....	399

Chapter 27	401
Windows Program structure	401
USING FORMS TO CREATE CODE	406
Chapter 28	419
Dialog boxes	419
OVERVIEW	419
GENERAL PURPOSE ROUTINES	419
FILE HANDLING	420
MESSAGES	421
USER INPUT	422
LISTS	424
TDX LIBRARY MODULE.....	425
GENERAL PURPOSE ROUTINES	427
FILE HANDLING	428
MESSAGES	430
USER INPUT	432
LISTS	434
PASSWORDS	436
FONTS	437
COLOR.....	438
PART V	439
Application Studies	439
Chapter 29	439
Worked examples	439
ICONEX (icon exchange)	439
BOXSTRINGS library module	449
INDEX.....	464

PART I

Getting Started

The latest edition (Version 6.007) of True BASIC for Windows is released on a CD-ROM disk. When you insert the disk into your CD disk drive a utility program called TB6007SETUP will automatically start up and will install the True BASIC editor and a number of other useful files such as demonstration programs.

If you have a downloaded copy of True BASIC then this will be contained in a file called TB6007.ZIP.

- (1) Click on the START button on your desktop
- (2) Select FIND or SEARCH for files.
- (3) Enter TB6007.ZIP as the search file name
- (4) When the computer has located the file double click on the icon.
- (5) The file will automatically unzip and will start the installer program

The opening page of the installer requires you to confirm that you understand the conditions of the license by clicking inside the small tick box.

The next page requires you to specify where you want to install this edition of True BASIC. The default location is c:\program files\TB. If you are using Windows Vista or above, you are advised to change the default location to c:\TB rather than using the program files folder, because these versions of Windows treat folders that have been created in Program Files in a different way to folders elsewhere, and you may find that some files are marked “read/write protected,” which may prevent your

programs from running. Once you have specified where you want to install True BASIC, click the NEXT button to continue with the installation.

On completion of the installation process you will be shown a "ReadMe" file that contains update notes about the latest version. There is also a shortcut icon that you can drag onto your desktop.

More recently, the editor package has been released inside an MS installer which includes a simple script that permanently sets "administrator" rights to the editor in its own installation folder, so the user can install it wherever they choose and thus bypass Windows' built-in permissions lockdown on C:\Program Files. In essence, it duplicates the "run as administrator" settings. However, it is crucial to change the default folder to something like "TBeditor" to avoid complications with the CHAIN statement.

WARNING:

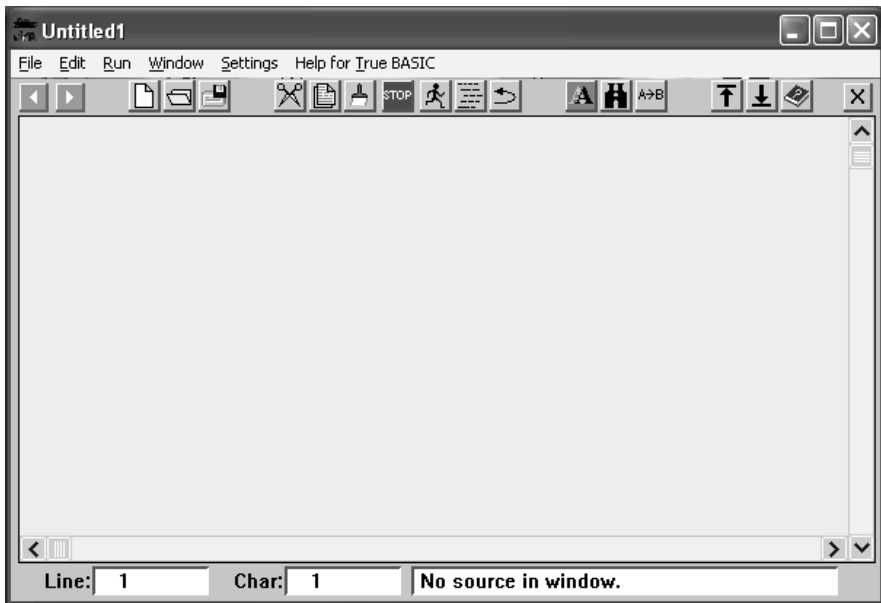
When a user RUNS a program, the TB editor chains to Tbsystem531.exe to COMPILE, check for errors and finally RUN your program. Spaces in folder names and filenames are not permitted. The same restrictions that applied to DOS also apply to True BASIC file and folder names, i.e., no spaces and a maximum of 8 characters plus an extension.

Chapter 1

The True BASIC Editor

Even if you are familiar with word or text processors you are still advised to read this section because the True BASIC editor contains a number of unique features that are not available in other text editors.

When you start True BASIC the screen will show a full screen empty window labeled “Untitled 1” in the top left corner of the window, which should look like the picture below, but probably in color, not black and white.



The central white area (you can change this color later) is your working window. Below this page are two information boxes. The box on the left tells you where the flashing cursor is located by giving you the line number and the character number counting from the left. On the right at the bottom of the window

is an information box. This box is also the command line where you can type instructions directly to the computer. If you click the mouse inside the information box it will turn blue, and you can begin typing your instructions. For example, if you type the word “version” in the blue box, the computer will immediately respond with the current version number of this edition of True BASIC. Most probably you will rarely use the command line because most commands are directly available from the main menu.

Along the top of the window there is a menu bar with six headings. If you click on any of the heading labels, then a drop down list of items will appear below the heading. Below the menu bar there is a toolbar displaying several icons that represent most of the menu options. If you click the mouse on an icon, then this is the same as clicking on the equivalent menu item. On the left of the toolbar there are two green arrow buttons that allow you to flip through other programs, and work just like the arrow buttons on an Internet browser.

Your program is not limited to the size of the window. Think of the window as being of infinite length, where you can scroll up and down the page. The True BASIC editor window is more like a notebook full of window pages, in that you can have more than one program open at the same time. In fact you can have ten open programs at the same time, which you can select directly from the WINDOWS menu list, or you can flip up and down through these programs with the green arrow buttons.

Each of the pages in the window is also of infinite width, so not only can you scroll up and down, but you can also scroll sideways as well. During normal typing, when you get close to the right hand side of the page, the editor will automatically scroll sideways so that you can keep on typing. When you get to the bottom line the editor will scroll the page upwards so that you can again keep on typing.

If you EXIT the editor, on purpose or by accident, then the editor will remind you to save any programs that you have not already saved. While we are on this subject, this might be a good time to select the SETTINGS menu and click on the HOTSTART label. When you shut down the editor, the HOTSTART feature will remember all the programs that you have opened, and all the cursor positions in each program. When you next switch on the editor, the HOTSTART will reload these previous programs ready for you to commence working where you left off.

Although writing computer programs is essentially the same as writing a list of instructions, you would be well advised to include the following items at the start of your program as standard practice (precede each line with an exclamation mark):

- (1) The name of the program
- (2) The date
- (3) The purpose of the program, i.e., what it does
- (4) A brief outline of the steps the program takes to achieve the objective

It is not mandatory that you do this, but believe me when I say that in the space of a month or two you will forget what the program does, or how it works, or you may even forget writing it in the first place. One day you will thank me for this advice.

An exclamation mark at the beginning of any line tells the computer to ignore these little notes because they are NOT program instructions. These reminder notes are called “comments” and they can be used anywhere in your program. Indeed it is good practice to make comments after certain lines of code to remind yourself what that line of code is actually doing, because it is not always blindingly obvious. As an alternative to the exclamation mark (!) you may type the word REM, short for reminder.

The date is important because it is very likely that you will make improvements or additions to the original program, or you may just correct a bug (an error). There is every chance that you will have several copies of the same program in various stages of development and you will need to know what the latest version is. Before making changes to a program always save the original, then make a copy and work on the copy. This way you can always go back to the original if the changes don't work out.

The most important difference between the True BASIC editor and other text editors is that when you type your instructions, the lines of text are NOT WRAPPED, i.e., when your typing reaches the right hand margin it just carries on and on. The text does not automatically drop to the next line down. The only way you can drop to the next line down is to press the RETURN key on your keyboard because this signifies the end of a line. The reason behind this method of operation is that True BASIC only allows one instruction per line, but that instruction may be too long to fit the width of the page, so the editor will always allow you enough space for your instruction regardless of how long it is. There is a scroll bar across the bottom of the editor page so that you can view anywhere along very long lines.

If long lines worry you, and you would prefer to see your all of your program without having to scroll across the page, then you can use the ampersand sign (&) to terminate a line as long as you begin the continuation line with an ampersand too.

When you reach the end of your program you must tell True BASIC by typing the word END on a line of its own. From the start of the program to the word END is called a PROGRAM UNIT. Your program can consist of several program units (see the section on EXTERNAL programs and LIBRARY MODULES).

True BASIC is very tolerant of the way you type the program instructions. You can use uppercase or lowercase, or both. You can also add spaces as often as you like if they make things clearer to read. Indeed there is a utility feature built into True BASIC that will “format” your code, i.e., it will indent certain keywords to make the program easier to read and easier to understand the way it is structured. In a way, it is a bit like using paragraphs and bullet points in ordinary text. You will find the format feature one of the most useful items on the True BASIC menu.

While True BASIC is tolerant of the way you set your program out, like all other computer languages it is not so tolerant about the instruction code itself. When you type an instruction, it has to be word perfect, and if there is any punctuation it has to be perfect too. It is not good enough to get it nearly right; it has to be perfect. Fortunately, True BASIC is wise enough to know that it is dealing with human beings that have a habit of making mistakes, so it has an extensive error detection system built in. When you attempt to RUN your program, if you have made any mistakes, then True BASIC will almost certainly find them.

When True BASIC compiles your program in order to convert it into something the computer can understand, it ignores comments, spaces, and empty lines. In other words, you can include comments and you can use empty lines as often as you like without affecting the speed or size of your program. In this respect I would advise that you use line spaces and comments to split your code into logical blocks. You can see many examples of this in the demonstration programs. Try to keep these blocks of code small enough to fit the monitor screen so that you can view a whole block of code without scrolling. You will find it much easier to read, understand and debug (find errors) your code this way.

You are free to use upper or lower case, but my advice would be to use lower case for all your variables and upper case for keywords. The built-in DO FORMAT option will automatically convert keywords to upper case for you, so you can use lower case for everything. There is a strong body of evidence that suggests lower case is much easier to read.

Readability is a crucial factor in writing good programs. You should aim to make your code as readable as any book. Typically, at school in algebra, you would write an expression as follows:

$$P = S - C - O$$

Whereas in True BASIC you could write:

`LET Profit = Sales – Costs – Overheads`

It is important to remember that True BASIC doesn't care whether you use single letters as variables or whether you use single words, so why not make your code self explanatory instead of using cryptic symbols. The golden rule about variable names is that there must be no spaces in the name and it must not begin with a number. Fortunately, True BASIC sees joined up words as being single variables, so you can use phrases too. For example:

`LET Gross_profit=Sales-variable_costs-
fixed_costs_and_expenses`

In the example, we are using the underscore as a way of joining words together. Alternatively, we could use upper and lower case to do the same thing:

`LET GrossProfit=Sales-VariableCosts-
FixedCostsAndExpenses`

Both of these methods of compounding words are frequently used in True BASIC library modules. The important message is, make your code easy to read—not just for you but for anybody else too. Make no mistake about it, as time goes by, you will forget everything about any program you have written, so making the code easy to read will help enormously to jog your memory. In commercial applications, you have to remember that somebody else may inherit the job of maintaining and updating your code, so it is important to establish good habits and practice right at the beginning.

Notice that each assignment instruction line begins with the word LET. Every instruction in True BASIC has to begin with a keyword or statement. A complete list of keywords can be found in the Reference Manual. Later you will learn how to avoid using the word LET, but for now it will be used for all assignment instructions. If you think of instructions as being similar to a sentence in English (without the full stop), then the keyword or statement is equivalent to a command verb, variables are like the subject and object, and functions are like additional verbs or adverbs.

SUMMARY

This is a convenient point to summarize what we know so far:

- ❑ A computer program is a list of instructions.
- ❑ The instructions are clear and exact because the computer is dumb.
- ❑ True BASIC only allows one instruction per line.
- ❑ Every instruction line must begin with a True BASIC keyword.
- ❑ Instructions are not case sensitive.
- ❑ Spaces, empty lines and comments are ignored by the computer.
- ❑ Every program must have an END statement.

- ❑ True BASIC source programs (TRU) are written in human readable plain text.
- ❑ Source programs are compiled in order for the computer to run them.
- ❑ Compiled programs (TRC) are not readable.

The True BASIC Editor menus

For your own benefit you should read this chapter about the features that are available in the editor, but you are probably familiar with text editors anyway, so you can skip this chapter temporarily and move on to PART 2, to learn the basics of programming.

Normally you would use the mouse to click on menu headings, and then to click on items under the heading. Alternatively you can press the ALT key on the keyboard to activate the menu bar. The side arrow keys can be used to drive the heading highlight backwards and forwards across the menu headings. The up and down arrow keys can then be used to highlight individual menu items. Pressing the <RETURN> key will select the current menu item.

FILE MENU

- **NEW** - this option opens a new empty editing page in the main window with the default title “Untitled” followed by a sequential number. A maximum of ten new and existing windows can be open at the same, and you can switch between them as often as you like.
- **OPEN** – will raise an open file dialog box where you can navigate through drives, folders and files to select the file of your choice. The list of files is limited to program files only, i.e., those files with the extension TRU or TRC. You can extend this by selecting ALL FILES in the file type box. When you select a file it will be displayed with the file name as the window title. The information box will display the total number of lines in the program.
- **CLOSE** – will close the current program in the main window. This action is identical to clicking the mouse on the close button (black cross on a gray background).

You will be asked if you wish to save the contents of the window. When a program is closed, the code is erased from the computer's memory.

- **SAVE** – will save the contents of the current window using the window title as the file name. The file will be saved in the same folder as the original version when the file was opened. In other words the new version will over-write the existing version.

If the program is being saved for the first time, i.e., it is untitled, then you should make sure you give your program a meaningful name because there is every chance that in a matter of weeks you will forget what it is called, so you will have to hunt through your programs folder to see if you recognize the name. For example, if your program calculates the time of sunrise and sunset at any geographical location, then `SUNSET.TRU` would be an appropriate name. Calling your program `MYPROG.TRU` or `ANYPROG.TRU` is not very helpful and will certainly not jog your memory as you glance through your program folder. Clearly this advice becomes more important the greater the number of program files you have saved. It is not unusual for True BASIC programmers to have hundreds, if not thousands, of saved program files on their hard drives, purely because it is so easy to write programs in this language.

- **SAVE AS** – will raise a save file dialog box that will let you specify any name for the file and will allow you to save the file in a folder of your choice. The default file name is the same as the window title, and the default destination folder is the same as the original file when it was first opened.
- **UNSAVE** – is a drastic measure because it will completely delete any file that you specify. You will be asked if you are sure you want to delete the named file.

Once you delete a file there is no way to recover it. This is NOT the same as dragging a file to the recycle bin.

- **RENAME** – changes the name of the current window. It does not send a copy of the current source text to a file. If the current source has already been saved to a file, then this existing file will remain unchanged. This corresponds to the RENAME command that executes exactly the same action.
- **PAGE SETUP** – this option presents you with a special dialog box that allows you to specify certain features of any printed output.
- **PRINT** – allows you to select all or a part of your program to be hard copy printed. You select the text by dragging the mouse to highlight the required text. You can also select text using the SHIFT KEY in combination with the DOWN-ARROW key. This procedure uses a high definition print method more suited to proportional fonts. At present this option only prints in COURIER 10 point font.
- **LISTING** - allows you to select all or a part of your program to be hard copy printed. You select the text by dragging the mouse to highlight the required text. You can also select text using the SHIFT KEY in combination with the DOWN-ARROW key. This procedure uses a standard print quality with 80 characters per line and 60 lines per page as default values. These default values can be changed under the SETTINGS menu. It is more suited to fixed pitch fonts such as COURIER and LUCIDA CONSOLE.
- **CHAIN TO....**- allows the user to select an executable file, i.e., with the extension .EXE, and to run this application directly from the editor. When the application is shut down, the editor is re-activated and continues where it left off.
- **CHAIN WINDOWS APP** – allows the user to select an application file, such as a WORD document file (with

the extension .DOC) or an Excel spreadsheet file (with the extension .CSV). The editor will run the main application and will automatically load the selected file. Both the Windows application and the editor continue to be active.

- EXIT – will close all windows and shut down the True BASIC editor. You will be asked if you wish to save the program as each window is closed if the SAVE ON CLOSE menu option has NOT been selected.

EDIT MENU

- UNDO – this option will re-instate the original program text prior to a CUT, PASTE, typing FORMAT or any other operation. In other words, if you perform a CUT or PASTE action and you decide that you have made a mistake and want to return to the original text before you made the mistake, then using UNDO will achieve this. There are now ten levels of UNDO available to the user. In other words you can back track ten times. The menu is labeled with the operation that can be undone, e.g., UNDO paste. The menu item is labeled “can’t UNDO” when you can no longer back track any more.
- REDO – this option allows you to effectively undo a previous undo operation. In other words you can re-instate the former text after you have done an UNDO operation. As with UNDO you can REDO repeatedly.
- CUT – will copy any highlighted text to the clipboard, and will then erase that portion of text. Portions of text held on the clipboard can be inserted back into your program with the PASTE option. CTRL-X can be used as an alternative way to execute CUT.
- COPY – will copy any highlighted text to the clipboard, but will not erase that portion of text from your program. CTRL-C can be used as an alternative. Portions of text held on the clipboard can be inserted back into your program with the PASTE option.

- **PASTE** – will transfer text from the clipboard to the point immediately after the current cursor position in your program. CTRL-V can be used as an alternative. You can position the cursor anywhere in your program by clicking the mouse at that point. The cursor location boxes at the bottom of the editor window indicate the current line and character position. If any text is highlighted then PASTE will replace the highlighted text with the text from the clipboard.
- **FIND** – will raise the find dialog box that allows you to specify and locate any word, part word or phrase in your program text. The search can be an exact match including upper and lower case, or the search can be independent of case. Normally the search begins at the current cursor position and proceeds to the end of your program. Alternatively you can “wrap” the search to include the whole of your program. The first instance of any text that matches your specification will be highlighted. After a FIND operation the FIND window stays on top, ready to be used again.
- **FIND AGAIN** – is a quick alternative to the FIND dialog box. Once the FIND search has located the first instance of a match, then you may use FIND AGAIN to progressively locate all the other instances.
- **CHANGE** – is similar to FIND except that when a match is found you have the option to replace the match with a specified alternative. You can replace the first instance of a match or you can replace all instances.
- **KEEP** – will retain the highlighted portion of your program and discard the rest. It is a quick way to delete large parts of your program.
- **INCLUDE** – will allow you to specify the name of a program file. The contents of this file will then be inserted in your program at the current cursor position.

- **SELECT ALL** – is a quick way to highlight the whole of your program text rather than dragging the mouse across all the text, which may occupy many pages.
- **MOVE TO** – this is a quick and useful way to place the cursor at a specific line or a specific word in your program. Alternatively you can select the name of a sub-routine from a list of all the sub-routines in your program, and the cursor will move to the start of that routine.
- **WRAP** – this option converts the current window to wrapped text, i.e., long lines are truncated at the edge of the window and are continued on the next line. In the WRAP mode the editor can be used as a general-purpose text reader. **CAUTION:** None of the options under the RUN menu will work while the window is in WRAP mode. Click the WRAP option again to restore normal programming mode.

RUN MENU

- **RUN** – this option will run the current program, i.e., the program in the front page of the editor window. When the program has finished running the title bar will tell you to click the mouse or press any key. Either action will return you to the editor. If True BASIC encounters any errors, these will be shown as a list. You may select any of the listed errors and the cursor will immediately go to the line and character position where the error occurred.

Prior to running your program, the editor adds a few extra lines of code to a copy of your source program (this preserves the original) and it is this copy that is run. These extra lines include adding any loaded libraries and aliases.

In the case of MODULES and EXTERNAL program units, no extra code is added and no loaded libraries are added.

Remember that the default directory is where the bound program is launched from.

CAUTION: If you are using line numbers then make sure to leave plenty of space between your line numbers to allow the editor to insert the extra code between your lines. Intervals of 10 are normally sufficient.

- **BREAKPOINT** – will mark your program at the line where the cursor is positioned. **BREAKPOINT** is normally disabled (grayed out) and only becomes active when you select **DEBUG MODE** from the **SETTINGS** menu. The breakpoint will appear as the word <<<**BREAKPOINT**>>> surrounded by angle brackets. This is a toggle action feature, i.e., if the line already has a breakpoint then it will be switched off, but if there is no breakpoint then one will be added. If you **RUN** your program with breakpoints marked, the program will stop at the first breakpoint. A dialog box will give you the opportunity to continue running your program. All breakpoints are cleared when you toggle **DEBUG MODE** again.

If you insert a series of variable names immediately after the breakpoint, e.g.

<<<**BREAKPOINT**>>>a,n,string\$,xyz,b

then when your program halts at the breakpoint a list of all these variables and their current values will be displayed. In this way you can track the changing values of any variable while the program is running. This is a valuable aid to debugging.

CAUTION: If you are using line numbers then make sure to leave plenty of space between your line numbers to allow the editor to insert extra code between your lines to achieve this breakpoint feature.

- **COMPILE** – will cause your program to be converted into a coded format that the computer understands. Unlike earlier editors, your source program will be preserved. The compiled version will be automatically saved with the same file name and in the same folder as your source program, but the extension will be changed to TRC instead of TRU.
Prior to the compiling process, the editor adds a few extra lines of code to a copy of your source program (this preserves the original) and it is this copy that is compiled. These extra lines include adding any loaded libraries and aliases. In other words a compiled program will run exactly like a source program. The exceptions to this rule are MODULES and EXTERNAL program units. In these two cases, no extra code is added and no loaded libraries are added. Remember that the default directory is where the TRC program is launched from.
CAUTION: If you are using line numbers then make sure to leave plenty of space between your line numbers to allow the editor to insert the extra code between your lines. Intervals of 10 are normally sufficient.
- **BIND** – is a special linking process that combines your program with any library modules and other resources to produce a stand-alone executable application. The default name of this application is the same as your original source code except the extension is changed to EXE instead of TRU. A dialog box allows you to change this name and to specify the folder where the executable file will be saved. NOTE: this feature is NOT available in the Bronze edition so the menu item is grayed out and disabled.
Prior to the binding process, the editor adds a few extra lines of code to a copy of your source program (this

preserves the original) and it is this copy that is bound. These extra lines include adding any loaded libraries and aliases, but DO NOT include the code that retains the output window. In other words when your program reaches the END statement, then the program will stop and the screen will clear. If you need to retain the output window then you must add the code yourself. For example, immediately before the END statement add the following line:

```
CALL Tbexitroutine
```

this will preserve your last screen until the user presses any key or clicks the mouse.

Remember that the default directory is where the bound program is launched from.

CAUTION: If you are using line numbers then make sure to leave plenty of space between your line numbers to allow the editor to insert the extra code between your lines. Intervals of 10 are normally sufficient.

- TRACE – is another feature that helps you locate errors in your program by stepping through your program line by line. Essentially TRACE puts a breakpoint on every line. TRACE is normally disabled (grayed out) and only becomes active when you select DEBUG MODE from the SETTINGS menu. All breakpoints are cleared when you toggle DEBUG MODE again.
- DO – is a general-purpose command in which you specify and run an EXTERNAL program unit. A file selector dialog box will assist you in locating the DO program of your choice. Note: The program called RUNDO.TRU is NOT a DO program and will generate errors if you attempt to run it. Do not move or delete this

file because you will no longer be able to run any DO programs.

- DO FORMAT – is a built-in routine that indents your program text depending on certain keywords in order to make the text more readable. It also helps you to locate errors because it aligns corresponding statements such as FOR...NEXT and DO....LOOP. If these statements are not perfectly aligned then there must be an error in the code between these statements. You will find that this menu option is one of the most frequently used features in the editor.
- DO UPPER – is a built-in routine that will convert the text of any True BASIC program to all upper case (capital letters).
- DO LOWER – is a built-in routine that will convert the text of any True BASIC program to all lower case (small letters).

WINDOW MENU

- RECENT FILES – this option displays a rolling list of the ten most recently closed files. As you close more files, older files will drop off the bottom of the list.

NOTE: At the bottom of the WINDOW menu there will be a list of all the program files that are currently OPEN. The list shows the full path name of each file. The current program file will be ticked. You may click the mouse on any of these file names to force the file to become the focus of the editor. When you close a program file it is removed from this list.

SETTINGS MENU

- SAVE ON CLOSE – this option sets an internal toggle action switch that automatically saves your program when you close the window. When the internal switch is active a tick will appear against this item. Click on this

item again to cancel the internal switch and the tick will be erased. The default condition is OFF. The True BASIC editor will try to help you avoid catastrophic mistakes by presenting you with a dialog box that asks if you wish to save your program every time you click on the close window button.

- **BACKUP ON SAVE** – this option allows you to set an internal switch that will automatically produce a back-up copy of any program at the time you save the program. The back-up copy has the same name as the original file except the extension is BAK instead of TRU. When the internal switch is active a tick will appear against this item. Click on this item again to cancel the internal switch and the tick will be erased. This is known as a toggle action switch; click once for ON and click again for OFF. The default condition is ON.
- **DEBUG MODE** – is a toggle action switch that enables the BREAKPOINTS and TRACE options under the RUN menu. When DEBUG MODE is switched ON the item is ticked. When the switch is OFF the tick is erased and your program will run as normal. All breakpoints are removed when DEBUG MODE is switched OFF. The default condition is OFF.
- **CONFIRM QUIT** – is a toggle action switch that causes a dialog box to appear whenever you attempt to shut down True BASIC. The dialog box requires that you confirm your intention to shut down. This option will avoid shutting down when you did not mean to do this. When the confirm switch is active a tick will appear against this item. Click on this item again to cancel the internal switch and the tick will be erased. The default condition is ON.
- **HOTSTART** – is a toggle action switch that causes all the open files that you were using in the previous session with True BASIC to be loaded automatically when you start up True BASIC in the current session.

When the hotstart switch is active a tick will appear against this item. Click on this item again to cancel the internal switch and the tick will be erased. The default condition is ON.

- **PREFERENCES** – raises a special dialog box where you can set the color of the editor window, and the name and color of the font used to print the text in the window. The default page color for the editor window is SAND. The default font for all editor windows is ANSI MONO, 10 point PLAIN (regular) or COURIER, 10 point PLAIN, and the standard default font color is BLACK. As an alternative LUCIDA CONSOLE 10 point PLAIN can be used as a fixed pitch font. The preferences dialog box also allows you to set the number of characters that will be printed across the hard copy page and the number of lines that will be printed per page. The default settings are 80 and 60 respectively. Code lines longer than 80 characters will be wrapped in the hard copy print.
- **BINDER** – allows you to select which binder you wish to use, i.e., the older version binder that requires DLL files in order for executable programs to run, or the new binder (5.5b19) which does not require DLL files to run executable programs. Note that the new binder has a number of residual bugs that prevent some TrueCtrl objects from working correctly.
- **ALIASES** – this menu option allows you to add or edit the list of alias pathnames used by the editor to locate filenames used in LIBRARY statements. Note that when the file is located in a sub-folder of the directory where the new editor is located, e.g., Tblibs, then only the sub-folder name is used in the alias list. If the file is located in a different directory altogether, then the full path to that directory must be given, e.g., C:\my documents\my pictures. Do not use a trailing backslash.

Aliases can also be used with the OPEN file statement provided the file already exists, i.e., CREATE OLD is specified. The filename must also be a string literal within quote marks and not a string variable.

Legacy programs that used curly brackets and an alias group name, e.g., {mygroup} will be handled by the new alias system, even though the group name is ignored.

Aliases that are added under the ALIAS menu are permanent, i.e., the editor will remember them so that when you shut down and re-start, the aliases will still be in effect. Note that temporary aliases can be added on the command line or by means of a SCRIPT file.

Temporary aliases are not remembered when the editor shuts down.

- **FUNCTION KEYS** – is a toggle action switch that enables or disables the function keys. This feature is now enabled in this version.
- **SHORT CUTS** – is a toggle action switch that shows or hides short cut keystrokes against each menu item. The default condition is ON, i.e., short cuts are shown.
- **COLORTEXT** – this feature uses different colors for line numbers (if any), key words, calls and sub-routines, definitions and functions, punctuation, aliases, strings and numeric variables. Two default color schemes are available depending on the background page color (light or dark). The user can also define a custom color scheme. This option allows the user to switch colortext on or off. Note that when colortext is ON then all current open source files will use color text except files that are wrapped. Colortext can be RUN, COMPILED, and BOUND in the normal way.

HELP FOR TRUE BASIC

- **HELP** – this option shows a small text window with a drop down index and an edit box that allows you to search the help file. You can resize this window to suit

your purpose and it will remain at this size for the remainder of the session while you are working with True BASIC. The help files contain details of all the functions and statements in True BASIC and how to use these features. There are a number of other useful items of information in the help files including extracts from this book. You can select which help file you want to use from the CONTENTS menu. This help file will remain current until you change to another help file. The full alphabetical index will be shown when you click on the down arrow button to the right of the topics title. When you select a topic from the index, the text related to the topic will be shown in the main text box.

If you are uncertain what you are looking for, you can type an associated word or concept in the search box then click on the green GO button. The program will then search the whole text in the current HELP file for a match and will display the results in the text box.

A unique feature of the HELP option is that you can edit, change or add items to the help file using the EDIT or INSERT options under the HELP WINDOW menu. COPY and PASTE options also allow you to copy code fragments contained in the help text box and transfer these fragments direct to your program

- FORMS – this option is grayed out (disabled) in all versions. It is a new option to True BASIC but must be purchased separately. The application automatically enables this menu option to make it fully integrated with the editor. FORMS is not available in any earlier versions. This program allows the user to design window layouts using a simple graphical drag-and-drop interface. Most importantly, FORMS generates the program code to reproduce your design, and includes this code in your own program. You can use FORMS repeatedly to create or modify as many windows as you like. Each window may contain as many controls and

objects as you need. The code generated by FORMS is a complete skeleton application that can be run immediately without further intervention by the user. Included in the code are comments to guide you to the point where you need to insert your own program code to respond to user input.

- **TBILT** – this option is grayed out (disabled) in all versions except Gold. It is a new option to True BASIC but must be purchased separately. The application automatically enables this menu option. This is a free standing program that allows the user to design window layouts using a simple graphical drag-and-drop interface. The editor automatically chains to TBILT. Most importantly, TBILT generates the program code to reproduce your design, and leaves this code on the clipboard for you to paste into your own program.
- **ABOUT TRUE BASIC** – will show you the version number, edition and release date of the version of True BASIC currently running.
- **MANUALS** – will display a selection list containing details of all the manuals available in the DOCS folder. When you select a manual from this list the program will automatically start up an Adobe PDF file containing the selected manual. When you close the Adobe Reader window, control is passed back to the True BASIC editor. You can also add your own manuals to the DOCS folder, provided the manual file itself is in PDF document format, and this manual will be automatically added to the list in the editor.

HELP WINDOW MENU

When you select **HELP** on the main menu, a separate window will appear (usually with a pink background page color). This new window has its own menu.

FILE

- PRINT – the current help file topic that appears in the text box will be copied to the hard copy printer.
- RUN DEMOS – first select a demo file by highlighting the name (drag the mouse across the name), then select RUN DEMOS. The file will be automatically loaded into editor window ready for you to run.
- CLOSE – this option closes the HELP window and returns the user to the main True BASIC editor.

EDIT

- CUT – this option is normally disabled (grayed out). It only becomes active when the MODIFY or INSERT options are selected. When active you can copy any highlighted text to the clipboard, and that portion of text will then be erased. Portions of text held on the clipboard can be inserted back into the help file with the PASTE option.
- COPY – will copy any highlighted text to the clipboard, but will not erase that portion of text from the help file. Portions of text held on the clipboard can be inserted into your program with the PASTE option on the editor menu.
- PASTE – this option is normally disabled (grayed out). It only becomes active when the MODIFY or INSERT options are selected. When active you can transfer text from the clipboard to the point immediately after the current cursor position in the help file. You can position the cursor anywhere in the text box by clicking the mouse at that point.
- MODIFY – this is a toggle action option that allows you to edit the existing help file text. For example, you may include additional explanatory notes or more examples to the existing topic, or you may correct mistakes if you find any. The options CUT and PASTE also become active. First select the topic you wish to modify from the

drop-down topics list, then click the MODIFY option. When you have completed your changes select the MODIFY option again. This will erase the active tick mark and will disable CUT and PASTE. At this point you will be given the option to SAVE your modified topic or DISCARD it. You must exit the HELP window for your modified topic to appear in the drop down list.

- ADD NEW – this is a toggle action option that allows you to add extra topics to the help file. First select the ADDNEW option to clear the text window ready for you to type in your topic. You must begin your topic with a title inside angle brackets, e.g., <TITLE> and this will ensure that your topic will then appear in the alphabetical drop down list. Your topic can be of any length. When you have finished, click the ADD NEW option again. At this point you will be given the option to SAVE your new topic or DISCARD it. The ADD NEW toggle option will then be turned OFF and CUT and PASTE will be disabled. You must exit the HELP window for your new topic to appear in the drop down list.
- IMPORT – is an alternative to ADD NEW. It allows you to import additional help information that has been saved in an external file. In this instance multiple help topics can be inserted in one operation. Each imported topic must begin with a title in angle brackets. The file import file can contain any number of topics. A dialog box will request the name of the file and its entire contents will be appended to the current help file. This is a very simple way to update your help file using files generated by others, e.g., True BASIC Forum Members or by True BASIC Inc. You must exit the HELP window for your imported topics to appear in the drop down list.

CONTENTS

Selecting any one of the following items determines which help file the editor will use. There are currently eight different help files that cover various aspects and library modules included with True BASIC. In turn this determines the list of topics that you can select from the drop-down list.

- USING THE EDITOR – is a series of topics related to using the editor and the help feature. The topics are arranged alphabetically. This item is common to all editions of True BASIC.
- FUNCTIONS – this section lists and explains all the built-in functions within True BASIC and again it is common to all editions. Most topics contain code that can be copied to your programs.
- STATEMENTS – this section lists and explains all the statements in True BASIC. Most topics contain code that can be copied to your programs. This section is common to all editions of True BASIC.
- TRUECTRL – this sections details all the sub-routines in the library module and explains the syntax and how to use each routine with code examples that can be copied directly to your programs. This option is not available to users of the Bronze edition, instead, BronzeTC is offered, that has a reduced set of windows controls and objects.
- TRUEDIAL – this sections details all the sub-routines in the dialog box library module and explains the syntax and how to use each routine with code examples that can be copied directly to your programs. This option is not available to users of the Bronze edition, instead, BronzeTD is offered, that has a reduced set of dialog boxes.
- TRUECTX – this sections details all the sub-routines in the extended color and text library module and explains the syntax and how to use each routine with code examples that can be copied directly to your programs.

This option is not available to users of the Bronze edition.

- TRUETDX – this section details all the sub-routines in the extended dialog box library module and explains the syntax and how to use each routine with code examples that can be copied directly to your programs. This option is not available to users of the Bronze edition.
- FORMS – this section describes how to use the FORMS program to create windows and objects and automatically generate code. This option is not available to users of the Bronze edition.

Note: The editor automatically reads all TXT files that reside in the TBhelp folder and creates the CONTENTS list from these files. To add another help file to this list, all you have to do is drop the file into the TBhelp folder and the editor will do the rest.

If you wish to add more help files, you may use Notepad or the TBeditor to create additional menu items.

SPECIAL EDITOR FEATURES

Apart from the special features available under the menus, the editor also has some special features that assist with programming. For example:

BLOCK COMMENTS

If you highlight a section of code lines and then press the exclamation mark key (!), then the whole block of text will be commented. Similarly, if you highlight a block of text that already has leading comment marks and you press the same key, then all the comment marks will be removed. This feature is particularly useful for temporarily “removing” a block of lines, doing a test run, and then replacing the same lines.

MODIFYING CODE LINES

Suppose you want to modify a few lines here and there in a large program, but just in case some of the modifications don't work out you need to remember which lines you added or changed. The editor has the answer to this problem because you can make all these changes in a different color, merely by adding a comment mark followed immediately by the percent sign (!%) at the end of the line of code. This will color the whole line in RED. If you use the hash sign (!#) at the end of a line of code then it will be colored BLUE. Erasing the comment and the percent or hash signs will restore the normal color. Your programs can be run in the normal way regardless of the color.

RENUMBERING CODE LINES

Although it is not recommended, some people still prefer to use line numbers at the start of each line of code. The editor has a built-in feature that allows users to renumber programs merely by putting the following comment on the first line:

```
100 !AUTOLINENUM
```

The first number (100) is chosen by the user and refers to the new start number of the first line of code, and the increments between following line numbers will be 10. In the example, this means the first line of code will be numbered 100, the second line will be 110 and the third line will be 120, and so on. As soon as you type AUTOLINENUM your line numbered program will be instantly re-numbered for you.

RIGHT CLICK MENU

Some of the most often used main menu items have been included in a right-click shortcut menu, which appears when and where you click the right hand button of the mouse.

COMMAND LINE

Earlier versions of True BASIC (e.g., Version 3.05) ran under the MS DOS operating system and the editor occupied the upper portion of the screen, with the lower portion reserved for commands. This editor was very simple but extremely effective, even though it had a very limited menu. Commands were abbreviated to three letters to simplify typing, and many commands were assigned to the function keys F1 to F12.

Nowadays, in the latest version of the editor, most of the commands are available as menu items so you hardly ever need to use the command line. However, some users may prefer to use typed commands over menu-accessed commands so this feature has been retained. The command line is located below the editing page on the right hand side. Click inside this information box and it will turn blue, allowing you to type a command.

NOTE: It is important to note that True BASIC commands are used only in the Command Box. They may not be used inside your True BASIC programs.

One instance where you might use the command line is to create a temporary ALIAS, i.e., a file path that only remains valid until you switch off the editor. Usually you will want to keep an ALIAS permanent, in which case you should use the ALIAS menu item.

ALIASES

The ALIAS command allows you to define aliases for different types of files used by True BASIC.

Aliases for library, help, do, and script files are already included as default settings. Aliases are used to tell True BASIC where to look for a certain type of file whenever a file of that type is needed. This is most easily visualized in terms of an example. Take the following command:

```
alias library, "", \TRUE\TBLIBS\
```

When entered, this command tells True BASIC where to look whenever a library file is needed. True BASIC assumes that any file specified in a LIBRARY statement or LOAD command is a library file. When such a file is needed, the above command will force True BASIC to look first in the current directory being used by the editor. This current directory is represented by the null string in the above command. If True BASIC cannot find the file in the current directory it will go to its home directory on the current drive, enter the subdirectory named "TBLIBS," and look there for the needed file. If the file is not there either then it will look in the other alias directories.

You can specify any valid file or directory path for True BASIC to search. You can also place the paths to be searched in whatever order you wish to speed up the search process when looking for frequently used files. For instance, if you have a directory which contains utility libraries that you use in most programs, place it first in the ALIAS command line. True BASIC stops searching once it has found the needed file. Once an ALIAS command has been executed, the defined alias will stay in effect until a new ALIAS command is executed for the same type of file, or until you exit True BASIC.

Personally I prefer not to use aliases. I prefer to type in the full path name of any file that I use. I know that this involves slightly more typing but at least I am certain that True BASIC will find my file. My advice to you would be to do the same.

PART II

Learning the basics

Most of this section is reasonably obvious. Don't worry about the technical names that this section contains. The important point is that you should understand how True BASIC uses these concepts.

In this book, where appropriate, I have attempted to illustrate all the concepts and strategies with examples of code. Sometimes these examples are not as clear on the printed page as they are in the True BASIC editor, because the lines of code are frequently wider than the printed page. I could have resorted to using the True BASIC trick of putting ampersand characters at the end of one line and the beginning of the next. In most cases it makes matters worse. The problem is also compounded when, for extra clarity, I have indented lines of code which also causes the lines to run off the edge of the page. For simplicity I have used the normal convention of wrapping the lines like normal text. I apologize in advance if this causes any confusion. At least you can always resort to the corresponding file that can be downloaded from the True BASIC website, related to this book, and view the code correctly with the TBeditor.

Chapter 2

DATA

Before you begin any programming you need to understand something about the data that your instructions will use. We already know that True BASIC, like us, recognizes numbers and strings (words). There are also special cases in these two types of data; variables, constants and arrays. In the case of variables, it is common practice to represent numbers and strings with names, i.e., letters or words, just as you would in algebra. These representative letters or words are called **VARIABLES**, because they are general-purpose representations and can be assigned any value while the program is running. A variable name must begin with a letter, must not contain spaces and must be less than 32 characters long. Other than those restrictions you are free to use whatever numbers and letters you like. As a general rule you would be well advised to make variable names meaningful and relevant to the data. If you use simple single character letters such as *(x)* and *(y)* then you will soon forget what these variables stand for, whereas if you use meaningful names such as *total* or *difference* then it is obvious at a glance what they mean.

NUMBERS

I mentioned earlier that some programming languages use a variety of types of numbers, e.g., integers, floating point decimals, etc., and of course you have to tell the computer up front which of these types you are using (C is like this).

True BASIC is much smarter because it only uses one type of number, the IEEE standard 64 bit format, otherwise known as double precision floating point. As far as you are concerned any numbers that you use will be converted to this standard where each number is represented within 8 bytes. Just 8 bytes doesn't

sound much but I seriously doubt if you will ever need numbers as big as $1.7876931 \times 10^{308}$ (which is monstrously large) or as small as $2.2250739 \times 10^{-308}$ (which is less than microscopic). Nor will you normally need an accuracy better than 14 decimal places.

Some of you will have seen fragments of computer code, or you can take a look at some of the example demonstration programs. You will probably say it looks suspiciously like math, or more precisely, algebra. This could easily put some of you off programming even before you start. Let me assure you that this is not the case. You do not need to be good at math to be a good programmer. What you see is just a shorthand notation for logic; it certainly is not math. Here is a simple illustration:

```
LET a=2
LET b=3
LET x=a+b
```

It certainly looks like algebra, but what these lines are saying is:

Let the new value of the variable (a) be equal to 2
Let the new value of the variable (b) be equal to 3
Let the new value of the variable (x) be equal to (a) plus (b)

If you are not yet convinced, how about the following example:

```
LET x=x+1
```

This is clearly not algebra because it is impossible, but it makes absolute sense in True BASIC:

Let the new value of (x) be equal to the present value of (x) plus 1

If the present value of x is 2 then the new value will be 3.

You can see from these very simple examples that we are using letters of the alphabet to represent numbers. In True BASIC you can use words or phrases rather than individual letters, but the principle remains the same. Letters or words that represent numbers are called NUMERIC VARIABLES. Numbers on their own are called NUMERIC CONSTANTS. The value of PI (3.14159265 approx.) is a good example of a constant.

STRINGS

Representing numbers with words or letters does not just apply to numbers; you can do the same with strings. This is something you definitely do not see in algebra. Firstly we need to tell True BASIC that we are dealing with strings so we add the dollar sign to STRING VARIABLES, e.g.,

```
LET a$="A"
```

In True BASIC this means:

Let the new contents of the string variable (a\$) be equal to the letter A.

Why do we need the quote marks, you ask? This all goes back to the computer being dumb. If we just typed, LET a\$=A, then the computer would read this as:

Let the new contents of the string variable (a\$) be equal to the numeric variable A.

This doesn't make sense to you and me, and it certainly does not make sense to the computer. Hence we use quote marks to show that A is a letter (string) rather than a numeric variable. Strings inside quote marks are called LITERAL STRINGS or STRING CONSTANTS.

We can also add strings together, e.g.,

```
LET b$="hot"  
LET c$="bed"  
LET a$=b$ & c$
```

In this case we use the ampersand sign rather than the plus sign because we are not really performing an arithmetic addition process on the strings, we are just running the two words together to make "hotbed". This is known as **CONCATENATION**. You can see from this simple illustration how casually we use our natural language because we assume that others will know that you cannot add words mathematically so it must mean just joining the words. The True BASIC translation reads:

Let the variable (b\$) be equal to the word hot.
Let the variable (c\$) be equal to the word bed.
Let the variable (a\$) be equal to the variable (b\$) joined to the variable (c\$), i.e., hotbed

The convention of using a dollar sign to denote that we are dealing with a string makes reading your code much easier. Some computer languages do not use this convention. Instead you have to define which variables are used for numbers and which variables are used for strings right at the start of your program. Reading code like this can be extremely difficult because you have to remember all the definitions.

Strangely, string variables can also look like numbers. It makes sense if you think about it. If a string can be composed of symbols that represent letters of the alphabet, then it is obvious that they could be composed of punctuation or even numerals, e.g.,

```
LET a$="abcDEF"  
LET b$="$%&*:/<>?"
```

```
LET c$="5.1467"
```

As far as the computer is concerned "5.1467" is not a number: it is a string. Just as we humans don't think of the words "five point one four six seven" as being a real number: at least we wouldn't use these words in a calculation. We would convert the words in our head into a real number, 5.1467, before we start any calculations. True BASIC has functions that let the computer do the same sort of thing, i.e., it can convert strings to numbers as well as numbers to strings.

PARTIAL STRINGS

True BASIC allows you to divide up whole strings into partial strings or sub-strings. For example, we could have the following string:

```
LET whole$ = "Mary had a little lamb, its fleece as  
white as snow."
```

Suppose we want to divide this up into two partial strings at the comma:

```
LET first$ = whole$[1:23]  
LET second$ = whole$[24:52]
```

The numbers inside the square brackets refer to the character positions within the string. In the first case the partial string consists of the first to the twenty-third letter. The second partial string consists of the twenty-fourth to the fifty-second letter.

Partial strings are no different to any other string, it is just a unique and convenient way that True BASIC uses to split strings into small pieces. This simple notation is extremely powerful and flexible and in terms of string handling it sets True BASIC apart from other dialects of BASIC and other languages.

You can use the partial string feature to substitute existing words or letters in a string with something else. For example, in the above `whole$` string we can substitute the following:

```
LET whole$[48:51] = "coal"  
LET whole$[39:43] = "black"
```

The string `whole$` will now read:
Mary had a little lamb, its fleece as black as coal.

ARRAYS

For convenience we sometimes arrange numbers or words together into groups with a common title or name. For example, the number of days in each month of the year could be grouped together in a list running from 1 to 12. Again for convenience we might use a single variable name for this list, e.g., *month*. This list is known as a NUMERIC ARRAY where each item in the list is identified by a sub-script inside brackets, e.g.,

The first item in the list is `month(1)` with a value of 31
The second item in the list is `month(2)` with a value of 28 or 29
The third item in the list is `month(3)` with a value of 31 and so on all the way up to 12.

Similarly we could group the names of the days of the week into a list represented by the single string variable name *day\$*, running from Sunday to Saturday, e.g.,

The first item in the list is `day$(1)`, i.e., Sunday
The second item in the list is `day$(2)`, i.e., Monday
The third item in the list is `day$(3)`, i.e., Tuesday
A list of string data using the same variable name is known as a STRING ARRAY.

Despite the unfamiliar word, array, the concept is not that difficult to grasp. Remember that arrays are just lists of data that share a common variable name. At this stage you may not

appreciate the significance or even the need for arrays, but later on you will get to know how useful they can be.

Let us take the concept of arrays a little bit further. So far we have only considered simple lists, but what about lists that look like tables or spreadsheets; in other words lists that have more than one column. Let us suppose we have a table or spreadsheet where the data is arranged in rows and columns. We can still use a single array variable name, e.g., *table*, to represent the whole thing. The convention we use to identify any item in the table is to use its row and column position, e.g., *table(2,6)*. In other words *table(2,6)* is the item in the second row in the sixth column. How simple and logical is that? An array like this is known as a TWO DIMENSIONAL ARRAY. True BASIC allows you to use multiple dimensions that are limited only by the memory capacity of your computer.

Arrays are the exception to the rule when it comes to defining them at the start of your program. You must 'dimension' or define any arrays you want to use before you use them. The best place to do this is right at the start of your program. You don't necessarily have to say how big the array is, i.e., how long the list is, because you can redimension the array just before you use it. Many computer languages do not have this ability to dynamically redimension arrays—even the earlier versions of C did not have this capability—but when you start to use arrays you will realize just how necessary and useful this feature is.

Chapter 3

OPERATORS

In True BASIC, a range of symbols are used to tell the computer that a particular operation is required. Some of these OPERATORS are obvious because we humans use them for the same purpose. There are three types of operators: arithmetic, relational, and logical.

ARITHMETIC OPERATORS

You have already seen some arithmetic operators in the above examples, i.e., symbols that carry out an arithmetic function, such as the plus and minus signs whose purpose is obvious. Operators occur between constants or variables or both in an expression, e.g., $a+b$. Less obvious is the asterisk (*), the multiplication symbol, which we have to use because the normal multiplication symbol (x) could be confused with being a variable. Even less obvious is the caret (^), which is used to indicate exponentiation (powers). For example, a^2 is written as a^2 .

Here is another opportunity to demonstrate how much we take for granted in our normal casual speech. Suppose we ask the question, "What is 3 plus 4 times 6?" One possible answer is "An arithmetical problem." This answer is correct but not what you are looking for because you neglected to say "What is the VALUE of 3 plus 4 times 6?" There are two possible answers: 42 and 27. This is because we have not specified the order in which the calculation should be done, e.g., $(3+4) \times 6$ or $3+(4 \times 6)$. Clearly we need to be more exact when we ask the computer such questions.

Here is a list of the arithmetic operators used in True BASIC in their order of priority, i.e., the order in which the computer will deal with them:

Symbol	function	True BASIC	English
()	brackets	$x=(a+b)/4$	$(a+b)/4$
^	exponentiation	$x=a^2$	$x=a^2$
/	division	$x=a/b$	$x=a/b$
*	multiplication	$x=a*b$	$x= a \times b$
+	addition	$x=a+b$	$x=a+b$
-	subtraction	$x=a-b$	$x=a-b$

Brackets are the top priority, meaning whatever is inside the brackets is the first to be processed. You can also use brackets to force the computer to deal with an operation of lower priority first, e.g., $x=(a+b)/4$ where we are forcing the addition process to be carried out before the division process.

RELATIONAL OPERATORS

Relational operators are used to compare two variables in a logical expression or conditional statement where the answer is either true or false. Expressions of this type are called **BOOLEAN EXPRESSIONS**. For example, if (a) is greater than (b) then do this action, but if not then do something else. In other words if the expression is true then do one action but if it is false then do something else. Most of these operators are obvious and easily recognizable.

Symbol	True BASIC	English
>	$a>b$	a greater than b
>=	$a>=b$	a greater than or equal to b
<	$a<b$	a less than b
<=	$a<=b$	a less than or equal to b
=	$a=b$	a equal to b
<>	$a<>b$	a not equal to b

The interesting point is that relational operators can be used with string variables as well as the more obvious numeric variables. The comparison of strings is done alphabetically but

with a subtle twist. As you know, True BASIC doesn't care whether you use upper or lower case but unfortunately the computer does know the difference. As far as the computer is concerned the alphabet is arranged upper case A to Z followed by lower case a to z. The computer recognizes a standard set of numbered keyboard characters called the ASCII character set, and it is this set of numbers that the computer uses to make comparisons. Since we as humans do not care about upper and lower case, you will often find it easier to simplify everything by converting string variables into upper or lower case when you make logical comparisons. This can be done with two very useful functions; `ucase$` and `lcase$`, but more about that later.

LOGICAL OPERATORS

In addition to the relational operators above, there are logical operators that allow you to construct logical expressions of greater complexity. In simple terms you can compare more than just a pair of logical expressions. Again, you will recognize and understand these operators even if you have never thought of them in this way before.

Operator	True BASIC	English
NOT	<code>a>b NOT c<d</code>	True if a greater than b but NOT if c less than d
AND	<code>a>b AND c<d</code>	True if a greater than b AND c less than d
OR	<code>a>b OR c<d</code>	True if a greater than b OR c less than d

Once again this applies to strings and numeric expressions. Brackets can also be used around expressions to define the order that the computer deals with them.

SUMMARY

This is a good point to summarize what you should know and remember:

- ❑ String is a general name for letters, words, phrases or blocks of text.
- ❑ Constants are values or strings that do not change during a program run.
- ❑ String constants must appear inside quote marks.
- ❑ Variables are values or strings that may change during a run.
- ❑ Variable names must begin with a letter and must contain no spaces.
- ❑ String variable names must terminate with the dollar sign (\$).
- ❑ True BASIC has a unique way to indicate partial strings.
- ❑ Arrays are lists of numbers or strings that share a common variable name.
- ❑ Arrays can have more than one dimension.
- ❑ Arrays must be defined (dimensioned) before they can be used.
- ❑ True BASIC uses special symbols for certain arithmetic operations.
- ❑ Arithmetic operators obey a strict order of precedence.
- ❑ Variables and expressions can be compared true or false.
- ❑ More than one pair of expressions can be compared true or false.

Chapter 4

INPUT and OUTPUT

This is where things start to get interesting; the point where you find out how to get stuff into the computer and how to get stuff out again. If you start with an untitled (blank) window, you can type the examples of code in the TEditor in order to follow what is going on. Technically, this section should come later, but it is introduced here so that you can follow examples by typing the code at your computer and running the code to see what happens. It is important at this very early stage for you to create programs just to reassure yourself that the process is quite simple.

SCREEN OUTPUT

The computer can send data or information (output) to any device; the monitor, a disk drive, a USB memory device or a printer. In this instance we will only consider the monitor.

To display data on the monitor screen we use the instruction PRINT. This is easy to remember because we are effectively printing stuff on the screen.

Let us take a very simple example of printing a string constant:

```
!Filename: WT4_001.TRU
!Author: J.R.Arscott
!Purpose: Simple Hello program

PRINT "My name is John."
END
```

By the way, you have just written your first computer program. Now run this example by clicking the RUN menu and selecting RUN, and you will see My name is John on the screen. In some earlier versions of True BASIC it was necessary to begin with the CLEAR instruction in order to initiate the output window. In

the latest version the output window will be automatically shown.

By the way, all examples of code are available to download from the True BASIC website. Save them in a directory on your hard drive so that you can copy them into the editor if you want to avoid typing. The examples are identified by WT followed by the chapter number, followed by an underscore, followed by a serial number.

At this point the output window will tell you to press any key (in the blue band at the top). Pressing any key will return you to the editor window. Erase the two program lines by clicking the EDIT menu, select SELECT ALL, then press the DELETE key on the keyboard. This will remove all the code ready for the next example.

Now let us look at a previous example of printing a string variable:

```
!Filename: WT4_002.TRU
!Author: J.R.Arcott
!Purpose: Sub strings program
```

```
LET whole$ = "Mary had a little lamb, its fleece
as white as snow."
PRINT whole$
END
```

Run this program in the same way as before. As you would expect, you will see this line from the well-known nursery rhyme. Press any key to return to the editor. Do not erase the code, instead use the arrow keys or the mouse to position the cursor just in front of the instruction END, then press return. This action will create an extra line for you to insert some more

code. Press return several times to create enough space then add the extra three lines:

```
!Filename: WT4_003.TRU
!Author: J.R.Arscott
!Purpose: Modified sub strings program
```

```
LET whole$ = "Mary had a little lamb, its fleece
as white as snow."
PRINT whole$
```

```
LET whole$[48:51] = "coal"
LET whole$[39:43] ="black"
PRINT whole$
```

```
END
```

Run the program and you will see the original line and below it you will see the revised line containing the substituted words. Programming is not so difficult after all.

There are several variations of the PRINT instruction but we will deal with those later. For the moment, you now know how to get stuff out of the computer.

KEYBOARD INPUT

At this stage we will only consider using the keyboard as our means of communicating with the computer, although you should know that there are a number of other ways, which we will deal with later.

Getting data into the computer is called INPUT and it will come as no surprise that True BASIC uses the same word. For example, suppose we have a program that orders shoes from a central depot. Erase any existing text from the editor window then enter the following code:

```
!Filename: WT4_004.TRU
!Author: J.R.Arscott
!Purpose: Simple input program
```

```
INPUT  shoe_size
PRINT  shoe_size
END
```

If you run this tiny program you will see a question mark on the screen. At the keyboard, type in any number and press the RETURN key. You will immediately see the shoe size printed below your input.

The first problem with the INPUT instruction is that it gives you no warning about what sort of data you are expected to input. You know, because you wrote the program, but nobody else would know. What we need is some sort of prompt for the user to tell them what the program expects in the way of input. True BASIC is way ahead of us here with the instruction INPUT PROMPT. Here is how we use it:

```
!Filename: WT4_005.TRU
!Author: J.R.Arscott
!Purpose: Input prompt sub strings program

INPUT PROMPT "Enter the shoe size ": shoe_size
PRINT "The shoe size you require is ";shoe_size
END
```

Note the space after the word 'size' and after the word 'is'. This is necessary otherwise the numerical value will appear immediately after these words and the result just won't look right. Make sure you put the colon in the INPUT PROMPT statement, and make sure the semi-colon is in the PRINT statement. If you get these wrong the computer will complain

with an error message. Remember, everything has to be word perfect.

Now let us try entering a string by adding the following code:

```
!Filename: WT4_006.TRU
!Author: J.R.Arscott
!Purpose: Modified input program

INPUT PROMPT "Enter the shoe size ": shoe_size
PRINT "The shoe size you require is ";shoe_size

INPUT PROMPT "Enter the color (Black/brown) ":
shoe_color$
PRINT "The shoe color you require is ";shoe_color$

END
```

When you enter the size or the color you must always press the RETURN key in every case, otherwise the program does not know when you have finished.

GET KEY

This statement waits for you to input a single character (letter or number) from the keyboard without the user needing to press the <RETURN> key. It returns a number that corresponds to the ASCII key code for the key that the user has pressed. You can print the result by converting this number to a letter of the alphabet, e.g.,

```
!Filename: WT4_007.TRU
!Author: J.R.Arscott
!Purpose: Using GET KEY for input program

    GET KEY keynum
    PRINT CHR$(keynum),keynum
END
```

GET MOUSE

You will know from your own experience when using NOTEPAD or WORD or even the True BASIC editor that you spend at least half of your time moving and clicking the mouse. We will deal with this method of inputting data only briefly here, but in much greater detail later.

```
GET MOUSE, x,y,button
```

Allows you to find out where the mouse is pointing and what the current button position is. Normally True BASIC only works with the LEFT mouse button, although there are library modules such as TrueCTRL that extend the range of buttons.

The button positions are indicated by the following numbers:

0= no button down

1= button down

2= button clicked at this point

3= button released at this point

4= button shift-clicked at this point

SUMMARY

I hope you will agree that your first expedition into programming has been simple and painless. In this section you have discovered how to:

- ❑ Use the editor to create and run a program.
- ❑ Use the PRINT instruction to show data on the monitor.
- ❑ Use the INPUT instruction to gather data from the user.
- ❑ Use the INPUT PROMPT instruction.
- ❑ Use the END instruction to stop the program.
- ❑ Use GET KEY.
- ❑ Use GET MOUSE.

Chapter 5

DECISIONS

You already know that relational operators allow the computer to do one thing if an expression is true, and another action if the expression is false. Essentially the flow of logic in your program can take either of two directions depending on whether an expression is true or false. In effect you are asking the computer to make a decision based on the comparison. True BASIC has two instructions for making decisions that control which of your instructions are carried out (executed) and which instructions are skipped.

IF....THEN....ELSE

This instruction uses relational operators to compare two or more values or strings. If the comparison is true, then the code that follows THEN will be executed. If the condition is false, then the code following ELSE will be executed. If ELSE is missing then nothing will happen if the condition is false. To make sure this concept is crystal clear let us take the following example:

```
!Filename: WT5_001.TRU
!Author: J.R.Arscott
!Purpose: Using decisions IF THEN ELSE program

INPUT  PROMPT "Enter the shoe size ": shoe_size
PRINT "The shoe size you require is ";shoe_size

INPUT  PROMPT "Enter the color (BLACK/BROWN) ":
shoe_color$

IF shoe_color$ = "BLACK" THEN
PRINT "The shoe color you require is BLACK "
ELSE
PRINT "The shoe color you require is BROWN "
END IF

END
```

When you run your program, you must enter the shoe color in capital letters. If you enter 'black' in lower case, then the condition is no longer true because 'black' is not the same as 'BLACK'. Since the condition is now false the program will print the shoe color is BROWN. To overcome this problem let us make a small addition to the conditional clause:

```
!Filename: WT5_002.TRU
!Author: J.R.Arscott
!Purpose: Using decisions IF THEN ELSE improved program
```

```
REM: Program WorldTour004.TRU
INPUT PROMPT "Enter the shoe size ": shoe_size
PRINT "The shoe size you require is ";shoe_size
```

```
INPUT PROMPT "Enter the color (BLACK/BROWN) ":
shoe_color$
```

```
IF shoe_color$ = "BLACK" OR shoe_color$="black" THEN
PRINT "The shoe color you require is BLACK "
ELSE
PRINT "The shoe color you require is BROWN "
END IF
```

```
END
```

The comparison now takes care of upper and lower case input. You would think that we have sewn up the case, but what happens if you type 'Black', i.e., a mixture of upper and lower case. We are back again with BROWN shoes for the same reason as before. You see how stupid the computer is. The best way around this problem is to use one of True BASIC's built-in functions; the one that converts strings into upper case:

```
!Filename: WT5_003.TRU
!Author: J.R.Arscott
```

!Purpose: Using decisions IF THEN ELSE improved more program

```
INPUT  PROMPT "Enter the shoe size ": shoe_size
PRINT  "The shoe size you require is ";shoe_size

INPUT  PROMPT "Enter the color (BLACK/BROWN) ":
shoe_color$

IF ucase$(shoe_color$) = "BLACK" THEN
PRINT  "The shoe color you require is BLACK "
ELSE
PRINT  "The shoe color you require is BROWN "
END IF

END
```

In this case whatever you type as a shoe color will be converted into upper case when the comparison is made. Job done, you might think. Well, not entirely, because a user might ignore the prompt about the colors and enter something like RED. The program will obviously tell the user that they have chosen BROWN shoes. As always, True BASIC has a way out. It is possible to make multiple comparisons using ELSEIF as follows:

```
!Filename: WT5_004.TRU
!Author: J.R.Arscott
!Purpose: Using decisions IF THEN ELSE better program

CLEAR
INPUT  PROMPT "Enter the shoe size ": shoe_size
PRINT  "The shoe size you require is ";shoe_size

INPUT  PROMPT "Enter the color (BLACK/BROWN) ":
shoe_color$

IF ucase$(shoe_color$) = "BLACK" THEN
PRINT  "The shoe color you require is BLACK "
ELSEIF ucase$(shoe_color$) = "BROWN" THEN
PRINT  "The shoe color you require is BROWN "
ELSE
```

```
PRINT "The shoe color you require is not available."  
END IF
```

```
END
```

You can use as many ELSEIF instructions as are necessary to do the job. This little exercise demonstrates very clearly that when you use this decision structure you need to think about ALL the options; not just the obvious ones, and make sure you have them all covered, because you cannot rely on the computer to be anything but dumb.

Note that we have to use END IF at the end of the decision structure in every case.

There is an alternative way to solve the problem that does not require us to use ELSEIF. Instead we can 'nest' IF...THEN inside another IF....THEN structure as follows:

```
!Filename: WT5_005.TRU  
!Author: J.R.Arscott  
!Purpose: Using decisions IF THEN ELSE nested program
```

```
CLEAR  
INPUT  PROMPT "Enter the shoe size ": shoe_size  
PRINT "The shoe size you require is ";shoe_size
```

```
INPUT  PROMPT "Enter the color (BLACK/BROWN) ":  
shoe_color$
```

```
IF ucase$(shoe_color$) = "BLACK" THEN  
PRINT "The shoe color you require is BLACK "  
ELSE  
IF ucase$(shoe_color$) = "BROWN" THEN  
PRINT "The shoe color you require is BROWN "  
ELSE  
PRINT "The shoe color you require is not available."  
END IF
```



```
END IF
```

```
END
```

The purpose in showing this alternative is to show that each IF...THEN structure needs to be terminated with END IF. This example also demonstrates in a simple way what happens to the appearance of your code when the logic gets more complicated. Once more True BASIC comes to the rescue with the DO FORMAT option under the editor RUN menu. Select this option and see what happens to your code. I think you will agree that it is easier to read and understand the logical sequences involved when the text is progressively indented.

IF KEY INPUT

There is one special case of the IF THEN clause which is used to determine if any key has been pressed on the keyboard. It is usually used in conjunction with GET KEY to find out which key has been pressed, e.g.,

```
!Filename: WT5_006.TRU
!Author: J.R.Arscott
!Purpose: Using KEY INPUT program

FOR x=65 to 92
    PRINT chr$(x)
    PAUSE 1
    LET keynum=0
    IF KEY INPUT then
        GET KEY keynum
        PRINT keynum
        PAUSE 5
    EXIT FOR
END IF
NEXT x
END
```

Obviously if a key has not been pressed, then the computer ignores GET KEY. The KEYNUM is a numerical value (NOT the actual letter). This number refers to the ASCII table of reference numbers for each letter of the alphabet (and punctuation). Note that capital letters have different numbers to lower case letters.

SELECT CASE

As an alternative to IF...THEN...ELSE we can use the instruction SELECT CASE, particularly when we have a large number of options. The big difference is that IF...THEN...ELSE allows us to make comparisons between variable expressions, whereas SELECT CASE can only make comparisons between one variable and any number of constants. Here is a very simple program that prints the name of the day of the week in response to the input of a number from 1 to 7.

```
!Filename: WT5_007.TRU
!Author: J.R.Arscott
!Purpose: Using SELECT CASE program
CLEAR
INPUT PROMPT "Enter a number (1 to 7) ":number

SELECT CASE number
CASE 1
    LET day$="Monday"
CASE 2
    LET day$="Tuesday"
CASE 3
    LET day$="Wednesday"
CASE 4

    LET day$="Thursday"
CASE 5
    LET day$="Friday"
CASE 6
    LET day$="Saturday"
CASE 7
```

```

        LET day$="Sunday"
CASE ELSE
        LET day$="not a weekday."
END SELECT

PRINT "The day is ";day$
END

```

Note the use of CASE ELSE to capture a number that is outside the limits we set. In this instance the decision structure covers seven options defined by constants not variables. However, we can use expressions for each case as long as the expressions do not contain variables, e.g.,

```

!Filename: WT5_008.TRU
!Author: J.R.Arscott
!Purpose: Using SELECT CASE ELSE program

```

```

CLEAR
INPUT PROMPT "Enter a number (1 to 7) ":number

SELECT CASE number
CASE 1
    LET day$="Monday"
CASE 2
    LET day$="Tuesday"
CASE 3
    LET day$="Wednesday"
CASE 4
    LET day$="Thursday"
CASE 5
    LET day$="Friday"
CASE 6 to 7
    LET day$="a weekend."
CASE ELSE
    LET day$="not a weekday."
END SELECT

PRINT "The day is ";day$
END

```

We could also have used `CASE 6 or 7`, or `CASE 6,7` or `CASE 6 to 7`.

SUMMARY

In this section you should know and understand how to:

- ❑ Create `IF...THEN...ELSE` decision structures that compare variables.
- ❑ Use `IF KEY INPUT` to determine if a keystroke has been made.
- ❑ Use `END IF` to terminate every `IF` decision structure.
- ❑ Use `ELSEIF` to create more options.
- ❑ Account for every possible option or rogue input.
- ❑ Nest one decision structure inside another.
- ❑ Use `DO FORMAT` to make the logic easier to read and understand.
- ❑ Create `SELECT CASE` structures that compare a variable with constants.
- ❑ Use `CASE ELSE` to account for rogue variable values.
- ❑ Use constant expressions in `CASE` comparisons.
- ❑ Use `END SELECT` to terminate every `SELECT` decision structure.

Chapter 6

LOGIC FLOW CONTROL

In addition to using decision structures to execute certain instructions and skip others, you can also repeat certain instructions. In the first case we will deal with repeating instructions a fixed number of times with FOR...STEP...NEXT, and in the second case we will look at unlimited repetition with DO...LOOP. All these structures come under the general heading of logic flow control, because they control how your program instructions are executed.

FOR...NEXT.....STEP

At some point in your program you may want to carry out the same action several times. Let us suppose you want to create a table of inch measurements with their corresponding values in centimetres. For simplicity we will only convert the measurements from 1 to 6 inches. Here is the code that will perform this task:

```
!Filename: WT6_001.TRU
!Author: J.R.Arscott
!Purpose: Using FOR NEXT STEP program
```

```
FOR i=1 to 6 STEP 1
LET centim=i*2.54
PRINT i,centim
NEXT i
END
```

In English the FOR line reads: starting with a value of (i)=1 and increasing (i) in steps of 1 for every loop, continue while (i) is less than or equal to 6. The keyword NEXT just means repeat the loop beginning at FOR. You can see that the value of (i) will

be greater than 6 when the loop stops. You can test this by printing the value of (i) immediately after the loop:

```
!Filename: WT6_002.TRU
!Author: J.R.Arscott
!Purpose: More FOR NEXT STEP program

FOR i=1 to 6 STEP 1
LET centim=i*2.54
PRINT i,centim
NEXT I
PRINT i
END
```

In fact (i) is representing the inch value, and the variable (centim) is representing the centimetre value (1 inch=2.54 centimetres). As the computer repeats the loop six times it prints the inch and centimetre values on a fresh line for each loop. Note that there is a comma between (i) and (centim). The comma tells the computer to leave a big space (tab) between the inch value and the centimetre value so that it looks nice on screen.

You will find that in the majority of cases where you use the FOR...NEXT loop, the step increment will always be 1. True BASIC assumes this is the default value so you don't need to include the word STEP 1. However, you still need to know how to use step because there will be times when you need it. Here are two examples:

```
!Filename: WT6_003.TRU
!Author: J.R.Arscott
!Purpose: Using STEP program

FOR i=1 to 6 STEP 1/2
LET centim=i*2.54
PRINT i,centim

NEXT i
```

END

In this first example instead of whole inches we are using half-inch intervals by using steps of half (or you can use 0.5).

```
!Filename: WT6_004.TRU
!Author: J.R.Arscott
!Purpose: Using negative STEP program

FOR i=6 to 1 STEP -1
LET centim=i*2.54
PRINT i,centim
NEXT i
END
```

In the second example we are counting backwards from six to one in steps of minus one to produce a list in descending order instead of ascending order.

Every FOR instruction must have an associated NEXT instruction to tell the computer which block of code lines are to be repeated.

In the same way that IF...THEN instructions can be nested, True BASIC allows you to nest FOR...NEXT loops. Here is a similar program that converts feet and inches into centimetres. The outer loop counts the feet and the inner loop counts the inches.

```
!Filename: WT6_005.TRU
!Author: J.R.Arscott
!Purpose: Using FOR NEXT nested program

FOR f=0 to 1
FOR i=0 to 11
LET centim=(i+12*f)*2.54
PRINT f;" ft",i;" ins",centim;" cm"

NEXT i
```

```
NEXT f
END
```

In a PRINT instruction a semi-colon causes the next item to be printed immediately, whereas a comma causes the next item to be printed with a TAB space.

Once again the whole program will look much better if you execute DO FORMAT so that you can see the nested FOR...NEXT loops more clearly.

The final feature that you need to know about FOR...NEXT loops is how to get out of them in an emergency. The instruction we use to quit from a FOR...NEXT loop is not surprisingly EXIT FOR.

```
!Filename: WT6_006.TRU
!Author: J.R.Arscott
!Purpose: Using EXIT FOR program

FOR f=0 to 10
  IF f>2 then
    EXIT FOR
  END IF
  FOR i=0 to 11
    LET centim=(i+12*f)*2.54
    PRINT f;" ft",i;" ins",centim;" cm"
  NEXT i
NEXT f
END
```

In this program we are printing all the centimetre values up to ten feet, but because our screen is too small to print all this data we have set a limit of two feet at which point we exit the loop.

DO LOOP

This type of loop is used when you don't know how many times you will have to use the loop. Since this loop will go on forever, you, the programmer, have to provide an escape route. Usually

a decision structure such as IF...THEN is used to determine when to escape and the instruction EXIT DO actually performs the escape. Here is a program that calculates the square of a number, starting at 1 and increasing by 1 each time it passes through the loop. At each pass you will be asked if you want to continue or exit.

```
!Filename: WT6_007.TRU
!Author: J.R.Arscott
!Purpose: Using DO LOOP program
```

```
LET number=0
DO
    LET number=number+1
    LET square=number^2
    PRINT number,"squared = ";square
    INPUT PROMPT "Continue? Y/N ": answer$
    IF ucase$(answer$)="N" THEN
        EXIT DO
    END IF
LOOP
END
```

Just like FOR...NEXT, DO...LOOP can be nested virtually as many times as you like. The next example shows a nested loop where the inner loop continues to double the squared number until the value is greater than 100. You can see that EXIT DO only applies to the loop that it is inside and not to any other.

```
!Filename: WT6_008.TRU
!Author: J.R.Arscott
!Purpose: Using DO LOOP nested program
```

```
LET number=0
DO
    LET number=number+1
    LET square=number^2
    PRINT number,"squared = ";square
```

```

LET double=square
DO
    LET double=double*2
    IF double>100 THEN
        EXIT DO
    ELSE
        PRINT "double = ";double
    END IF
LOOP
INPUT PROMPT "Continue? Y/N ": answer$
IF ucase$(answer$)="N" then
    EXIT DO
END IF
LOOP
END

```

This example demonstrates very clearly how you can exit from a loop either with user input or automatically using the value of program variables. You can also see how a very few instructions can be combined into complex and powerful logical sequences.

Apart from EXIT DO there are two other methods of escaping from the DO...LOOP. We can set escape conditions on either the keyword DO or on the keyword LOOP. The conditions are similar to IF...THEN decision structures except we use either WHILE or UNTIL. Here is the 'square' program again, but this time we are using LOOP UNTIL to escape the loop.

```

!Filename: WT6_009.TRU
!Author: J.R.Arscott
!Purpose: Using DO LOOP UNTIL program

```

```

CLEAR
LET number=0
DO
    LET number=number+1
    LET square=number^2
    PRINT number,"squared = ";square
LOOP UNTIL square>100

```

```
INPUT PROMPT "Continue? Y/N ": answer$
LOOP UNTIL ucase$(answer$)="N"
END
```

Notice how the escape line at LOOP is almost readable English. By now you should be able to read each line of this program as if it was written in English.

Alternatively we could have used WHILE to do the same job, but we would have to reverse the decision clause using “Y” instead of “N”.

```
!Filename: WT6_010.TRU
!Author: J.R.Arscott
!Purpose: Using DO LOOP WHILE program
```

```
LET number=0
DO
    LET number=number+1
    LET square=number^2
    PRINT number,"squared = ";square
    INPUT PROMPT "Continue? Y/N ": answer$
LOOP WHILE ucase$(answer$)="Y"
END
```

Now it will be obvious to you, because I am drawing your attention to it, that in both of these examples the program flow passes through the loop at least once. There will be times when you want to skip the loop entirely unless certain conditions are true. You don't need to be a genius to recognize that putting the decision structure on the DO keyword will achieve this. You are becoming familiar with the square program so we will use it again to illustrate this point.

```
!Filename: WT6_011.TRU
!Author: J.R.Arscott
!Purpose: Using DO UNTIL LOOP program
```

```

LET number=0
DO UNTIL ucase$(answer$)="N"
    LET number=number+1
    LET square=number^2
    PRINT number,"squared = ";square
    INPUT PROMPT "Continue? Y/N ": answer$
LOOP
END

```

We can use the keyword WHILE in a similar fashion.

```

!Filename: WT6_012.TRU
!Author: J.R.Arscott
!Purpose: Using DO WHILE LOOP program

```

```

CLEAR
LET number=0
DO WHILE ucase$(answer$)="Y"
    LET number=number+1
    LET square=number^2
    PRINT number,"squared = ";square
    INPUT PROMPT "Continue? Y/N ": answer$
LOOP
END

```

If you are paying attention, you will realize that this version of the program doesn't work, and worse still, there are no error messages. The DO line does exactly what it says, but the reason it doesn't work is because the very first time you enter the loop, the variable (answer\$) has no value; it is neither (Y) nor (N). Hence the program skips the loop altogether. To get the program to work you need to insert,

```
LET answer$="Y"
```

at the very beginning. This will give the variable (answer\$) an initial value that will get it past the first decision point. The process of assigning a starting value to a variable is known as initialising. Some computer languages require that you initialise all variables before you start. True BASIC sets all numeric

variables to zero and all string variables to nothing as default conditions.

SUMMARY

In this section you have seen and should know and understand how to:

- ❑ Create a FOR...STEP...NEXT limited repeat loop structure.
- ❑ Use the STEP feature to increment upwards and downwards.
- ❑ Use a decision structure to EXIT FOR.
- ❑ Use NEXT to terminate every FOR structure.
- ❑ Nest one FOR..NEXT loop inside another.
- ❑ Create a DO...LOOP unlimited repeat loop structure.
- ❑ Use a decision structure to EXIT DO.
- ❑ Use the WHILE option to quit at DO or LOOP.
- ❑ Use the UNTIL option to quit at DO or LOOP.
- ❑ Nest one DO loop inside another.
- ❑ Use LOOP to terminate every DO loop.
- ❑ Use DO FORMAT to help read and understand the logic in loops.

At the end of Part 2 you have 21 core instructions (see below) at your fingertips to help you construct a program with any logical sequence you require.

CORE INSTRUCTIONS

So far you have learnt the following minimum list of core programming instructions that will allow you to write programs that actually work, and don't cause your computer to crash:

```
LET
DO
EXIT DO
WHILE
UNTIL
LOOP
FOR (STEP)
EXIT FOR
NEXT
SELECT CASE
CASE
END SELECT
CASE ELSE
IF THEN
ELSEIF
ELSE
END IF
END
INPUT
INPUT PROMPT
PRINT
```

Here is a very useful little routine that allows you to input stuff at the keyboard to illustrate how to use KEY INPUT and GET KEY and the other core instructions that you have learnt so far.

Make sure you read all the comments so that you understand how and why this routine works.

```
!Filename: WT6_013.TRU
!Author: J.R.Arscott
!Purpose: General purpose KEYBOARD input program

! Give the computer a simple task

FOR n=1 to 100

    PRINT n
```

```

! Zero all the variables
LET event$=""
LET word$=""
LET z1=0
LET z2=0
LET start=0
LET delay=0
LET type=0

REM: type=0 (string):type=1 (char)
LET delay=3
IF start=0 then
    LET start=time                ! sets the timer
going
END IF

DO

    IF key input then             ! looking for a key
stroke
    GET KEY z1                    ! get the ASCII
number
    IF type>0 then                ! only a single
letter expected
        LET event$="KEYPRESS"
        PRINT event$,z1
        EXIT DO
    ELSEIF z1=8 then              !back space
        !The BACK SPACE key=8 so take one letter
off the end
        LET length=len(word$)
        IF length>=1 then        ! remove last
character
            LET word$=word$(1:length-1)
        END IF
    ELSEIF z1=13 or z1=27 then
        ! The RETURN key=13 so end the string
        ! sends back string but NOT return
character
        LET event$=word$
        ! sends back length of string
        LET z2=len(word$)
        PRINT event$,z1,z2

```

```

        EXIT DO
    ELSE                                     ! a string expected
        ! Add another letter to the string
        LET word$=word$ & chr$(z1)
    END IF

    ELSEIF word$="" then
        ! no key press so exit
        ! nothing in the pipeline
        EXIT DO
    ELSEIF word$>"" and (time-start)>delay then
        ! The timer has timed out
        ! sends back string but NOT return character
        LET event$=word$
        ! sends back length of string
        LET z2=len(word$)
        PRINT event$,z1,z2
        EXIT DO
    END IF
LOOP
IF z1=27 then
    ! the ESC key=27 has been pressed so end
    EXIT FOR
END IF
PAUSE 1                                     ! slow things down
NEXT n
END

```

What this program does is to keep an eye on the keyboard and waits for some input from the user. The input can be either a single keystroke or a string.

In the case of single keystroke the program prints “KEYSTROKE” followed by the ASCII key number, followed by zero. In the case of a string the program prints the string, followed by the ASCII last pressed key number, followed by the length of the string.

If the value of (delay) is set greater than zero (e.g., 3 secs) then the user doesn’t even need to press <return> after typing a string.

We could have made this slightly less complicated by removing the stuff about (delay), but there is method behind this madness. Let's look at a similar general purpose program for checking what is happening to the mouse.

```
!Filename: WT6_014.TRU
!Author: J.R.Arscott
!Purpose: General purpose MOUSE input program

! First set the co-ordinates of the window
ASK PIXELS xpix,ypix
SET WINDOW 0,xpix-1,ypix-1,0

LET event$=""
LET word$=""
LET z1=0
LET z2=0
LET button=0
LET start_time=0
LET delay=0

! Keep watch on the mouse (LEFT button only)
! to see if the user has clicked it

PRINT " Press ESC key to end"

DO
  LET button=0
  DO

    GET MOUSE: z1,z2,button
    ! give the user time to release the button
    PAUSE 0.3
    IF button=1 then
      ! the mouse button is DOWN
      LET event$="MOUSE DOWN"
      ! set the computer to pause for a time to
allow
      ! the user to release the mouse button
      ! if it is still down then the user must
      ! be dragging the mouse
```

```

        EXIT DO
    ELSEIF button=3 then
        ! if the user has released the button then
        ! the button is UP
        LET event$="MOUSE UP"
        EXIT DO
    END IF

LOOP

PRINT CLEAR
PRINT event$, z1,z2

GET KEY key
IF key=27 then
    EXIT DO
END IF
LOOP
END

```

If you keep your finger on the mouse button then this program prints MOUSE DOWN, followed by the x and y screen co-ordinates in pixel units, where the mouse click was made.

If you click AND release the mouse button then the program will print MOUSE UP, followed by the x and y screen co-ordinates in pixel units, where the mouse was released.

In the WTLIB library module, both of these programs have been combined into one single sub-routine that looks after both keyboard and mouse input called:

```
SUB user(delay,event$,z1,z2)
```

Don't bother to type out this combined program in the editor because it has been done for you. Instead, OPEN the program called WT6_015.TRU into the editor, where you will find the routine, which you can RUN to see how it works.

You can use this sub routine in any program instead of INPUT or GET KEY or GET MOUSE. Later on I will introduce you to another, almost identical routine called TC_event, which you will find is virtually indispensable if you are using the TrueCTRL library module.

PART III

Beyond the Basics

Chapter 7

PROCEDURES

Now is the time to peek into the toolbox of goodies to see the range of additional True BASIC statements, functions and sub-routines, that will help you achieve virtually any task you set yourself. I am using the term ‘tools’ here as an umbrella for statements, functions and sub-routines. You don’t have to remember all these tools because you can always refer to the Reference Manual, you just need to know that they exist and how to use them. There are 238 additional built-in tools. The good news doesn’t stop there, because True BASIC has a LIBRARY feature that allows you to import even more tools written by other people. At the last count there were 1443 more library tools and the list is growing by the day. The best news is that you don’t need to know anything about the code that makes these tools work. You have already used one of the built-in functions, `ucase$`, in some of the examples. You have no idea of the code that makes it work, but you used it successfully nevertheless.

For convenience, because there are so many tools, we will divide them into smaller groups based on what they do. If I tried to give you details of the whole list in one go, you would be bored out of your mind long before we hit the half way mark, and you would forget most if not all of it. Not all of the tools will be discussed in this book, but they are included in the Reference Manual, so if you want to use them, you can. The purpose of this book is to encourage novices, not to frighten

them off. Even experienced programmers frequently refer to the Reference Manual to check the syntax and parameters of functions and sub-routines. It is not a crime: it is a very prudent precaution.

FUNCTIONS

In case you have any doubts about the difference between functions and sub-routines, let me define them. The next couple of paragraphs are crucial to writing good programs so make sure the concepts are firmly embedded in the grey matter between your ears before reading any further.

If the code that makes up a function or sub-routine occurs within your program, i.e., the code is located before the END statement, then this known as an INTERNAL procedure. If the code occurs after the END statement, or is in a separate program unit or module, then it is an EXTERNAL procedure.

Functions are blocks of code that perform specific tasks with your data and return a value to your program. Think of it this way; your program wants to do something to a piece of data so it uses a ready-made block of code (the function) to do the job. The block of code that provides this function lives somewhere else, it could be built-in to True BASIC, it could be in a library module, or it could even be part of your program (such as the VALSTR example). The items of data that the function works on are called the parameters and must be included in brackets after the function name, e.g.,

```
LET upper$=UCASE$(word$)
```

In the example above, UCASE\$ is the name of the function.

The string variable, word\$ is the piece of data you are sending to the function as a parameter. When the block of code inside the function has done its job it sends a value back to your

program either as a numeric value or as a string. The function does NOT change or modify your data (word\$). The reason for this is that when True BASIC executes a function it takes a copy of your data to work on. For this reason it is not a good idea to use arrays in functions because it may take a long time to copy a big array. If the value calculated by the function is a string then the name of the function must end with a dollar sign. If the value is numeric then the name must not end with a dollar sign. In the example the value is a string of upper case letters which we assign to the string variable upper\$.

Before you attempt to use a function that is not built into the True BASIC language you must define its name. The best place to do this is at the top of your program, e.g.,

```
DECLARE DEF valstr
```

From time to time you may want to find out the numerical value of a string. True BASIC has a built in function that carries out this task, called VAL(string\$). However, this function has an annoying side effect which causes an error if the string\$ contains any non numeric characters. The above example VALSTR works in exactly the same way as VAL except that it doesn't result in an error when non numeric characters are encountered.

Let me give you a simple example. Suppose your program calculates the circumference of a circle, so you ask the user to enter the diameter in inches. Sadly the user doesn't know how you have designed your program, so he or she could enter the value 5.25 or they could enter 5.25 inches. If you use VAL to determine the numerical diameter then the first option will be accepted but the second will result in an error because of the presence of the word "inches". The function VALSTR will accept both options without any errors. This function is included in the WTLIBS.TRU library module in the files that can be downloaded from the True BASIC website.

You do not need to name built-in functions like VAL before you use them, but you do need to put:

```
DECLARE DEF valstr
```

At the start of your program in order to use VALSTR.

SUB-ROUTINES

Sub-routines are blocks of code that perform specific tasks with your data, but do NOT return a value to your program. Instead, the sub-routine changes the value of your data. It follows that the names of all sub-routines do not end with dollar signs because they are just convenient labels, not values.

In the same way as functions, the block of code that makes up the sub-routine can be located anywhere; it could be built-in to True BASIC, it could be in a library module, or it could even be part of your program. If the sub-routine block of code is not located inside your own program, i.e., it is not located before the END statement, then you must include the data items as parameters inside brackets after the name of the sub-routine. If the sub-routine is located before the END statement in your program, you do not need to include data items as parameters because the code in the sub-routine already knows the values of all the data in your program.

Arrays can be used as parameters in both functions and sub-routines but only the array name is required followed by empty brackets, e.g., anyarray(). File channel numbers (see the section on file handling) can be used as sub-routine parameters but not for functions.

If you are paying attention you might want to know how it is possible for somebody to write some code for a function or a

sub-routine without knowing what names you are going to use as variables. The person who originally wrote the built-in function UCASE\$ cannot possibly know that we are using the variable name word\$. The answer is very simple, when you write a function or sub-routine you define what variable names you are going to use as parameters and you write the block of code accordingly. When a user calls the sub-routine or function, True BASIC matches the user's parameters with those that are used in the block of code. Obviously it is very important that when you include parameters, they must be of the same type (string or numeric) and in the same order as defined by the function or sub-routine.

In order to make the sub-routine perform its task, you must CALL the routine by its name (plus any parameters inside brackets), e.g.,

```
CALL myroutine(a,b,word$)
```

The sub-routine itself will be headed with the name, e.g.,

```
SUB myroutine(x,y,string$)
! lines of code
END SUB
```

Note that the sub-routine uses the temporary parameters (variables) *x*, *y*, and *string\$*. True BASIC matches these with your own personal parameters contained in the CALL, i.e., *a*, *b*, *word\$*. Now you can see why it is so important to use parameters in the correct order and of the correct type, because the code within the sub-routine will take the value of variable *a* as being equal to *x*, and the value of *b* as *y*, and the value of *word\$* as *string\$*.

When you are writing your own functions or sub-routines you may want to include a rapid exit or some means to by-pass the

code in the routine under some circumstances. EXIT DEF and EXIT SUB respectively can be used for this purpose.

Sub-routines and to a lesser extent functions are going to occupy a very large portion of all the programs that you write, even though at this stage you may not fully appreciate their usefulness or relevance. To illustrate this point, suppose we have a program that includes a HELP file. Access to the help file can be by selection from a menu, or by clicking on a help icon, or by a special keystroke (ALT-H). Your program will contain some code that detects which of these options the user has selected. Do you now write three identical blocks of code that reads the help file itself and generates a window to view the text in the help file? Of course not; you only write one block of code and make it into a sub-routine called something like Show_Help. Each of the above options; selecting the menu, clicking the icon or a special keystroke, just has a single line of code, all pointing to the same sub-routine:

```
CALL Show_Help
```

Ultimately, many of your programs will consist almost entirely of calls to sub-routines and functions. This strategy makes debugging your code so much easier. It also makes modifying your code much easier, with less risk of interaction with other parts of your program. You will also see the advantage of grouping associated sub-routines into self-contained units called library modules. In this way your program will be a logical structure in its own right, quite apart from the logic of the program code. For example, the True BASIC editor uses eleven library modules containing hundreds of sub-routines totalling almost twenty thousand lines of code. The main program in the editor only has less than 500 lines of code.

STATEMENTS

Statements are built-in instructions that consist of one or more keywords sometimes followed by one or more data items. The data is not enclosed in brackets as is the case with functions and sub-routines. Statements are part of True BASIC and you cannot create extra statements of your own.

SUMMARY

Here is what you need to know about functions and sub-routines:

- ❑ Functions are blocks of program code that perform a specific task.
- ❑ You must DECLARE a function before you use it except those built into the language.
- ❑ Functions return a string or numeric value.
- ❑ Function names are like variables, numeric or strings.
- ❑ Functions do not change the value of your variables.
- ❑ The function code can be located inside or outside your program.
- ❑ Sub-routines are blocks of program code that perform a specific task.
- ❑ Sub-routines do not need to be declared before you use them.
- ❑ To use a sub-routine you must use CALL plus a name.
- ❑ Sub-routines do not return a value.
- ❑ Sub-routine names are just convenient titles.
- ❑ Sub-routines can change the value of your variables.
- ❑ Sub-routines can be located inside or outside your program.
- ❑ Parameters must be used for routines outside your program.

Chapter 8

CONVERSION FUNCTIONS

Here are a group of functions that convert numbers into strings and strings into numbers.

Num\$(N)

converts a number N to an internal string representation that cannot be displayed but is recognized by the True BASIC language system as a string. This means you can send this string to a file and read it back too.

Num(A\$)

reverses the action of Num\$(N) by restoring the numeric value stored in the string A\$. This means that you can read A\$ from a file and convert it to a number for use in calculations.

ORD(string)

You already know that the alphabet that the computer recognizes is the ASCII table and not the alphabet that we all know, because the computer sees upper and lower case letters as being different. This function lets you find out the reference number in the ASCII table of any SINGLE letter or character. Here are three examples in one program that demonstrate the use of the function ORD.

```
!Filename: WT8_001.TRU
!Author: J.R.Arscott
!Purpose: Using ORD program
```

```
LET refnum1=ORD("A") ! the reference number of a
constant
PRINT refnum1          ! should print 65
```

```
LET letter$="w"
```

```
LET refnum2=ORD(letter$) ! the reference number of a
variable
```

```

PRINT refnum2                ! should print 119

LET word$="True BASIC"
LET partial$=word$[5:5]
PRINT partial$               ! should print B
LET refnum=ORD(partial$)
! the reference number of an ASCII letter
PRINT refnum                 ! should print 66

END

```

In the last example we could put:

```

LET refnum=ORD(word$[5:5])
! the reference number of an ASCII letter

```

But it is easier to understand if it is set out in stages rather than directly.

Note the use of comments to explain the code.

CHR\$(number)

This function is the exact opposite of ORD(string) in that it converts a reference number in the ASCII table into a string character or letter. The number must be a whole number (integer) and must be in the range 0 to 255.

```

!Filename: WT8_002.TRU
!Author: J.R.Arscott
!Purpose: Using CHR$ program

LET letter1$=CHR$(67) ! the character of a constant
PRINT letter1$        ! should print C

LET number=83

LET letter2$=CHR$(number) ! the character of a variable
PRINT letter2$           ! should print S

LET letter3$=CHR$(number+5) ! the character of an
expression

```

```
PRINT letter3$           ! should print X

END
```

STR\$(number)

The previous function converted reference numbers into characters in the ASCII table, but here is a function that changes any number (not a reference number) into a string. Why would you want to do this, you ask? Well, let us suppose you want to compose a line of text, but one item in the text is a number that you obtained from some user input. In this case you would need to convert the number into text, e.g.,

```
!Filename: WT8_003.TRU
!Author: J.R.Arscott
!Purpose: Using STR$ program

INPUT  PROMPT "Enter the shoe size ": shoe_size
LET size$=STR$(shoe_size)! Convert the number to a
string
LET text$="Your shoe size is " & size$

PRINT text$

INPUT  PROMPT "Enter any number e.g. 1.543 ": value

LET number$=STR$(value)! Convert the number to a string
LET text$="The number you entered is " & number$

PRINT text$

END
```

In this case the number can be any size and can contain decimals.

VAL(string)

This function is the opposite of STR\$. At some point in your programs you will often find it necessary to convert strings into

numbers. This might be because the source of your data is a text file, where some of it may be text and some will be numbers. Your program will read the data as text so the text data that is really a number will need to be converted.

```
!Filename: WT8_004.TRU
!Author: J.R.Arscott
!Purpose: Using VAL program
```

```
REM: Program WorldTour028.TRU
```

```
LET number=VAL("1.234") ! converting a constant
PRINT number
INPUT PROMPT "Enter a number ":value$
LET number=VAL(value$) ! converting a variable
PRINT number
END
```

DEF VALSTR

As I mentioned above, there is a big problem with VAL; if the string is not a number or the string contains letters or anything that is not a number then an error occurs and your program will stop. This can be very annoying. What can we do about this problem? One solution is to be really creative and write our own function that does not suffer from this problem. Essentially you are going to add another function to True BASIC, but it will be your very own. Up until now all the examples have had no real purpose other than demonstrating how things are done. In this case we are going to create something useful that you can include in all your programs.

First of all we need to plan what we want to do. We already know, because I told you so, that VAL(string) only fails if the string contains anything other than a number, so all we need to do is remove everything in the string that is not a number.

Note: computer numbers do not contain commas, for example; a million is often written as 1,000,000 but the computer will only accept 1000000.

```

!Filename: WT8_005.TRU
!Author: J.R.Arscott
!Purpose: DEF VALSTR program

```

```

DECLARE DEF valstr
Input prompt "Enter a mixed number with punctuation
plus letters":mix$
LET value=valstr(mix$)
PRINT value
END

```

```

DEF VALSTR(string$)
    LET temp$="" ! create a temporary
working variable
    LET length = len(string$) ! measure length
    FOR n=1 to length !compare each letter with ASCii
LET letter=ORD(string$(n:n))
        SELECT CASE letter
            CASE 35,44 ! hash/comma
                ! do nothing
            CASE 48 to 57 ! these are numerals 0 to 9
                LET temp$=temp$ & chr$(letter)
! build new string
            CASE 45 ! minus sign
                LET temp$=chr$(letter) & temp$ ! add
to front
            CASE 46 ! decimal point
                LET temp$=temp$ & chr$(letter) !
build new string
            CASE ELSE ! must be a letter
                IF n=1 THEN
                    LET temp$="0"
                END IF
                EXIT FOR
            END SELECT
        NEXT n

! temp$ only has numbers and no letters now so use
VAL
    IF LEN(temp$)=1 and CPOS(temp$,"0.-")>0 THEN
        LET temp$="0"
    END IF
    LET valstr = val(temp$)

```



```
END DEF
```

Now we will walk through the code line by line see how it works. The first line just creates an empty variable (temp\$) so that we can mess with the (string\$) variable without affecting the original. The second line finds out how many letters (characters) are in the string. The next line sets up a FOR...NEXT loop that counts through each character in the string one by one so we can check if it is numerical or not. First of all we convert the single character (temp\$[n:n]) into the ASCII table number using ORD, then we use SELECT CASE to compare the ASCII table number with the numbers 48 to 57 which represent the numerals 0 to 9. These are added to the temp\$ variable. If the ASCII table number is not in the range 48 to 57 then it must be a non numeric character or punctuation. If it is a comma, (character 44) or a hash sign then we can ignore it without altering the value of the number. If it is the minus sign then we add it to the front of temp\$. As soon as we hit a non numeric character then we don't need to check any more letters because we have found the first non numeric character. We know we have removed all the non numeric characters so it is safe to use VAL.

What we have done here is to create a defined function, hence the DEF and END DEF statements. Note that we have given this function a name; VALSTR. You could use any name you like but VALSTR reminds you that it is variation of VAL that works for all strings. This whole block of code can be placed anywhere in your program but I would suggest you put it after the END statement to keep it out of the way. The next thing you need to do is tell the computer that you are going to use this function, so right at the top of the program you type:

```
DECLARE DEF valstr
```

You can now use VALSTR in your programs just as you would VAL but without worrying about a crash when you use a string containing non-numeric characters.

```
!Filename: WT8_006.TRU
!Author: J.R.Arscott
!Purpose: Using VALSTR program

LIBRARY "C:\Tbbok\WTLlib.tru"
DECLARE DEF valstr

INPUT PROMPT "Enter a number ": value$
LET number=VALSTR(value$) ! converting a string
PRINT number

END
```

For your convenience, this particular routine has been included in the WTLIB library module, which contains many other useful routines. You don't have to write out the function each time you want to use it. Use the LIBRARY reference (with your own path name) then merely DECLARE DEF VALSTR at the top of your program and then you can use this function at any time in preference to VAL. At least your programs won't crash every time you try to convert an alphabetical string into a number, which I can assure you, will happen.

Chapter 9

STRING FUNCTIONS

One of the significant strengths of True BASIC is its range of string handling functions that are as good as the best that other languages can offer, and in some respects better. You have already used UCASE\$ in previous examples so you are aware of how simple it is to use functions like this. Here are some functions that help you play around with strings.

LEN(string)

This function counts the number of letters or characters in a string. You will often need to know how many letters there are in a string so you can set the limits of a FOR...NEXT loop for example.

```
!Filename: WT9_001.TRU
!Author: J.R.Arscott
!Purpose: Using LEN program

LET count=LEN("True BASIC") ! length of a constant
PRINT count                ! should print 9
LET anystring$="Mary had a little lamb."
LET count=LEN(anystring$) ! length of a variable
PRINT count                ! Should print 24 including
period

LET partial$=anystring$[12:17]
LET count=LEN(partial$) ! length of a partial string
PRINT count
! should print 6

END
```

UCASE\$(string)

As you have seen, this function converts all alphabetical characters in a string into upper case. This is very useful when you are comparing a string entered by a user with known string constants. The user can be careless, so you never know

whether they will use upper case, or lower case or both. Another use for UCASE\$ could be to ensure that the first letter in a sentence is always a capital letter.

LCASE\$(string)

This function is similar to UCASE\$ except that it converts all alphabetical characters in a string into lower case. This is also very useful when you are comparing a string entered by a user with known string constants. The user can be careless, so you never know whether they will use upper case, or lower case or both.

RTRIM\$(string)

This function removes all the spaces from the trailing end of a string, i.e., the right hand end of a string.

LTRIM\$(string)

This function removes all the spaces from the leading end of a string, i.e., the left hand end of a string.

TRIM\$(string)

This function cleans out all the spaces from both ends of a string.

REPEAT\$(string,number)

For the first time we are dealing with a function that has two parameters (the variables inside the brackets after the function name). The first parameter is a string but more often that not it is a single character. The second parameter is a number that defines how many times you want the string (or character) repeated, e.g.,

```
!Filename: WT9_002.TRU
!Author: J.R.Arscott
!Purpose: Using REPEAT$ program
```

```
PRINT repeat$("abc",3) ! should print abcabcabc
PRINT repeat$("1234",2) ! should print 12341234
```

```
PRINT repeat$("x",10) ! should print xxxxxxxxxxxx
END
```

String\$a:b]

In other dialects of BASIC you will often find the functions; LEFT\$, MID\$ and RIGHT\$. However, in True BASIC, there is no need for these functions because the unique partial strings notation can be used to create these functions with greater simplicity. Take my word for it that you will use this simple notation countless times in your programs. For example, let us take the string "unobtainable". The three characters on the left hand side of this string of characters are "uno". We can write this as:

```
!Filename: WT9_003.TRU
!Author: J.R.Arscott
!Purpose: Using partial strings program

LET anystring$="unobtainable"
LET left$=anystring$[1:3]
PRINT left$ ! prints "uno"
END
```

It follows that (n) characters on the left of a string is given by anystring\$[1:n]

Similarly for the middle three characters in a string starting at the 5th character and ending at the 7th character:

```
LET anystring$="unobtainable"
LET mid$=anystring$[5:7]
PRINT mid$ ! prints "tai"
```

In the case of the three characters on the right of the string, we first need to measure how long the string is using the LEN function:

```
LET anystring$="unobtainable"

LET length=LEN(anystring$)
```

```
LET right$=anysting$[length-2:length]  
PRINT right$ ! prints "ble"
```

The above examples have been expanded so as to clarify the individual steps in the process, but they can be reduced to a single line, for example:

```
LET right$=anysting$[LEN(anysting$)-  
2:LEN(anysting$)]
```

This last example clearly demonstrates how economy has made the code infinitely more difficult to read and understand. As a general rule, the only advantage in compressing code is that it involves less typing, whereas expanding your code significantly improves its readability.

Chapter 10

STRING SEARCHING

In many of your programs you will find a need to split strings into smaller pieces or you may want to find out where certain characters or words are located within a string. Let us use a small program we have seen before to illustrate this point:

```
!Filename: WT10_001.TRU
!Author: J.R.Arscott
!Purpose: String searching program

LET number=0
DO
    LET number=number+1
    LET square=number^2
    PRINT number,"squared = ";square
    INPUT PROMPT "Continue? Y/N ": answer$
    IF ucase$(answer$)="N" THEN
        EXIT DO
    END IF
LOOP
END
```

The INPUT PROMPT statement requires the user to enter Y or N representing YES or NO. If the user actually types YES or NO then the program doesn't work properly, so we are depending on the user obeying the instruction Y/N. One thing you can count on is that users will disobey instructions with alarming regularity. What we really need here is a method of analysing the response by the user that doesn't depend on the user keeping to the rules. This sort of strategy is often called making a program 'bomb-proof'. The simple solution is to check whether the user's response contains the letter 'N' regardless of any other letters. To do this we use the CPOS function:

```
!Filename: WT10_002.TRU
!Author: J.R.Arscott
```

```

!Purpose: CPOS searching program

LET number=0

DO
    LET number=number+1
    LET square=number^2
    PRINT number,"squared = ";square
    INPUT PROMPT "Continue? Y/N ": answer$
IF CPOS(ucase$(answer$),"N")>0 THEN
    EXIT DO
END IF

LOOP
END

```

This line is saying, if the position of the letter 'N' in the user's answer (converted to upper case) is greater than zero then exit the DO...LOOP. In plain English this means if the user has typed anything that contains the letter 'N' then exit the DO...LOOP.

Don't worry about remembering the detail of each of the following string searching functions because you can always look them up in the index when you need them. For now you just need to remember that they exist and what they do and that they are all variations of the same basic function.

CPOS(target\$,sample\$)

The function CPOS tells you where the first occurrence of ANY character in the string *sample\$* is located in the string *target\$*. The location of the first occurrence counts from the first letter in the string *target\$*. If none of the letters in the string *sample\$* appears in the string *target\$* then the value of CPOS will be zero. Here are some examples.

```

!Filename: WT10_003.TRU
!Author: J.R.Arscott
!Purpose: more CPOS searching program

```



```

PRINT CPOS("computation","put") ! should print 4
PRINT CPOS("computation","pot") ! should print 2
PRINT CPOS("computation","case") ! should print 1
PRINT CPOS("computation","native") !should print 6
PRINT CPOS("computation","level") ! should print 0
PRINT CPOS("computation","!:;,?.") !should print 0

END

```

Notice in the last example we are effectively checking to see if the target string contains any punctuation marks.

CPOS(target\$,sample\$,n)

This is a variation of CPOS in which you can ignore any number of leading characters in *target\$* by specifying the value of (*n*), in other words, the first occurrence in *target\$* of any character in *sample\$* at or after the *n*th character. Here are some more examples:

```

!Filename: WT10_004.TRU
!Author: J.R.Arscott
!Purpose: CPOS start searching program

```

```

PRINT CPOS("computation","put",5) ! should print 5
PRINT CPOS("computation","pot",5) ! should print 6
PRINT CPOS("computation","case",5) !should print 7
PRINT CPOS("computation","native",8)!should print9
PRINT CPOS("computation","level",3)!should print 0
PRINT CPOS("computation","!:;,?.",2)!should print0

```

CPOSR(target\$,sample\$)

This function is similar to CPOS but in this case any character in the string *sample\$* is compared with *target\$* starting from the back end and working backwards. The actual location is still given as a number counting from the front of *target\$*. Here are some more examples that explain how this function works:

```

!Filename: WT10_005.TRU

```

```
!Author: J.R.Arscott
!Purpose: String searching program
```

```
PRINT CPOSR("computation","put") ! should print 8
PRINT CPOSR("computation","pot") ! should print 9
PRINT CPOSR("computation","case") ! should print 7
PRINT CPOSR("computation","native")!should print11
PRINT CPOSR("computation","level")!should print 0
PRINT CPOSR("computation","!:;,?.")!should print 0
```

END

NCPOS(target\$,sample\$)

Let us look at yet another variation of CPOS that slightly more difficult to understand. In this case the function locates the first occurrence of ANY character in the string *target\$* that does NOT appear in the *sample\$* string. The location of the first occurrence counts from the first letter in the string *target\$*. If all of the letters in the string *target\$* also appear in the string *sample\$* then the value of NCPOS will be zero. Here are some examples.

```
!Filename: WT10_006.TRU
!Author: J.R.Arscott
!Purpose: String searching program
```

```
PRINT NCPOS("computation","put") ! should print 1
PRINT NCPOS("computation","pot") ! should print 1
PRINT NCPOS("computation","case") ! should print 2
PRINT NCPOS("computation","comic") ! should print 4
PRINT NCPOS("computation","computer") ! should print 7
```

END

NCPOSR(target\$,sample\$)

In this variation of the CPOS, the function locates the first occurrence of ANY character in the string target\$ that does NOT appear in the *sample\$* string starting at the back end of target\$ and working backwards. Here are some examples.

```
!Filename: WT10_007.TRU
!Author: J.R.Arscott
!Purpose: String searching program
```

```
PRINT NCPOSR("computation","put") ! should print 11
PRINT NCPOSR("computation","not") ! should print 9
PRINT NCPOSR("computation","notice") ! should print 7
PRINT NCPOSR("computation","caution") ! should print 4
```

```
END
```

POS(target\$,sample\$)

This function is similar to CPOS but in this case the WHOLE of the string sample\$ is compared with target\$. In other words, POS locates the first occurrence of the whole string sample\$ in the string target\$. Here are some more examples:

```
!Filename: WT10_008.TRU
!Author: J.R.Arscott
!Purpose: String searching program
```

```
PRINT POS("computation","put") ! should print 4
PRINT POS("computation","pot") ! should print 0
PRINT POS("computation","at") ! should print 7
PRINT POS("computation","on") ! should print 10
PRINT POS("computation","comp") ! should print 1
```

```
END
```

POS(target\$,sample\$,n)

We can also specify where the function starts in the same way as CPOS. In other words, POS locates the first occurrence of the whole string sample\$ in the string target\$ at or after the *n*th character. Here are some more examples:

```
!Filename: WT10_009.TRU
!Author: J.R.Arscott
!Purpose: String searching program

PRINT POS("computation","put",4) ! should print 4
PRINT POS("computation","pot",4) ! should print 0
PRINT POS("computation","at",3) ! should print 7
PRINT POS("computation","on",2) ! should print 10
PRINT POS("computation","comp",5) ! should print 0

END
```

POSR(target\$,sample\$)

This function is similar to POS but in this case the WHOLE of the string sample\$ is compared with target\$ starting from the back end working forwards. The actual location is still given as a number counting from the front of target\$. Here are some more examples that explain how this function works:

```
!Filename: WT10_010.TRU
!Author: J.R.Arscott
!Purpose: String searching program

PRINT POSR("computation","put") ! should print 4
PRINT POSR("computation","pot") ! should print 0
PRINT POSR("computation","at") ! should print 7
PRINT POSR("computation","on") ! should print 10
```

```
PRINT POSR("computation","comp") ! should print 1
```

```
END
```

POSR(target\$,sample\$,position)

This function is similar to POSR but in this case the starting position from the left is also specified. In other words, POSR locates the first occurrence of the whole string *sample\$* in the string *target\$*, starting from *position* and working towards the left. Here are some more examples:

```
!Filename: WT10_011.TRU  
!Author: J.R.Arscott  
!Purpose: String searching program
```

```
PRINT POS("computation","put",5) ! should print 4  
PRINT POS("computation","pot",2) ! should print 0  
PRINT POS("computation","at",5) ! should print 0  
PRINT POS("computation","on",5) ! should print 0  
PRINT POS("computation","comp",2) ! should print 1
```

```
END
```

Note that there are no equivalent functions to NCPOS or NCPOSR. It is important to understand that when you specify a starting position with POSR, any characters BEFORE the starting position are the only characters analyzed. The result returned by POSR is the character counting from the left of the original target string, and not counting from the right.

Both CPOSR and POSR count in the same manner, always from the left, regardless of the fact that they analyze the target string from the right

This is an appropriate point to test how well you have understood the string manipulation functions we have discussed so far. Let us take the case of a simple phone book

where we need to collect four items of data from the user; surname, first name, area code and phone number. We could use 4 separate INPUT PROMPT statements, but let's be a little more adventurous and use just one LINE INPUT PROMPT statement to collect all the data in one go. We can then use our string handling functions to separate the individual bits of data because this will give you a chance to see how these functions are used in practice.

In this example we ask the user to enter each data item followed by a comma (hence LINE INPUT). We could use any character as the means of separating the data; such as a slash (/), but in this simple case we will use a comma, which happens to be character 44 in the ASCII table.

```
!Filename: WT10_012.TRU
!Author: J.R.Arscott
!Purpose: Analyzing multiple input program

LINE INPUT PROMPT "Enter the surname, first name, area
code, phone number all separated by commas.": data$

LET length=LEN(data$) ! find out how long the data
string is

REM: now separate out the individual parts using string
functions
LET comma1=CPOS(data$,"") ! look for first comma
LET surname$=data$[1:comma1-1]
LET comma2=CPOS(data$,"",comma1+1) ! look for second
comma
LET first$=data$[comma1+1:comma2-1]

LET comma3=CPOS(data$,"",comma2+1) ! look for third
comma
LET area$=data$[comma2+1:comma3-1]
LET number$=data$[comma3+1:length]

REM: print the results to check the analysis works
PRINT surname$
```

```

PRINT first$
PRINT area$
PRINT number$
END

```

So it works, but only if the user sticks to the rules, and we know that doesn't always happen. Lets bomb-proof the program by allowing the user to enter colons or slash marks, or any other sort of punctuation to separate out the bits of data, e.g.,

```

!Filename: WT10_013.TRU
!Author: J.R.Arscott
!Purpose: Analyzing multiple input program

LINE INPUT PROMPT "Enter the surname, first name, area
code, phone number": data$

LET length=LEN(data$) ! find out how long the data
string is

REM: now separate out the individual parts
LET comma1=CPOS(data$,"./ \:;") ! look for first
punctuation
LET surname$=data$[1:comma1-1]
LET comma2=CPOS(data$,"./ \:;",comma1+1) ! look for
second punctuation
LET first$=data$[comma1+1:comma2-1]
LET comma3=CPOS(data$,"./ \:;",comma2+1) ! look for
third punctuation
LET area$=data$[comma2+1:comma3-1]
LET number$=data$[comma3+1:length]
REM: print the results to check the analysis works

PRINT surname$

PRINT first$

PRINT area$

PRINT number$

END

```

This is an improvement, but we are not yet out of trouble. People tend to be careless with spaces so we ought to clean up the variables with the TRIM\$ function.

```
!Filename: WT10_014.TRU
!Author: J.R.Arscott
!Purpose: Analyzing multiple input program

LINE INPUT PROMPT "Enter the surname, first name, area
code, phone number": data$

LET length=LEN(data$) ! find out how long the data
string is

REM: now separate out the individual parts and clean up
spaces
LET comma1=CPOS(data$,"./ \:;") ! look for first
punctuation
LET surname$=TRIM$(data$[1:comma1-1])
LET comma2=CPOS(data$,"./ \:;",comma1+1) ! look for
second punctuation
LET first$=TRIM$(data$[comma1+1:comma2-1])
LET comma3=CPOS(data$,"./ \:;",comma2+1) ! look for
third punctuation
LET area$=TRIM$(data$[comma2+1:comma3-1])
LET number$=TRIM$(data$[comma3+1:length])
REM: print the results to check the analysis works

PRINT surname$

PRINT first$

PRINT area$

PRINT number$

END
```

We could even go further and check that there are four pieces of data, i.e., the user has not missed anything out. We could even check that the data has been entered in the correct order, but we have already done enough to demonstrate the principle.

How does an artist know when he or she has finished a picture? You have exactly the same dilemma – how far do you go with bomb-proofing your program, before you make it so complicated that it ceases to be useful?

The program as it stands, only allows you to enter one set of subscriber details, but by putting a DO...LOOP around the whole program you can collect as many subscriber details as you want by assigning data\$ to elements in an array or by writing data\$ to progressive records in a file.

Did you notice that this was a two stage process? Find the commas separating the data items and then starting the search for the next data item one character beyond the most recent comma.

Chapter 11

NUMERIC FUNCTIONS

The inventors of True BASIC were both brilliant mathematicians, so it is hardly surprising that the greatest strength of True BASIC is its mathematical capabilities. For the moment we will concentrate on the simpler stuff. On the surface it looks like True BASIC thinks of numbers the same way that we humans do, but behind the scenes True BASIC actually works with double precision floating point numbers. As a user you don't even need to know what this definition means, unless you are doing some serious mathematics at fantastic accuracy. The normal level of accuracy of True BASIC numbers is way beyond what you would normally need for practical purposes so you will frequently need to round off numbers.

Here are some useful numeric functions, although don't be surprised if you hardly ever use any of them:

ABS(number)

This returns the absolute value of a number.

CEIL(number)

This returns the least integer that is greater than or equal to the number, and is equal to $-\text{INT}(-\text{number})$

Divide(number,div,quotient,remainder)

This function divides a (number) by (div) and returns the quotient and any remainder.

EPS(value)

This returns the smallest number that can be added to value such that $\text{value} <> \text{value} + \text{EPS}(\text{value})$

EXP(number)

This returns the exponential of number.

Log(number)

This returns the natural logarithm of number

Log10(number)

This returns the logarithm of number to the base 10.

ROUND(number,places)

This function will round off any number to a given number of decimal places, e.g.,

```
!Filename: WT11_001.TRU
!Author: J.R.Arscott
!Purpose: Using ROUND program

PRINT ROUND(45.234567,3) ! should print 45.235
LET number=217.876
LET places=2
PRINT ROUND(number,places) ! should print 217.88

END
```

ROUND(number)

This function will round off any number to the nearest whole number, e.g.,

```
!Filename: WT11_002.TRU
!Author: J.R.Arscott
!Purpose: Using ROUND program

PRINT ROUND(45.234567) ! should print 45
LET number=217.876
PRINT ROUND(number) ! should print 218
END
```

INT(number)

This function returns the integer part (whole number) of a number. For positive numbers this means knocking off all the

decimal places to leave the whole number. Things get trickier when you are dealing with negative numbers. In this case it is the next lower whole number, e.g.,

```
PRINT INT(3.456) ! should print 3
PRINT INT(-3.456) ! should print -4
```

IP(number)

This is similar to INT except it works the same way for positive and negative numbers, i.e., the decimal places are removed, e.g.,

```
PRINT IP(3.456) ! should print 3
PRINT IP(-3.456) ! should print -3
```

FP(number)

You don't need to be a rocket scientist to guess that this function returns the fractional part of a number, e.g.,

```
PRINT FP(3.456) ! should print .456
PRINT FP(-3.456) ! should print -.456
```

It follows that $x = IP(x) + FP(x)$, where x is any positive or negative number.

SGN(number)

This function returns the sign of a number using the convention -1 for negative numbers, zero for a number that equals zero, and +1 for a positive number.

MOD(x,y)

This function has two parameters and returns the remainder when x is divided by y . You might ask why you would need such a function because it is not immediately obvious. Suppose you want to find out if a number (x) is an exact multiple of the number seven then:

```
IF MOD(x,7)=0 then ! x is an exact multiple of 7
```

MAX(x,y)

This function determines which is the larger of the two numbers x and y.

MIN(x,y)

This function determines which is the smaller of the two numbers x and y.

MAXNUM

This returns the largest positive number available in True BASIC.

RND

This function has no parameters but returns a random number between 0 and 1 each time you use the function. If you want to simulate throwing a dice then:

```
LET dice=INT((6*RND)+1)
```

If you want to generate a random number between 0 and 68 then:

```
LET random_number=INT(68*RND)
```

If you want a random number between 1 and 68 then:

```
LET random_number=INT((67*RND)+1)
```

CAUTION: RND returns the same group of random numbers every time you use the function, which is hardly 'random', is it? To work around this problem you must include the statement,

```
RANDOMIZE
```

At the start of your program before you use the RND function. This will make sure that RND uses a different set of random numbers every time you use it.

PI

This function has no parameters. It returns the value of the famous mathematical constant that is usually written as a Greek symbol. Its value is 3.14159265 (approx.).

SQR(number)

This returns the square root of (number)

TRUNCATE(number,N)

This returns the value of (number) truncated to N decimal places.

Chapter 12

TRIGONOMETRIC FUNCTIONS

For normal folk, the number of times that you encounter trigonometry in your normal daily life is extremely rare. Unless you are a mathematician, then the number of times you will need to use 'trig' functions in your programs will be equally rare.

For the benefit of those that need to know, the angular measure in True BASIC trigonometric functions may be expressed in units of either radians or degrees. The default unit is radians, but you can change to degrees by including the statement:

```
OPTION ANGLE degrees
```

Once this statement has been executed, all subsequent trigonometric functions will expect their angles to be in degrees. You can change back to radians by including the statement,

```
OPTION ANGLE radians
```

and all subsequent functions then expect angles in radians.

Deg(X)

This function returns the angle X in degrees, where X is measured in radians.

Rad(X)

This function returns the angle X in radians, where X is measured in degrees.

Acos(X)

This returns the arccosine of X

Angle(X,Y)

This returns the angle of a line from the origin to point X,Y with respect to the X-axis.

Asin(X)

This returns the arcsine of X

Atn(X)

returns the arctangent of X

Cos(X)

returns the cosine of angle X

Cosh(X)

returns the hyperbolic cosine of angle X

Cot(X)

returns the cotangent of angle X

Csc(X)

returns the cosecant of angle X

Sec(X)

returns the secant of angle X

Sin(X)

returns the sine of angle X

Sinh(X)

returns the hyperbolic sine of angle X

Tan(X)

returns the tangent of angle X

Tanh(X)

returns the hyperbolic tangent of angle X

Chapter 13

DATES & TIME

Inevitably you will need to use the current date at some point in many of your programs. As you probably know, there are several conventional ways of expressing the date in numeric form, largely depending on how you express the date in common speech. In the U.S. and many other nations the month comes first, followed by the day, then the year. In the U.K. along with other countries, the day is put first, followed by the month and year. True BASIC puts the year first, followed by the month, then the day, without separations between them. Here are some examples for the seventh of March in the year 2006:

U.S. date convention 03/07/06

U.K. date convention 07/03/06

True BASIC convention 20060307

The advantage of the True BASIC convention is that dates can be compared numerically to determine which is sooner or later.

Here are some standard functions that deal with the date:

DATE\$

This function generates a string containing the date in the format (yyyymmdd), e.g.,

```
PRINT DATE$ ! prints something like 20080327 (27 March 2008)
```

This format is useful if you want sort events in date order.

DATE

This function generates a number consisting of the year followed by the day number counting from 01 January as 1, e.g.,

```
PRINT DATE ! prints something like 8087 (87th day of 2008)
```

This format is very useful if you want to find out how many days there are between two events.

TIME\$

This function generates a string containing the time in the format (hh:mm:ss) e.g.,

```
!Filename: WT13_001.TRU
!Author: J.R.Arscott
!Purpose: Time$ program
```

```
PRINT TIME$ ! should print something like 10:21:43
LET hour$=TIME$[1:2]
LET minute$=TIME$[4:5]
LET second$=TIME$[7:8]
PRINT hour$ ! should print 10
PRINT minute$ ! should print 21
PRINT second$ ! should print 43
END
```

TIME

This function counts the number of seconds since midnight (accurate to 3 decimal places). It is a very useful function for measuring the elapsed time between two events, e.g.,

```
!Filename: WT13_002.TRU
!Author: J.R.Arscott
!Purpose: Elapsed Time program
```

```
LET first=time
FOR n=1 to 100000
REM: just a loop to waste some time
let x=n^2
NEXT n
LET second=time
LET elapsed_time=second-first
PRINT elapsed_time ! should print something like 0.015
END
```

By now your head will be stuffed full of information about features that you are unlikely to remember. Don't worry, even experienced programmers have to refer to the Reference Manual from time to time to refresh their memories about how particular functions and statements are used. There is no point in trying to guess, because computers are too stupid to work out what you really mean. Look it up and get it right.

Chapter 14

More Input

LINE INPUT

The normal INPUT statement is not particularly useful as you have no doubt recognized. One its major problems is the way it deals with commas, e.g., if you type “fruit,trees,days” in response to an INPUT statement, the computer will assume you have enetered three separate responses. Likewise if you INPUT a line of text from a file that contains a comma, then the computer will assume the line is in fact two lines separated by a comma. This is confusing rather than helpful. Fortunately there is a simple solution: the LINE INPUT statement. This statement takes commas in its stride. Take the case of a normal address line:

14 Main Street, Windsor, Vermont.

INPUT would consider this to be three separate responses. LINE INPUT considers it to be a single response, which is how you and I think of it. As a general rule I would always use LINE INPUT in preference to INPUT. As you will see later, the same applies to reading data from TEXT files with LINE INPUT.

READ & DATA

Very often you will find the need to use look-up tables or lists of standard data. Take the case of dates: not everyone will recognize or understand the True BASIC date convention, YYYYMMDD. Users of your programs would probably prefer a more human readable format. At the same time you might want to show an abbreviated version of the date, e.g., DD MMM YYYY, i.e., the month in 3 letter format. Rather than asking the

user to type all this data, we can include it inside your program with a DATA statement, e.g.,

```
DIM month$(12,2)
DATA JAN,January
DATA FEB,February
DATA MAR,March
DATA APR,April
DATA MAY, May
DATA JUN,June
DATA JUL,July
DATA AUG,August
DATA SEP,September
DATA OCT,October
DATA NOV,November
DATA DEC,December
```

I have set out the DATA statements this way to make it easier to understand, but you could have entered this data on one single line and it would make no difference to the outcome. As you can see we have prepared an array (month\$) that we can use to assign this data, in which the first column contains the abbreviation and the second column contains the full month name. You could easily extend the columns in this array to cover foreign language equivalents of these month names.

We could create this array the hard way, e.g.

```
LET month$(1,1)="JAN"
LET month$(1,2)="January"
LET month$(2,1)="FEB"
LET month$(2,2)="February" etc.
```

Or we can use the READ statement:

```
For a=1 to 12
For b=1 to 2
READ month$(a,b)

NEXT b
```

```
NEXT a
```

Or better still:

```
MAT READ month$
```

True BASIC doesn't care where you put DATA statements because it reads through your whole program and makes a separate reference list of the data. True BASIC never loses track of where it is when it READs data, but we humans can easily lose track, so take my advice: put all of your DATA statements in one place and READ them into arrays in the same place. The reason for this advice is that when True BASIC READs data it carries on from the last item of data.

To illustrate this point, let's look at another example, a simple menu with three headings, fruit, trees and days, and under each heading there are 7 items.

```
!Filename: WT14_001.TRU
!Author: J.R.Arscott
!Purpose: MAT READ program

DIM menu$(3,0:7)
MAT READ menu$
DATA fruit,apple,cherry,orange,lemon,lime,"",""
DATA trees,ash,oak,beech,elm,"","",""
DATA days, Mon, Tues, Wed, Thur, Fri, Sat, Sun
PRINT menu$(2,3) ! should show beech
PRINT menu$(3,4) ! should show Thur
END
```

As you can see it doesn't matter where we put the READ statement: either in front or after the DATA statement. What does matter are the empty spaces (empty quotes). If we didn't have these "spaces" then the fruit list would acquire two trees and the trees list would get a few "days" in it, because the private list of data that True BASIC keeps doesn't match the menu\$ array.

Another interesting use of the DATA statement is when you want to embed an image inside your program without having to supply a separate image file. The True BASIC toolbox icon is embedded in the executable program this way. An image consists of a large number of bytes of data where each byte can be converted to an ASCII decimal code number. If you have the patience then any image can be converted to a series of numerical DATA statements, which can be written into an external program unit. When you bind your program into an executable version, this external unit will be included in the package.

Obviously this is a tedious task because images tend to use a lot of bytes. To save you all this hassle, I have included a program in the files that can be downloaded from the True BASIC website called BOXDATA.EXE that performs this task for you, i.e., it reads any BMP file, converts it to the True BASIC internal image format suitable for displaying on screen with a BOX SHOW statement. At the same time it also converts this image into a series of DATA statements inside an external program unit, e.g.,

EXTERNAL

```
SUB Read_BoxStringData(boxstring$)

DATA 0,0,0,0,0,0,0,0,0,148,0,0,0,151,0,0,0
DATA 64,0,0,0,212,44,0,0,40,0,0,0,148,0,0,0
DATA 151,0,0,0,1,0,4,0,0,0,0,0,212,44,0,0
DATA 206,14,0,0,196,14,0,0,0,0,0,0,0,0,0,0
DATA 0,0,0,0,0,0,128,0,0,128,0,0,0,128,128,0
DATA 128,0,0,0,128,0,128,0,128,128,0,0,192,
192,192,0

LET boxstring$=""

FOR n=1 to 96
READ number
```



```
LET boxstring$=boxstring$ & chr$(number)
NEXT n
END SUB
```

In your own program, all you need is a one line CALL to this routine to read the data and send back an image string that you can use directly with BOX SHOW, e.g.,

```
CALL Read_BoxStringData(boxstring$)
BOX SHOW boxtring$ AT x,y
```

The larger the image, the more data statements you need. Normally I don't use this technique for large images, instead I mainly use it for toolbar icons or similar small images. The toolbar in the True BASIC editor is done this way. A typical example of this technique can be seen in the file STOPDATA.TRU in the files that can be downloaded from the True BASIC website.

RESTORE

In effect this statement instructs True BASIC to move the pointer, which keeps track of its own data statement list, back to the start of the data. If you follow my advice, then you will probably never need to use this statement. I have never used it.

Chapter 15

More Output

SET CURSOR

You may have as many SET CURSOR statements as you wish in a program. You may move the cursor up or down, right or left on the screen. You cannot, however, move the cursor off the screen. On a typical PC monitor, the column value is between 1 and 80 while the row value is between 1 and 25.

If you don't know the size of your screen, run the following program that introduces the ASK MAX CURSOR statement:

```
!Filename: WT15_001.TRU
!Author: J.R.Arscott
!Purpose: TEXT WINDOW SIZE program
```

```
CLEAR
ASK MAX CURSOR Height, Width
PRINT "Height of output window ="; Height; "rows"
PRINT "Width of output window ="; Width; "columns"
END
```

This program gives the screen size if the output window is made large enough to fill the entire screen.

If you run this program on a PC with the normal monitor, you should get the following output:

```
Height of output window = 25 rows
width of output window = 80 columns
```

On the other hand, if you run the program on a Mac with a built-in monitor, you see a different output, as follows:

```
Height of output window = 19 rows
```

width of output window = 72 columns

PRINT USING

Let me take this opportunity to remind you that True BASIC has been around for a good many years. In the early days, not only did you have to write your own programs, but you also had to concern yourself with how you were going to display the output and how you intended to make a hard copy print. This was in the days of the dot matrix printer, or if you were lucky it might have been a black and white ink jet printer. What was on your screen didn't necessarily come out the same way on the printer.

Obviously things have come a long way since those far off days, and if you have any sense you will ship your data into a CSV file and then chain directly to Excel or any other similar spreadsheet. Once your data is in a spreadsheet you can re-order the columns and perform countless other acts of wizardry, including drawing graphs and printing the results.

Old fashioned legacy code requires you to know and use statements like:

```
PRINT USING "####.##": value
```

This was a method of formatting data so that it appeared aligned in tabular form, which was helped by the fact that proportional fonts were not widely available, so you had to use fixed pitch fonts. In the above example the formatting hash signs ensured that value always occupied four digits in front of the decimal point and two digits after.

The Reference manual contains a detailed list of the various forms of this statement, so there is little point in my repeating them here, particularly as you are unlikely to use them. All you

need to know is that the formatting hash code allows you to configure any number in terms of digits before the decimal point as well as the accuracy following the decimal point.

Here is a summary of the PRINT USING statement, which you can skip if you take my advice and instead use the features that most spreadsheets can offer.

The general syntax of the PRINT USING statement is:

```
PRINT USING Format$: Item1, Item2...
```

The expression Format\$ is either a string variable or string constant containing special characters called format characters. Here is a list of the numeric format characters:

#	any digit, print leading zeroes as spaces
%	any digit, print leading zeroes as zeroes
*	any digit, print leading zeroes as stars
+	print number with leading plus or minus sign
	print number with leading space or minus sign
\$	print number with leading dollar sign
^	print the exponent field of a number

String format characters are as follows:

#	any character
<	any character, left-justify string
>	any character, right-justify string

These format characters are used to control the appearance of the items appearing in the PRINT USING statement. As with the regular PRINT statement, the items can be constants, variables, or expressions. Note that the items in the PRINT USING statement are separated by commas. These commas are used only as separators; they do not cause the item values

to be displayed in different print zones. Semicolons cannot be used as separators.

The format string in a PRINT USING statement may be a string variable instead of a string constant, as illustrated by the following program fragment:

```
LET Format$ = "####" PRINT using Format$: 125
```

The quantity to be printed may be a variable instead of a constant, as in the following example:

```
LET Number = 125
LET Format$ = "####"
PRINT using Format$: Number
```

Any characters included in the format string, other than format characters, are displayed as entered. For example, the statement

```
PRINT using "The total is ###,###.": 56752
```

displays the sentence

The total is 56,752.

The letters, spaces, and period are not format characters and are displayed as entered. The comma behaves a little differently — it is only displayed if there are four or more digits in the number.

A common use of the PRINT USING statement is to display numbers representing dollars and cents with the decimal point properly aligned. For this purpose, the dollar sign (\$) is used as a format character. A period is included in the format string to mark the column where the decimal point is located. Commas or other characters cannot be inserted between the dollar signs.

If you wish to display an amount with a leading dollar sign, use a statement like the following:

```
PRINT using "$$$$$$.##": N
```

with dollars signs acting as format characters to the left of the decimal point and sharp signs to the right of the point.

The dollar sign preceding the displayed number “floats,” remaining always just to the left of the first digit. All the dollar signs except the first one are place holders for digits, so the largest number that can be printed using this format string is 99999.99. This type of format string can only be used with numeric constants, variables, or expressions, it cannot be used to format strings.

If N has a value of 10, the statement displays the following output:

```
$10.00
```

with three spaces are printed to the left of the dollar sign.

If you want the dollar sign to appear always in the same column, use a statement such as this:

```
PRINT using "$#####.##": N
```

If N still has a value of 10, the output is displayed as

```
$ 10.00
```

The regular PRINT statement places a blank space in front of a number if it is positive and a minus sign in front of the number if it is negative. The PRINT USING statement doesn't work this way. A plus sign (+) as a leading format character will display a

plus sign if the number is positive and a minus sign if the number is negative. Multiple plus or minus signs in the format string cause a single “floating” plus or minus sign to be displayed. These characters behave like the dollar sign format character, all the plus and minus signs except the first one are place holders for digits.

```
!Filename: WT15_002.TRU
!Author: J.R.Arscott
!Purpose: PRINT USING program

LET N = 12
LET X = -35
PRINT using "-###": N
PRINT using "-###": X
PRINT
PRINT using "+++#": N
PRINT using "+++#": X
PRINT
PRINT using "--#": N
PRINT using "--#": X
PRINT
PRINT using "##": N
PRINT using "##": X

END
```

Here is the output produced by this program:

```
12
35

+12
-35

12
-35

12
**
```

Note that the last PRINT USING statement displays error symbols (*) because the number -35 requires a format string of at least three characters.

If a PRINT USING statement has more variables than there are format character groups in the format string, the format string is used over and over again until all the variable values have been printed. Any remaining characters in the format string are ignored after the last variable is displayed.

The usual format character (#) displays numbers with leading blank spaces. An alternate format character, the percent sign (%), displays numbers with leading zeroes. This format is sometimes used when displaying such items as part numbers or zip codes. These two format characters cannot be mixed in a single format character string.

For example, the statement

```
PRINT using "%%%" : 32
```

displays
0032

but the statement

```
PRINT using "%##" : 32
```

is not allowed and displays an error message.

The exponential notation for numbers is seldom used in business applications, but often appears in scientific or technical documents. The PRINT USING statement can display numbers in this form and a few examples are shown. The caret (^) is used as a format character to indicate the size of the exponent field, including the letter "e" and the numeric sign. The

exponent field must be a least three characters long.

Another example program displays this format:

```
!Filename: WT15_003.TRU
!Author: J.R.Arscott
!Purpose: EXPONENTIAL NUMBERS program

! Display numbers in exponential form.

PRINT using "###^^^": 2.5e+7
PRINT using "#.##^^^": 2.5e7
PRINT using "#.##^^^^^": 2.5E7
PRINT using "##^^^": 87654321
END
```

It produces the following display:

```
250e+5
2.50e+7
2.50e+007
88e+6
```

Note in the last value displayed how the original value is rounded and six significant figures are lost. This format accepts a number containing either an “e” or an “E” but displays all results with the lowercase “e.”

String variables may also be formatted with PRINT USING statements. The same format character (#) is used to indicate the field in which the string value is displayed. If the field is longer than the string value, the value is centered in the field. If the field is shorter, a string of asterisks (*) is printed.

If the leftmost format character (#) is replaced by another format character, the left angle bracket (<), then the string value is left-justified in the field. A right angle bracket (>) in the same position right-justifies the string value.

Angle brackets cannot be used to justify numeric values.

Consider the following example program:

```
!Filename: WT15_004.TRU
!Author: J.R.Arscott
!Purpose: FORMATTED STRINGS program

! Display a string variable.

LET Title$ = "abc"
PRINT using "|#####|": Title$
PRINT using "|<#####|": Title$
PRINT using "|>#####|": Title$
PRINT using "|##|": Title$
END
```

Here is the output:

```
|  abc  |
|abc   |
|   abc|
|**|
```

I have used vertical bars to show the limit of format strings. Note that the first display of "abc" is centered, the second is left-justified, and the third is right-justified. The fourth display is an error display because the format string is too short.

A format string can be used to control the display of a combination of strings and numbers. Here is an example that displays lines from a typical price list, showing the number, description, and price of certain items:

```
!Filename: WT15_005.TRU
!Author: J.R.Arscott
!Purpose: FORMATTED STRINGS & NUMBERS program
```

```
LET Format$ = "%%%<##### $$$##"
```

```

DO while More data
    READ PartNum, Item$, Price
    PRINT using Format$: PartNum, Item$, Price
LOOP

DATA 245, "Cleat, 4 inch", 12.50
DATA 1705, Bow Light, 34.50
DATA 12, Boat Cushion, 3.95
DATA 717, Shackle, 5.50
DATA 718, Pin, 2.25
END

```

This program displays the following price list:

0245	Cleat, 4 inch	\$12.50
1705	Bow Light	\$34.50
0012	Boat Cushion	\$3.95
0717	Shackle	\$5.50
0718	Pin	\$2.25

The format string contains several numeric format characters, then a group of string format characters, and finally more numeric format characters. In between are blank spaces which are not format characters and thus display as blank spaces.

USING\$

This statement is very similar to PRINT USING, but it doesn't print anything. Instead, in this case the statement just allows you to format numbers or strings as strings, in preparation for sending data to a file, again with the option of how many leading and trailing characters before and after the decimal point. For example, if you have a CSV file containing financial data, then you might want the financial column to be aligned to probably two decimal places. This is a case where you would use this statement, e.g.,

```
LET finance$=USING$ ("#####.##", 169.88)
```

```
LET money$=USING$ ("####.##",expenses)
```

In terms of its formatting rules, USING\$ is exactly the same as PRINT USING.

In normal everyday life, normal folk rarely need to use more than two decimal places, unless you are a stock market currency dealer, in which case you would be interested in exchange rates with many more decimal places.

TAB

Again, this particular statement relates back to the early days of fixed pitch fonts and is of limited usefulness, except for legacy code.

There are two forms of this statement, e.g.,

```
TAB (25)
```

and

```
TAB (3,25)
```

The latter is equivalent to the statement:

```
SET CURSOR (3,25)
```

As you can see, TAB allows you to position the cursor or the print field at particular locations based on the screen being divided into rows and columns based on a fixed pitch font. It is therefore of limited use when using proportional fonts.

You need to determine how many rows and columns that your screen supports before using TAB. This can be done with:

```
ASK MAX CURSOR, row,column
```

You cannot use TAB with a PRINTER.

DATA TABLE

As I mentioned earlier, there is little point in spending your time attempting to format your output either to the screen or to the printer. Sending your data as a CSV file to Excel or any other spreadsheet is a much more satisfactory solution and involves you in no effort. You can do this directly from the TEditor.

For those of you that have a copy of the CTX library (included in the GOLD edition), there is also a Windows object called DATA TABLE. This object allows you to display a two dimensional array with multiple columns, just like a spreadsheet without all the whistles and bells. A side scroll bar lets you view a screenfull of data at one time, anywhere in the list. The vertical column separators can also be dragged aside to expand or reduce the column width. The data for this table is provided by an array, e.g.,

```
CALL CT_Table_ReadArray(id,array$(,))
```

The two dimensional array must be dimensioned as follows:

```
DIM array$(0:rows,columns)
```

the zero row must contain the heading titles, e.g.,

```
array$(0,1)="name"  
array$(0,2)="street"  
array$(0,3)="town/city"  
array$(0,4)="state"  
array$(0,5)="zip"
```

As an alternative, the DATA TABLE can be populated from a CSV file, e.g.,

```
CALL CT_Table_ReadFile(id, csvname$)
```

The executable file called WT_TABLE_DEMO.EXE uses an array of data in the file called NAMES.TXT to provide a two dimensional display of names, heights, weights and Date of birth.

Chapter 16

Matrices

The word matrix is another name for an array, normally an array of numbers.

A set of special statements in True BASIC, called MAT statements, allows you to manipulate matrices according to the rules of matrix arithmetic. Manipulating matrices in matrix arithmetic is in many ways similar to manipulating numbers in ordinary arithmetic. MAT statements allow you to manipulate arrays of strings as well as arrays of numbers.

Most ordinary folk know little or nothing about matrix arithmetic, apart from mathematicians. It is not something that you come across in daily life, so you may wonder why you should be concerned about it in relation to programming. Well, you will be pleased to know that like in real life, you can get by without matrix arithmetic and the MAT statements. However, it does make some tasks very much easier and hugely simplifies the amount of code you need to write.

A matrix is an array and as you know from Chapter 2, every array must be dimensioned in a DIM statement before it can be used or referenced in any other program statement. The DIM statement specifies the number of dimensions and the size of each dimension. This size can be specified by a single number or by a lower boundary and an upper boundary. Most matrices in example programs have a lower bound of one. For example, here are two matrix declarations:

```
DIM anyarray(1 to 100)
DIM Transform(3,3)
```

I recommend the first format over the second because it reads

more like normal English.

The second declaration implies,

```
DIM Transform(1 to 3,1 to 3)
```

This can also be written,

```
DIM Transform(1:3,1:3)
```

DIM statements can also be used with arrays having any legal lower bound and upper bound, such as the following array:

```
DIM Spreadsheet(0 to 100, 0 to 50)
```

Or

```
DIM Spreadsheet(0:100, 0:50)
```

MAT REDIM

One of the most useful matrix statements is MAT REDIM because it allows you to dynamically redimension any array on the run, as and when you need more elements.

For example, suppose you want to enter an unknown number of names into an array. One possible solution is to dimension the array large enough to hold the maximum likely number of names. A much neater solution is to redimension the array as it needs to get bigger.

```
!Filename: WT16_001.TRU
!Author: J.R.Arscott
!Purpose: more MAT REDIM program

! Start with just one element
DIM name$(1)
PRINT "After the prompt, press Return key to stop."
LET count = 0
DO
LINE INPUT prompt "Name? ": string$
```



```

IF string$ = "" then
PRINT count; " names have been entered."

GET KEY dummy
EXIT DO

ELSE
LET count=count+1
! increase array size
MAT REDIM name$(count)
! assign string$ to array list
LET name$(count)=string$
END IF
LOOP
PRINT "Total names=";count
END

```

For example, suppose your friend has sent you an e-mail with an attached text file that lists all the files in one of his directories. You are only interested in the files with TRU as the extension so you want to make a separate list of all the TRUfiles but you don't know how many file names there will be in your list. As above, the best solution is to use MAT REDIM as you go along.

```

!Filename: WT16_002.TRU
!Author: J.R.Arscott
!Purpose: MAT REDIM with files program

LIBRARY "WTLIB.tru"

DIM file$(1)
! use the library to read the e-mail attachment
! Assume the name of the attachment file is
! ALL_FILES.TXT
! Read the contents into an array called file$()

CALL
Read_anyfile("All_files.TXT","TEXT",file$(),0,error$)
! get how many files are in the list

LET total=ubound(file$)

```

```

! now dimension an array to hold the TRU files
DIM TRUfile$(1)
! this sets the array list to one element
LET count=0          ! this sets a counter to zero

FOR n=1 to total  ! check every filename
    IF POS(ucase$(file$(n)), ".TRU>")>0 then
! File must be TRU type so add to list
        LET count=count+1 ! this adds 1 to the counter
        MAT REDIM TRUfile$(count)
! this re-dimensions the array to match the count
LET TRUfile$(count)=file$
! this assigns the name to the array list
        END IF
NEXT n
PRINT "Total TRU files=";count
END

```

In this way we end up with a list that is correctly dimensioned to fit the number of filenames in the list, even though we didn't know what that number was to start with. Although it allows you to increase the number of elements in the array, you cannot change the number of dimensions, e.g.,

```

DIM array(a,b) ! two dimensions
! You cannot use
MAT REDIM array(x,y,z) ! three dimensions

```

Now that you understand how MAT REDIM works, let's look at a much more interesting and useful routine that is included in the EXEClib library module, because it is so useful. Suppose you want to make a list of all the files in a particular folder on your hard drive. To begin with you don't know how many file names there will be in your list. Here is how you solve this particular problem:

```

!Filename: WT16_003.TRU
!Author: J.R.Arscott

```

!Purpose: MAT REDIM program

```
LIBRARY "EXEClib.trc"

DIM name$(1)    ! this sets our array list to
!only one filename
! DIM other dummy variables
DIM s(1)
DIM d$(1)
DIM m$(1)
DIM t$(1)

INPUT PROMPT "Enter path of directory ": dir$
! Change directory to dir$

CALL EXEC_ChDir(dir$)

INPUT PROMPT "Enter extension or (*) ": extn$
! normally 3 letters or wildcard (*)
! Use EXEClib to read all the files in dir$

CALL EXEC_readDir(extn$,name$(),s(),d$(),m$(),t$(),v$)

! get total number of files in directory
LET total=ubound(name$)
! we now know how many files there are
PRINT "Total files of type " & extn$ & "= " &
str$(total)
END
```

Note that the routine called EXEC_readDir is self dimensioning so all you need to do is check how big the array is with UBOUND. You are very likely to need this routine many, many times so remember how it works.

MAT READ

To illustrate how matrix statements can simplify your programming tasks, let's look at constructing a simple menu that consists of three headings with up to seven items under each heading. We can include the title or heading in the zero

element, so we can dimension the array as,

```
DIM menu$(3,0:7)
```

In this array we have three headings; FRUIT, TREES and DAYS. Under each heading there are up to seven items (although we don't have to use all seven). The first menu list only has five items, the second list has four items and the last list has seven items. The unused elements in the array are filled with empty spaces.

Now we could assign strings to each of the 24 elements in the array, e.g.,

```
!Filename: WT16_004.TRU
!Author: J.R.Arscott
!Purpose: DIM ARRAYS program
```

```
DIM Menu$(3,0:7)
```

```
LET menu$(1,0)="fruit"
LET menu$(2,0)="trees"
LET menu$(3,0)="days"
```

```
LET menu$(1,1)="apple"
LET menu$(2,1)="ash"
LET menu$(3,1)="Monday"
```

```
LET menu$(1,2)="cherry"
LET menu$(2,2)="oak"
LET menu$(3,2)="Tuesday"
```

```
LET menu$(1,3)="orange"
LET menu$(2,3)="beech"
LET menu$(3,3)="Wednesday"
```

```
LET menu$(1,4)="lemon"
LET menu$(2,4)="elm"
LET menu$(3,4)="Thursday"
```

```

LET menu$(1,5)="lime"
LET menu$(2,5)=""
LET menu$(3,5)="Friday"

LET menu$(1,6)=""
LET menu$(2,6)=""
LET menu$(3,6)="Saturday"

LET menu$(1,7)=""
LET menu$(2,7)=""
LET menu$(3,7)="Sunday"
END

```

As you can see, this method will involve a lot of typing. Alternatively, we can set out the array as DATA statements, e.g.,

```

!Filename: WT16_005.TRU
!Author: J.R.Arscott
!Purpose: MAT READ DATA program

DATA fruit,apple,cherry,orange,lemon,lime,"",""
DATA trees,ash,oak,beech,elm,"","",""
DATA days, Mon, Tues, Wed, Thur, Fri, Sat, Sun

```

At this point we simply use a one line MAT statement to fill the array with DATA, e.g.,

```
MAT READ menu$(3,0:7)
```

I think you have to agree that using MAT READ is so much easier.

The same strategy can be used if the data is in a file or is being typed at the keyboard.

I hope you noticed that the empty elements are denoted by two sets of quote marks with nothing in between. If you think about, it is quite logical. In the case of numerical data the concept of

nothing is denoted with a zero. In the case of string data how would you denote nothing? A space perhaps? Unfortunately a space is not nothing: it is the ASCII character 32. So to be absolutely sure the computer knows that we mean nothing we use quote marks (because we want to tell the computer that it is a string constant) with nothing in between.

MAT INPUT

The fundamental statement used to read matrix values from the keyboard or from a text file is the MAT INPUT statement. This statement reads in values and assigns them to the matrix elements. It assigns the first value to the first element of the first row and continues assigning values in that row, from left to right, until all elements in the row have been assigned values. It then starts assigning values to the elements in the second row and continues in this manner until all values have been assigned to all matrix elements. This method of accessing elements and assigning values is often called odometer order, meaning that the items index varies faster than the headings index.

Here is another example program that reads a matrix:

```
!Filename: WT16_006.TRU
!Author: J.R.Arscott
!Purpose: MAT INPUT program

DIM Ratio(1 to 3, 1 to 2)
PRINT "Type in six element values."
PRINT
MAT INPUT Ratio
PRINT "The matrix has been filled."
END
```

The MAT INPUT statement is similar to the regular INPUT statement. A question mark is displayed to prompt the user when to enter element values. The first number entered is assigned to element Ratio(1,1), the second to element Ratio(1,2), and so forth.

Multiple element values, separated by commas, can be entered on the same line. In this example, six numbers must be entered to fill the array. If you press the <return> key before all numbers have been entered, you receive the message “Too few input items. Please reenter input line.” and another question mark prompt.

Here is an example of user interaction with the program:
Type in six element values on one line separated by commas.

```
? 3,4,1,7
Too few input items. Please reenter input line.
? 3,4,1,7,3,6
The matrix has been filled.
```

Later we will look at reading data from a file and filling an array with that data. To do this we essentially use a similar statement, MAT INPUT #n.

A powerful attribute of the MAT INPUT statement is its ability to redimension an array. The number of dimensions cannot be changed, but the size of each dimension can be modified. For example, the statement,

```
MAT INPUT Ratio(5 to 8, 8 to 10)
```

changes the size of the Ratio array from 3 by 3 to 4 by 3. The following statements are all legal ways of changing the size of the array in our last example:

```
MAT INPUT Ratio(2,2)
MAT INPUT Ratio(1 to 3, 10 to 15)
MAT INPUT Ratio(I, J to K)
```

In the first example, Ratio is redimensioned to an array with two rows and two columns. In the second example, Ratio is

redimensioned to an array with three rows, numbered 1 through 3, and six columns, numbered 10 through 15.

The third case is particularly interesting because it redimensions the array on the basis of variable values. These values can be assigned from within the program. In this example, Ratio is redimensioned to an array with I rows, numbered from 1 to I, and columns numbered from J to K. The values stored in I, J, and K are used to redimension the matrix when the MAT INPUT statement is executed.

The MAT INPUT statement has a special form of redimensioning for arrays of one dimension. If a MAT INPUT statement is written as

```
MAT INPUT NumList(?)
```

the array NumList is redimensioned to hold as many numbers as are typed in. This special form of redimensioning applies only to one-dimensional arrays of numbers.

Here is an example program using this form of the MAT INPUT statement:

```
!Filename: WT16_007.TRU
!Author: J.R.Arscott
!Purpose: SINGLE DIMENSION ARRAYS program

! Use of variable array dimensioning in
! a one-dimensional array.

DIM NumList(1)

PRINT "Type in a list of numbers separated by commas
after the ? prompt."
MAT INPUT NumList(?)
PRINT
```



```
PRINT "Number of items in the list is";  
Ubound(NumList)  
MAT PRINT NumList;  
END
```

The size of NumList is set initially to one, knowing that it will be redimensioned later. This program allows a user to type in a sequence of numbers, separated by commas, and the size of the array is adjusted to conform to the number of items entered. The process of entering items terminates when the Return key is pressed. The size of the array is then displayed on the screen and the array itself is printed.

Program output appears as follows:

Type in a list of numbers after the ? prompt.
? 1,2,3,4,5,6,7,8,9

Number of items in the list is 9
1 2 3 4 5 6 7 8 9

The general form of the MAT INPUT statement is

```
MAT INPUT Array1, Array2,...
```

In most cases, a program is easier to use and understand if the MAT INPUT statement is used to enter only a single array, and that is my usual practice, although in real life you are unlikely to use MAT INPUT or even INPUT in practical applications. There are much better ways of gathering input.

A prompt can be added to the MAT INPUT statement, using the format

```
MAT INPUT prompt Prompt$: Array1, Array2,...
```

where Prompt\$ is any string expression, variable, or constant. This statement displays the prompt and expects the user to enter a sequence of numbers, separated by commas. If all the numbers cannot be entered on a single line, add a trailing comma and the program will respond with a single question mark (?) prompt. You can then continue to enter one or more additional lines, terminating every line except the last line with a trailing comma.

```
!Filename: WT16_008.TRU
!Author: J.R.Arscott
!Purpose: SINGLE DIMENSION ARRAYS PROMPT program

! Demonstrate how multiple lines
! of numbers can be entered.

DIM Ratio(1 to 3, 1 to 3)
MAT INPUT prompt "Enter nine numbers: ": Ratio
PRINT
PRINT "Here is the resulting matrix."
PRINT
MAT PRINT Ratio
END
```

The output results might look as follows:

```
Enter nine numbers: 1,2,3
? 4,5,6,7,8,9
```

Here is the resulting matrix.

```
1 2 3
4 5 6
7 8 9
```

MAT LINE INPUT

If an array has string elements, the MAT LINE INPUT statement is allowed and preferred. Characters are entered into each string element until a carriage return is encountered. The general forms are as follows:

```
MAT LINE INPUT Array1$, Array2$,...  
MAT LINE INPUT prompt Prompt$: Array1$, Array2$,...
```

The MAT LINE INPUT PROMPT statement is not very useful, because it displays the prompt string only for the first item entered. A single question mark is displayed for each subsequent item (also see the preceding example program). For this reason, you should usually display a line of general input instructions rather than a prompt and then use the plain MAT LINE INPUT statement.

The next example program enters names into an array. Once again the initial size of the array Name\$ is set to one. The MAT LINE INPUT statement will accept lines containing commas, such as “John J. Williams, Jr.” Note that this statement is used both to redimension the array (by the variable Number in parentheses) and to accept string input.

```
!Filename: WT16_009.TRU  
!Author: J.R.Arscott  
!Purpose: SINGLE DIMENSION LINE INPUT program  
  
! Enter names into an array.  
  
DIM Name$(1)  
INPUT PROMPT "Number of names to enter? ": Number  
PRINT "Enter one name at each ? prompt."  
"  
  
MAT LINE INPUT Name$(Number)  
PRINT Number; "names have been entered."  
  
END
```

The following output is produced:
Number of names to enter? 3
Enter one name at each ? prompt.
? John
? Betsy
? Linda
3 names have been entered.

Note that the counter (count) is initialized to one and incremented only if a valid name is entered. If you enter a null value for Name\$ to stop the entry process, the flag Finished\$ is set to true. After exiting the loop, Count will be one larger than the number of names entered and must be decremented by one to equal the actual number of names. Program output looks as follows:

Maximum number of names? 20
Press Return key after prompt to stop. Name? John
Name? Betsy Name? Linda Name?
3 names have been entered.

Special redimensioning using the question mark — as demonstrated in a preceding section — is not allowed with the MAT LINE INPUT statement.

MAT PRINT

Having entered information into an array, you can use the MAT PRINT statement to display this information on the screen. The MAT PRINT statement displays the element values by rows, in odometer order.

```
!Filename: WT16_010.TRU  
!Author: J.R.Arscott  
!Purpose: MAT PRINT program
```

```
DIM Ratio(1 to 3, 1 to 3)
```

```

PRINT "Type in nine element values."
MAT INPUT Ratio

PRINT
PRINT "Here is the resulting matrix."
PRINT
MAT PRINT Ratio
END

```

Element values are displayed one to a print zone, just as if a comma had been placed between the elements in an ordinary PRINT statement. A trailing comma at the end of the MAT PRINT statement produces the same output format. Each time the computer finishes displaying a complete row, the cursor moves to the beginning of the next row. If six single-digit numbers are entered in response to the MAT INPUT statement, here is how the output might appear:

```

Type in nine element values
? 3,4,1,7,3,6,5,-8,4
Here is the resulting matrix.

```

```

3 4 1
7 3 6
5 -8 4

```

This example was chosen carefully so that each row fits on the page. If you have a matrix with more columns, you can either reduce the size of the print zone using the SET ZONEWIDTH statement or place a trailing semicolon after the MAT PRINT statement. Here is the program modified by adding a trailing semicolon.

```

!Filename: WT16_011.TRU
!Author: J.R.Arscott
!Purpose: MAT INPUT & MAT PRINT program

! Read a matrix from the keyboard
! and display it on the screen.

```

! Add a trailing semicolon to the MAT PRINT statement.

```
DIM Ratio(1 to 3, 1 to 3)
PRINT "Type in nine element values."
MAT INPUT Ratio
PRINT
PRINT "Here is the resulting matrix."
PRINT
MAT PRINT Ratio;
END
```

The results now look slightly different:

Type in nine element values

? 3,4,1,7,3,6,5,-8,4

Here is the resulting matrix.

```
3 4 1
7 3 6
5 -8 4
```

MAT READ

The MAT READ statement is used to read array element values from one or more DATA statements. The general syntax is

```
MAT READ Array1, Array2,...
```

If there are more values in the DATA statements than are required by the MAT READ statement, the extra values will be ignored. If there is an insufficient number of values, the error message "Reading past end of data" is displayed.

Here is an example program where only the first four data values are used. The fifth number in the DATA statement is never read.

```

!Filename: WT16_012.TRU
!Author: J.R.Arscott
!Purpose: MAT READ program

! Read element values from data statements.

DIM Table(1,1)
PRINT "An array read from DATA statements."
PRINT
MAT READ Table(2,2)
MAT PRINT Table;
DATA 1,2,3,4,5

END

```

When this program is executed, the following table is displayed:

```

1 2
3 4

```

The MAT READ statement can be used like the MAT INPUT statement to redimension an array.

Other MAT PRINT Statements

The general syntax of the MAT PRINT statement is

```
MAT PRINT Array1, Array2,...
```

This statement first displays Array1, then displays Array2, and so forth. To simplify formatting problems, you should use the MAT PRINT statement to display only a single array.

Commas not only separate the array names — they also specify that the elements in a row be displayed in different print zones.

Another form of the MAT PRINT is

```
MAT PRINT Array1; Array2;...
```

which creates an array whose row elements are separated by only one or two spaces. Note the use of semicolons as separators, specifying that array elements be displayed with only 1 or 2 spaces between them.

It is even possible to use both comma and semicolon separators in the same statement, as follows:

```
MAT PRINT Array1; Array2, Array3;
```

In this case, the elements in the rows of Array1 will be separated by 1 or 2 spaces, those in the rows of Array2 by 16 spaces (or the current print zone width), and those in the rows of Array3 by 1 or 2 spaces. In most cases, however, it is better to display only a single array with each MAT PRINT statement.

A format string, used to control the format of array rows, can be included in the MAT PRINT USING statement. The syntax is

```
MAT PRINT USING Format$: Array1, Array2,...
```

where Format\$ is a string constant or variable containing format specifications for displaying the elements in each row.

Here is an example program that displays a two-column array consisting of part numbers and unit prices:

```
!Filename: WT16_013.TRU
!Author: J.R.Arscott
!Purpose: MAT PRINT USING program

! Display an array of part numbers and prices.

DIM Inventory(1 to 6, 1 to 2)
```



```

LET F1$ = "#####"
PRINT using F1$: "PART NO.", "PRICE"
PRINT
MAT READ Inventory
LET F2$ = "##### $$$##"
MAT PRINT using F2$: Inventory
DATA 163151,12.85,177762,33.40,183926,18.75
DATA 205055,7.55,234965,81.60,309835,2.25

END

```

The output looks as follows:

PART NO. PRICE

```

163151 $12.85
177762 $33.40
183926 $18.75
205055 $7.55
234965 $81.60
309835 $2.25

```

MAT PRINT #n

MAT statements can also be used to read matrices from files and to write matrices on files.

The general statement to write a matrix on a text file is

```
MAT PRINT #N: Array1, Array2,...
```

This statement writes matrix elements on file #N in the same format that they would be displayed on the screen. The statement has limited usefulness because a file written with this statement cannot be read by a subsequent MAT INPUT statement. As a general rule you should write data to a text file one line at a time, whereas the MAT PRINT statement violates this principle. Thus the MAT PRINT statement is not used in the example programs that write arrays to text files.

MAT LINE INPUT #n

Matrix element values can be read from a text file instead of from the keyboard, provided that they were written on the file in an appropriate format. You can assume one element value per line. The general statements are

```
MAT INPUT #N: Array1, Array2,...  
MAT LINE INPUT #N: Array1$, Array2$,...
```

Trying to handle multiple arrays in this way is a waste of time. Take my advice and keep things simple and only handle one array at a time. Why bother to complicate matters just for the sake of saving one line of code?

As mentioned earlier, it is usually preferable to read only a single array with either of these statements. The number N is the file or channel number, assigned to a file by the OPEN statement.

In general, the MAT INPUT statement is also of limited usefulness for reading information from sequential text files. You need to know the number of entries in a file so you can dimension an array, read the file entries, and assign them to array elements with a MAT INPUT statement. In order to count the number of entries, you essentially have to go through the file sequentially and read each entry, so you might as well assign its value to an array element at that point. MAT statements are much more useful with record files, as discussed in the following paragraphs.

MAT WRITE #n

MAT statements are available to write information on and read information from record files. The statement

```
MAT WRITE #N: Array1, Array2,...
```

writes arrays on a record file, one array element per record. The MAT WRITE statement is usually used with only a single array. The array is written by rows, in odometer order, with element(1,1) being written on the first record, element(1,2) on the second record, and so forth.

MAT READ #n

The general statement for reading information from a record file is

```
MAT READ #N: Array1, Array2,...
```

and it is also normally used with only a single array.

Here is a simple program to read an array of numbers from DATA statements and write the array on a record file. The record size is set to 8 because the array elements are numbers and each number occupies 8 bytes on a record file.

```
!Filename: WT16_014.TRU
!Author: J.R.Arscott
!Purpose: MAT READ & MAT WRITE program

! Write an array on a record file.
DIM Array(1 to 6)
INPUT prompt "Name of file? ": FileName$
OPEN #1: name FileName$, create newold ERASE #1 ! if it
already exists
SET #1: RECSIZE 8
MAT READ Array
MAT WRITE #1: Array
CLOSE #1
PRINT "File "; FileName$; " has been written."
```

```
DATA 2,5,3,7,5,9
END
```

The preceding program does require that the size of the array and the number of its dimensions be specified, and that these values be compatible with the number of data elements. It produces the following output:

```
Name of file? WT_NUMS.REC
File WT_NUMS.REC has been written.
```

A companion program reads the record file and displays the array:

```
!Filename: WT16_015.TRU
!Author: J.R.Arscott
!Purpose: MAT READ & MAT WRITE program

! Read an array from a record file.

DIM Array(1)
INPUT prompt "Name of file? ": FileName$
OPEN #1: name FileName$
ASK #1: FILESIZE Size MAT REDIM Array(Size)
MAT READ #1: Array
PRINT
MAT PRINT Array;
END
```

Note in this case that it is not necessary to specify a record size because the file has already been created. A one-dimensional array is assumed. The trailing semicolon in the MAT PRINT statement displays the array in a compact format.

```
Name of file? WT_NUMS.REC
```

```
2 5 3 7 5 9
```

The size of the array can be determined if it is a one-dimensional array, but it is not possible to determine the number of dimensions. The contents of the test file, WT_NUMS.REC, could just as well have been the element values of a 2-by-3 array as a 6 by-1 array.

Three points on using MAT statements are worth repeating:

1. I recommend using a single array with each of these statements. The resulting program is simpler and easier to understand.
2. In some cases, MAT statements create the best and simplest program; in other cases, a loop is required. You should use whichever structure is most appropriate.
3. A powerful feature of the MAT statements is their ability to redimension arrays, using a constant, variable, or expression to change the size of any dimension.

MATRIX ARITHMETIC

A set of mathematical rules exists that allows you to manipulate matrices in much the same way that you manipulate numbers. You can assign a value to a matrix, which means assigning a value to every element of the matrix. You can use the common arithmetic operations of addition, subtraction, multiplication, and division with matrices, but with some restrictions and differences from number arithmetic.

Matrix arithmetic is useful in solving certain scientific and specialized business problems. Some of the simpler operations of matrix arithmetic and their implementation in True BASIC are presented. If you are interested in learning more, I suggest that you read a book on matrix arithmetic like Thomas and Finney, *Calculus and Analytic Geometry*, 7th Edition (Addison-Wesley,

1988).

MATRIX ASSIGNMENTS

The element values of one matrix can be assigned to the corresponding elements of another matrix. For example, if both A and B are matrices with two rows and columns, the statement

```
MAT A = B
```

assigns the value of B(1, 1) to A(1, 1), the value of B(1,2) to A(1, 2), and so forth. Here is an example program that demonstrates matrix assignment:

```
!Filename: WT16_016.TRU
!Author: J.R.Arscott
!Purpose: MATRIX ASSIGNMENT program

DIM A(2,2), B(2,2)
PRINT "Type in the 4 element values for matrix B."
MAT INPUT B

MAT A = B PRINT

PRINT "Here is matrix A after assigning B to A."
PRINT
MAT PRINT A;

END
```

The following output is produced:

```
Type in the element values for matrix B.
? 7,1,-2,8
```

Here is matrix A after assigning B to A.

```
7 1
```

-2 8

You must watch out for one special feature of the matrix assignment statement. The matrix assignment statement can redimension a matrix. Remember that redimensioning can change only the size of a dimension (which it does by changing one or both bounds), not the number of dimensions. The best way to demonstrate this feature is to write the previous example with a different matrix B. Here is the new example program:

```
!Filename: WT16_017.TRU
!Author: J.R.Arscott
!Purpose: MATRIX ASSIGNMENT program

! Matrix assignment with redimensioning.

DIM A(2,2), B(2,3)
PRINT "Matrix A is 2 x 2, matrix B is 2 x 3."
PRINT "Type in the 6 element values for matrix B."
MAT INPUT B
MAT A = B PRINT
PRINT "Here is matrix A after assigning B to A."
PRINT
MAT PRINT A;

END
```

Note that matrix B is dimensioned as a 2-by-3 matrix. When you run the program, you have to enter six numbers to assign values to all the elements in B, as shown:

Type in the element values for matrix B.
? 7,1,5,-2,8,3

Here is matrix A after assigning B to A.

```
7 1 5
-2 8 3
```

The matrix assignment statement (MAT A = B) redimensions matrix A so that it is changed to a 2-by-3 matrix. The program then displays matrix A.

Matrix redimensioning is both a blessing and a curse. The ability to redimension a matrix with the MAT INPUT or MAT READ statements is a powerful and useful feature of True BASIC. Automatic redimensioning in a matrix assignment statement, however, can lead to unexpected results. When assigning the values of one matrix to another, you should make sure that both matrices are of the same size so that redimensioning does not occur.

You can also use the matrix assignment statement to assign the same value to every element in an array. The value can be a constant or the value of a variable or an expression. Here is an example program:

```
!Filename: WT16_018.TRU
!Author: J.R.Arscott
!Purpose: MATRIX ASSIGNMENT program

! Assign a value to each array element.

DIM Table$(3,3)
MAT Table$ = ("xxxx")
MAT PRINT Table$
END
```

This program produces the following display:

```
xxxx xxxx xxxx xxxx xxxx xxxx xxxx xxxx
```

The parentheses around "xxxx" are not required in this case, but they are required if the expression on the right side of the assignment statement is anything other than a constant or a simple variable.

MATRIX ADDITION and SUBTRACTION

Adding two matrices means adding the values of the corresponding elements of the two matrices. If B and C are both 2-by-2 matrices, you create the sum matrix A with the statement

```
MAT A = B + C
```

where $A(1, 1) = B(1, 1) + C(1, 1)$, $A(1, 2) = B(1, 2) + C(1, 2)$, and so forth.

Matrices B and C must have the same number of dimensions and each dimension must be of the same size. If needed, matrix A will be redimensioned to the size of B and C, but as in the case of assignment, I recommend avoiding such redimensioning. Here is an example program:

```
!Filename: WT16_019.TRU
!Author: J.R.Arscott
!Purpose: MATRIX ADDITION program

! Matrix addition.

DIM A(2,2), B(2,2), C(2,2)
PRINT "Read matrix B from a DATA statement."
MAT READ B
PRINT "Type in 4 element values for matrix C."
MAT INPUT C
MAT A = B + C
PRINT
PRINT "Matrix B"
MAT PRINT B;
PRINT "Matrix C"
MAT PRINT C;
PRINT "Matrix A after adding matrices B and C."
MAT PRINT A;

DATA 3,3,3,3
END
```

The following program output is displayed:
Read matrix B from a DATA statement. Type in 4 element values for matrix C.

? 1,2,-3,5

Matrix B

3 3

3 3

Matrix C

1 2

-3 5

Matrix A after adding matrices B and C.

4 5

0 8

Subtraction is similar to addition, so here is a program that does both addition and subtraction:

```
!Filename: WT16_020.TRU
!Author: J.R.Arscott
!Purpose: MATRIX ADDITION & SUBTRACTION program

! Matrix addition and subtraction.

DIM A(2,2), B(2,2), C(2,2), D(2,2)
PRINT "Read matrices B, C, and D."
MAT READ B, C, D
PRINT "Matrix B"
MAT PRINT B;
PRINT "Matrix C"
MAT PRINT C;
PRINT "Matrix D"
MAT PRINT D;
MAT A = B + C
MAT A = A - D
PRINT "Matrix A = B + C - D."
```

```
MAT PRINT A;  
  
DATA 1,2,3,5  
DATA 2,1,1,2  
DATA 3,7,1,4  
END
```

This program displays the resulting matrix A, as shown:
Read matrices B, C, D.

Matrix B

1 2

3 5

Matrix C

2 1

1 2

Matrix D

3 7

1 4

Matrix A = B + C - D

0 -4

3 3

Note that a single expression like $\text{MAT } A = B + C - D$ is not allowed. Only two matrices are permitted on the right side of the MAT expression and thus two MAT statements are needed to accomplish the addition and subtraction. You should check the results yourself by hand or calculator.

MATRIX MULTIPLICATION and DIVISION

Multiplication of two matrices is allowed only under certain conditions. Division of one matrix by another is not defined, but there is an equivalent process of multiplying one matrix by the inverse of another matrix. These operations really cannot be understood without some background in matrix arithmetic and

will not be discussed further.

You should know, however, that True BASIC does have statements for multiplying two matrices and for inverting a matrix. If you need to use either of these operations, consult a book on matrix arithmetic and the True BASIC Reference Manual.

Multiplying a matrix by a single constant, variable, or expression means multiplying each element of the matrix by the same value. The syntax is

```
MAT A = N * B
```

where N is a constant, variable, or expression and B is a matrix. Note that N must come before B; the statement `MAT A = B * N` is not allowed. If N is an expression, it must be enclosed in parentheses.

Here is an example program where a list of prices is read from DATA statements and each price is increased by 20 percent — that is, multiplied by 1.2:

```
!Filename: WT16_021.TRU
!Author: J.R.Arscott
!Purpose: MATRIX MULTIPLICATION program

DIM Price(4)
LET Increase = 20 ! percent increase
MAT READ Price
MAT Price = (1 + Increase/100) * Price
MAT PRINT Price
DATA 12.45, 13.95, 8.40, 2.95
END
```

The following results are displayed:
14.94 16.74 10.08 3.54

Summary of Important Points

- Accessing array elements in odometer order means accessing the elements so that the last index varies the fastest. In a two-dimensional array, this means accessing the elements by row, from left to right.
- The MAT INPUT and MAT READ statements assign values to array elements in odometer order.
- Any MAT INPUT or MAT READ statement can be used to redimension an array.
- The MAT PRINT and MAT WRITE statements read values from array elements in odometer order.
- The statement MAT INPUT Array(?) is a special form of redimensioning allowed only with one-dimensional arrays of numeric values.
- The MAT LINE INPUT PROMPT statement displays its prompt only before the first element value is entered, not before subsequent values.
- The MAT PRINT # statement has only limited usefulness because a text file written with this statement cannot be read by a MAT INPUT # statement.
- We recommend using only a single array variable with the MAT INPUT, MAT READ, MAT PRINT, and MAT WRITE statements.
- The matrix assignment statement can be used to redimension an array.

Common Errors

- Reading or writing more than one matrix in a single MAT statement. This is valid but can cause confusion. It is poor programming practice.
- Allowing automatic matrix redimensioning to occur in a MAT assignment statement when you do not want the size of the matrix to change.
- Trying to add two matrices of different sizes, or trying to add more than two matrices in a single MAT statement.
- Using the wrong order of items when multiplying a matrix by a constant. The constant must pre-multiply (be to the left of) the matrix.

Chapter 17

MATRIX and ARRAY functions

Idn

returns a square numeric array that is an identity matrix, as in the program statement

```
MAT(A) = Idn
```

The identity matrix is a square matrix (same number of rows and columns) with element values of one along the principal diagonal (from upper left corner to lower right corner) and element values of zero elsewhere. This matrix is created by the Idn function.

```
!Filename: WT17_001.TRU
!Author: J.R.Arscott
!Purpose: MATRIX IDENTITY program

! Create an identity matrix.

DIM A(3,3)
MAT A = Idn
MAT PRINT A;
END
```

Here is the program output:

```
1 0 0
0 1 0
0 0 1
```

The value of any matrix remains unchanged when it is multiplied by the identity matrix.

Trn

The transpose of a matrix is that matrix with its rows and columns interchanged. This matrix is created by the Trn function, e.g.,

```
MAT B=Trn(A)

!Filename: WT17_002.TRU
!Author: J.R.Arscott
!Purpose: MATRIX TRANSPOSE program

! Create the transpose of a matrix.

DIM A(2,3), B(3,2)
MAT READ A
MAT B = Trn(A)
PRINT "Original matrix"
MAT PRINT A;
PRINT "Transpose matrix"
MAT PRINT B;

DATA 2,5,1,3,6,2
END
```

This program produces the following output:

Original matrix

```
2 5 1
3 6 2
```

Transpose matrix

```
2 3
5 6
1 2
```

Note that the arrays A and B must be properly dimensioned for one to be the transpose of the other.

Det(A)

returns the determinant of the numeric array A

A square matrix has a value associated with it called its determinant. If the determinant of a matrix is zero, there are certain restrictions on how that matrix may be used. The function Det(A) returns the determinant of the matrix A.

```
!Filename: WT17_003.TRU
!Author: J.R.Arscott
!Purpose: MATRIX DETERMINANT program

REM: Program WorldTour057.TRU
! Calculate the determinant of a matrix.

DIM A(3,3)
MAT READ A
PRINT "Matrix A"
MAT PRINT A;
PRINT "Its determinant is "; Det(A)

DATA 2,5,1,3,6,2,2,5,7
END
```

Program output is shown: Matrix A

```
2 5 1
3 6 2
2 5 7
```

Its determinant is -18

Con

returns a numeric array all of whose elements are one, as in the program statement

```
MAT(A) = Con
```

Dot(A, B)

returns the dot product of the one-dimensional numeric arrays

A and B

Inv(A)

returns the inverse of the square numeric array A

Lbound(A, N)

returns the lower bound in dimension N of array A

```
!Filename: WT17_004.TRU
!Author: J.R.Arscott
!Purpose: LBOUND program
```

```
DIM A(2:5,-3:10)
DIM F$(1:20)
LET x=LBOUND(A,1)
LET y=LBOUND(A,2)
LET z=LBOUND(F$)
PRINT x,y,z
```

END

Ubound(A, N)

returns the upper bound in dimension N of array A

```
!Filename: WT17_005.TRU
!Author: J.R.Arscott
!Purpose: UBOUND program
```

```
DIM A(2:5,-3:10)
DIM F$(1:20)
LET x=UBOUND(A,1)
LET y=UBOUND(A,2)
LET z=UBOUND(F$)
PRINT x,y,z
```

END

Nul\$

returns a string array all of whose elements are null strings, as

in the statement

```
MAT(A$) = Nul$
```

Size(A, N)

returns the number of elements in dimension N of array A
SIZE can be defined as UBOUND(A,N)-LBOUND(A,N)+1

```
!Filename: WT17_006.TRU  
!Author: J.R.Arscott  
!Purpose: SIZE program
```

```
DIM A(2:5,-3:10)  
DIM F$(1:20)  
LET x=SIZE(A,1)  
LET y=SIZE(A,2)  
LET z=SIZE(F$)  
PRINT x,y,z  
END
```

Zer

returns a numeric array all of whose elements are zeroes, as
in the statement

```
MAT(A) = Zer
```

Chapter 18

File Handling

For a few moments let me remind you about the fundamental principles of how computers work. They work and communicate using an electronic language that only has two words: on and off. In mathematical terms we represent these two conditions with the symbols one and zero, hence the name binary code. Each brain cell in the memory of a computer can either be on or off. In other words the computer thinks in terms of 'bits' of data which are either 1 or zero.

On the face of it, this seems very limited compared to our own language capabilities. However, we humans are an ingenious bunch, so we devised a way to make it possible to communicate with computers. If we group eight 'bits' of data into 'bytes' we can create a unit of data that has 256 different possible combinations. By assigning each one of these combinations to a different letter of the alphabet, and to punctuation and to numbers, then we arrive at the basis for a common language that we both understand.

So there you have it: we communicate by means of bytes of data. In fact the byte is a common unit for specifying computer memory size. The term kilobyte (KB) equals 1024 bytes; the term megabyte (MB) equals one thousand bytes, the term gigabyte (GB) equals one thousand megabytes.

OPENING FILES

Before you can use any file (including a printer) you have to open the file using the following general format:

```
OPEN #channel: NAME:CREATE:ORGANIZATION:ACCESS
```

Where (#channel) is a numeric constant, e.g., #1 or #2 or #3

etc., which is effectively a shorthand reference number rather than using the full name of the file again and again. Once you have opened a file, then you must always use the channel number whenever you want to perform an instruction on that file rather than using the file name. The channel number must be an integer from 1 to 1000.

File #0 is reserved for reading from the keyboard or writing on the screen; this channel is always open. If N has a noninteger value, it is rounded to the nearest integer.

NAME

Following the label NAME you must insert the name of your file. If this file resides in the current working directory then a simple name such as "MyFile.ABC" can be used, but if it is located anywhere else then the full path is required, e.g., "c:\True BASIC\Tblibs\Myfile.ABC" or you can use an ALIAS, e.g., "{library}MyFile.ABC" or you can use a string variable such as filename\$ as long as you have previously assigned the real name to this variable, e.g.,

```
LET filename$="c:\True BASIC\Tblibs\Myfile.ABC"  
NAME filename$:  
or  
NAME "c:\True BASIC\Tblibs\Myfile.ABC":
```

When you OPEN a file you must specify the file name or path. The other parts of the specification are optional, because True BASIC will assign default values to these options.

CAUTION: If the editor is using Tbsystem531 then do not use long filenames. Instead, use the old fashioned DOS convention of no more than 8 characters in the filename of a directory or sub-directory. If your filename is too long then you will probably get the error message "No such file".

Now let's deal with the other three option labels:

FILE TYPES (ORGANISATION)

The default option is TEXT.

BYTE

When we store data on a hard drive or floppy disk we do so as bytes of data, usually as magnetic traces (which are the direct equivalent of electronic signals). The files of these bytes of data are naturally called BYTE files. All files are BYTE files, in which the files consist of a series of bytes of data recorded as a nose to tail stream on the storage disk.

You send or retrieve data to and from a BYTE file with:

```
WRITE #channel:  
READ #channel:
```

Following the ORGANIZATION label you need to enter what type of file it is. You may use ORG as an abbreviation. True BASIC only recognizes BYTE, TEXT, RECORD and RANDOM, e.g.,

```
ORG Byte:
```

TEXT

This is not the end of the story because we humans don't always find the byte method of storing data as being particularly helpful or convenient. For example, we are accustomed to reading lines of text in books and elsewhere, so it would be nice if computer files worked this way too. Hence we have TEXT files. These are BYTE files in which there is a hidden symbol attached to the end of each line of text. In fact on a Windows PC there are two symbols; CHR\$(10) and CHR\$(13), otherwise known as <Line Feed> and <Return>. This convention is almost universally recognized by most applications such as

NOTEBOOK and MS WORD, as well as the TBeditor. Files that are formatted in this way are usually designated with the filename extension TXT.

You send or retrieve data to and from TXT or CSV files with:

```
PRINT #channel:  
LINE INPUT #channel:
```

There is one other aspect of TEXT files that you need to know about. You cannot start reading somewhere in the middle of a sequential text file, only at the beginning. You cannot go to the beginning or the middle of an existing text file and write new information. If you try to do either of these things, you will receive an error message. However, you can APPEND data to the file by moving the pointer to the end of the file, e.g.,

```
RESET #1: end
```

You can also ERASE a text file and re-write it. This doesn't delete the file: it just erases the data, e.g.

If you want to write new information on an existing file, use the statement

```
ERASE #1
```

CSV

There is also a special version of the TXT file that in the early days of computing was very popular for storing data used by spreadsheets. Even today MS Excel can still read this type of file, which is known by the extension CSV. In this type of text file the lines of text are further sub-divided into individual data items separated by commas, hence the name CSV; Comma Separated Variables. These files adopted a convention that the very first line of text contained the titles or headings for each column in a spreadsheet.

You send or retrieve data to and from TXT or CSV files with:

```
PRINT #channel:
LINE INPUT #channel:
```

Following the ORGANIZATION label you need to enter what type of file it is. You may use ORG as an abbreviation. True BASIC only recognizes BYTE, TEXT, RECORD and RANDOM, e.g.,

```
ORG Text:
```

RECORD

We also have a need for yet another different method of formatting files. Take the case where we have huge amounts of data, such as the personal details of every single person in the nation. We are talking of many gigabytes of data. If we want to see the personal detail of just one person, then we don't really want to read through all this data just to get to one record. We want to be able to pick out that particular record by calculating where it is or by means of some sort of reference number. Hence the name RECORD files. Again they are BYTE files, but this time the data is stored in blocks of bytes, where all the blocks contain a fixed number of bytes. This way we can easily calculate where any particular block starts and finishes.

You send or retrieve data to and from RECORD or RANDOM files with:

```
WRITE #channel:
READ #channel:
```

Unfortunately each record can only hold one data item, which can be either a numeric value or a string, but you need to know which of these two data types each record contains. When you read data from this type of file you have to make sure that you

don't try to read a string when the record contains a number. These limitations mean that this type of file is very rarely used. It is only included in True BASIC to satisfy legacy (old fashioned) code.

RANDOM

Fortunately, in True BASIC there is a variation of the RECORD file format in which the block of bytes is again equal for each record, but the data inside can be mixed numbers and strings with no limitation on how many data items as long as they fit inside the block size. This type of file is called a RANDOM file, but otherwise it works the same way as RECORD files.

A word of caution is required here. RECORD and RANDOM files are not universally recognized. These files will only work with True BASIC and not with any other language or application, whereas TEXT and BYTE files are more or less universal. Other computer languages have their own ways of creating and handling RECORD and RANDOM files and the format is not the same as that used by True BASIC.

You send or retrieve data to and from RECORD or RANDOM files with:

```
WRITE #channel:  
READ #channel:
```

CAUTION: If you used mixed numbers and strings when WRITING to a record, then you must mirror the same data type and order when READING a record.

Following the ORGANIZATION label you need to enter what type of file it is. You may use ORG as an abbreviation. True BASIC only recognizes BYTE, TEXT, RECORD and RANDOM, e.g.,

ORG Random:

CREATING FILES

Default option is OLD.

Now you must define whether you want to create a (NEW) file or open an existing (OLD) file, or whether you want both options (NEWOLD).

Obviously the NEWOLD is the safe option because if the file exists it will be opened, and if it doesn't exist then it will be created as a new file. Add the appropriate option after the label CREATE, e.g.,

```
CREATE newold:
```

ACCESSING FILES

Default option is OUTIN.

Next we need to complete the specification with how we intend to access the file. In this case True BASIC only recognizes INPUT, OUTPUT, or OUTIN (both), e.g.,

```
ACCESS outin:
```

A complete OPEN specification looks like:

```
OPEN #2:NAME file$:CREATE new:ORG text:ACCESS out
```

CLOSING FILES

When you have finished with a file then you must close it with:

```
CLOSE #channel
```

This will release the channel number which can then be used

for another file.

WTLIB library

I think you will agree with me that most of this is obvious even if the OPEN statement is a bucket full of typing. It is also obvious that reading and writing stuff to files is a prime candidate for a general purpose library of sub-routines, just to save all that typing everytime you want to use a file.

Here we go with two general routines that read and write to any of the file types. Before we begin let's think about how we can simplify the task. Firstly, unless you are a mathematician or maybe a physicist, you will probably never need to use high precision numbers in the internal IEEE format. Instead we can opt to use ordinary numbers in string format, e.g.,

```
Let number$=str$(number)
```

This way we are always dealing with reading and writing data as strings.

Lastly we need to think about how we want the data that we collect from the file sent back to our program from the sub-routine. We know that there is likely to be a lot of data and we know that the data type will all be strings. Does the word arrays ring any bells for you? In the case of CSV files and RANDOM files, the data string is likely to consist of several sub-strings, but for simplicity we can break these down with a separate routine. This will not be true for TEXT files or RECORD files.

As a minimum requirement the parameters we need to send to the sub-routine are the name of the file and its type. In the case

of RANDOM files we ought to allow for a record number as a parameter too. The minimum data it can send back to your program is an array, e.g.,

```
SUB read_anyfile(file$,type$, array$(),rec,error$)
```

Where type\$ can be “T” for text, or “C” for csv, or “B” for byte or “R” for random or “REC” for record. The variable (rec) will be zero for T and C and for R if we want to read the whole RANDOM file. If (rec)>0 then we only want to read one record.

What the sub-routine sends back to your program is a string array that contains complete lines of text in each element for TXT files, or an array of rows of data items for CSV files, or a similar array for whole RANDOM files or just one row for a single record.

You get all this for just one line of code, and no worries about debugging either. A fair exchange I would say. This is an excellent demonstration of how to make life easy for yourself by writing sub-routines to do the donkey work.

By the way, I have written the code in Wtlib.TRU so that you can see how to read data from each of the file types individually, just in case you want to do something extra special. For example, you may want to read through a TEXT file and if you find a particular item of data then you may want to stop reading any more. In this case you can copy the text reading part of the routine and add:

```
IF data$=(some specific string) then  
EXIT SUB  
END IF
```

You can now create your own sub-routine and use that instead of the general purpose one.

Much the same goes for the routine that writes data to files, e.g.,

```
SUB write_anyfile(file$,type$,array$(),rec, err$)
```

In this case your program has to provide the array and then the sub-routine will sort out all the nitty gritty stuff concerned with separating out the array into data items in the file.

I have also added two very useful twists to this routine, i.e., the ability to append data to files, just by setting the value of (rec) equal to -1, and the ability to erase and overwrite an existing file by setting (rec) equal to -2.

Notice that you don't need to worry about channel numbers, nor about opening or closing the channel. So in future you can use these routines without bothering to know about all the stuff earlier in this chapter. You may be interested to know that the TBeditor uses almost identical routines for all its file handling.

Both of these routines are listed in the example program WTLIB.TRU, which is actually a library module, and also contains a routine to check if files exist, as well as a routine for splitting up data rows into columns. You can use these routines with any file types.

The provisional outline code for these two routines is as follows:

```
!Filename: WT18_001.TRU
!Author: J.R.Arscott
!Purpose: READ ANY FILES program
REM: typical example text file
DIM file$(1)
LET rec=0
LET name$="c:\TBbook\All_files.TXT"
CALL Read_anyfile(name$,"T",file$(),rec,error$)
PRINT error$
PRINT rec ! total bytes
END
```

```

SUB Read_anyfile(file$,type$,any$(),rec,error$)
  WHEN EXCEPTION IN
    REM: Check if file exists
  OPEN #99:name file$,access outin,create old,org byte
    ASK #99: FILESIZE total
    LET rec=total
    IF total>0 then ! file exists so proceed
      REM: File type is upper case first letter.
      LET org$=ucase$(type$[1:1])
      SELECT CASE org$
        CASE "B"
          REM: must be BYTE file
        CASE "C","T"
          REM: must be CSV or TEXT file
        CASE "R"
          REM: Random or Record
          REM: Check first three letters
          IF ucase$(type$[1:3])="REC" then
            REM: must be RECORD file
          ELSE
            REM: must be RANDOM file
          END IF
        CASE ELSE
          LET error$=" File type not recognized."
        END SELECT
      ELSE ! file not there so abort
        LET error$=error$ & "|Cannot find " & file$
      END IF
    USE
    CLOSE #99
    LET error$=error$ & "|Trouble reading from " &
filename$ & ".Process aborted"
  END WHEN

END SUB

```

What we have done here is to simplify the problem by sorting out which type of file we are dealing with. All we have to do now is insert a block of code to deal with each CASE.

If you have not already noticed, the CREATE option in

Read_anyfile is OLD. In other words the file must already exist for this routine to work.

Need I say that we can use exactly the same structure for writing to a file, except that the blocks of code for each CASE will be obviously different, e.g.,

```
SUB Write_anyfile(file$,type$,any$(),rec,error$)
```

You will notice, because I am pointing it out, that in both the reading and writing routines, an error handling system is included that effectively stops the library module from crashing. In addition there is a parameter (error\$) that will report back to your program what type of error has occurred. This makes debugging much easier because all you have to do is add the following line in your program after you have called these routines to reveal the errors:

```
IF error$>"" then
PRINT error$
GET KEY dummy
STOP
END IF
```

GENERAL READING

If you want to read anything from any type of file, then CALL the general purpose routine in the WTLIB library

```
CALL Read_anyfile(file$,type$,any$(),rec,error$)
```

I have written this routine so that you can copy and paste any section into your own program as a template if you wish to do something special that isn't already covered by the routine.

This means that the OPEN file statement is sometimes repeated unnecessarily.

For example, here is the routine for reading BYTE files:

```
!Filename: WT18_002.TRU
!Author: J.R.Arscott
!Purpose: READ BYTE FILES program
LET rec=0 ! read whole file
REM: must be BYTE file
OPEN #99:NAME filename$,create old,access outin,org
byte
ASK #99: FILESIZE bytes

REM: read whole file
MAT REDIM any$(1)
SET #99: RECSIZE bytes
READ #99:any$(1)
ELSEIF rec>0 then ! only read recsize bytes

LET length=len(type$)
IF length>1 then
LET newtype$=type$[2:length]
LET recsize=VAL(newtype$)

SET #99: RECSIZE recsize
MAT REDIM any$(1)
READ #99:any$(1)
ELSE
LET error$=error$ & "|No recsize"
END IF
END IF
CLOSE #99

END
```

If rec=0 then this routine allows you to read the whole file into just one single element of the array (any\$(1)).

If rec>0 then this determines how many bytes you wish to read from the start byte onwards is defined by adding this the type\$, e.g "B100", i.e., you want type\$ to be a BYTE file and you want to read 100 bytes from the start. This is just convenient shorthand notation that increases the flexibility of this routine.

In the case of TEXT and CSV files, the reading process is as follows:

```
!Filename: WT18_003.TRU
!Author: J.R.Arscott
!Purpose: READ TEXT FILES program

LET file$="c:\TBbook\All_files.txt"
DIM any$(1)
CLOSE #99      ! safety precaution
OPEN #99: name file$, create newold, org text, access
outin
DO while more #99
LET count=count+1
MAT REDIM any$(count)
LINE INPUT #99: any$(count)
LOOP
CLOSE #99
PRINT ubound(any$)
END
```

In the case of CSV files each element in the array will consist of a series of variables separated by commas. I will deal with how to disassemble each element in the CSV array into separate data items later.

Finally we have the case of reading a record:

```
!Filename: WT18_004.TRU
!Author: J.R.Arscott
!Purpose: READ RECORD/RANDOM FILES program

REM: Check first three letters
IF ucase$(type$[1:3])="REC" then
REM: must be RECORD file
OPEN #99:NAME filename$,create old,access outin,org
record
ELSE
REM: must be RANDOM file
OPEN #99:NAME filename$,create old,access
outin,org random
```

```

END IF
ASK #99: RECSIZE recsize

IF rec=0 then      ! read whole file
SET #99: RECORD begin
FOR v=1 to recsize
MAT REDIM any$(v)
READ #99: any$(v)
NEXT v
ELSEIF rec>0 then
REM: only read one record
MAT REDIM any$(1)
SET #99: RECORD rec
READ #99:any$(1)
END IF
CLOSE #99

```

If rec=0 the whole file is read into the array.
If rec>0 then only this one record is read into the array and the array size is reduced to one element (any\$(1)).

Once again, in the case of RANDOM files the process is exactly the same except that each array element may contain several data items, which we can separate later.

FIELDS and ARRAYS

Here is a routine that will UNPACK a data\$ array and transfer it to a two dimensional array in which the rows of data are also split up into columns or fields. This is the ideal format for spreadsheets.

```

!Filename: WT18_005.TRU
!Author: J.R.Arscott
!Purpose: UNPACK ARRAYS program

SUB Unpack_array(array$(,), data$(,),tab$,error$)

```

```

REM: breaks data$ array rows into array$ rows and
columns
REM: after reading from CSV or RANDOM files
REM: Assume all data$ have the same number of columns
REM: splits data$ array string into field$
REM: assigns each field$ element to array$ second
dimension

WHEN EXCEPTION IN
DIM field$(1)          ! iniitializing

LET top=UBOUND data$
FOR d=1 to top
REM: clean up data line and remove spaces
LET origin$=trim$(data$(d)) & tab$
LET start=1
LET total=0

DO
REM: search for the first occurence of tab$
LET position=cpos(origin$,tab$,start+1)
IF position>0 then
LET total=total+1
REM: redim field$ array first time around
IF d=1 then
MAT REDIM field$(total)
END IF

REM: clean up the data item

LET field$(total)=trim$(origin$[start:position-1])
REM: set the start position for the next search
LET start=position
END IF
LOOP while position>0

IF d=1 then

REM: only redimension array$ first time around
REM: allow enough for all field$()
MAT REDIM array$(top,total)

END IF

```

```

FOR f=1 to total
LET array$(d,f)=field$(f)
NEXT f

NEXT d

USE
LET error$=error$ & "|Trouble with unpack array."
END WHEN

END SUB

```

Here is another routine that does the reverse of UNPACK. In other words it takes a two dimensional array consisting of rows and fields (columns) and compacts it into a simple array of rows with the data fields separated by the parameter tab\$. CSV files use commas to separate the fields but other file types use symbols, such as the vertical bar (|).

```

!Filename: WT18_006.TRU
!Author: J.R.Arcscott
!Purpose: PACK ARRAYS program

SUB Pack_array(array$(,),data$(),tab$,error$)

REM: compacts array$ rows and columns into data$ rows
REM: ready to write to CSV or RANDOM files

WHEN EXCEPTION IN

LET dtop=UBOUND(array$,1)      ! number of elements in
first dimension
LET ftop=UBOUND(array$,2)      ! number of fields in
second dimension

REM: make enough space in data$ array

MAT REDIM data$(dtop)

FOR d=1 to dtop

REM: accumulate fields into one data$ element
LET data$(d)=array$(d,1)      ! first field

```

```

FOR f=2 to ftop
REM: add a separator (tab$) between each field
LET data$(d)=data$(d) & tab$ & array$(d,f)
NEXT f
NEXT d
USE
LET error$=error$ & "|Trouble with pack array."
END WHEN

END SUB

```

We only need these two routines when we are dealing with CSV or RANDOM files. In practice what you do is use the general purpose routine to read any file which puts all the data into an array and then you process this array to UNPACK the fields. Or if you are writing stuff to a file, you again use the general purpose routine but this time you PACK your data fields into a simple array first.

The WTLIB library also contains two more similar routines that UNPACK a single item of data into fields, or PACK several fields into a single item of data. In effect these two routines allow you to break down or build a single row of the sort of data you might get in a CSV or RANDOM file rather than a whole array, e.g.,

```

CALL Unpack_data(field$(), data$,tab$,error$)

CALL Pack_data(field$(),data$,tab$,error$)

```

GENERAL WRITING

If you want to write to any type of file, then CALL the general purpose routine in the WTLIB library

```

CALL Write_anyfile(file$,type$,any$(),rec,error$)

```

I have written this routine so that you can copy and paste any section into your own program as a template if you wish to do something special that isn't already covered by the routine. This

means that the OPEN file statement is sometimes repeated unnecessarily.

Here is the routine for writing to a BYTE file:

```
!Filename: WT18_007.TRU
!Author: J.R.Arscott
!Purpose: WRITE BYTE FILES program

DIM any$(1)
LET any$(1)="Mary had a little lamb."
LET rec=0 ! write whole file
LET filename$="SAMPLE.BYT"

IF rec=-2 then      ! delete and re-create
OPEN #99:NAME filename$,create newold,access outin,org
byte
CLOSE #99
UNSAVE filename$ ! file deleted
REM: now create new file
OPEN #99:NAME filename$,create newold,access outin,org
byte
LET length=len(any$(1))
SET #99: RECSIZE length
WRITE #99:any$(1)
CLOSE #99

ELSEIF rec=0      ! then erase and overwrite

OPEN #99:NAME filename$,create newold,access outin,org
byte
ERASE #99
LET length=len(any$(1))
SET #99: RECSIZE length
WRITE #99:any$(1)

ELSEIF rec=-1 ! append
OPEN #99:NAME filename$,create newold,access outin,org
byte
ASK #99: FILESIZE total
SET #99:RECORD total+1
LET length=len(any$(1))
SET #99: RECSIZE length
```

```

WRITE #99:any$(1)
ELSEIF rec>0 then ! replace existing bytes

OPEN #99:NAME filename$,create newold,access outin,org
byte
SET #99:RECORD rec
LET length=len(any$(1))
SET #99: RECSIZE length
WRITE #99:any$(1)
END IF
CLOSE #99
END

```

Note that this routine allows for various values of the variable (rec):

When rec=-2 then the whole file is deleted and re-written.

If rec=-1 then the data is added to the end of the file.

If rec=-0 then the file is erased and re-written.

If rec>0 then this will set the pointer to the byte at position rec, and the recsize will be determined by the size of the data in array\$(1). **This new data will overwrite the existing bytes.**

The general purpose write-to-file routine allows for an array of data to be written, but in the case of BYTE files the array can only have one element (any\$(1)). In other word\$ ALL the data you want to write has to be assigned to this single element, e.g.,

```
LET any$(1)="all my data regardless of how much"
```

This the general routine for writing to TEXT and CSV files.

```

!Filename: WT18_008.TRU
!Author: J.R.Arscott
!Purpose: WRITE TEXT FILES program
LET filename$="SAMPLE.TXT"
LET rec=0 ! erase and overwrite
DIM any$(1)
LET any$(1)="Mary had a little lamb."

IF rec=-2 then      !delete existing and re-create

```

```

CLOSE #99
OPEN #99: name filename$, create newold, org text,
access outin
CLOSE #99
UNSAVE filename$
OPEN #99: name filename$, create newold, org text,
access outin
SET #99: MARGIN maxnum
MAT PRINT #99: any$

ELSEIF rec=-1 then      ! append
OPEN #99: name filename$, create newold, org text,
access outin
SET #99: MARGIN maxnum
SET #99: POINTER end
PRINT #99: any$(1)
ELSE IF rec=0 then      ! erase and overwrite
OPEN #99: name filename$, create newold, org text,
access outin
ERASE #99
SET #99: MARGIN maxnum
SET #99: POINTER begin
MAT PRINT #99: any$
END IF

CLOSE #99
END

```

Finally, here is the the routine that writes to RECORD or RANDOM files:

A word of caution here. When you are writing to Record or Random files you need to specify the size of the records **BEFORE** you start to use the file. The WTLIB library module also contains a routine that lets you do this, called:

```
SUB SetFileRecord(filename$,type$,recsize)
```

Once you have set the record size then you don't need to bother about that file anymore, because the general purpose writing routine checks this size each time before writing anything


```

!Filename: WT18_009.TRU
!Author: J.R.Arscott
!Purpose: WRITE RECORD or RANDOM FILES program
LET filename$="SAMPLE.REC"
LET rec=0 ! erase and overwrite
DIM any$(1)
LET any$(1)="Mary had a little lamb."
LET recsize=50
LET type$="REC"

REM: Check first three letters
IF ucase$(type$[1:3])="REC" then
REM: must be RECORD file
OPEN #99: name filename$, create newold, org
record, access outin
SET #99: RECSIZE recsize
ELSE
REM: must be RANDOM file
OPEN #99: name filename$, create newold, org
random, access outin
END IF
SET #99: RECSIZE recsize

IF rec=0 then
REM: Erase and re-write file but keep recsize
ERASE #99
SET #99: POINTER begin
MAT WRITE #99: any$

ELSEIF rec=-1 then
REM: append one record to end of file
SET #99: POINTER end
WRITE #99: any$(1)
ELSEIF rec>0 then
REM: overwrite existing record
SET #99: RECORD rec
WRITE #99: any$(1)
ELSE
REM: Invalid rec.
LET error$="Invalid (rec) value"
END IF

```

```
CLOSE #99  
END
```

If `rec=-1` then the data is appended to the end of the file.
If `rec=0` then the file will be erased but the `recsize` will remain and **the whole file will be overwritten** by the array data.
If `rec>0` then only **this record will be overwritten** by the new data.

THE PRINTER

True BASIC thinks of the PRINTER as just another file, which is quite logical if you think about it. The PRINTER is just another peripheral device like a hard drive or a floppy disk. Instead of writing data in the form of a magnetic trace it sprays jets of ink at a piece of paper.

A printer is identified by the keyword PRINTER and is assigned a channel number using the statement

```
OPEN #N: printer
```

True BASIC tries to send an error message to the screen if an attached printer is not turned on or properly connected, but on some computer systems the message is not displayed. The computer may wait for a while and then continue executing the program. All information sent to the printer is lost. If this happens to you, all you can do is interrupt the program by pressing the STOP button on the editor toolbar. A warning message is included in the example program to remind the user to turn on an attached printer.

Here is a simple program that prints a text file specified by the user. It reads the file using the WTLIB library and sends back any error messages. This example program shows the code that sends data from a previously created file called WTtext.TXT to the printer.

```
!Filename: WT18_010.TRU
!Author: J.R.Arscott
!Purpose: WRITE to PRINTER program
```

```
REM: Send text to a printer
LIBRARY "WTlib.tru"
LET name$="WTtext.TXT"
LET rec=0
CALL Read_anyfile(name$,"T",data$(),rec,err$)
IF err$>" " then
PRINT err$
GET KEY dummy
END IF
```

```
REM: how many lines of text?
Let end_data=ubound(data$)
```

```
PRINT "Be sure printer is attached and turned on."
PRINT "Press any key to continue."
GET KEY dummy
REM: could use WTLIB Write_anyfile routine
OPEN #1: printer
IF rec>0 then
    PRINT #2: Data$(rec)
ELSE
For n=1 to end_data
    PRINT #2: Data$(n)
NEXT n
END IF
CLOSE #1
END
```

Note the GET KEY statement after the two PRINT statements that display a message to the user. You have seen this statement before — it freezes the screen display so a message can be read. Pressing any key allows program execution to continue. You cannot see the output from the TEXT file on the screen because all output is redirected to the printer.

The WTLIB library also recognizes the PRINTER as a text file that you can write to with:

```
CALL Write_anyfile("PRINTER","P",data$(),rec,error$)
```

The code that handles writing to a printer is as follows:

```
!Filename: WT18_011.TRU
!Author: J.R.Arscott
!Purpose: WRITE to PRINTER program

WHEN EXCEPTION IN

OPEN #99: printer
IF rec=0 then      ! print whole file
LET top=ubound(data$)
FOR n=1 to top
PRINT #99: any$(n)
NEXT n
ELSEIF rec>0 then  ! print one line of data
PRINT #99: any$(rec)
ELSE
LET error$="Invalid rec value"
END IF
CLOSE #99

USE
CLOSE #99

LET error$=error$ & "|Trouble printing " & filename$ &
"|Process aborted"

END WHEN
```

If `rec=0` then the whole array is printed whereas if `rec>0` then only this one line of `data$(rec)` is printed.

As with all the routines in the WTLIB library, if there are any problems then the parameter (`error$`) will contain an appropriate message and your program will not crash.

Immediately after a call to a WTLIB routine it is advisable to check for errors with

```
IF error$>"" then
PRINT error$
GET KEY dummy
STOP
END IF
```

These few lines will print any errors on the screen. You only need to press any key and this will STOP your program to allow you to sort out the error problem.

With the aid of another special library module called Pwlib we can even persuade True BASIC to think of a printer as being a WINDOW instead of a file. This means you can reproduce graphics as well as text, but more about that later.

Personally I have never experienced any problems with EPSON printers, but this not necessarily true of other makes of printer. The problems basically boil down to the driver used by the printer. A printer driver is a piece of software that translates data coming from your computer into something that the printer understands. For example; if your computer sends the ASCII character 10 then the printer driver will translate this into an instruction that causes the paper to advance by one line. If you send the ASCII character 13 then the printer will be instructed to move to the start of a line. It stands to reason that if you have the wrong driver installed in the printer, then it may not do what you and your computer expect it to do.

Summary of Important Points

- When a text file containing text is opened for writing, you can start to write only at the end of the file.
- It is good practice to close a file after you are through using it, especially if you have written new information on it.
- I recommend that you use the PRINT # statement with only a single variable, expression, or constant when

writing on a file.

- You cannot start reading somewhere in the middle of a sequential text file, only at the beginning.

Common Errors

- Forgetting that the default create value is OLD when trying to open a new file without specifying a create value.
- Trying to write new information in the middle of an existing text file.
- Opening a text file for INPUT access and then trying to write information on it.
- Failing to close a file after writing on it. This may not hurt but you never know.
- Opening a file for OUTPUT access and then trying to read information from it.
- Forgetting to erase an existing file before writing new information on it.
- Writing more than one item per line on a text file, and then failing to tell every user of the file what you have done.
- Using the LINE INPUT statement to read digits from a text file and assign them to a numeric variable. Use the INPUT statement instead, or use the Val function to convert a string of digits to a numeric value.
- Trying to pass a file number as a parameter to a function.
- Failing to set a wide margin when writing long strings on a text file, thus breaking up each string into multiple substrings.
- Using long filenames or pathnames when running with TBsystem531

EXEClib

One of the goodies included with True BASIC is a neat library module that contains some very useful directory and file handling routines that can save you a lot of time and effort.

SUB Exec_AskDir

This returns the name of the current folder or directory

SUB Exec_ChDir

This allows you to change directories

SUB Exec_MkDir

This allows you to create a new folder or directory

SUB Exec_ReadDir

This is an extremely useful routine because it will list all the files in the specified folder. You can even customize this list by specifying the type of file you want to list.

dir\$: name of top directory to look through

template\$: selects subset of available files

name\$(): array containing file names

size(): array containing the file sizes in bytes.

dln\$(): array containing last-modified date.

tlm\$(): array containing last-modified times.

type\$(): array containing the file's type.

Type\$ is a 4-character string 'drwx' with 'd' for directory, 'r' for read-permission, 'w' for write permission, 'x' for execute-permission, and '-' for otherwise.

SUB Exec_ClimbDir

This is the same as the routine above except that it also searches through any sub-directories as well.

SUB Exec_Rename

This renames folders.

SUB Exec_RmDir

This routine removes folders.

SUB Exec_DiskSpace

This tells you how much disk space you have used and how much you have left.

SUB Exec_SetDate

This routine sets the current date like you can do in Windows Control Panel.

SUB Exec_SetTime

This routine sets the current time like you can do in Windows Control Panel.

For more details of these routine consult the Reference Manual.

DATABASE PROGRAMS

I don't propose going into the theory behind database structure here, because there is not enough room to do the subject justice, and besides there are many elegant tomes available that deal extensively with this subject. You just need to know what a database is and what they can do.

Essentially a data base is like an oversized spreadsheet that is specifically designed to be searched very quickly, because they tend to be extremely large, and they are designed to present the user with useful and relevant data in an easy to read form.

Although it is certainly possible to create a database using TEXT files, there are serious flaws when it comes to using them in such applications, particularly if the data content is large. The right way to approach this application is to use RANDOM access files. Even RECORD files are not suitable because they are limited to one data item per record, although we can work around this problem by packing several sub-strings or fields into a single data item.

In case you have not realized, the big advantage of RANDOM or RECORD files is that you can read and write anywhere in the file, whereas with other files you always have to start at the

beginning.

As an example of the use of RANDOM files, let's develop a telephone directory program. This program maintains a file of telephone subscriber names and numbers. Such a file is commonly called a database, although that term more properly denotes a collection of several related data files.

The telephone directory program is a very simple example of one of the most important uses of computers, i.e., maintaining information in files and allowing that information to be searched, updated and extended. I don't want to bore you with all the fine detail. The important thing is for you to understand the principle, because in all probability you won't write your own database: instead you will use a commercial version, a free one downloaded from the Internet, or the Btrees version issued by True BASIC.

You also need to get into the habit of using sub-routines and library modules. Let these do the donkey work for you. You get no extra points for inventing the wheel again. Whenever you can, benefit from the experience and effort of others.

In practice, using a database involves not just simply reading and writing data, but you also need to be able to create a data file, add data, search the data, modify or change data, delete data and view the data. In this context we will be using the WTLIB library module because it does most of the file handling jobs required by this application without you getting involved in the fine detail. Let's face it, you don't need to know how the internal combustion engine works in order to drive a car.

If you glance through the WTLIB module, you will see that you can read and write stuff in bulk or just one record at a time. You can also overwrite existing data or you can add new data. These are all the sorts of activity that you need when you use a

database.

Record numbering starts with record 1. You cannot move the record pointer before the beginning of the file or beyond the end of the file.

Records are of a fixed length, defined initially by you. You cannot write any data larger than this record size, but you can write less. Often your data string will consist of sub-strings which were described earlier as fields.

There are times when you would like to know how many records are being used in a record file. This information is available from the WTLIB module with

```
CALL GetSize(filename$,type$,rec,size,error$)
```

Where rec is the length of each record in bytes and size is the total number of records in the file. Note that this routine only works for RECORD and RANDOM files

If a record file is used to store general information, each record usually consists of a single string. Remember that a string cannot be over 65,000 characters long. The record is often subdivided into several fields, each field containing a different piece of information.

The simplest way to create fields is to use variable-length substrings for each field. It is then necessary to place a special character as a separator between adjacent fields. The separator character must be one that is never used in a field value such as the vertical bar (|) rather than a comma. This is how the Pack_data and Unpack_data routines work in the WTLIB module, for example:

```
!Filename: WT18_012.TRU  
!Author: J.R.Arscott
```

```
!Purpose: PACK SUB STRINGS program
```

```
SUB Pack_data(field$(),data$,tab$,error$)
REM: Assembles an array of field$ in one single data
item
```

```
WHEN EXCEPTION IN
LET ftop=UBOUND(field$)
LET data$=field$(1)
FOR f=2 to ftop
REM: add a separator (tab$) between each field
LET data$=data$ & tab$ & field$(f)
NEXT f
USE
LET error$=error$ & "|Trouble with pack data."
END WHEN
```

```
END SUB
```

You just send an array of fields to this routine and it sends back the fields all joined together as the string data\$.

Likewise the Unpack_data routine does the reverse, i.e., you send it a string and it splits it into separate fields, e.g.,

```
!Filename: WT18_013.TRU
!Author: J.R.Arscott
!Purpose: UNPACK SUB STRINGS program
```

```
SUB Unpack_data(field$(), data$,tab$,error$)
REM: breaks up a single data item into field$ array
```

```
WHEN EXCEPTION IN
```

```
REM: clean up data line and remove spaces
```

```
LET origin$=trim$(data$) & tab$
LET start=1
LET total=0
```

```
DO
```

```
REM: search for the first occurence of tab$
```

```

LET position=cpos(origin$,tab$,start+1)
IF position>0 then
LET total=total+1
MAT REDIM field$(total)
REM: clean up the data item
LET field$(total)=trim$(origin$[start:position-1])
REM: set the start position for the next loop
LET start=position
END IF

LOOP while position>0

USE
LET error$=error$ & "|Trouble with unpack data."
END WHEN

END SUB

```

The following record string consists of the concatenation of variable-length field strings and separators. It is also of variable length, but must be no longer than the fixed length of the record. For example, a typical name-and-address record, using a vertical bar (|) character as the separator, might look as follows:

```
John Butler|13 South St.|Worcester|MA|03120
```

Apart from reading and writing to files, the WTLIB module also allows you to re-order or sort data in an array or in a file, and allows you to search through sorted or unsorted arrays and files to locate a specific item. These are the areas that we will concentrate on next.

You might be asking, ‘what is the big deal with sorting, why do we need to sort stuff anyway?’. The answer is quite simple: we need to sort data in arrays or files because it is so much quicker and easier to search through sorted lists than unsorted lists. This is the reason why dictionaries or an index at the back of a book are sorted alphabetically.

SORTING

There are various methods of sorting collections of data into a defined order, such as ascending or descending alphabetical or numerical order. The simplest method is called the Bubble Sort, which involves taking one item of data and comparing it with all the other items. If the number of data items is (n) then the time taken to perform a Bubble Sort is clearly a function of (n) squared. There are several other sorting algorithms that are much faster where the time to sort is a function of $n \log(n)$. The True BASIC library module SORTLIB.trc describes these other methods in more detail and includes several of the faster sorting routines, so I will not deal with them here.

Arrays

In programming terms the Bubble Sort involves using two nested FOR NEXT loops, e.g.,

```
!Filename: WT18_014.TRU
!Author: J.R.Arscott
!Purpose: SORTING ARRAYS program

REM: ASCENDING ALPHABETICAL STRING SORT

LET total=UBOUND(array$)      ! total items
FOR n=1 to total-1
  FOR m=n+1 to total
    REM: use ASCii values to compare
    IF ucase$(array$(n))<ucase$(array$(m)) then
      REM: swap over these two elements
      LET temp$=array$(n)
      LET array$(n)=array$(m)
      LET array$(m)=temp$
    END IF
  NEXT m
NEXT n
END
```

The WTLIB library module contains bubble sort routines for arrays that are based on the above example, e.g.,

```
SUB Sort_array(array$(),method,error$)
```

If you set the METHOD as 1 then the original array would be re-arranged with the highest value data (based on the ASCII table) in the first element of the array. If you set the METHOD as zero then the array would be sorted with the lowest value data in the first element at the top.

For additional flexibility, in the case of data arrays containing fields, as would be the case of CSV files, then there is a more complex sort routine that allows you to specify the field number on which to base the comparison test. The default case is field(1), i.e., the array is sorted on the contents of the first field. You may also specify sorting in ascending or descending order.

This bubble sort is called:

```
SUB Sort_fields(field$(,),field,method,error$)
```

In this routine the user must specify which field is to be sorted. Suppose you had a phone directory array that was divided into the following fields:

- | | |
|---|----------------|
| 1 | First name |
| 2 | Surname |
| 3 | Street address |
| 4 | Town |
| 5 | City/county |
| 6 | State |
| 7 | ZIP code |
| 8 | Phone number |

You might want this array sorted by SURNAME, in which case the field number would be 2, and if you set the METHOD as 1

then the original array would be re-arranged with the highest value surname (probably beginning with (Z)) at the top). If you set the METHOD as zero then the array would be sorted with the lowest value name (probably beginning with (A)) at the top.

There is also a built-in feature to cope with CSV arrays, i.e., where the first row in the array is a set of field titles. Obviously you want to leave these titles intact. In this case you make the field number negative. This tells the routine to leave the titles out of the sorting process. In the above example you would set the field number to -2, so the routine would still sort the array according to the second field, but would ignore the titles.

Files

In the case of sorting files, the method that you use depends on how much data is in the files. If it is fairly small (up to 500 items) then you might as well read the contents of the file into an array, and then sort the array, and finally re-write the file in sorted order. However, if the file is large (as it would be in a database) then I would suggest that you should sort the file records directly, e.g.,

```
!Filename: WT18_014.TRU
!Author: J.R.Arscott
!Purpose: SORTING ARRAYS program
REM: DESCENDING ALPHABETICAL STRING SORT
DIM ndata$(1)
DIM mdata$(1)
REM: Get number of file records
CALL GetSize(file$, "R", rec, size, error$)
FOR n=1 to size-1
CALL Read_anyfile(file$, "R", ndata$( ), n, error$)
FOR m=n+1 to size
CALL Read_anyfile(file$, "R", mdata$( ), m, error$)
REM: use Ascii values to compare
IF ucase$(ndata$(1)) > ucase$(mdata$(1)) then
REM: swap over these two elements
CALL Write_anyfile(file$, "R", mdata$( ), n, error$)
CALL Write_anyfile(file$, "R", ndata$( ), m, error$)
```

```
END IF  
NEXT m
```

```
NEXT n  
END
```

WTLIB has a single file sorting routine, based on the above example which reads two records and compares them. If these records are out of sequence then they are swapped over. Note that we don't need a temporary variable this time. We can just switch copies of the original data and write them back to their new record locations.

```
SUB Sort_file(file$,type$,method,field,tab$,err$)
```

If the data in the file is separated into fields, then this routine allows the user to specify which field the data order in the file is to be based on. If (field) is 1 or zero and tab\$ is empty, then the file is organized on the basis of a simple array of records. If the (field) is negative then the first row of titles is not included in the sort.

By now, it must be evident to you that sub-routines and library modules such as WTLIB can be extremely helpful. My advice would be that before you write any code yourself, always check to see if anyone else has already done the job or something similar.

THE BINARY SEARCH

Returning to the problem of searching a sorted list, we can now develop a program that uses the binary search method. For a moment, consider how you use a dictionary to find a particular word. First of all you open the dictionary anywhere and then look at the words on the open page. Immediately you can tell whether you need to back track towards the beginning (A) or whether you need to proceed forwards towards the end of the book (Z). Suppose you need to go towards the end, so you pick

a point half way between the current page and the back of the book. Once again we can immediately see in which direction we need to go next. By progressively picking the halfway point each time we eventually home in on the word we are looking for.

This is the basic principle behind the Binary Search, and it is surprisingly quick, but it only works with sorted lists. If you have a dictionary of 65,000 words, it only takes 16 attempts to find any word (fractions of a second). Now you know how modern spell-checkers keep up with your typing. In the case of predictive text, the tiny pauses between each letter that you type are enough for the computer to have located the most probable full word in the dictionary long before you have finished typing.

Arrays

To begin a binary search, you first need to establish how big the list is, i.e., the lowest element and the highest element number. For the moment let us call these two numbers the BASE and the APEX. We now take a value which is MIDWAY between these two values. At this point we check the midway element in the array against our target string to see if we need to go up towards the apex or down towards the base.

You need to pay attention here because it is very easy to get confused over which direction to take. It all depends on whether your data has been sorted in ASCENDING or DESCENDING order. If the order is ascending then this follows the same order as the elements in the array, i.e., the largest ASCII characters (the Z end) will be the highest elements. If the order is descending then the largest ASCII characters (the Z end) will be the lowest elements in the array.

As a result, if you compare your target string with a smaller array element AND the array is ASCENDING then you need to go towards the upper end of the array to find a match.

If we need to go up, then we change the value of BASE to MIDWAY+1 and go back to the beginning to repeat the process.

if you compare your target string with a larger array element AND the array is ASCENDING then you need to go towards the lower end of the array to find a match.

If we need to go down, then we change the value of APEX to MIDWAY-1 and go back to the beginning to repeat the process.

Conversely, if you compare your target string with a smaller array element AND the array is DESCENDING then you need to go towards the lower end of the array to find a match.

If we need to go down, then we change the value of APEX to MIDWAY-1 and go back to the beginning to repeat the process.

if you compare your target string with a larger array element AND the array is DESCENDING then you need to go towards the upper end of the array to find a match.

If we need to go up, then we change the value of BASE to MIDWAY+1 and go back to the beginning to repeat the process.

Have you got that? Maybe not, so here is a list of names arranged in ascending order and our target is VICTOR.

- 1 Adam
- 2 Charles
- 3 Eric
- 4 Frank
- 5 Harvey
- 6 Jack (this is the MIDWAY element)
- 7 Liam
- 8 Peter

- 9 Ronald
- 10 Thomas
- 11 Victor
- 12 William

The first binary trial hit (MIDWAY) is JACK which is less than VICTOR in the ASCII table, so we need to go DOWN the list (numerically) towards the BASE (William) in order to find a match for VICTOR.

Now here is the same list but in DESCENDING order.

- 1 William
- 2 Victor
- 3 Thomas
- 4 Ronald
- 5 Peter
- 6 Liam (this is the MIDWAY element)
- 7 Jack
- 8 Harvey
- 9 Frank
- 10 Eric
- 11 Charles
- 12 Adam

The first binary trial hit (MIDWAY) is LIAM which is again less than VICTOR in the ASCII table, so we need to go UP the list (numerically) towards the APEX (William) in order to find a match for VICTOR, whereas previously we had to go DOWN.

My advice is to stick with ASCENDING order to avoid confusion. It is a much more natural arrangement because it runs in the same direction as the numbered elements in an array.

In effect we are making the list shorter and shorter by changing the values of BASE and APEX. Eventually we will hit a match for the target string. At some point BASE and APEX might cross over, in which case we can stop the search because a match for the target string could not be found.

In the WTLIB library, the ASCENDING method=0 and the DESCENDING method=1.

Here is the code from the WTLIB routine:

```
!Filename: WT18_015.TRU
!Author: J.R.Arscott
!Purpose: BINARY SORTING ARRAYS program

SUB Search_sorted_array(array$( ),target$,rec,error$)

    REM: array must be one dimensional
    REM: use binary search

    LET rec=0
    LET total=UBOUND(array$)
    LET base=1
    LET apex=total
    REM: check sort method (ascending or descending)

    IF UCASE$(array$(apex))>UCASE$(array$(base)) then
    LET method=0
    ELSE
    LET method=1
    END IF

    DO
        LET midway=ROUND((apex+base)/2)

        REM: use ASCii values to compare
        REM: is target$ anywhere in array$(midway)?

        IF POS(ucase$(array$(midway)),ucase$(target$))>0
then
            LET rec=midway
        EXIT DO
```

```

                ELSEIF
UCASE$(array$(midway))<UCASE$(target$) then

                LET base=midway+1
                ELSEIF
UCASE$(array$(midway))>UCASE$(target$) then
                LET apex=midway-1

                END IF

        LOOP until base>apex
        IF rec=0 then
                LET error$=target$ & " not found."
        END IF

END SUB

```

It is assumed that the list (array\$) has already been pre-sorted. Note that this routine checks to see whether the array is ascending or descending by comparing the first and last elements. As you can see, the POS function is used to test whether the target string is anywhere inside array\$(midway). As soon as a match is found then the search is halted and the current position of MIDWAY is assigned to the parameter (rec).

If the array list has not been pre-sorted then you can use the much slower routine in WTLIB, e.g.,

```

!Filename: WT18_016.TRU
!Author: J.R.Arscott
!Purpose: SEARCH UNSORTED ARRAYS program

SUB Search_unsorted_array(array$(),target$,rec)

        REM: array must be one dimensional
        REM: compare all array elements

        LET rec=0
        LET total=UBOUND(array$)

        FOR n=1 to total

```

```

    REM: use ASCii values to compare
    REM: is target$ anywhere in array$(n)?
        IF
    POS(ucase$(array$(n),ucase$(target$))>0 then
        LET rec=n
        EXIT FOR
    END IF

    NEXT n

    IF rec=0 then
        LET error$=target$ & " not found."
    END IF

    END SUB

```

The WTLIB library also has two more routines that let you search through two dimensional lists (field\$(,)) such as you would find in CSV files, e.g.,

```

Search_sorted_fields(field$(,),target$,rec,field,err$)

Search_unsorted_fields(field$(,),target$,rec,field,err$
)

```

In each case you must specify the field you want to search.

The file WTdata.CSV allows you to test drive these routines with ready-made data arrays. You can then see how fast or slow the routines are.

As an extra bonus there is an executable program in the files that can be downloaded from the True BASIC website called WT_table_demo.exe that demonstrates the use of the “table” object in the CTX library module. This object allows you to construct a spreadsheet style table showing a two dimensional array on screen as rows and columns of data. This whole table only takes a couple of code lines to create.

Files

Once again, files are basically no different to arrays. If the file is small, then you might as well read the file into an array and search the array. In the case of databases, the file is likely to be very large so it is probably better to read and match individual (MIDWAY) records one at a time. By clicking on the heading column and then the SORT button, that column will be sorted. Beware, the FORMAT button only works with numerical data.

As usual, the WTLIB module offers the user routines that search both sorted and unsorted files, although the latter could be very slow indeed. These routines can search simple records and records with fields, e.g.,

```
SUB Search_sorted_file(file$,  
type$,target$,field,tab$,rec,err$)
```

```
SUB Search_unsorted_file(file$, type$,target$,rec,err$)
```

Obviously the WTLIB library cannot do everything. There are bound to be instances when you need to do some special file reading, or writing, or sorting, or searching, but rather than starting from scratch you might be better off taking one of the WTLIB routines and inserting a few extra lines of code. This way there is much higher probability that you will produce working code more quickly. Let the existing code guide you towards achieving your objective.

Now is the time to put your knowledge of sorting and searching to the test. It will also be an opportunity for you to marvel at what can be done with just a few lines of code, and to appreciate the power that True BASIC brings to your finger tips.

Let us suppose you have a directory folder on your hard drive that contains possibly thousands of files that you have accumulated over the years. You don't want to throw them away

because you know that buried in there somewhere are absolute pearls of wisdom on virtually every subject under the sun.

You recall reading an interesting article about the 1952 Studebaker automobile, but you have no idea what the file name was, nor when you filed it. Are you going to read through ALL those files to try and find it? For most folk this would be such a tedious and daunting task that they would be reluctant to even begin. However, you are a programmer and that makes all the difference in the world. With speedy True BASIC you write the following twelve lines of code and then let your computer do the legwork:

```
!Filename: WT18_017.TRU
!Author: J.R.Arscott
!Purpose: SEARCHING FILES program

LIBRARY "WTLIB.TRU"

DIM file$(1)
LET rec=0
SET DIRECTORY "C:\myfoolder"
CALL Get_all_files("C:\myfolder","*",file$())
FOR n=1 to ubound(file$)
CALL Search_unsorted_file(file$(n),"BYTE","Studebaker",
rec,error$)
IF rec>0 then ! must have found target word
PRINT "Target File=" & file$(n)
EXIT FOR
END IF
NEXT n
END
```

As you know, ALL files are byte files regardless of their method of organization. So we don't care whether the files are text files or CSV or record or random: BYTE fits all. We just search for the keyword and stop when we've found it.

So much for so little effort. That's True BASIC for you.

So far we have briefly looked at the sort of tools you need to operate a database. In the early days this was how it was done, until somebody observed that in practice many of the fields in a database were empty. You might have a name and a phone number, but no address details. Or you might have some of the address but maybe the zip code was missing. Imagine a database where 90% of the fields are missing. In a RANDOM file you have to allow space for this data in your original record size, even if that field is currently empty. You now have files that are ten times bigger than they need to be, and activities like reading, sorting and searching also take ten times long than necessary. There had to be a better way.

Eventually some very bright individuals devised a way to overcome this problem, the relational database. In this solution, much shorter length records were used in a simple tabular format. For example, the simple table might just consist of a name and a phone number. Later on when you had the address details, you created another table that only contained addresses. There were no empty spaces because you only entered data when you had the details. The end result was a series of much shorter files and no wasted space. The crucial part was how to link these random data lists together so that related information could be identified as belonging together.

The solution turned out to be the use of a unique “key” for each entry. As a simple illustration let’s use the actual record number in the original table as the key, i.e., the record number of the original name and phone number table. Let’s assume you call this file “original_name.dat”. A typical record entry in this file would be:

159|Jones|T|234|77654372

where 159 is the record number (key), Jones is the surname, T is the initial, 234 is the area code and 77654372 is the phone

number.

We can sort this list alphabetically in ascending order based on the second field (surname) and we can save this file as “sorted_name.dat”. In this file Mr Jones may be the 62nd person in the list, but the data entry will still be the same as the original complete with the original key.

Some time later we find out Mr Jones address, so we start another table file called address.dat. In this file Mr Jones may be the tenth entry that we have, e.g.,

159|14 Main Street|Windsor|Vermont

Later still we might find out that Mr Jones is a truck driver, so we start another table called “occupation.dat” and in this file maybe Mr Jones’s occupation is the twenty-third record and appears as:

159|Truck driver”

Like all the other files, this one can be sorted by “key number”. Retrieving data from these files would be extremely quick and efficient. We can now search the “sorted_name” file for the name Jones and the key (159) will allow us to locate his address, occupation or any other related data in any other file. It may not seem like much, but this concept was revolutionary in its time and spawned most of the relational databases in current use.

As I said earlier, you are unlikely to create a data base from scratch, using the WTLIB library and the techniques I described, because there are specialist database languages and systems available that do all the things that you are likely to need.

Summary of Important Points

- When strings are compared in logical expressions, they are compared character by character using the ASCII values of the characters.
- A third temporary variable is needed when interchanging the values of two variables.
- A logical flag can be used to indicate whether or not a certain task has been accomplished.
- A sequential search is the simplest technique for searching an unsorted list.
- The binary search technique works only for a list in sorted order.
- A record file contains fixed-length records. A string record can be divided into two or more fields.
- A random file contains fixed-length records with one or more string or numeric fields.
- Any record in a record or random file can be accessed directly and information can be either read from or written on the record.
- Unlike text files, you cannot read a string of numeric characters from a record file and assign them to a numeric variable.
- A True BASIC program can often be written to convert one text file format to another format.

Common Errors

- Modifying the wrong statement when changing the bubble sort algorithm from an ascending sort to a descending sort.
- Attempting to sort a text file directly on the disk.
- Writing a long text file to an array for sorting when there is insufficient memory for the array. One solution is to convert the text file to a record file and then sort the file directly on disk.
- Comparing two strings representing numeric values rather than comparing the numeric values themselves. Note that “+12.7” is less than “12.3” — because “+” is less than “1” — but the number +12.7 is greater than 12.3.
- Searching an unsorted list with the binary search algorithm.
- Trying to display a record file with a text editor.
- Failing to consider the differences between lowercase and uppercase characters during a search.

Chapter 19

Program Units

Modern computer languages allow a program to be designed as a series of independent program units or procedures. Two types of procedures are used frequently in True BASIC, the external function and the external subroutine.

Arguments and parameters allow information to be passed to external procedure and, in the case of external subroutines, to return information to the original program. Local variables provide isolation between the main program and procedures, and between individual procedures. Libraries of procedures can be constructed, saved on disk, and subsequently incorporated into new True BASIC programs. Libraries with special properties, called modules, are useful in larger programs.

MAIN PROGRAM

As a program becomes larger, it is advantageous to divide it into smaller units. These units are called program units and are further identified as the main program and additional procedures. The main program (or one of the procedures) can make use of one or more other procedures as it carries out its task, and it is then identified as the calling program unit. Any procedure can be called from any other procedure or from the main program.

There are several benefits to be derived from writing programs in this fashion.

Well-designed programs

Programs designed as collections of smaller units are easier to write and debug, easier to read and understand, and easier to maintain and modify. This type of design is called modular program design; dividing a large program into smaller modules

or program units. The term “modular” refers to dividing a large program into smaller units, it does not refer exclusively to True BASIC modules,(see page 260).

For example, suppose that you want to write a program to develop a classroom timetable schedule. That is not an easy problem to solve and you have not yet learned all the techniques required to write such a program. You can, however, analyze the problem and see how it can be divided into subtasks.

Task: Develop a classroom timetable schedule.

Subtask A: Enter list of classes with time and size of each class. Enter list of classrooms with size of each room.

Subtask B: Sort classes by hour of the day. For each hour, sort classes by descending size. Sort classrooms by descending size.

Subtask C: For each hour, assign class to room, starting with the largest size class. Note any class that cannot be assigned.

Subtask D: Print list of classes not assigned to rooms. Print list of rooms showing hours not assigned.

Whereas the task is complex and stated in general terms, each subtask is more specific and thus easier to understand. There may be a further division of one of the listed subtasks; for example, a general sorting routine might be a separate subtask within subtask B. After more experience, you will be able to write a program based on such a modular design.

Isolation

Isolation implies that each program unit is separate and distinct from the other units. Complete isolation means that a variable

in one program unit is completely different from a variable with the same name in another program unit. Changes in the value of a variable are confined to a single program unit.

The importance of isolation is that a change in a variable value in one program unit can never cause an unexpected change in another program unit. Lack of program unit isolation can be a major and hard-to-find source of errors in large programs.

In practice, complete isolation is not possible when you need to return or pass back information from a called program unit to the calling program unit. The goal of isolation is still desirable, however, and to the extent that if this goal can be achieved, the chance of errors is reduced.

Libraries

Libraries of well-designed and error-free procedures can be created and used in many different programs. A procedure that does a particular job and does it well should not be written over and over again from scratch. It should be placed in a library and used when appropriate in any new programs. These library functions and subroutines serve as language extensions to True BASIC and are discussed at the end of this chapter.

In view of all the preceding benefits, you will see modular program design used in many of the large example programs in this and subsequent chapters.

Here is a list of program units:

- main program
- external functions
- external subroutines
- external pictures
- internal functions
- internal subroutines

The main program is the type of True BASIC program that you have written in previous chapters. Its last statement is always an END statement. Other statements in the main program must name and identify all external procedures.

External functions and subroutines are discussed in the next two sections. They must be located after the main program's END statement or in a separate library file.

External pictures are mentioned here for completeness but are not discussed until Chapter 21.

Internal functions and subroutines are introduced at the end of this chapter. They are not used in any of the example programs in this book.

EXTERNAL FUNCTIONS

You are already familiar with built-in functions and single-line user-defined functions, defined in Chapter 7. A similar definition describes an external function unit. It is a procedure consisting of a separate block of program statements identified by name. It can have values passed to it from the calling program unit. The external function unit performs some calculation and returns a string or numeric value that is assigned to the function name.

An example might be a function to calculate the volume of a rectangular box, given the length, width, and height. That specific function is developed in this section and used in the following examples.

The external function is invoked or called when its name is used in an expression. A value must be assigned to the function name before control returns to the calling program unit. When I refer to a function in this section, I mean an external function.

For example, a function named `Volume` can be designed to calculate the volume of a box. When you call or invoke this function, you have to pass to it — make it aware of — the length, width, and height values of a specific box. To display the volume of a box that is 10 inches long, 5 inches wide, and 3 inches high, you might use the statement

```
PRINT Volume (10, 5, 3)
```

The numeric values in parentheses are called arguments of the function `Volume`. The external function unit itself consists of the following statements:

```
EXTERNAL FUNCTION Volume (L, W, H)
LET Volume = L * W * H
END FUNCTION
```

The function named `Volume` is assigned a value equal to the volume of the box. The variables `L`, `W`, and `H` are called parameters of the function `Volume`.

Function Unit Heading

The first statement in a function unit is

```
EXTERNAL FUNCTION Name (Parameters)
```

I prefer to use `DEF` (definition) rather than `EXTERNAL FUNCTION` because that is what we are doing. We are defining the function, where `Name` is the function name and `Parameters` is a list of variable names, separated by commas. The function name must follow the same rules that apply to variable names (see Chapter 2). The function value may be either numeric or string and if the function has a string value, the function name must end with a dollar sign.

The word `EXTERNAL` is optional. This word is required in `ANS`

BASIC but may be omitted in True BASIC. If the word EXTERNAL is omitted, any procedure listed after the main program END statement or in a library file is considered an external procedure.

In True BASIC the word FUNCTION can be replaced by DEFINITION (or the short form DEF) because this is exactly what it is, i.e., a definition of the FUNCTION. Personally I prefer DEF.

In the preceding example, the heading statement is

```
DEF Volume (L, W, H)
```

and the parameters are the variables L, W, and H.

Function Parameters

The parameters in a function heading statement are variables, separated from one another by commas, and are of type string or numeric. These variables are called local variables, meaning that they are known and can be used only within the function unit itself. Variables with the same name in any other part of the program are considered different from these local variables.

Values are assigned or passed to parameter variables when the function is called. This method of passing information through parameters is called pass by value and the parameters are called value parameters or sometimes, ONE-WAY parameters.

The concept of local variables is important because they provide isolation between an external function and other program units. In addition to the parameter variables, any other variable introduced and used in the function unit is a local variable.

Consider the case of a main program with a variable named

Reply\$ and an external function with a variable named Reply\$. These two variables have the same name but they refer to different mailboxes or memory locations. Thus a value assigned to Reply\$ in the function is placed in one mailbox, while the value of Reply\$ in the main program is contained in another mailbox. The two variables may have the same name but they are really completely different variables.

As an analogy, post office box 572 in the main post office contains entirely different letters than post office box 572 in a branch post office, although they both have the same identifying number or name and are in the same town.

Other FUNCTION Statements

The last statement in a function unit must be

```
END FUNCTION
```

True BASIC also allows

```
END DEF as an equivalent.
```

The unit which calls the function — often the main program — must have a declaration statement containing the function name. One declaration statement may contain several function names and there may be more than one declaration statement. The syntax is

```
DECLARE DEF Name1, Name2,...
```

Finally, there must be a specific assignment statement — a LET statement — in the function unit that assigns a value to the function name. This value may be either a number or a string, depending on the function type. The value must be assigned before the END DEF statement is executed. The complete example program looks as follows:

```

!Filename: WT19_001.TRU
!Author: J.R.Arscott
!Purpose: FUNCTIONS program

! Short program that uses an external function.

! Main program
DECLARE DEF Volume
PRINT "Volume of box is"; Volume(10, 5, 3)
END

! Function procedure
DEF Volume (L, W, H)
LET Volume = L * W * H
END DEF

```

The following output is displayed:

Volume of box is 150

It is possible to exit a function at points other than the END FUNCTION by using the phrase

```
EXIT DEF
```

An error occurs if the function has not been assigned a value before exiting. I recommend that you use this phrase sparingly and normally exit through the END DEF statement.

When a function is called, the values passed to it can be the values of variables or constants or expressions, as demonstrated in the next example program:

```

!Filename: WT19_002.TRU
!Author: J.R.Arscott
!Purpose: FUNCTIONS program

! Using different types of arguments
! in an external function.

DECLARE DEF Volume
LET L = 10

```

```

PRINT "Volume of box is"; Volume(L, L/2, 3)
END

DEF Volume (Length, Width, Height)
LET Volume = Length * Width * Height
END DEF

```

The function call in the PRINT statement obtains its first argument value from the variable L, its second value from the expression L/2, and its third value from the constant 3. The output is the same as that of the previous example program, even though we are using completely deferent variable names. This illustrates the principle of isolation.

Writing and Using Functions

Here are some other example programs that use functions. The first program uses a function to calculate the factorial value of a non-negative integer entered from the keyboard. For example, the factorial value of 5, denoted 5! by mathematicians, is equal to $1 \times 2 \times 3 \times 4 \times 5$ or 120. By definition, zero factorial equals 1. This program prompts the user to enter a number from the keyboard. If the number is negative or not an integer, the program displays an error message and stops. Otherwise, the factorial value of the number is calculated and displayed:

```

!Filename: WT19_003.TRU
!Author: J.R.Arscott
!Purpose: FUNCTIONS program

! Calculate the value of a factorial number.

DECLARE DEF Factorial

INPUT prompt "Enter number: ": Number
IF Number < 0 then
    PRINT "The number must be positive or zero."
ELSEIF Number <> Int(Number) then
    PRINT "The number must be an integer."
ELSE

```

```

        PRINT "The factorial value of"; Number;
        PRINT "is"; Factorial(Number)
    END IF
END

DEF Factorial (X)
! Calculate the factorial value of parameter X.
    LET Result = 1 ! initialize the variable Result
FOR I = 1 to X
    LET Result = Result * I
NEXT I
    LET Factorial = Result
END DEF

```

If the number is positive or zero and an integer, the function Factorial is called by the PRINT statement. The factorial value is calculated in the function procedure and the result is returned to the main program. If X has a value of zero, the FOR loop never executes and a value of one is assigned to Factorial.

In the main program, the value of the entered number is stored in the variable Number and this value is passed to the variable X in the function. Variable X is a local variable defined only in the function.

Here are three examples of typical program runs:

```

Enter number: -5.2
The number must be positive or zero.

```

```

Enter number: 5.2
The number must be an integer.

```

```

Enter number: 5
The factorial value of 5 is 120

```

Another example program uses string functions to find palindromes. A palindrome is a sentence or sequence of letters that reads the same backward and forward. To test a palindrome, all letters must be converted to the same case (say,

uppercase) and all nonalphabetic characters (including spaces or blanks) must be removed from the sentence.

The main program asks the user to enter a string of characters from the keyboard. The standard function Ucase\$ is called to convert all letters to uppercase. The user defined function Remove\$ is called to remove all nonalphabetic characters. Finally, the user-defined function Reverse\$ is called to write a new string that is the reverse of the original string:

```
!Filename: WT19_004.TRU
!Author: J.R.Arscott
!Purpose: FUNCTIONS program

! Find a palindrome.

DECLARE DEF Remove$, Reverse$
LINE INPUT prompt "Enter string: ": String$
LET String$ = Ucase$(String$)
LET String$ = Remove$(String$)
LET Compare$ = Reverse$(String$)
IF String$ = Compare$ then
    PRINT "String is a palindrome."
ELSE
    PRINT "String is not a palindrome."
END IF
END

DEF Remove$ (Item$)
    ! Remove nonalphabetic characters from string A$.
LET Copy$ = "" ! a null string
    FOR I = 1 to Len(Item$)
        IF Item$[I:I] >= "A" and Item$[I:I] <= "Z"
then
            LET Copy$ = Copy$ & Item$[I:I]
        END IF
    NEXT I
    LET Remove$ = Copy$
END DEF
DEF Reverse$ (Item$)
    ! Reverse the order of characters in a string.
LET Copy$=""
```

```

    FOR I = Len(Item$) to 1 step -1
        LET Copy$ = Copy$ & Item$[I:I]
    NEXT I
    LET Reverse$ = Copy$
END DEF

```

If you try this program yourself with the well-known palindrome “Madam, I’m Adam,” you should get the following results:

```

Enter string: Madam, I'm Adam
String is a palindrome.

```

Note that the string Copy\$ is initially assigned a null value in each function. The selected characters are then added to Copy\$, one by one. The resulting string, Compare\$, is then compared with the original string, String\$, and if they are the same, the original string is a palindrome.

True BASIC allows you to use the word DEF in place of the standard word FUNCTION in a function heading. The word FUNCTION is more descriptive but the word DEF is shorter. You may use either word in your programs. Remember that the word EXTERNAL is optional in the function unit heading.

Functions may be nested, meaning that one function may call another function. The calling function must have a DECLARE statement naming the called function.

Each time a function is called, all local parameter variables have values passed to them. Other local variables are re-initialized, numeric variables to zero and string variables to the null string.

The next example program illustrates how to pass a local variable (sum) to an external function.call:

```

!Filename: WT19_005.TRU
!Author: J.R.Arscott

```



```

!Purpose: SUM FUNCTION program

! Display the sum of a sequence of numbers
! entered from keyboard.

DECLARE DEF Add
LET total=0
PRINT "Display the sum of a sequence of numbers."
PRINT "Enter zero to stop input."
INPUT prompt "Number? ": Number
DO until Number = 0
    LET total = Add(Number,total)
    INPUT prompt "Number? ": Number
LOOP
PRINT "The sum is"; total
END

DEF Add (N,total)
    ! Add the number N to total
    LET total = total + N
    LET Add = total
END DEF

```

The following results are obtained:

```

Display the sum of a sequence of numbers. Enter
zero to stop input.
Number? 5
Number? 3
Number? 0
The sum is 8

```

EXTERNAL SUBROUTINES

In contrast to a function, the general purpose of a subroutine is to perform some task or carry out some action, such as printing a standard business form.

Subroutine Unit Heading

Like functions, external subroutines are separate blocks of

program statements. They are identified by name, and the first statement in a subroutine program unit must be

```
EXTERNAL SUB Name (Parameters)
```

In True BASIC, the word EXTERNAL may be omitted and thus the syntax

```
SUB Name (Parameters)
```

is also allowed.

Unlike a function, no value is associated with a subroutine name. A subroutine name cannot represent a string value and the dollar sign character is not allowed. The name is used only to identify the subroutine and never has any value assigned to it.

Unlike functions, it is not necessary to DECLARE subroutines.

The parameters in a subroutine heading statement are local variable names, separated from one another by commas. These parameters are DIFFERENT from the value parameters of a function. Their behavior is discussed in detail in the next section. Any other variables used in the subroutine are also local variables.

The last statement in a subroutine block must be

```
END SUB
```

and it is the normal exit point. Any other exit point must be identified by the phrase

```
EXIT SUB
```

The same caution applies here about use of the EXIT SUB statement when it is not absolutely necessary.

The following subroutine displays a string parameter named Value\$ on the screen, centered between the left and right margins. The parameter Width is the width of the screen in columns.

```
SUB Center (Value$, Width)
    PRINT Tab((Width-Len(Value$))/2); Value$
END SUB
```

Calling a Subroutine

A subroutine is called from another program unit by the statement

CALL Name (Arguments)

Arguments in the CALL statement may be variables, constants, or expressions. They must agree in number and type (numeric or string) with the parameter list in the subroutine heading statement. The calling program unit may be the main program, a function, or another subroutine. Using the preceding example subroutine, the call statement might be:

```
!Filename: WT19_006.TRU
!Author: J.R.Arscott
!Purpose: CENTER SUB-ROUTINE program

LET value$="2015 Earnings Analysis"
ASK MAX CURSOR height,width ! gets width of screen
CALL Center(value$,width)
END

SUB Center (Value$, Width)
    PRINT Tab((Width-Len(Value$))/2); Value$
END SUB
```

It creates the following centered display:

Subroutine Parameters

A major difference between functions and subroutines is the way that information is passed through parameters. In the CALL statement, names of argument **variables** refer to specific mailboxes or memory locations. In the SUB heading statement, names of parameter variables are **local names for these same mailboxes**. Thus if a change is made in the value of one of the local parameter variables in a subroutine, the value of the corresponding variable in the calling program is also changed.

Using the previous analogy, your post office box may be known officially as Box 572, but when you forget the key and ask the postmaster for your mail, you usually refer to it as John Blair's box. The public knows this box as 572, but the postmaster knows it as John Blair's box. These are just two names for the same box. Obviously, the contents of Box 572 and John Blair's box are the same.

Passing information in this manner does not provide complete isolation between the subroutine and the calling program unit, because both argument and parameter variable names refer to the same mailbox and the value it contains. In fact, the information that is passed is the address of the memory location or mailbox. On the other hand, this type of parameter does allow information to be passed back to the calling unit because the value in the mailbox is accessible to the calling unit — the address of the mailbox is known. This method of passing information through parameters is called pass by reference and the parameters are called variable parameters or sometimes, TWO-WAY parameters.

Remember that a subroutine passes information back to the calling program through variable arguments and parameters. On the other hand, a function passes

information back to the calling program through the function name.

Here is a simple program that uses a subroutine to halve a numeric parameter. The program contains lots of PRINT statements that show a user what is going on at various points in the program. I suggest that you run this program if you are uncertain about just how variable parameters behave in subroutines.

```
!Filename: WT19_007.TRU
!Author: J.R.Arscott
!Purpose: VARIABLE PARAMETERS program

! A demonstration of how variable parameters work.

INPUT prompt "Assign a value to variable In...": In
PRINT "Before calling subroutine, In ="; In
CALL Halved (In)
PRINT "After returning from subroutine, In ="; In
END

SUB Halved (New)
    ! Divide the value of New in half.

    PRINT "Start of subroutine named Halved"
    PRINT "Entering the subroutine, New ="; New
    LET New = New / 2
    PRINT "After division, New ="; New
END SUB
```

The preceding example is a good illustration of how you can use PRINT statements to see what is happening inside a program. These results are displayed:

```
Assign a value to variable In...10
Before calling subroutine, In = 10
Start of subroutine named Halved
Entering the subroutine, New = 10
After division, New = 5
After returning from subroutine, In = 5
```

The variable (In) had an original value of 10. It was passed to the variable New in the subroutine, divided by two, and thus its value was changed to 5. Back in the main program, the value of (In) also changed to 5. Why? Because (In) and (New) are just two different names for the same memory location and the value stored in that location is now 5.

Those arguments in the CALL statement that are constants or expressions have their values assigned to temporary variables when the CALL statement is executed. These variables can then be referenced by the corresponding parameter variables in the subroutine and the values used in the subroutine. The temporary variables are deleted, however, when control returns to the calling program and the information in them is lost. **Values cannot be passed back to the calling program through temporary variables.** In effect, constant and expression arguments in subroutines behave just like function arguments.

For example, the statement

```
CALL Blank (Num, a+b,25, List$)
```

can result in the subroutine changing the values of Num and List\$ in the calling program (both are variables) but not the value of a+b, because a+b is an expression, and the value of the expression is assigned to a temporary variable that is

deleted when control returns to the calling unit.

If you want a variable used as an argument to behave like an expression, surround it with parentheses. In the statement

```
CALL Blank ((Num), (a+b), 25, (List$))
```

all arguments behave like one-way or value parameters and values are not passed back to the calling program.

Just the simple act of enclosing a variable in parentheses makes it an expression. This technique does provide complete isolation between the subroutine and the calling program.

Look at the next two example programs to get a better idea of how parameters pass information. In the first program, the argument in the CALL statement behaves as a value parameter because of the extra set of parentheses.

```
!Filename: WT19_008.TRU
!Author: J.R.Arscott
!Purpose: VARIABLE PARAMETERS program

! Subroutine with value parameter.

PRINT "Subroutine with a value parameter."
LET Number = 5
PRINT "Before calling subroutine, Number is"; Number

CALL New ((Number)) ! note double parentheses

PRINT "After calling subroutine, Number is"; Number
END

SUB New (X)
    LET X = 10
END SUB
```

The following output is produced:
subroutine with a value parameter. Before calling
subroutine, Number is 5
After calling subroutine, Number is 5

This program displays a final value of 5 because the new parameter value 10 is not passed back to the main program.

On the other hand, the second program demonstrates the use

of a variable parameter:

```
!Filename: WT19_009.TRU
!Author: J.R.Arscott
!Purpose: VARIABLE PARAMETERS program

! Subroutine with variable parameter.

PRINT "Subroutine with a variable parameter."
LET Number = 5
PRINT "Before calling subroutine, Number is"; Number
CALL New (Number)
PRINT "After calling subroutine, Number is"; Number
END

SUB New (X)
    LET X = 10
END SUB
```

In this case, a different output is produced:

Subroutine with a variable parameter.

Before calling subroutine, Number is 5

After calling subroutine, Number is 10

A final value of 10 is displayed by the program because this value is passed back to the main program from the subroutine. The variables Number and X both refer to the same memory location. Study these two examples carefully until you understand thoroughly the different behavior of value and variable parameters.

Substrings used as arguments are considered expressions and thus cannot be changed. For example, the statement

```
CALL Blank (12, Sum, List$[2:5])
```

allows only the value of Sum in the calling program to be changed because 12 is a value and List\$[2:5] is an expression.

All variables in a subroutine are local variables and except for parameter variables, are re-initialized each time the subroutine is called.

Summary of Argument and Parameter Behavior

I think the hardest part of learning procedures is understanding how arguments and parameters work.

Remember these five points:

- Arguments are part of the calling statement, parameters are part of the procedure heading.
- Parameters are always variables.
- Function arguments always provide only one-way communication through arguments and parameters — a value can be passed back through the function name.
- Subroutine arguments that are variables provide two-way communication — the address of the argument variable is passed to the subroutine parameter.
- Subroutine arguments that are expressions or values provide one-way communication — only the value of the expression argument or value argument is passed to the subroutine parameter.

LIBRARIES

External functions and subroutines can be kept in special files called libraries. A library file is not a program that can be run, but rather a collection of functions and subroutines that can be included in and made available to any program.

A library file must start with the statement

`EXTERNAL`

on a line by itself as the first statement. This statement identifies

the file as a library file, not a True BASIC program. The remainder of the file is a sequence of external procedures. An END statement is neither required nor allowed.

Caution: There is a special case of a LIBRARY called a MODULE in which case the library must begin with the keyword MODULE plus an arbitrary identifying name, e.g.,

```
MODULE multifile
```

Unlike EXTERNAL libraries a MODULE must include.

```
END MODULE.
```

The library unit called Wtlib.TRU in the files that can be downloaded from the True BASIC website is a module. Modules and their special properties will be discussed later.

Here is an example of a library file containing some useful functions and subroutines for handling strings:

```
EXTERNAL !to identify this file as a library file
```

```
!Filename: WT19_010.TRU
```

```
!Author: J.R.Arscott
```

```
!Purpose: LIBRARY program
```

```
! Procedure units to manipulate strings.
```

```
DEF Ans$(Question$)
```

```
! Check Y/N answer to a question.
```

```
DO
```

```
    PRINT Question$;
```

```
    LINE INPUT prompt "": Reply$
```

```
    LET Reply$ = Ucase$(Reply$[1:1])
```

```
    SELECT CASE Reply$
```

```
    CASE "Y", "N"
```

```
        LET Ans$ = Reply$
```

```
        LET Finished$ = "true"
```

```

        CASE else
            PRINT "Answer YES or NO."
            PRINT
            LET Finished$ = "false"
        END SELECT
    LOOP until Finished$ = "true"
END DEF

DEF Reverse$ (String$)
    ! Reverse the characters in a string.
    LET Temp$ = "" ! null string
    FOR Index = Len(String$) to 1 step -1
        LET Temp$ = Temp$ & String$(Index:Index)
    NEXT Index
    LET Reverse$ = Temp$
END DEF

DEF More$(Question$)
    ! Check Y/N/Q answer to a question.
    DO
        PRINT Question$;
        LINE INPUT prompt "": Reply$
        LET Reply$ = Ucase$(Reply$(1:1))
        SELECT CASE Reply$
            CASE "Y", "N", "Q"
                LET More$ = Reply$
                LET Finished$ = "true"
            CASE else
                PRINT "Answer YES, NO or QUIT."
                PRINT
                LET Finished$ = "false"
        END SELECT
    LOOP until Finished$ = "true"
END DEF

SUB Center (String$)
    ! Print a centered string.
    ASK margin Width
    LET Space = (Width-len(String$))/2
    PRINT Tab(Space); String$
END SUB

SUB Outline (String$)
    ! Print string in box.

```

```

    ASK margin Width
    LET Space = (Width-len(String$))/2
    PRINT Tab(Space, 2); "+";
    PRINT Repeat$("-", Len(String$) + 2);
    PRINT "+"
    PRINT Tab (Space, 2); "| "; String$; " |"
    PRINT Tab(Space, 2); "+";
    PRINT Repeat$("-", Len(String$) + 2);
    PRINT "+"
END SUB

SUB Vertical (String$, Column)
    ! Print a vertical string in a specified column.
    FOR I = 1 to Len(String$)
        PRINT Tab(Column); String$[I:I]
    NEXT I
END SUB

```

Note that the word DEF is used to identify a function rather than the word FUNCTION. The main advantage is that it produces a slightly shorter statement line.

After a library file is written, it must be saved as a disk file. The library file may be given any legal file name. I recommend that whenever possible, you use a name containing the letters LIB to identify the file as a library file and to distinguish it from a True BASIC program.

Let's assume that you have saved your example library file under the name WT19_010.TRU. If this file is in the current directory, you can use it in a program by including the statement

```
LIBRARY "WT19_010.TRU"
```

as one of the first statements in your program. If the library file is in another directory or on another disk, the name in quotation marks must be its complete path name.

A library function must be declared before it can be used. If you want to access the function Reverse\$, for example, you must

include the statement

```
DECLARE DEF Reverse$
```

or the equivalent statement

```
DECLARE FUNCTION Reverse$
```

in your program.

The library statement must be placed ahead of any reference to a procedure in that library. A program may contain more than one library statement and each library statement may refer to one or more library files with their individual file names in quotation marks and separated by commas.

A library subroutine is called by a CALL statement in the program that wants to use that subroutine. Two procedures in a program cannot have the same name, whether they are located in a library attached to the program or written as part of the program itself.

Here is an example of a short program using the library of string procedures. It is designed to test single names and determine if they are palindromes.

```
!Filename: WT19_011.TRU
!Author: J.R.Arscott
!Purpose: PALINDROME program

! A simplified palindrome program
! for use with names.
! It assumes that the library file.
! is in the current directory.

LIBRARY "WT19_010.TRU"
DECLARE DEF Reverse$
LINE INPUT prompt "Enter a name: ": Name$
LET Palindrome$ = Reverse$ (Name$)
IF Ucase$(Palindrome$) = Ucase$(Name$) then
    PRINT Name$; " is a palindrome."
ELSE
    PRINT Name$; " is not a palindrome."
END IF
```

END

This program recognizes palindromes only if they contain no spaces or punctuation marks. The following output is produced in two program runs:

```
Enter a name: Hannah
Hannah is a palindrome.
```

```
Enter a name: Jasmine
Jasmine is not a palindrome.
```

Several library files are currently distributed with the True BASIC system and other library files will undoubtedly become available in the future. As an example, the file FNMLIB.TRU contains many mathematical and statistical functions.

MODULES

A module is a library of external functions and subroutines with some special properties. Like any other library file, it is included in a program by naming it in a LIBRARY statement. The format of a module is as follows:

```
MODULE Name
    module header
    module initialization
    module functions and subroutines
```

END MODULE

The module Name is an identifier, used primarily by True BASIC to identify error messages that apply to a specific module. The module itself is a separate file, similar to a library file, with its own file name. We recommend that you use a file name containing the letters MOD to remind you that the file contains a True BASIC module. Note that an EXTERNAL statement is not needed in the file, it is identified as a library file by the MODULE statement.

The module header specifies which module variables are shared by all module procedures. The `SHARE` statement identifies variables (including array variables and channel (file) numbers whose values and file associations are known everywhere within the module but not outside the module). These variables are sometimes called static variables. As you move from one procedure to another within the module, these shared variables keep their values and the files remain open. Note that this behavior is quite different from that of local variables and channel numbers in ordinary library procedures.

The module header also specifies which of the module's variables or procedures are public and which ones are private. The `PUBLIC` statement lists those variables that can be accessed from outside the module. These variables are often called global variables. Their values are known everywhere throughout the program. Any program unit that wishes to use one of these variables must include its name in a `DECLARE PUBLIC` statement.

The `PRIVATE` statement lists certain procedures in the module that may not be accessed from outside the module. These are usually procedures that the module's author wishes to keep hidden in the module because they can cause damage if used indiscriminately in the program. Only a few, if any, procedures in a module are normally declared private.

The code for module initialization — if any such code is needed — appears after the module header and before the first module procedure. `OPTION` statements, if any, in the initialization section apply to all procedures of the module. The module procedures, both functions and subroutines, are just like those in an ordinary library file. Any procedure that is not private can be used like any other external procedure. Functions must first be declared by a `DECLARE` statement in your program. If you

want to use a public variable in your program, it must first be identified in a DECLARE PUBLIC statement.

The module WTlib.TRU is a typical module in that it is a collection of sub-routines that help you read and write to any sort of file, and to manipulate strings and sub-strings.

Here is a list of all the procedures available in this module:

DEF valstr (converts strings to numbers)

Read_anyfile

Write_anyfile

Get_size (finds out the record size of a file)

Search_unsorted_file

Search_sorted_file

Sort_file (not suitable for byte files)

Sort_array

Sort_fields

Search_unsorted_array

Search_unsorted_fields

Search_sorted_array

Search_sorted_fields

Is_file_there

Copy_file

Unpack_data

Unpack_array

Pack_data

Pack_array

SetFileRecord

Get_all_files

Vtext

Bytepair

Makepair

ValueToByte

User

Note: the three routines bytepair,makepair and valuetobyte

could have been `DECLARED PRIVATE` because they are only used within the module and have no meaning outside the module.

INTERNAL FUNCTIONS and SUBROUTINES

Procedures may be written within the main program unit — before the `END` statement — and then they are called internal functions or internal subroutines. They share one of the advantages of external procedures, the ability to create a modular program that is easier to understand.

The most serious disadvantage of internal procedures is that they do not maintain the desired isolation between procedure variables and main program variables. Values of variables declared in an internal procedure are known throughout the main program unit, including all its internal procedures. Thus the change of a variable value in one procedure will affect all variables with the same name in any other internal procedure and in the main program. Suppose you have a habit of using `(n)` as a counter in `FOR NEXT` loops or `(temp$)` as a temporary string variable then it is quite possible for internal procedures to change these variables without you being aware of it. To avoid unwanted interaction between variables, I recommend that you use only external functions and subroutines in your programs.

CHAIN

The `CHAIN` statement is not technically a program unit in the same way as subroutines or library modules, but I have included it here because it allows you to access other executable programs, and to use their features while you are still running your own program. For example, your own program may have produced a file of data that you may want to inspect using MS Excel. You can `CHAIN` to this application to view your

data and then RETURN to your own program. There are two ways to do this:

```
CHAIN "!&rundll32.exe url.dll,FileProtocolHandler " &
filename$
```

Where filename\$ is the actual you want use such "myfile.doc" or "mydata.csv", not the program ,e.g., MS WORD or MS Excel.

Or

```
CHAIN "!& c:\myfolder\anyprog.exe",RETURN
```

The syntax for the CHAIN statement is

```
CHAIN strex arglist$, RETURN
```

If the target program named in (strex) is not accessible, an exception does NOT occur. Instead a black DOS type SYSTEM COMMAND window opens with a white default OUTPUT window.

To avoid this happening you are advised to first check that *strex* exists BEFORE attempting to CHAIN to it. IF *arglist\$* is present then there MUST be a matching PROGRAM statement in the target program with the same number of variables as *arglist\$* or the program must be able to accept external parameters.

If the PROGRAM statement is missing or the number of variables does not match *arglist\$* then an exception does NOT occur. Instead the *arglist\$* is ignored if the PROGRAM statement is missing, or if the number of variables is incorrect then only those that are correct will be filled. For example if there are 3 parameters in *arglist\$* and only 2 in the PROGRAM parameters then the first two from *arglist\$* will be transferred to the PROGRAM parameters.

CAUTION:

You cannot use spaces in a file path name.

If the individual parameter is a filename then it is likely the filename may contain spaces. CHAIN considers such spaces as extra parameters, e.g.,

```
LET arglist$="C:\Program files\myfolder\myfile"
```

This will count as 2 parameters "C:\Program" and "files\myfolder\myfile".

The parameters in *arglist\$* consist of variables separated by SPACES, or one string variable that includes spaces in the text, e.g.,

```
LET arglist$ = "Mary had a little lamb" (5 parameters)
LET arglist$ = arg1$ & " " & arg2$ & " " & arg3$
(3 parameters)
```

The parameter passing mechanism in CHAIN is by value, the same as defined for functions.

We can use the fact that CHAIN must have a valid filename in order to work correctly otherwise it drops down to the DOS command level, to our advantage. If we give CHAIN a valid DOS command instead of a valid filename, then it will execute our command. For example, suppose we use the VOL command, which essentially asks the computer to reveal the hard drive serial number. Suppose we also divert the normal screen print to a simple text file in the root directory (C:\):

```
VOL>HD.ser
```

The outcome is that a file will be created in the root directory called HD.ser which contains a single line message, something like "Volume serial number is 1234-FA5D". The name of the file is entirely your choice, so you could use "RESTORE.DOC" or something equally innocuous. If you now embed this serial

number inside your program, you now have a means of preventing your programs being copied or pirated because your program will only run on one computer. I will deal later with how you can embed stuff into your program after you have bound it into an executable file.

```
!Filename: WT19_012.TRU
!Author: J.R.Arscott
!Purpose: HD SERIAL NUMBER program

CHAIN("!vol>hd.ser"),RETURN
OPEN #1:name "hd.ser",access outin,create newold,org
byte
ASK #1: FILESIZE total
SET #1: RECSIZE total
READ #1: txt$
CLOSE #1
LET location$= "Volume Serial Number is "
LET location=pos(txt$,location$)
LET volume$=txt$[location:maxnum]
PRINT volume$
END
```

Summary of Important Points

- Each program unit in True BASIC (main program, external function, external subroutine, or external picture) may have its own local variables.
- All external functions used in a program unit must be declared in one or more DECLARE statements.
- A value must be assigned to the function name before control returns to the calling program unit.
- Function parameters are one-way or value parameters.
- No value is associated with a subroutine name.
- Subroutine parameters are two-way or variable parameters.
- A subroutine passes information back to the calling program through its parameters.

- A function passes information back to the calling program through its function name.
- Values cannot be passed back through temporary variables from an external subroutine to the calling program.
- A library file (not a module) must start with the statement `EXTERNAL` on a line by itself.
- Variables declared in internal functions and subroutines are known throughout the main program.

Common Errors

- Writing a long main program without any procedures.
- Leaving a function procedure before assigning a value to the function name.
- Using an `EXIT` statement to exit a procedure that could just as easily have been exited through the `END` statement.
- Passing an address to a subroutine parameter rather than passing a value when maximum isolation is desired between the calling unit and the subroutine.
- Passing a value to a subroutine parameter rather than passing an address when information must be passed back through the parameter to the calling unit.
- Forgetting to place an additional set of parentheses around a simple variable argument in a subroutine when you wish to pass information by value.
- Trying to calculate the factorial value of a negative number.
- Testing a string to determine if it is a palindrome without converting all letter characters to the same case.
- Forgetting to place an `EXTERNAL` statement or a `MODULE` statement at the beginning of a library file.

Chapter 20

Error Handling

ERROR TRAPPING AND HANDLING

Program errors can be divided into several general classes. Compile errors (also called syntax errors) are errors in statement syntax. These errors are generated when your program is being compiled and are identified immediately. They are typically caused by mistakes in syntax. You can use the editor to correct your program and then compile the program again.

Logical errors are the result of mistakes in logic when the program was designed or written. The program may run perfectly but produce meaningless results. These errors are sometimes hard to find, and can be located and corrected only by thoroughly testing the program.

Run-time errors occur while the program is executing. These errors may be caused by logical errors or by other mistakes made when writing the program. They are usually fatal and cause the program to stop. Computer programmers say the program has “crashed.” True BASIC provides a facility for trapping these run-time errors, allowing the user to make corrections and continue executing the program.

The syntax for trapping errors is as follows:

```
WHEN EXCEPTION IN
protected block
USE
exception handler block
END WHEN
```

THE PROTECTED BLOCK of STATEMENTS

EXCEPTION is the standard ANS BASIC word for an error, but True BASIC allows the word ERROR to be used in place of the

standard word. The protected block can be any sequence of statements in the main program unit or in a subprogram unit. In most applications, the protected block is a short sequence of statements.

THE ERROR HANDLER BLOCK

The error handler block is another sequence of statements that is executed only if there is a run-time error in the protected block. Under normal circumstances when there is no error, the program skips over the error handler block and executes the first statement after the END WHEN statement.

EXTYPE and EXTEXT\$ FUNCTIONS

Two system functions are particularly useful in the error handler block. The numeric function Extype returns an error number when a run-time error occurs. These numbers are listed in the Reference Manual. The value of Extype can be used to make a decision in the error handler block.

The string function Extext\$ returns the specific error message and is also listed in the Reference Manual. When error trapping is in operation, this message is not printed automatically. The value of Extext\$, the error message string, can be displayed or used in any other manner.

Here is a sample program using error trapping:

```
!Filename: WT20_001.TRU
!Author: J.R.Arscott
!Purpose: ERROR HANDLING program

! Trap any errors while calculating
! the square root of a number.

LET NoError$ = "false"
```



```

DO
LINE INPUT prompt "Enter number: ": Value$
WHEN error in ! protected block
LET Number = Val(Value$)
LET Result = Sqr(Number)
PRINT "Square root of "; Value$; " is"; Result
LET NoError$ = "true"
USE ! error handler block
IF Extype = 3005 or Extype = 4001 then
PRINT "Error: "; Extent$
ELSE
PRINT "Unexpected error: "; Extent$
END IF
PRINT "Please try again."
END WHEN
LOOP until NoError$ = "true"
END

```

There are plenty more examples inside the WTlib.TRU library module.

Note the use of the LINE INPUT statement to accept any sequence of characters, with checking for a proper number taking place when the Val function is invoked. Error 3005 means trying to take the square root of a negative number. Error 4001 means using the Val function on a string that is not a proper number. In either case, the appropriate error message is printed. If another, unexpected run-time error occurs, a general error message is printed.

Here is an example of program output:

```

Enter number: -9
Error: SQR of negative number
Please try again. Enter number: 9
Square root of 9 is 3

```

Error trapping is an example of good defensive programming. You should anticipate the possibility of run-time errors and develop plans to handle them when they occur. Nothing is more discouraging to the user of an application program than to have

the program suddenly stop and display an error message that seems to make no sense.

Exline

returns the line number in a program where the most recent error occurred

Exline\$

returns a string giving the location in a program where the most recent error occurred

Extext\$

returns the error message associated with the most recent error trapped by an error handler

Extype

returns the error number of the most recent error trapped by an error handler

Chapter 21

Graphics

There is one particular issue that we need to address, before we begin a general discussion about graphics, and that is the problem of copying graphics to a file.

BMP FORMAT

This method makes an exact copy of every pixel color on your screen, in rows and columns just like a very large spread sheet. It uses up a lot of file space because each pixel takes 24 bits of data. However, in its favor it does maintain its accuracy regardless of how many times you edit and save the file. The scale of BMP images cannot be changed easily.

JPG FORMAT

In this format your image is subjected to compression to save space. These files can easily be 20 times smaller than the equivalent BMP file, but they are only an approximation of the original. In the case of photographs you cannot tell the difference, whereas line drawings are clearly less accurate. The more you edit and save a file the more it deteriorates.

VECTOR FILES

In these files there is no copy of your drawing or image, instead there are a series of drawing instructions that will reproduce your drawing exactly. Moreover, you can change the scale and still the accuracy will be preserved. Many fonts are drawn this way, as well as CAD/CAM drawings. Indeed the True BASIC library module Pwlib also allows you to draw this way on a printer. In general vector files are only used by specific applications.

The choice of file format largely depends on what sort of graphics you are doing. If you are messing with photographs

then JPG is fine but for everything else I would recommend the BMP format.

THE GRAPHICS SCREEN

The next concept you need to grasp is that the graphics created by the TrueCTRL library module sit on top of the normal True BASIC graphics. Imagine that your screen has an extra layer of glass onto which the TrueCTRL graphics are projected, whereas normal True BASIC graphics are displayed on the normal screen beneath. Users of the CTX library do not need to bother with this difference because CTX graphics appear on the same screen as True BASIC graphics.

Before you start using graphics, you need to find out how big the current window is, and then you need to decide what system of units you intend to use. You have a choice: pixel units which run left to right and top to bottom with the origin at TOP LEFT, or user units which run from 0 to 1 left to right and bottom to top with the origin at BOTTOM LEFT. Personally, I prefer to use pixel units because you are always working in integers and it is easier to position images and fonts which are measured in pixels. Whichever system you adopt, then stick with it. If you want to make life difficult for yourself then mix these units and see how far you get.

WINDOW SIZE

To determine the pixel size of the current window then use:

```
ASK PIXELS xpix,ypix
```

Remember that user units are always 0 to 1 in both the x-axis and the y-axis.

```
ASK PIXELS xpix,ypix
```

```
SET WINDOW 0,xpix-1,ypix-1,0
LET midx=int((xpix)/2)      ! mid x axis
LET midy=int((ypix)/2)      ! mid y axis
```

In earlier versions of True BASIC it was necessary to use SET MODE to either graphics or text, but that is no longer necessary. It is possible to use graphics and text modes simultaneously, but each of these modes uses a different system to locate the text. My advice is do not mix these two modes because you will end up confused. If you wish to have text displayed within your graphics then always use the PLOT statement, e.g.,

```
PLOT TEXT, AT x,y: text$
```

This statement will display text\$ starting at point x pixels from the left hand edge of the window on the x-axis with the bottom of the text line sitting on an invisible line y pixels down from the top of the window. This sort of precise location is not possible in text mode because SET CURSOR row,col only works within the strict limits of the TEXT screen where rows and columns are in fixed positions.

SETTING THE CURSOR POSITION

If you insist on using text location rather than pixels, then you first need to find out how big the text area is, i.e how many lines are there and how many printing columns (characters across the screen). This can done with:

```
ASK MAX CURSOR height,width
```

Similarly you set the printing position for text with:

```
SET CURSOR row,column
```

True BASIC provides extensive and advanced graphics

statements that allow you to draw points, lines, rectangles, polygons, circles and ellipses. You can save and retrieve any graphics that you have created. You can also import photographic images and combine them with your own graphics. The 24 bit internal color definition of images gives you access to millions of colors. In addition True BASIC provides you with a similar structure to the external sub-routine, called a PICTURE procedure, which is called with the keyword DRAW. Graphics figures can be transformed using either user-designed or built-in transformation routines, such as SCALE, SHIFT and SHEAR.

The files that can be downloaded from the True BASIC website also contain a very useful library module called BOXlib.TRU that contains a routine for drawing rubber boxes and dragging it around the screen. You can use the rubber box to accurately enclose an area of the screen, and then using the co-ordinates of the corners, you can BOX KEEP that area as an image.

FUNDAMENTALS OF DRAWING

At first sight it would appear that the drawing capabilities are somewhat limited; points, lines, areas and if you include BOX statements; circles and ellipses. Not much you might think, but with the aid of a little math, you can also plot curves and arcs and practically anything you want.

POINTS

The True BASIC statement to draw or plot a point at the position X,Y on the graphics screen is:

`PLOT POINTS: X,Y`

In many applications, you want to plot several points and then the statement becomes

```
PLOT POINTS: X1,Y1; X2,Y2;...
```

The position identifiers X1, X2, Y1, and Y2 can be numeric variables, constants, or expressions. Each position pair, X and Y, represents the coordinates of a point to be plotted. The pairs are separated by semicolons. There is no trailing semicolon when you plot a single point.

You can, of course, use the PLOT POINTS statement in a loop to plot a sequence of points. Here is a program that demonstrates that procedure:

```
!Filename: WT21_001.TRU
!Author: J.R.Arscott
!Purpose: PLOTTING POINTS program

! Plot a sequence of points.

ASK PIXELS xpix,ypix
SET WINDOW 0, xpix-1, ypix-1,0
LET midx=xpix/2
LET midy=ypix/2
FOR X = 1 to 20
LET Y = X*X
PLOT POINTS: 5*X, Y
NEXT X
PRINT "Press any key to quit."
GET KEY dummy
END
```

This program plots the equation $Y = X^2$, a parabolic curve. After you run the program, press any key to quit.

LINES

Here is the statement for drawing a line between two points:

```
PLOT LINES: X1,Y1; X2,Y2
```

You can connect several points together with lines by using a

modified PLOT LINES statement, as shown here:

```
PLOT LINES: X1,Y1; X2,Y2; X3,Y3...
```

If the coordinates of the first point are the same as those of the last point, you will produce a closed figure — that is, the line will close on itself.

Here is the previous example, rewritten to display a line plot rather than a point plot:

```
!Filename: WT21_002.TRU
!Author: J.R.Arscott
!Purpose: DRAWING LINES program

! Plot a line between points.

ASK PIXELS xpix,ypix
SET WINDOW 0, xpix-1, ypix-1,0
LET midx=xpix/2
LET midy=ypix/2
FOR X = 1 to 20
LET Y = X*X
PLOT LINES: 5*X, Y;
NEXT X
PRINT "Press any key to quit."
GET KEY dummy

END
```

Note the trailing semicolon in the PLOT LINES statement which causes a line to be drawn from one point to the next point. In this case.

AREAS

Having learnt to draw lines, you can now connect lines to produce a shape, still using the PLOT LINES statement. This technique produces an outline figure. The PLOT AREA statement can be used to fill in the area enclosed by the outline.

Here is an example program that uses both statements. This program first draws an outline of a square and then, after a short pause, fills in the square.

```
!Filename: WT21_003.TRU
!Author: J.R.Arscott
!Purpose: DRAWING ENCLOSED AREAS program

ASK PIXELS xpix,ypix
SET WINDOW 0, xpix-1, ypix-1,0
LET midx=xpix/2
LET midy=ypix/2

! Display an outline square.
SET COLOR -1 ! black
PLOT LINES: 100,100; 300,100;300,300;100,300;100,100
PAUSE 2

SET COLOR 12 ! red
! Display a filled square in red.
PLOT AREA: 100,100; 300,100;300,300;100,300;100,100
SET COLOR -1 ! black
PRINT "Press any key to quit."
GET KEY dummy

END
```

COLOR

In the previous example the statement:

```
SET COLOR 12
```

This statement was used to set the “pen” color to red.

I could have used:

```
SET COLOR Name$
```

where Name\$ contains the name of the desired color. In

general, True BASIC supports at least the following colors:
RED CYAN MAGENTA YELLOW WHITE GREEN BLACK and
BLUE.

Personally, I find this list is too limited so I prefer to use color
numbers rather than names.

A similar statement is used to define the background or
“paper” color:

```
SET COLOR BACKGROUND Name$
```

This last statement can be abbreviated to:

```
SET BACK Name$
```

Note that the color BACKGROUND means the figure will be
displayed in the same color as the background, thus effectively
erasing an existing figure. Here is an example of this technique:

```
!Filename: WT21_004.TRU
!Author: J.R.Arscott
!Purpose: ERASING DRAWINGS program

ASK PIXELS xpix,ypix
SET WINDOW 0, xpix-1, ypix-1,0
LET midx=xpix/2
LET midy=ypix/2

! Display an outline square.
SET COLOR -1 ! black
PLOT LINES: 100,100;
300,100;300,300;100,300;100,100
PAUSE 2

SET COLOR -2 !white BACKGROUND ! erases drawing
```

```

PLOT LINES: 100,100;
300,100;300,300;100,300;100,100
SET COLOR -1 ! black
PRINT "Press any key to quit."
GET KEY dummy
END

```

One of the most important statements related to color is:

```
SET COLOR MIX(N) r,g,b
```

In effect this statement allows you to create any color you wish and to assign any reference number to that color. If you wanted to, you could create an entire palette of custom colors.

The r,g,b values are decimal (or fractional) values for the red, green and blue components. Here are the first 16 VGA colors with their respective r,g,b values:

Black	0,0,0
Blue	0,0,.5
Green	0,.5,0
Cyan	0,.5,.5
Red	.5,0,0
Magenta	.5,0,.5
Dark yellow	.5,.5,0
Dark gray	.5,.5,5
Light gray	.75,.75,.75
Bright blue	0,0,1
Bright green	0,1,0
Bright cyan	0,1,1
Bright red	1,0,0
Bright magenta	1,0,1
Bright yellow	1,1,0
White	1,1,1.

Here is a small program that generates a gradient of colors from top to bottom of your screen. The “start” color begins at the top of the screen and fades into the “finish” color at the bottom of the screen. The program uses color number 99 as a temporary color for the gradient and resets this color to its original status at the end. This program works by drawing lines across the screen and then progressively reducing the r, g, b color values towards zero (black) as the line moves down the screen. You could modify this program to increase the r,g,b values towards one (white).

```
!Filename: WT21_005.TRU
!Author: J.R.Arscott
!Purpose: COLOR GRADIENT program
```

```
ASK PIXELS xpix,ypix
SET WINDOW 0, xpix-1, ypix-1,0
LET midx=xpix/2
LET midy=ypix/2
```

```
ASK COLOR MIX(99) ro,go,bo
REM: set start color as bright blue 0,0,1
LET r=0
LET g=0
LET b=1
SET COLOR MIX(99)r,g,b
```

```
For y=0 to ypix-1
```

```
SET COLOR 99
PLOT LINES: 0,y; xpix-1,y
IF r>0 then
let rfade=r*(1-(y/ypix))
ELSE
LET rfade=r
END IF
IF g>0 then
LET gfade=g*(1-(y/ypix))
ELSE
LET gfade=g
```

```

END IF
IF b>0 then
LET bfade=b*(1-(y/ypix))
ELSE
LET bfade=b
END IF

SET COLOR MIX(99) rfade,gfade,bfade
NEXT y
SET COLOR MIX (99) ro,go,bo ! reset color
PRINT "Press any key to quit."
GET KEY dummy
END

```

TEXT

It is often desirable to display text in a graphics window. Displayed text is needed for such purposes as labeling figures or displaying instructions for a user.

As I mentioned earlier, the familiar PRINT statement can be used but it may not place the text just where you want it. A more useful statement is

```
PLOT TEXT, AT X,Y: Label$
```

where Label\$ is a string variable containing the desired text.

Drawing with BOX Statements

PLOT statements can be used to draw any figure consisting of straight lines. A set of BOX statements, allows simple figures to be drawn more easily and quickly. The additional drawing speed makes BOX statements a good choice for animation programs where drawings must change rapidly.

All the BOX statements use a common rectangular co-ordinate system that extends horizontally from Xmin to Xmax and

vertically from Ymin to Ymax. In pixel co-ordinates this means left, right, bottom and top, e.g.,

```
BOX LINES (left,right,bottom,top)
```

BOX LINES

Draws a rectangular box similar to that produced by the PLOT LINES statement.

BOX AREA

Draws a solid rectangle in a selected color.

BOX CLEAR

Erases the area defined by the co-ordinates.

BOX CIRCLE

Draws a circle or ellipse that exactly fits the boundaries defined by the co-ordinates.

BOX ELLIPSE

Is exactly the same as BOX CIRCLE.

BOX SHOW string\$ AT left,bottom

This statement will display an image string with its bottom left corner located at the co-ordinates given. The string must be in the True BASIC internal image format. BMP and JPG file formats can be converted into this internal format with:

```
Read_image(format$,boxstring$,filename$)
```

Where format\$ can be “MS BMP” or “JPEG”

BOX KEEP left,right,bottom top IN string\$

This statement will copy the graphical contents of the rectangular area as an image string in the internal image format. This image can be saved to a file in BMP format only with:

```
Write_image(format$,boxstring$,filename$)
```

Where format\$ must be "MS BMP"

CAUTION:

BOX KEEP and BOX SHOW only work in True BASIC logical window graphics layer. In the case of graphics produced by the TrueCTRL library in the top graphics layer, the following conversions may be used:

From an image in a file to:

Boxstring	use Read_image
TB graphics layer	use Read_image then BOX SHOW
TC graphics layer	use TC_graph_SetImageFromFile

From a BOX KEEP string to:

Image file	use Write_image
TB graphics layer	use BOX SHOW
TC_graphics layer	use TC_graph_SetImageFromBox

From the TB graphics layer to:

Image file	use BOX KEEP, Write_image
Boxstring	use BOX KEEP
TC_graphics layer	BOX KEEP,TC_graph_SetImageFromBox

From the TC_graphics layer to:

Image file	use TC_GetImageToBox, Write_image
Boxstring	use TC_GetImageToBox
TC_graphics layer	use TC_GetImageToBox, BOX SHOW

This complication does not apply to the CTX library module, where all objects and images are drawn on the same layer as True BASIC graphics.

FLOOD

The FLOOD statement lets a user “fill in” an outline figure with solid color. The coordinates in the FLOOD statement can be any point within the outline figure. Here is an example program similar to the preceding one, but the ellipse is now drawn within a rectangle, not a square, and is filled in with the default color:

```
!Filename: WT21_006.TRU
!Author: J.R.Arscott
!Purpose: FLOOD program

! Use the BOX statement to draw an ellipse
! and flood the figure with the default color.

ASK PIXELS xpix,ypix
SET WINDOW 0, xpix-1, ypix-1,0
LET midx=xpix/2
LET halfx=midx/2
LET midy=ypix/2
LET halfy=midy/2

! Draw a rectangle.
BOX LINES midx-halfx,midx+halfx,midy+halfy,midy-halfy
PAUSE 2

! Erase the rectangle.
BOX CLEAR midx-halfx,midx+halfx,midy+halfy,midy-halfy
PAUSE 2

! Draw an ellipse.
BOX ELLIPSE midx-halfx,midx+halfx,midy+halfy,midy-halfy
PAUSE 2

! Flood the ellipse with red.
SET COLOR 12 ! red
FLOOD midx,midy
END
```

Note that the coordinates of the FLOOD statement must refer to a point inside the ellipse. Try running this program with

different FLOOD statement coordinates and see what happens.

VERTICAL TEXT

The following example shows a series of horizontal histograms with days of the week as label\$. Usually histograms are drawn vertically so I have written a sub-routine in the WTLIB library module that allows you to set the font for PLOT TEXT, and another that allows you to print vertically, and you can set the pen and paper colors too.

```
!Filename: WT21_007.TRU
!Author: J.R.Arscott
!Purpose: VERTICAL TEXT program

! Display a bar graph with labels.

LIBRARY "c:\TBbook\WTLlib.tru" ! insert your own
pathname

ASK PIXELS xpix,ypix
SET WINDOW 0, xpix-1, ypix-1,0
LET midx=xpix/2
LET midy=ypix/2
DIM string$(7)
DIM Y$(7)
DATA MONDAY,TUESDAY,WEDNESDAY,THURSDAY
DATA FRIDAY,SATURDAY,SUNDAY
MAT READ Y$

REM: horizontal plot
LET count=0
FOR Y = 60 to 340 step 40
LET count=count+1
IF count>7 then
EXIT FOR
END IF

SET COLOR 9
PLOT AREA: X+50,Y; X+50+Y,Y; X+50+Y,Y+20; X+50,Y+20
LET string$(count) = Y$(count)
```

```

SET COLOR 14
PLOT TEXT, at 50,Y+16: string$(count)
BOX KEEP X+50,X+50+Y,Y+20,Y IN string$(count)
NEXT Y

REM: vertical plot

LET n=0
FOR X = midx to midx+280 step 40
LET n=n+1
IF n>7 then
EXIT FOR
END IF
SET COLOR 9
PLOT AREA: X,ypix-50; X+20,ypix-50; X+20,ypix-(X/2);
X,ypix-(X/2)
LET Label$ = Y$(n)

CALL vtext(string$(n),rotate$,-1,14,9)

! rotates is string$(n) rotated anticlockwise by 90
degrees

BOX SHOW rotate$ AT x,ypix-70
NEXT X

PRINT "Press any key to quit."
GET KEY dummy

END

```

Note that the sub-routine called “vtext” requires the original horizontal boxstring and the direction of rotation i.e., direction>0 is clockwise and direction <0 is anticlockwise. The last two numerical values are for the pen and paper colors (yellow on blue). The sub-routine returns the boxstring rotate\$ which must be shown with the BOX SHOW statement using the bottom left co-ordinates.

RUBBER BOXES

Here is a little program to demonstrate how to generate a rubber or elastic box which you can drag across the screen. It illustrates the principle of animation by copying the background, directly drawing a graphic shape, re-showing the background and then re-drawing the shape in a new position. The actual drawing routines are included in the BOXlib.tru library module.

```
!Filename: WT21_008.TRU
!Author: J.R.Arscott
!Purpose: RUBBER BOXES program

REM: rubber box routines

REM: USER NOTES
REM: Click mouse button anywhere in the window and drag
REM: dotted blue box will expand/contract from start
point
REM: release mouse button when box is desired size
REM: place mouse inside box and drag box around screen
REM: when box is in correct position then release mouse
button
REM: use the arrow keys to accurately locate the box.

REM: use your own path names
LIBRARY "C:\TBbook\BOXlib.tru"
LIBRARY "C:\website\TRUEctrl.trc"

CALL TC_init
CALL TC_setunitstopixels
CALL TC_show(0) ! default window
CALL TC_win_switch(0)

ASK PIXELS xpix,ypix
SET WINDOW 0,xpix-1,ypix-1,0 ! a logical window
for graphics

CALL BX_init_box
REM: set attributes to a dotted rubber box in blue (9)
```

```

CALL BX_color_box(9,"DOT")
REM: drag the box into desired size and shape
CALL BX_Make_box
REM: now move the box by dragging with the mouse
REM: finally nudge the box with arrow keys
CALL BX_Move_box
PRINT "Press RETURN to print locations"
DO
  get key keynum
  CALL BX_Nudge_box(keynum)
  IF keynum=13 then
  EXIT DO
END IF
LOOP

REM: get the coordinates with
CALL BX_getrect(x1,x2,y1,y2)
PRINT "Box location in pixel units:"
PRINT "left=";x1
PRINT "right=";x2
PRINT "bottom=";y2
PRINT "top=";y1
PRINT " Press any key to exit"
GET KEY kkk
CALL BX_reset_box
CALL BX_erase_box
CALL TC_Cleanup

END

```

PICTURE PROCEDURES

True BASIC offers users a special graphics program unit called a PICTURE. It is the graphics equivalent of a True BASIC subroutine. In all respects it works in exactly the same way as normal subroutines with respect to parameters. The general syntax is:

```
PICTURE identifier name (parameter list)
```

The parameters are local variable names, separated from one another by commas. Information is passed to these parameters by reference, just as with subroutines. In fact, the

use of temporary variables, passing of file or channel numbers and so forth, all behave with pictures as they do with subroutines.

The last statement in a picture unit must be END PICTURE and other exit points are identified by the EXIT PICTURE phrase.

Here is an example program that uses a picture unit to draw a circle at a specified location. The user is asked to enter the radius of a circle to be drawn on the screen. If the radius is zero, the program stops. The user is then asked to enter the center coordinates of the circle and an external picture is called to draw the figure.

```
!Filename: WT21_009.TRU
!Author: J.R.Arscott
!Purpose: DRAWING PICTURES program

! Use picture unit to draw a circle.
DO
CLEAR
PRINT "Enter zero to stop the program."
INPUT prompt "Radius of circle? ": Radius
IF Radius > 0 then
INPUT prompt "Center coordinates (X,Y)? ": X, Y
ASK PIXELS xpix,ypix
SET WINDOW 0,xpix-1,ypix-1,0
DRAW Circle (Radius, X, Y)
PAUSE 2
ELSE IF Radius < 0 then
PRINT "You cannot have a negative radius."
PAUSE 2
END IF
LOOP until Radius = 0
END
PICTURE Circle (R, X, Y)
! Draw a circle of radius R at point X, Y
BOX CIRCLE X-R, X+R, Y-R, Y+R
END PICTURE
```

The user is asked to specify the radius and center coordinates of the circle. An external picture named Circle draws the figure, with the radius and center coordinates passed as parameters.

A picture routine is called with the keyword DRAW. The general syntax is:

```
DRAW identifier name WITH transform
```

Permitted transforms are:

```
SCALE (nx, ny)
```

Where nx is the scale factor for the x-axis and ny is the scale factor for the y-axis.

```
ROTATE (angle)
```

Where the picture is rotated in an anticlockwise direction by (angle) radians or degrees if these units have been specified.

```
SHIFT (deltax, deltay)
```

Where the picture is shifted along the x-axis by deltax and along the y-axis by deltay.

```
SHEAR (angle)
```

Where vertical lines in the picture are tilted clockwise through (angle) radians or degrees, leaving horizontal lines intact.

It also possible to compound these transformations, e.g.

```
DRAW square WITH SHIFT(2,0) * ROTATE(45)
```

The transformations are applied left to right, i.e., shift first and then rotate afterwards.

USING TRANSFORMS with PICTURES

If you are going to transform a picture unit, it must contain PLOT statements, not BOX statements. Figures drawn with BOX statements cannot be transformed. Here is an example program using a picture unit with transformations. It first draws a simple square and then redraws the same figure with several different transformations applied:

```
!Filename: WT21_010.TRU
!Author: J.R.Arscott
!Purpose: PICTURE TRANSFORMS program
! Using transformations with a picture.
  OPTION ANGLE degrees
LET Ratio = 1.4
SET WINDOW -(50*Ratio), (50*Ratio), -50, 50

DRAW Square(10, -35, 25)
PLOT TEXT, at -60, 5: "Original square"

DRAW Square(10, 35, 25) with shear(10)
PLOT TEXT, at 5, 5: "Sheared 10 degrees clockwise"

DRAW Square(10, 0, 0) with rotate(45)*shift(35, -25)
PLOT TEXT, at 5, -45: "Rotated 45 degrees"

DRAW Square(10, -25, -25) with scale(1.5, 1)
PLOT TEXT, at -60, -45: "Widened 50 percent"
END

PICTURE Square (Size, X, Y)
! Draw a square centered at the point X, Y.
LET P = Size/2
PLOT LINES: -P+X, P+Y; P+X, P+Y;
PLOT LINES: P+X, -P+Y; -P+X, -P+Y; -P+X, P+Y
END PICTURE
```

The window coordinate system is centered on the screen. To rotate a square, draw it so its center coincides with the center of coordinates. When you rotate the square, it rotates about its own center and after rotation, you can move it to the desired

position.

PICTURE LIBRARIES

Library files of pictures can be created in the same manner as library files of functions or subroutines. A file named GRAPHLIB.TRU is distributed with True BASIC and contains picture units for drawing polygons, arcs of circles, window axes, line graphs, and bar graphs.

To use any of the pictures in this library, add the statement at the top of your program:

```
LIBRARY "graphlib"
```

COMPUTER AIDED DESIGN

The process of using computer graphics to assist architects and engineers who create drawings is called computer-aided design (CAD). Most CAD systems produce drawings that are very sophisticated and detailed. Some commercial applications not only produce accurate technical drawings, but can also render 3D views, and some will even allow you to vary the direction and intensity of the light to produce shadows.

CAD systems for architects usually have libraries or collections of drawings for such items as doors and windows. An architect may be able to choose from several dozen door and window designs and “drag and drop” these items where needed on the drawing.

Nowadays it is very common to find CAD systems in furniture stores, and stores selling kitchen appliances or bathrooms. Within minutes you can plan an entire kitchen to make sure cookers, dishwashers, washing machines and cupboards all fit in properly. Similarly in furniture stores you use a similar system

to select wall and carpet colors and other décor to give you a better visualization of how your room will look.

In other industries, CAD is used to produce manufacturing drawings. In some cases the drawings are not printed out; instead, the vector data files are sent directly to a computer-controlled machining process, such as milling, turning, or punching. This is called CAD/CAM. In some CAD applications it is possible to partially animate the drawing to show the leg room when reclining seats are operated, or to show the clearance when the car trunk is opened, or how wide the car doors open.

Despite all the frills, CAD is just a convenient and quick way to produce lines, circles and curves. Take the case of a rectangular table top. If you now want to round off the corners, you only have to tell the computer that you want a half inch radius and it will find all four corners. In the old days it would take four operations with a compass to achieve the same result. Similarly, if you want to punch out a series of complex shapes from a flat sheet of steel then it could easily take hours to work out the most economical way to arrange these shapes to achieve the least wastage. With CAD you just let the computer work out how the shapes should be arranged in seconds.

Years ago, it was common practice in large drawing offices to keep master copies of drawings, usually on semi-translucent paper 3 feet by 4 feet (1m x 1.5m). These drawings were done in pencil or more usually in ink because it reproduced better.

Paper copies were produced on light-sensitive paper by contact printing and were developed using a blue toner – hence the name “blueprints.” The storage space required by this procedure was astronomical. Some companies even micro-filmed the master copies to save space. All this changed with the arrival of CAD. At first master copies were still produced

using a king size printer or plotter with a turret of steel nibbed pens on the print head instead of an ink jet. Pen plotters have essentially become obsolete, and have been replaced by large-format ink jet printers and LED toner-based printers. Many of these newer devices still understand the vector languages originally designed for plotter use.

There are plenty of freeware examples of CAD applications available on the Internet, so it is unlikely that you will attempt to write such a complex program yourself, unless you have a very specific requirement that is not provided already.

If you do attempt the task, then you will need to think very carefully about how you are going to reproduce your drawing. Fortunately, True BASIC has a library module called Pwlib that can help in this respect.

PWlib

Just as there are vector file systems for saving graphical data in such a way that they can be reproduced accurately at any scale, True BASIC offers you an alternative printing method that essentially configures your printer as if it is a window, using a library module called PWlib.

Anything that you can draw or print in a screen window can also be drawn in this simulated printer-window.

When I use ASK PIXELS on my A4 ink jet printer-window, the printer returns the value 2892 for the x-axis and 3969 for the y-axis. This is almost three times better pixel definition than my screen definition, i.e., approximately 350 dots per inch. At this definition, circles actually look like circles and fonts down as small as one point (approx 1mm high upper case) can be read with ease. This is more than adequate for quick prints of engineering drawings of machine parts.

Here is a simple demonstration of what the Pwlib library module can do.

```
!Filename: WT21_011.TRU
!Author: J.R.Arscott
!Purpose: Pwlib PRINTING program

LIBRARY "pwinlib.trc"
CLEAR

CALL Prt_Create(pwid,"Printer Window Demo",result)

ASK PIXELS xpix,ypix
SET WINDOW 0,xpix-1,ypix-1,0
REM: you can print the values of xpix and ypix
REM: to see the dot density of your printer

PRINT xpix,ypix

IF result = 0 then
    WINDOW #0
    PRINT "Could not open printer."
    GET KEY xxx
    STOP
END IF

CALL Prt_NextBand(pwid,result)
CALL Prt_SetFont(pwid,"Times",12,"ITALIC")
PLOT TEXT, AT 100,100:"This is just a test for
plotting."
CALL Prt_SetFont(pwid,"Times",12,"BOLD")
PLOT TEXT, AT 500,1000:"This is another test for
plotting."

REM: insert the filename of your own image here
CALL read_image("MS BMP",string$,"crown_text.bmp")
BOX SHOW string$ at 200,2500

!SET COLOR -1
!BOX CIRCLE 500,1000,800,300
```

```

!BOX LINES 500,1000,800,300
!CALL Prt_SetFont(pwid,"Comic Sans MS",12,"BOLD")
!PLOT TEXT, AT 100,600:"This is the final test for
!plotting."
CALL Prt_Close(pwid)

WINDOW #0
PRINT "Done..."
GET KEY xxx
END

```

INTERACTIVE COMPUTER GRAPHICS

There are countless excellent examples of commercial and freeware applications that come under the general title of Interactive Computer Graphics, such as Photoshop and MS Paint. CAD is concerned with precise technical or engineering drawings, whereas interactive graphics are more like artistic sketching or painting or retouching photographs. There is of course a great deal of overlap between these two applications. In MS Paint for example, you can draw geometric shapes using screen co-ordinates. On the other hand it is unlikely that you will find “blur” or “smudge” in the toolbox of a CAD application.

There is little point in writing an equivalent to MS Paint using True BASIC code, even though it is not difficult to do so. The TBeditor allows you direct access to MS Paint, so why not use it. Obviously a free application like this does not have the range and capabilities of Photoshop, but there are many other freeware applications that come close.

Basically, an interactive graphics program allows you to draw a figure by moving the mouse or stylus to add color or edit the existing color of a pixel or group of pixels on the screen. Very often this will be done in the “zoom” mode, but the ultimate objective is to produce a photographic type image rather than a precision engineering drawing. In fact these applications are basically optical illusions as opposed to geometrical drawings.

This is best illustrated in the case of text fonts where many letters of the alphabet are curved. Not so many years ago, large scale letters such as “C” and “O” clearly had saw-tooth edges where the black pixels didn’t quite match the geometric curve. Nowadays the problem is solved by slightly blurring these edges with shades of gray to deceive the eye into seeing a smooth curve. Similarly, if you plot a single pixel point in red and immediately next to it you plot a yellow pixel, then your eye will tell you that the color is orange.

To better illustrate the power of optical illusions, let us look at the case of a simple typical Windows push button, i.e., a gray rectangular area with a text legend in black. If you look carefully at the code and at the screen you will see that there are white highlights and dark gray shadows on the edges of the button that make it appear to stand out from the screen as if the light is coming from over your left shoulder. When you click on the push button with your mouse, these shadows will invert making the button appear to be depressed into the screen. The reality is that a flat rectangular shape has been modified with a few white and dark gray lines: the illusion is that the screen is no longer flat, but has depth as well. Our brains know that the world is 3D and the brain only needs a few hints from your eyes to be able to “visualize” the rest. There was an old saying that “seeing is believing” – not anymore.

```
!Filename: WT21_012.TRU
!Author: J.R.Arscott
!Purpose: PUSH BUTTON DRAWING program

REM: Push Button illusion
LIBRARY "TrueCTRL.trc"

CALL TC_init

CALL TC_Win_setfont(main,"times",16,"bold")
```

```

ASK PIXELS xpix,ypix
SET WINDOW 0, xpix-1, ypix-1,0
LET midx=xpix/2
LET midy=ypix/2
LET text$="BUTTON"
SET BACK 2
CLEAR

CALL draw_button(midx-60,midx+60,midy-20,midy-
60,0,text$)

CALL draw_button(midx-
60,midx+60,midy+60,midy+20,1,text$)

DO

GET MOUSE x,y,button

IF button<>0 then

IF x>midx-60 and x<midx+60 then
IF y>midy-60 and y<midy-20 then ! top button

CALL draw_button(midx-60,midx+60,midy-20,midy-
60,1,text$)
pause 0.2
CALL draw_button(midx-60,midx+60,midy-20,midy-
60,0,text$)

ELSEIF y>midy+20 and y<midy+60 then ! bottom
button

CALL draw_button(midx-
60,midx+60,midy+60,midy+20,0,text$)
pause 0.2
CALL draw_button(midx-
60,midx+60,midy+60,midy+20,1,text$)
END IF
ELSE

```

```

EXIT DO

END IF
END IF
let button=0
LOOP

CALL TC_cleanup
END

SUB draw_button(tx1,txr,tyb,tyt,mode,text$)

LET brush=7 ! gray button
LET pen=-1 ! black legend
LET shadow=8
SET COLOR brush
BOX AREA tx1,txr,tyb,tyt
SET COLOR pen
LET xstart= tx1+20-2

LET tbase=(tyb+tyt)/2+6
PLOT TEXT, AT xstart,tbase:text$
CALL draw_shadows(tx1,txr,tyb,tyt,shadow,mode)
END SUB

SUB draw_shadows(tx1,txr,tyb,tyt,shadow,mode)
IF mode=<0 then ! proud
SET COLOR shadow
PLOT tx1,tyb;txr,tyb
PLOT tx1+1,tyb-1;txr-1,tyb-1
PLOT txr,tyb;txr,tyt
PLOT txr-1,tyb-1;txr-1,tyt+1

SET COLOR -2
PLOT tx1,tyb;tx1,tyt
PLOT tx1,tyt;txr,tyt

SET COLOR -2
PLOT tx1+1,tyb-2;tx1+1,tyt+1

```

```

PLOT txl+1,tyt+1;txr-2,tyt+1

ELSE                                     ! depressed
SET COLOR shadow
PLOT txl,tyb;txl,tyt
PLOT txl+1,tyb-1;txl+1,tyt+1
PLOT txl,tyt;txr,tyt
PLOT txl+1,tyt+1;txr-1,tyt+1

SET COLOR -2
PLOT txl,tyb;txr,tyb
PLOT txr,tyb;txr,tyt

SET COLOR -2
PLOT txl+1,tyb-1;txr-1,tyb-1
PLOT txr-1,tyb-1;txr-1,tyt+1
END IF
END SUB

```

This demonstration shows two push buttons, one above the other. If you click your mouse on the “proud” button then it will depress momentarily. If you click on the lower “depressed” button then it will momentarily turn “proud” and then return to depressed. This is all done just by changing the shadows. Click anywhere outside the buttons to end the program.

ANIMATION

There are basically two strategies for achieving animation:

- (a) A computer version of the “movies” where you display a succession of pictures one after the other, in which objects within the pictures occupy very slightly different positions. In the movies the system depends on “persistence of vision” of the human eye so that while the film reel is indexed forwards the eye does not notice the black space caused by the shutter between frames.

In practice the frame is shown three times at 24 frames per second. As we all know is that this gives an extremely good illusion of smooth movement.

In the computer version, there is no shutter so we can reduce the frame speed considerably without affecting the smoothness of movement. This is well illustrated in the example program called WT_enginex.TRU , in which 15 images of a complex rotating machine with 16 different cam movements are shown.

A sliding scale allows you to speed up and slow down the machine. Both the RPM and the frames per second are indicated.

Note: you must modify this program with your own path names for the libraries TrueCTRL and TrueDIAL

- (b) The second method, sometimes called “sprites,” involves moving a small object across a stationary background, or moving the background slowly past a small stationary object. In order to avoid colors from the background showing through the object it is necessary to “mask” the object, i.e., to show a black and white image, where the background portion is colored black and the remainder is colored white. Now you can show the “sprite image” on top of the “mask” knowing that the black portion will allow the background to pass through unchanged, whereas the white portion blanks out the background and prevents it interfering with the colors of the “sprite” image. In the case of photographic sprite images it is very difficult to produce a perfect mask, so some blurring can be expected.

The WT_vine.TRU program is an example of this technique in which a growing grape vine produces leaves and grapes and stem growth randomly. Obviously at some stage leaves will grow on top of places that already have leaves, so each new leaf or bunch of grapes must be masked to prevent former growth showing through. To exit this program press the ESC key.

Note: you must modify this program with your own path names for the libraries TrueCTRL and TrueDIAL.

Summary of Important Points

- A text cursor is not displayed when you are in the graphics mode.
- If you are drawing a geometric figure, you should specify a window coordinate system that conforms to the aspect ratio of your monitor screen.
- You can erase an existing figure by drawing the same figure again using the background color.
- Transformations take place about the center of the coordinate system, not necessarily the center of the figure.

Common Errors

- Failing to use SET WINDOW prior to plotting or using BOX SHOW.
- Rotating a graphics figure without moving the center of coordinates to the desired center of rotation.

- Trying to transform a picture unit containing BOX statements.

Chapter 22

Sound and Music

Frankly, the provisions for sound and music in True BASIC are at best rudimentary. True BASIC only supports the internal motherboard speaker, which on my computer is barely audible, so I have to use independently powered extension speakers. True BASIC does not support any conventional sound or music file formats such as .WAV or .MP3.

SOUND

It is possible to generate sounds in True BASIC with:

```
SOUND frequency, duration
```

For example, here is a small program fragment that produces a telephone ring tone.

```
!Filename: WT22_001.TRU
!Author: J.R.Arscott
!Purpose: PHONE RING TONE program

FOR ring=1 to 6
FOR f=1 to 20
SOUND 600,0.03
SOUND 1500,0.03
NEXT f
PAUSE 1.5
NEXT ring
END
```

MUSIC

Again it is possible to generate musical sounds with the PLAY statement, but there is no control over the type of instrument playing, so the output sound is crude and indeterminate.

The simplest solution, if you want good quality music in your program or in the background while you are programming, is to use Windows Media Player or some other third party freeware. You can CHAIN to this software while still running your own program.

TEXT TO SPEECH

True BASIC supplies a speech to text library module called SPEAK.TRC with certain editions, e.g., True BASIC Gold. The library only has one routine:

```
SPEAK (text$)
```

If this routine is called, then whatever is contained within the string (text\$) will be spoken. The main limitation is that there is no control over the speed of the speech, nor the voice (male or female).

Once again the practical solution is to CHAIN to third party freeware such as 2nd SPEECH CENTER, to do the work for you. You need to download a copy of this program to make the following program work.

```
!Filename: WT22_002.TRU
!Author: J.R.Arscott
!Purpose: TEXT TO SPEECH program
```

```
REM: Tests with third party Speech to Text
REM: direct text - note no quote marks
REM: note hyphens because CHAIN does not accept spaces
REM: -e 1 is a male voice -s 120 slows it down
REM: Speech from a file tempts.txt
REM: -e 30 is a female voice
CHAIN "!C:\iisc\iisc.exe /tts" & " temptss.txt -e 30 -s
120", RETURN
```

```
END
```

Obviously you need to download a free copy of 2nd Speech Center, and you need to change the CHAIN pathname location of the executable program iisc.exe to the folder where you have placed the download.

Chapter 23

Program Structure

Most of the chapters in this book have dealt with using the many True BASIC language features but little has been said about how you arrange your lines of code into a coherent whole. Rather than learning about the methods for finding and correcting program errors, would it not be better to write programs that have no errors in the first place.

When you say that a program is error-free, you mean that extensive testing has revealed no errors. There is no way of knowing absolutely that every single error has been found and corrected. Unfortunately, some errors are never found until the program has been used by many different people in many different applications. Probably some errors are never found at all!

Notice the mention of program testing. Programs written by beginning programmers are seldom tested thoroughly. As much, or even more time should be spent testing a program as was spent writing it. In practice, however, a program is often completed just before the deadline and is barely tested at all.

WRITING STRUCTURED PROGRAMS

Throughout the book I stress logical program design, development of a program in small modules, and the use of clear structures for looping and branching. Keep these points in mind as I discuss some hints for writing better programs.

UNDERSTAND the PROBLEM

You cannot write a program to solve a problem until you

completely understand the problem. This is an obvious statement, but a lot of people start programming before they understand clearly what the program must do. Keep asking questions or investigate further until you do understand the problem.

PLAN FIRST, PROGRAM LATER

A common mistake made by beginning programmers is to start writing their programs too soon. This doesn't mean to do nothing until just before the program is due, but it does mean to do a lot of thinking and planning before writing your first program statement. Here are two steps you can take.

First, you must decide how you will solve the problem. The method of solution is called an algorithm. You want an efficient algorithm that will solve the problem accurately and quickly. You learn algorithms by reading books and articles, by reading other programs, and by discussing methods of solution with experienced programmers.

Second, you must write an outline of your program. An outline can be written as one or more descriptive paragraphs, or in the more traditional outline form. Try to divide your problem solution into separate tasks and then outline a solution for each task. When you learn about the subprogram capabilities of True BASIC in Chapter 19, you will be able to organize your program into a collection of program units. The more detailed your outline, the easier it is to write the computer program.

Only when you have taken these two steps are you ready to start writing the program itself.

WRITE SIMPLE PROGRAMS

Use all the tools you have available to write a program that is easy to read and understand. Use variable names that are as descriptive as possible. Use indentation to identify the statements in a branch or a loop. Use comments to explain the logical action of the program, not what an individual statement does.

Avoid program structures that are tricky or hard to understand. There is usually no justification for writing programs that are difficult to understand in order to make them more efficient. Remember that computer memory is becoming less expensive and computers are becoming faster. In general, programs should be written to minimize the time required for maintenance rather than to minimize the amount of memory used or the execution time.

Most important, write your program as a series of small units, as independent from one another as possible. No unit should be larger than what you can see on the screen. Divide your code into subroutines or functions whenever possible. I recommend strongly that you use this technique.

TEST and DEBUG CAREFULLY

Thoroughly test each block of code, subroutine, or function before adding it to your program. Small units tend to be easier than large programs to write and debug. If each unit is free of errors, the resulting program should also be error-free.

If you are careful in writing your program, it should contain few errors. Test your program for as many conditions as possible that might cause an error. Look carefully at extreme conditions, such as the largest and smallest values of key variables. Ask someone else to run your program and see if they can make it

fail. As mentioned previously, you should budget as much time for testing your program as for writing it. Most program failures are due to inadequate testing.

A common type of error is called the “off by one bug.” This is the error that occurs, for example, when you are counting and the sum turns out to be one less or one greater than it should be. Look carefully at your logic and you should be able to find the mistake. If you start counting at one and the computer starts counting at zero, you may produce this type of error.

Your ability to write error-free programs will improve quickly if you follow these suggestions. There will be times, however, when in spite of your best efforts, you cannot get a program to work. You should then go back and examine your program outline. If you are not satisfied with it, you may save time by starting over again. It is difficult to write a good program from a bad outline.

PROGRAM STYLE

I have included a discussion of program style in this chapter because good style reduces programming errors. Some of these styles are my personal preference. There is no reason you have to follow every one of them. But whatever you do, be consistent.

Indentation

You should always indent the statements in the body of a loop. Indentation makes it easier to see where the loop starts and stops. The usual indentation is two to six spaces. An indentation of one space may not be noticed, while an indentation of more than six spaces tends to make program statements too long. Long statements may not display on one line of the screen or print on paper of normal width.

Indentation becomes especially important when you have nested loops — that is, when one loop is part of the body of another loop. In fact, it is extremely difficult to read a computer program that has multiple nested-loops and no indentation.

Compare the following two example programs:

```
!Filename: WT23_001.TRU
!Author: J.R.Arscott
!Purpose: NESTED LOOPS program
```

```
! Nested loops without proper indentation.
```

```
OPEN #1: name "READINGS.DAT"
FOR I = 1 to 2
FOR J = 1 to 3
FOR K = 1 to 3
INPUT #1: Reading
PRINT Reading;
NEXT K
PRINT
NEXT J
PRINT
PRINT
NEXT I
```

```
! Nested loops with proper indentation.
```

```
OPEN #1: name "READINGS.DAT"
FOR I = 1 to 2
    FOR J = 1 to 3
        FOR K = 1 to 3
            INPUT #1: Reading
            PRINT Reading;
        NEXT K
        PRINT
    NEXT J
    PRINT
    PRINT
NEXT I
END
```

I think most readers would agree that the indented part of the program is easier to read and understand.

The statements that make up each branch of a branching structure should also be indented. Your program is more readable and you can see more clearly what the computer program does when a particular branch is selected.

Use of Comments

Comments are another feature that make a program more readable. A block of comments at the beginning of the program should explain what the program does. Comments can be placed between blocks of program statements or after individual statements. These comments should be used to explain program logic, not program syntax. You can assume that the reader already knows True BASIC syntax.

It is possible, but not common, to overuse comments. Certainly there should seldom be a need to place a comment after every statement in a program. On the other hand, if you are using a complex algorithm that is difficult to understand, lots of comments may make the program easier to read.

Variable Names

Descriptive variable names should be chosen. In True BASIC there is no penalty attached to using long variable names. In many cases, the most descriptive variable name consists of two or more words. As you know, you cannot include space characters in variable names. You can, however, make up a variable name of two or more words and use capital letters to mark the beginning of each word. For example, the variable name; MyBigChance might be chosen. It is certainly more readable than the format; mybigchance, although both variable

names refer to the same memory location.

The Do Format Command

After you have finished writing a program, you can select Do Format from the EDIT menu.

This command converts all program statements in the current program to a standard format. Keywords in statements are capitalized but variable names are not changed. Program structures are indented by appropriate amounts. Comments following program statements are aligned vertically.

Summary of Important Points

- Learn to use the Do Trace command to help locate program errors.
- Always execute the Do Trace command from the command line or from the editor menu so that watch variables can be specified.
- Never modify a program statement or add a new statement when a breakpoint is executed.
- I recommend that you avoid using numerical equalities as logical tests in computer programs.
- As much or more time should be spent testing a program as was spent writing it.
- You must understand the problem before writing a program to solve the problem.
- Always write down a program outline before starting to write program statements.
- Develop your program as a series of small, simple units that have been thoroughly tested.
- Avoid using tricky or complicated logic in order to make a program shorter or more efficient. Even if it involves more typing, make sure your logical steps are clear.
- Always indent statements in the body of a loop. Let DO FORMAT do this for you, because it is so quick.

- Add comments to explain what you are doing, but don't comment too much to the point where you obscure the logic of the code.

Chapter 24

Problem Solving

Now would be a good time to check how good you are at problem solving, because that is why you are learning to program: to get the computer to perform tasks that are either too difficult, too tedious, or too repetitive to do any other way.

Cast your mind back less than 50 years to the decade before calculators and personal computers arrived on the scene. Some computers were around at the time, but they were as big as a house, impossibly expensive with ridiculously small memories, and very rare. I personally learnt to program a Ferranti Mercury with 16K memory way back in 1961.

In those far off days, a few adventurous Americans crossed the Atlantic to spend vacations in Europe, and for many the first stop-over on the arduous journey would have been the United Kingdom. It had a reputation for being quaint, eccentric, and steeped in history. Visitors were not disappointed. Motor cars were very small and drove on the wrong side of the road. There were no straight roads anywhere and no skyscrapers. The people were friendly and polite and more or less spoke the same language, but with a huge variety of strange accents. There was no doubt that it was different.

The real killer was the currency system: an impossible problem for all foreigners, not just Americans. At that time the U.K. was about the only nation that didn't have decimal currency. Technically the system was vigesimal-duodecimal; it counted to the bases 10, 20, and 12, and used fractions rather than decimals. Worse still, there was a bewildering array of coins in circulation. For example, if you wanted buy something in the U.S. for 8 cents then you only have two choices: 8 one cent coins or a nickel (5 cents) and three one cent coins. In contrast,

in the U.K. there were more than 50 ways to combine coins to make 8 pence from the 8 different coins in circulation at that time.

Despite this absurd complication, the local population could perform calculations as easily as those with decimal currency, even very young children were extremely adept. I am not claiming that the British were in some magical way more intelligent than anybody else. They just had more practice, on a daily basis. To demonstrate this point, here are some questions from a real 1962 Arithmetic examination paper set for 11 year old pupils in a national test that all children of this age sat every year.

Arithmetic (1962 London County Council 11 plus examination) Part 2

Question 3.

Add

£	s	d
3	18	3 ½
	2	6 ½
1	5	11 ½

Your task is to write a program that solves this addition problem, or more precisely solves any monetary addition problem using this old-fashioned currency.

UNDERSTANDING THE PROBLEM

Firstly you need to understand that three-column accounting is required instead of just two columns, not just dollars and cents, but in this money system the columns were headed £ - short for the Latin word *libra* (pound), s – short for the Latin word *solidus* (shilling), and d – short for the Latin word *dinarius* (pence). The pound column is a normal decimal system, i.e., it counts to the base 10 and only contains integers (whole numbers); the

shillings column counts to the base 20 and only contains integers; and finally the pence column counts to the base 12 and contains whole numbers and fractions NOT decimals. To begin, you need to get into your mind that these three columns might just as well be labeled pineapples, pears, and cherries, where a pear is worth 12 cherries and a pineapple is worth 20 pears. In effect, each column has a different exchange rate.

It is interesting to note that the examiners who set this question expected 11 year old children to be able to do this sum in their heads. As additional guidance the answer to the sum is:

£5 6s 9 ½d

PLAN FIRST

The solution is to recognize that this is a problem of presentation rather than numerical addition, because the computer can only print numbers in decimal notation. We are not dealing with decimal numbers; we are dealing with three separate groups of integers, which is why I suggested that you should think of these groups as fruit. The program is mainly about handling strings and sub-strings. Essentially, we need to enter a monetary value that consists of 3 separate components, with fractions inside brackets, so that we can easily separate each monetary input string into its three component parts plus fractions. For example, we could use:

```
INPUT PROMPT "£/s/d(fractions)": value$ ! e.g.  
3/18/3(1/2)
```

WRITE SIMPLE PROGRAMS

In its simplest form your program could look like:

```
!Filename: WT24_001.TRU
```

```

!Author: J.R.Arscott
!Purpose: MONETARY PROBLEM outline program

DO
INPUT PROMPT "£/s/d(fractions)": value$
! e.g. 3/18/3(1/2)
IF value$<>"+" then
REM: analyze_input(value$)
REM: split into pounds,shillings,pence,fraction

REM: do additions(pounds,shillings,pence,fraction)

REM: assemble_output(result$)

PRINT "Addition so far= ";result$
ELSE
! no input value so print result
PRINT "Complete addition= ";result$
! the answer we are looking for is 5/6/9(1/2)
EXIT DO
END IF
LOOP
END

```

In other words, we are asking the user to enter a monetary value separated by two slashes and a fraction in brackets when required, e.g., 3/18/6(1/2). This request will be repeated until the user enters a plus sign. The computer will print the cumulative sum and then exit the loop. Note that when there are zero pounds, only shillings and pence are entered as the input string, e.g., 2/6(1/2).

SUB-ROUTINES

Firstly we need a routine to separate out the real numeric values of the pounds, shillings, and pence from the input string. To do this, we must find out where the brackets and slash symbols are located starting at the back end of the input stream. The brackets will tell us the value of fractional pence, if any. The

slashes will tell us the pence, shillings, and pounds values respectively.

Once these values have been separated out, we can process the pounds as normal decimals; the shillings can be added by counting in 20s, and the pence can be added by counting in 12s. The individual calculations are done in four separate operations: fractions, pence, shillings, and pounds in that order, and each group has its own cumulative total.

Finally we need a routine that re-assembles the cumulative total values back into the monetary string result\$.

The full code can be seen in the source file:

```
!Filename: WT24_002.TRU
!Author: J.R.Arscott
!Purpose: FULL LSD PROBLEM program

REM: Program to demonstrate addition with pre-decimal
UK currency

REM: Program notes
REM: Only the code between lines of asterisks concerns
numeric addition

REM: The remainder is concerned with string
manipulation

REM: Set all cumulative totals to zero
LET total_pounds=0           ! sets the pounds to
zero
LET total_shillings=0        ! sets the shillings
to zero
LET total_pence=0           ! sets the pence to
zero
LET total_fractions=0        ! sets the fractions
to zero
```

```

REM: ask the user for input
PRINT "Type in the values you wish to add together."
PRINT
PRINT "Type the plus sign to print and exit."
PRINT

DO
  INPUT PROMPT "£/s/d(fractions)": value$ ! e.g.
3/18/3(1/2)
  IF value$<>"+" then
    ! perform cumulative addition
    REM: look for fractions
    LET bracket=cpos(value$,"(")
    IF bracket=0 then ! there are no
fractions
      LET fraction=0
      LET newvalue$=value$
    ELSE
      LET end_bracket=cpos(value$,")")
      LET fraction$=value$[bracket+1:end_bracket-1]
      LET newvalue$=value$[1:bracket-1]
      ! convert fraction to decimal value

      SELECT CASE fraction$
      CASE "1/4"
        LET fraction=0.25
      CASE "1/2"
        LET fraction=0.5
      CASE "3/4"
        LET fraction=0.75
      CASE ELSE
        LET fraction=0
      END SELECT

      !*****ADD FRACTIONS OF A
PENNY*****
      LET total_fractions=total_fractions+fraction
      IF total_fractions>=1 then ! update the
pence column
        LET total_pence=total_pence+1
        LET total_fractions=total_fractions-1
      END IF

      !*****

```

```

END IF
SELECT CASE total_fractions
CASE 0.25
    LET cum_fraction$="1/4"
CASE 0.5
    LET cum_fraction$="1/2"
CASE 0.75
    LET cum_fraction$="3/4"
CASE ELSE
    LET cum_fraction$=""
END SELECT
LET length=len(newvalue$)

REM: look for pence

LET slash=cposr(newvalue$,"/")
IF slash=0 then          ! no shillings and no
pounds
    LET pence$=newvalue$[1:length]
    LET pence=VAL(pence$)
    LET newvalue$=""
ELSE

    LET pence$=newvalue$[slash+1:length]
    LET pence=VAL(pence$)
    LET newvalue$=newvalue$[1:slash-1]
END IF

!*****ADD PENCE AND UPDATE SHILLINGS*****
LET total_pence=total_pence+pence      !
cumulative pence
IF total_pence>=12 then          ! update the
shillings column
    LET total_shillings=total_shillings+1
    LET total_pence=total_pence-12      ! counting
by 12
END IF

!*****
LET length=len(newvalue$)

REM: look for shillings
IF newvalue$>"" then          ! must have some more
values

```

```

    LET slash=cposr(newvalue$,"/")
    IF slash=0 then          ! no pounds
        LET shilling$=newvalue$[1:length]
        LET shilling=VAL(shilling$)
        LET newvalue$=""
    ELSE
        LET shilling$=newvalue$[slash+1:length]
        LET shilling=VAL(shilling$)
        LET newvalue$=newvalue$[1:slash-1]
    END IF

    !*****ADD SHILLINGS AND UPDATE
POUNDS*****
        LET total_shillings=total_shillings+shilling
    ! cumulative shillings

        IF total_shillings>=20 then    !UPDATE the
pounds column
            LET total_pounds=total_pounds+1
            LET total_shillings=total_shillings-20    !
counting by 20
        END IF
        !*****

    END IF
    LET length=len(newvalue$)

    REM: look for pounds
    IF newvalue$>"" then          ! must have some more
values
        LET pound$=newvalue$[1:length]
        LET pound=VAL(pound$)

        !*****ADD
POUNDS*****
            LET total_pounds=total_pounds+pound    !
cumulative pounds

    !*****

    END IF

    REM: reconvert totals to output format

```

```

    IF total_pounds>0 then
        LET result$=STR$(total_pounds) & "/"
    END IF
    IF total_shillings>=0 then
        LET result$=result$ & STR$(total_shillings) &
"/"
    END IF
    IF total_pence>=0 then
        LET result$=result$ & STR$(total_pence)
    END IF
    IF cum_fraction$ >" " then      ! modify output by
adding fractions
        LET result$=result$ & "(" & cum_fraction$ &
") "
    END IF
    PRINT "Addition so far= ";result$
ELSE                                ! no input value so
print result
    PRINT "Complete addition= ";result$      ! the final
answer we are looking for is 5/6/9(1/2)
    EXIT DO
END IF
LOOP
END

```

You will notice that 95% of the code is involved with string manipulation, which we humans do naturally. Only half a dozen lines are actually concerned with adding up the three columns using different bases in each column. Although the original problem appears to be complex, in practice the computation is very simple.

Just think that an entire nation of 50 million people went through this complicated calculation process many, many times every day of their lives. Fortunately for everybody, the British adopted decimal currency on 15 February 1971. The British are also members of the European Union, and as such are obliged to adopt the metric system. Forty-five years later the metrication process is still far from complete. Meanwhile in America, men went to the Moon and back.

PART IV

Programming for Windows

Chapter 25 Principles

In other parts of this book, you have been shown how to write “top-down” programs and routines. In other words, the programs start at the beginning and finish at the end, and how the user proceeds through this sequence of events is controlled entirely by the person who wrote the program.

Programming for windows requires a completely different approach because a window can contain many objects, such as menus, push buttons, edit fields, lists, etc., and the user has complete freedom to activate any of these objects at any time. You, as the programmer, must accept that you **no longer control the process**. Your job is to write a program that takes account of whatever the user wants to do in whatever order they elect to do it.

This is a profound difference in the way you think about and approach writing programs, so make sure you understand this fundamental concept before you try to work with windows.

In True BASIC this is not as difficult as it sounds. Firstly because a special library module called TrueCTRL (or for the Bronze edition it is called Bronze_TC) has been constructed that makes it very easy to create windows and objects. In general, just one single line of code will produce a whole window or an object such as a list or push button.

At the start of all your windows programs you need to access this library with:

```
LIBRARY "{library} TrueCTRL.trc"
```

Or if you are using the Bronze version, use:

```
LIBRARY "{library} Bronze_TC.trc"
```

And then you must also initialize the library before you use it with:

```
CALL TC_init
```

The Bronze version works exactly the same as the Silver and Gold versions except that there are a limited number of objects, namely windows, menus, edit field, list button, and push button.

In the case of the Gold and Silver editions, there is an extended third-party version of this library called CTX.trc which offers a greater range of objects with improved control over colors and fonts. The extra objects include, elevator buttons, bar scales, data tables, images and right click menus. This extended library is written in True BASIC and generates all these objects from True BASIC graphics. This is a clear demonstration of the power and scope of what can be done with the True BASIC language.

EVENT HANDLING

The TrueCTRL library contains a sub-routine called TC_event that solves the problem of finding out what actions the user has made with the mouse or the keyboard. Indeed, you will find that calling this sub-routine from inside a DO....LOOP structure in your programs makes programs much easier and quicker to write, because you can use the same '*skeleton*' program every time. The files that can be downloaded from the True BASIC

website related to this book contain such a *'skeleton'* program called:

WT_skeleton.TRU. (also WT25_001)

This skeleton was created entirely automatically by the FORMS program in which windows are created and objects are selected by a drag-and-drop process, and all the code is written for you.

It is important to recognize that the TC_event routine is able to queue the events to allow you plenty of time to process each one in turn. The event routine describes in words each event (*event\$*), gives a reference to the window where the event is happening (*window*), and gives two data values that further describe the event (*x1* and *x2*). Programming for windows essentially forces the programmer into a systematic and highly structured programming style, e.g.,

```
!Filename: WT25_002.TRU
!Author: J.R.Arscott
!Purpose: TC_EVENT program

DO
LET x1=0
LET x2=0
CALL TC_event(0,event$,window,x1,x2) `
! What type of event was it?
IF event$="CLOSE" then
! Closing window - perhaps EXIT
ELSEIF event$="SIZE" then
! Window is resized. Check if any
! objects need redrawing
ELSEIF event$="MENU" then
    CALL checkMenu
ELSEIF event$="KEYPRESS" then
! could be Txed or just plain keypress
    ELSEIF event$="CONTROL DESELECTED" then
        CALL check_control_objects
ELSEIF event$="CONTROL SINGLE" then
    CALL CheckLists
ELSEIF event$="SINGLE" then
```

```

! just a mouse click somewhere
ELSEIF event$="DOUBLE" then
! just a double mouse click somewhere
ELSEIF pos(event$,"MOUSE")>"" then
! check other mouse events
ELSE
! raise an error message

```

```

        END IF
LOOP

```

In the example above the event called CONTROL DESELECTED means that a push button has been pressed AND released, or that an edit field has been selected, the user has entered some data, AND has pressed the <return> key to signal that they have finished entering data.

WINDOWS COORDINATES

The next concept that you need to understand before you go any further is the difference between physical and logical windows. Physical windows are those that usually have a border, a title bar, and buttons for closing and resizing the window. Logical windows are zones or areas within a physical window. Logical windows have no borders, title bars, or any control buttons. Every time you create a physical window, a logical window is also created to fill that window. Why have two windows? The reason is very simple: physical windows are where you put windows objects such as push buttons, edit fields, and list buttons, and logical windows are where you use all the other True BASIC features such as PRINT, PLOT, and image handing statements such as BOX SHOW.

It will help you to visualize how things will appear on the monitor screen if you think of the physical window as being on top. For example, if you color an area of the logical window with a BOX AREA statement, then you place a push button in the physical

window, the button will appear on top of the colored area. If you change the colored area with another BOX AREA statement, then the button will still be on top.

The size and location of all windows are defined by four coordinates in this order: left, right, bottom, top.

The coordinates for windows are always relative to the **whole screen**.

The coordinates for objects within the window are always relative to the **window**.

There are basically two units of measurement, **USER** units and **PIXEL** units. To avoid any confusion, you are advised to select one type of working unit and stick with it. The moment you make this choice you have reduced the learning process by half. In practice if you can plot x,y points on a piece of graph paper, then you can just as easily plot points on the screen.

User Units

USER units run from 0 to 1 left to right across the screen and from 0 to 1 up the screen, i.e., a whole screen window in the standard notation:

```
SET WINDOW 0,1,0,1
```

(N.B., this is the default condition)

In the case of physical windows, the location coordinates are relative to the whole screen. In other words, the bottom left of the **screen** is the origin in user units.

The great advantage of user units is that you don't have to worry about how big the screen is in terms of pixels, so your program will run and look the same whatever the screen definition.

The disadvantage is that location coordinates for windows less than full screen size are always in decimals and often to 3 places, e.g.,

```
SET WINDOW 0.552,0.783,0.254,0.672
```

The biggest disadvantage is that many windows objects, such as push buttons, edit fields, and lists, contain text that is independent of user units. As a result you may have a push button with a label that looks perfectly alright on a screen 1024x800 pixels, but at 640x480 the button will be too small to show the text label even though the button is correctly proportioned and positioned with respect to the window. Another problem area is the menu bar, which is a fixed size (20 pixels) regardless of the screen dimensions. For these reasons my personal preference is to use pixel units.

When you come to place **objects** such as push buttons inside a window, the location coordinates of the object are always relative to the **physical window** itself. In other words, the bottom left of the **window** is the origin in user units.

Pixel Units

If you intend to use pixel units then you must tell True BASIC that this what you want by using:

```
CALL TC_SetUnitsToPixels
```

Screen sizes vary depending on the type of computer monitor you have and the operating system that drives it. To establish how big the screen is in pixel units, you have to use:

```
CALL TC_GetScreenSize (left,right,bottom,top)
```

PIXEL units run from left to right across the screen and from top to bottom down the screen, e.g., a whole screen (640x480) window in the standard notation:

```
SET WINDOW 0,639,479,0
```

In the case of physical **windows**, the location coordinates are relative to the whole screen. In other words, the top left of the **screen** is the origin in pixel units.

To find out how big a logical window is and to set its units in pixels you can use:

```
ASK PIXELS xpix,ypix  
SET WINDOW 0,xpix-1,ypix-1,0
```

The advantage of using pixel units is that location coordinates are always in whole numbers, and it is easier to work with different font sizes because the point size is directly related to pixels.

The big disadvantage is that pixel units are dependent on the screen size, so you need to account for this in your program to make sure your program runs and looks the same regardless of the screen definition. In practice you will begin all your programs with a few standard program lines that look after the gritty stuff. For example, in the True BASIC editor, the location coordinates of the text area are relative to the four sides of the window, whereas the edit fields at the bottom are relative to the bottom edge of the window. As a result as you increase the window size, the text area gets bigger but the edit fields remain constant. It is much more flexible and easier than you think. One trick is to position all objects relative to the center of the window so that everything looks balanced regardless of the screen dimensions.

When you come to place **objects** such as push buttons inside a window, the location coordinates of the object are always relative to the **physical window** itself. In other words, the top left of the **window** is the origin in pixel units.

My personal preference is to work in whole numbers with pixel units, so all the examples will also adopt this convention. There is nothing to stop you working in any other units if you prefer.

IDENTITIES

The final concept that you need to get your head around is that all windows and all objects inside windows each have a unique identity number. The screwball part is that you don't need to know the value of that number, because you can use a variable name instead. For example, suppose you want to put a push button in a window, then you can use the variable name *button* as the ID of that button. You are free to use any variable name you like. Suppose you have a button that is labelled QUIT, then you might use the variable name *quit* as the ID for this button.

To create this button you would use:

```
CALL TC_PushBtn_create(quit,"QUIT",xl,xr,yb,yt)
```

To show this button you would use

```
CALL TC_show(quit)
```

To erase this button you would use

```
CALL TC_erase(quit)
```

Strange as it may seem, at no time do you ever need to know the value of the variable name (quit).

TrueCTRL keeps its own internal array list of all the objects that you create and their appropriate variable names. So you don't need to know what these internal reference numbers are.

Chapter 26

Creating Controls and Objects

WINDOWS

Just one line of code will generate an entire window just where you want it:

```
CALL TC_win_create(wid,option$,xl,xr,yb,yt)
```

wid is a variable name for the window ID but you can use any name of your choice.

option\$ is a string that contains a series of single word references to particular features you want in the window. Here is a list of some of these single word references:

TITLE The window will have a title bar at the top but you must supply the title

SIZE A resize button will be provided

CLOSE A close window button will be provided

Typically you would specify *option\$* as:

```
LET option$="TITLE|SIZE|CLOSE"
```

Note the vertical separator bar between each reference word.

The title itself can be added to a window with:

```
CALL TC_win_SetTitle(wid,"My title")
```

The rectangular coordinates *xl,xr,yb,yt* can be in user units or pixels.

TC_Event Responses

Event\$=SINGLE

X1=Mouse x co-ordinate

X2=Mouse y co-ordinate

Description: Single mouse click inside window

Event\$=DOUBLE

X1=Mouse x co-ordinate

X2=Mouse y co-ordinate

Description: Double mouse click inside window

Event\$=EXTEND

X1=Mouse x co-ordinate

X2=Mouse y co-ordinate

Description: Single mouse click with SHIFT inside window

Event\$=MOUSE UP

X1=Mouse x co-ordinate

X2=Mouse y co-ordinate

Description: Mouse button released at this point

Event\$=MOUSE MOVE

X1=Mouse x co-ordinate

X2=Mouse y co-ordinate

Description: Mouse position has changed

Event\$=SIZE

X1=

X2=

Description: Window has been resized

Event\$=REFRESH

X1=

X2=

Description: Window has been redrawn

Event\$=SELECT

X1=Mouse x co-ordinate

X2=Mouse y co-ordinate

Description: Window has been selected

Event\$=HIDE

X1=

X2=

Description: Window close button has been clicked

In order to see this window you must show it with:

`CALL TC_show(wid)`

To remove the window from view use:

`CALL TC_erase(wid)`

This routine just hides the window by erasing it from the screen. The window and all the objects inside it still exist and can be seen using `TC_show`

To make things even easier, True BASIC creates a default window for you that has the identity zero.

Note that every sub-routine uses the ID name of the window.

Before we move on you need to understand a bit more about how windows work.

TARGET WINDOWS

When you create a new window it becomes the current target window. It may not be visible because it may not have been shown, but nevertheless it is the target window. This means that any objects you create will be attached to the current target window. It also means that any output from your program, such as printed text, will be sent to the target window. If the window isn't visible, you will not be able to see the output.

You can force a window to become the target with:

```
CALL TC_win_target(wid)
```

ACTIVE WINDOWS

The active window is the one in front of any other windows. If you click the mouse inside any window it will become the active window. The color of the title bar will also change when the window becomes active. Making a window active just brings it to the front so that it is not partially obscured by other windows. Beware, just because the active window is in front it doesn't necessarily mean that this is the target window.

You can force a window to become active with:

```
CALL TC_win_active(wid)
```

At first this all seems very complicated. Keeping track of which windows the user has clicked with the mouse has all the hallmarks of a nightmare. Fortunately there is a very simple solution:

```
CALL TC_win_switch(wid)
```

This routine makes the specified window the current target and it makes it active. All the program output will appear in this window and it will be in front of all others. In your programs if you are in any doubt about which window is the target or active, then use TC_win_switch to make certain.

MENUS

You can attach a menu structure to a window with a single line call to the routine:

```
CALL TC_Menu_Set(wid,menu$(,))
```

Note the use of the window ID name again.

The array menu\$ is two dimensional and has some strict rules about how it is constructed.

- (1) The first dimension signifies how many menu headings appear on the menu bar
- (2) The second dimension indicates how many items there are in each menu list
- (3) The second dimension must begin at zero. This zero element is the menu heading.
- (4) All menu item lists must be of the same length. In other words there must be the same number of items in each

list, even if you have to pad out the list with “empty” items.

To illustrate the point, here is a menu array with three headings and seven items, i.e., menu\$(3,0:7)

Note that the second dimension has eight elements (0:7) because the title of each main grouping occupies element zero.

```
MAT READ menu$(3,0:7)
DATA fruit,apple,cherry,orange,lemon,lime,"",""
DATA trees,ash,oak,beech,elm,"","",""
DATA days, Mon, Tues, Wed, Thur, Fri, Sat, Sun
```

TC_Event Response

Event\$=MENU

X1=Menu title number

X2=Menu item number

Description: Mouse click on a menu

EDIT FIELDS

In previous chapters you have been instructed how to use INPUT, INPUT PROMPT, and GET KEY in order to gather user input. Here is another very simple one-line call to a subroutine that will collect user input for you. An edit field is just like a one line word processor, and it will remember what the user has entered so that you can recall this information at any time. The big advantage is that you don't have to write any code to do all this. The edit object is specifically designed to gather general purpose input.

To create an edit field you must use:

```
CALL TC_edit_create(eid,text$,xl,xr,yb,yt)
```

eid is the variable name of the edit field ID

text\$ is the initial text that you want to show in this field.

The rectangular coordinates can be in user or pixel units.

In older versions of True BASIC, the edit field is surrounded by a single black lined box. In newer versions the box is replaced by a recessed frame.

If you click on an edit field it becomes active and you can type anything in that field. When you press the <return> key the edit field is deactivated but retains the text you have typed.

TC_Event Responses

Event\$=CONTROL SELECT

X1=

X2=Edit field ID

Description: User has clicked on the edit field

Event\$=CONTROL DESELECTED

X1=

X2=Edit field ID

Description: User has clicked the <return> key

You can find out what a user has entered in the edit field with:

```
CALL TC_Edit_GetText(eid,text$)
```

You can also send and display text to the edit field with:

```
CALL TC_Edit_SetText(eid,text$)
```

LISTBTN

The list object is designed to show the user a simple list of data, similar to the list of items when you click on a menu heading. This object can be used to collect user input, where there are a limited number of options available to the user. Suppose you want the user to select one fruit from a group of six fruits. You could print the names of all the fruits and ask the user to type the name of their selected fruit in an edit field. However, it would

be much easier to show the user a list of fruit and to ask the user to click the mouse on the one they want to select.

You can make a list with:

```
CALL TC_ListBtn_create(lid,list$(),xl,xr,yb,yt)
```

lid is the variable name of the list ID

list\$() is an array of text items that will be printed in the list.

A scroll bar will be provided if the list is too long to fit inside the co-ordinates

The rectangular co-ordinates can be in user or pixel units.

The list appears as a box containing the text from the first list item - list\$(1) and alongside this box there is a push button with a “down” arrow on the face of the button. If you click on the button then the whole list drops down. The rectangular co-ordinates define how big the list is when it is in the dropped position. When the list retracts it only occupies one line of text (about 20 pixels).

If you click your mouse on any item in the list, then the list will retract and the item you selected will appear in the text box.

TC_Event Response

Event\$=CONTROL SINGLE

X1=

X2=List Btn ID

Description: User has clicked on a list button item

You can find out what item in the list the user has selected with:

```
CALL TC_ListBtn_Get(lid,selection)
```

where *selection* is the number of the item in the list.

You can also highlight a particular item in the list with:

```
CALL TC_ListBtn_Set(lid,selection)
```

At any time you can change the list array with:

```
CALL TC_SetList(lid,list$())
```

PUSH BUTTONS

The push button object can be used to initiate certain actions in your program, such as QUIT, BEGIN, HELP, etc., or they can be used as yet another means of collecting user input. As input devices, push buttons are usually limited to simple decisions such as YES and NO, or sometimes CANCEL or QUIT.

You can produce a push button with:

```
CALL TC_PushBtn_create(pid,text$,xl,xr,yb,yt)
```

pid is the variable name of the push button ID

text\$ is the legend that will appear on the face of the button.

The rectangular coordinates can be in user or pixel units.

Push buttons appear as a gray box with the legend text in black. Shadows are added to the box to make it appear to stand out proud of the screen. When you click on a push button, the shadows change so that it appears to be momentarily depressed.

TC_Event Response

Event\$=CONTROL SELECT

X1=

X2=Push button ID

Description: User has clicked on a push button.

Event\$=CONTROL DESELECTED

X1=

X2=Push button ID

Description: User has released the mouse button.

At any time you can change the button text with:

```
CALL TC_SetText(pid,text$)
```

TEXT and FONTS

The TrueCTRL library module also allows you to select different fonts in any size and in various styles. When you use the PRINT or PLOT TEXT statements, they will be executed in whatever happens to be the current font, in the current size and in the current style.

You can change any of these details with:

```
CALL TC_win_SetFont(wid,fontname$,fontsize,fontstyle$)
```

Make sure you specify the name of a font that exists on your computer.

The font size is specified in units called points. One point is approximately one pixel, so uppercase (capital) letters in a 10-point font are 10 pixels high. However, when you are designing space for text, you must remember that some lowercase letters such as “p” and “g” have descenders that go below the line. Moreover, some vertical space must be allowed between lines, i.e.,

Total font height = spacing + point size + descender

For practical purposes you may use the following rule of thumb:

Total font height = INT((point size + 1)*3/2)

Descender height = ROUND(point size/4)

The style of the font can be BOLD, PLAIN, ITALIC, or BOLD ITALIC, e.g.,

```
CALL TC_win_SetFont(wid,"Times",12,"Bold italic")
```

You may change the font details any number of times, so it is possible to fill the screen with a variety of fonts in different sizes and styles, and in different colors, e.g.,

```
!Filename: WT26_001.TRU
```

```

!Author: J.R.Arscott
!Purpose: TC_FONTS program
LIBRARY "C:\TBSilver\TBlibs\TrueCTRL.trc"
CALL TC_init
CALL TC_show(0)
CALL TC_win_switch(0)
CALL TC_Win_SetTitle (0,"Font Test")
CALL TC_win_SetFont(wid,"Times",12,"Bold italic")
Set Color 9 ! blue
PRINT "This is an exaple"
CALL TC_win_SetFont(wid,"Arial",16,"Bold")
Set Color 12 ! red
PRINT "of different fonts"
PRINT
CALL TC_win_SetFont(wid,"Helvetica",10,"Plain")
Set Color 13 ! magenta
PRINT "in various colors"
GET KEY dummy
CALL TC_cleanup
END

```

The Silver and Gold versions of True BASIC offer an additional range of controls and window objects.

STATIC TEXT

In the TrueCtrl library, this object is created by:

```
TC_Stext_create(id,text$,l,r,b,t)
```

It prints text\$ in the SYSTEM font in black in a white box regardless of the background color of the screen. TC_event does not respond to mouse clicks inside this object.

At any time you can change the text with:

```
TC_SetText(id,text$)
```

You can also justify the text with:

```
TC_SetTextJustify(id,justify$)
```

Where justify\$ can be LEFT, CENTER or RIGHT.

Provided you have allowed enough vertical space, the program will format text into multiple lines. In all other cases, if the text is too long for the space allowed, then it will be truncated on the right. You can avoid this problem by using -99999 as a default value for either top (t) or bottom (b) co-ordinates.

CHECK BOX

In the TrueCtrl library this object is created by:

```
TC_CheckBox_create(id,text$,l,r,b,t)
```

It prints text\$ in the SYSTEM font in black on a white background regardless of the background color of the screen. The checkbox itself is a small recessed square containing a tick mark when activated, with the text label alongside.

TC_Event Response

Event\$=CONTROL SELECT

X1=

X2=Checkbox ID

Description: User has clicked on the checkbox.

Event\$=CONTROL DESELECTED

X1=

X2=Checkbox ID

Description: User has clicked released the mouse button.

The checkbox is already sensitive to mouse clicks and shows a tick mark when clicked with the mouse. You can also make the text alongside the checkbox mouse sensitive, i.e., a dotted outline is shown around the text when it is clicked with the mouse. This is achieved with:

```
TC_sensitize(id,flag)
```

When the flag=1 the text is made sensitive to mouse clicks.

When the flag=0 the text is not sensitive to mouse clicks

At any time you can change the checkbox text with:

```
TC_SetText(id,text$)
```

To find out if the checkbox has been clicked with the mouse use:

```
TC_CheckBox_Get(id,status)
```

When status=1 the checkbox has a tick mark

When status=0 the checkbox is not ticked

To set the status of a checkbox use:

```
TC_CheckBox_Set(id,status)
```

When status=1 the checkbox has a tick mark

When status=0 the checkbox is not ticked

RADIO GROUP

In the TrueCtrl library this object is created by:

```
TC_RadioGroup_create(id,radio$(),l,r,b,t)
```

The radio buttons are already sensitive to mouse clicks and show a black spot when clicked with the mouse. You can also make the text alongside the radio button mouse sensitive, i.e., a dotted outline is shown around the text when it is clicked with the mouse. This is achieved with:

```
TC_sensitize(id,flag)
```

When the flag=1 the text is made sensitive to mouse clicks.

When the flag=0 the text is not sensitive to mouse clicks

TC_Event Response

Event\$=CONTROL SELECT

X1=

X2=Radio Group ID

Description: User has clicked on a radio button.

Event\$=CONTROL DESELECTED

X1=

X2=Radio Group ID

Description: User has clicked released the mouse button.

At any time you can change the text associated with a particular radio button with:

`TC_RadioGroup_SetText(id,button,newtext$)`

Where id is the radio group identity

Where button is the button number starting with one at the top

Where newtext\$ is the new text you want to insert.

You can check which radio button is active with:

`TC_RadioGroup_On(id,button)`

The variable (button) is the button number starting with one at the top of the radio group.

You can set any button in the radio group to be ON with:

`TC_RadioGroup_Set(id,button)`

The variable (button) is the button number starting with one at the top of the radio group. If button=0 then all radio buttons are set to OFF

SCROLL BARS

In the TrueCtrl library this object is created by:

`TC_Sbar_create(id,type$,l,r,b,t)`

Type\$ can be VSCROLL or HSCROLL. The scroll bar consists of a silver gray background with arrow buttons at each end and a slider or thumb button located somewhere along the scroll bar. The end buttons advances the slider by a “single” unit and the areas on either side of the slider advance the slider by “page” units. The slider itself can also be dragged along the bar with the mouse.

TC_Event Response

Event\$=HSCROLL

X1=

X2=Scroll ID

Description: User is using the thumb slider.

Event\$=END HSCROLL

X1=

X2=Scroll ID

Description: User has finish using the thumb slider.

Event\$=LEFT

X1=

X2=Scroll ID

Description: User has clicked the scroll LEFT arrow button.

Event\$=RIGHT

X1=

X2=Scroll ID

Description: User has clicked the scroll RIGHT arrow button.

Event\$=PAGELEFT

X1=

X2=Scroll ID

Description: User has clicked the PAGELEFT trough.

Event\$=PAGERIGHT

X1=

X2=Scroll ID

Description: User has clicked the PAGERIGHT trough.

Event\$=VSCROLL

X1=

X2=Scroll ID

Description: User is using the thumb slider.

Event\$=END VSCROLL

X1=

X2=Scroll ID

Description: User has finish using the thumb slider.

Event\$=UP

X1=

X2=Scroll ID

Description: User has clicked the scroll UP arrow button.

Event\$=DOWN

X1=

X2=Scroll ID

Description: User has clicked the scroll DOWN arrow button.

Event\$=PAGUP

X1=

X2=Scroll ID

Description: User has clicked the PAGEUP trough.

Event\$=PAGEDOWN

X1=

X2=Scroll ID

Description: User has clicked the PAGEDOWN trough.

Any system of units can be assigned to the scroll bar. The lowest value of these units (*srange*) is when the slider is at the top or left end of the scroll bar. The highest value (*erange*) of these units is when the slider is at the bottom or right end of the scroll bar. The slider and the end buttons are all square so the variable *prop* is not used by the system. The units and range of the scroll bar is defined by:

```
TC_Sbar_SetRange(id,srange,erange,prop)
```

You can find out what the range and units are with:

```
TC_Sbar_GetRange(id,srange,erange,prop)
```

You can set how far the slider moves with the end buttons or by mouse clicks either side of the slider with:

```
TC_Sbar_SetIncrements(id,single,page)
```

Similarly you can find out what these values are for any scroll bar with:

```
TC_Sbar_GetIncrements(id,single,page)
```

The current position of the slider can be found with:

```
TC_Sbar_GetPosition(id,position)
```

Where position is measured in the same units defined by CT_Sbar_SetRange.

You can also determine the position of the slider with:

```
TC_Sbar_SetPosition(id,position)
```

LIST BOX

In the TrueCtrl library this object is created by:

```
TC_ListBox_create(id,mode$,l,r,b,t)
```

TC_Event Response

Event\$=CONTROL SINGLE

X1=

X2=List box ID

Description: User has selected an item.

Event\$=CONTROL DOUBLE

X1=

X2=List box ID

Description: User has finish selecting from the box.

The list itself is provided by:

`TC_SetList(id,list$())`

The list is printed in the SYSTEM font in black on a white background with no ruling. A vertical scroll bar appears when the list is long enough.

The selected item in the list can be found with:

`TC_ListBox_Get(id,select())`

and any item can be set as the highlight with:

`TC_ListBox_Set(id,selection)`

LIST BUTTON

In the TrueCtrl library this object is created by:

`TC_ListBtn_create(id,list$(),l,r,b,t)`

It prints the item selected from the list in the SYSTEM font in black on a white background regardless of the background color of the screen.

The list itself is printed in the SYSTEM font in black on a white background with no ruling. A vertical scroll bar appears when the list is long enough. The original text in the button field is the same as list\$(1). The button shows a black down arrow on a gray button.

TC_Event Response

Event\$=CONTROL SINGLE

X1=

X2=List ID

Description: User has selected a list item.

The list itself is provided by:

`TC_SetList(id,list$())`

The selected item in the list can be found with:

`TC_ListBtn_Get(id,selection)`

and any item can be set as the highlight with:

`TC_ListBtn_Set(id,selection)`

LIST EDIT

In the TrueCtrl library this object is created by:

`TC_ListEdit_create(id,list$(),l,r,b,t)`

It prints the item selected from the list in the SYSTEM font in black on a white background regardless of the background color of the screen.

The list itself is printed in the SYSTEM font in black on a white background with no ruling. A vertical scroll bar appears when the list is long enough. The original text in the button field is the same as list\$(0). The button shows a black down arrow on a gray button.

TC_Event Response

Event\$=CONTROL SELECT

X1=

X2=Edit button ID

Description: User has clicked the list edit button.

Event\$=CONTROL DESELECTED

X1=

X2=Edit button ID

Description: User has finish using the thumb slider.

Event\$=KEYPRESS

X1=Keypress ASCII value

X2=Edit button ID

Description: User pressed a key.

The list itself is provided by:

```
TC_SetList(id,list$())
```

The selected item in the list can be found with:

```
TC_ListEdit_Get(id,selection)
```

and any item can be set as the highlight with:

```
TC_ListEdit_Set(id,selection)
```

Unless you set EXIT CHAR to <RETURN>, <TAB> and <ESC> there is no way to tell when the user has finished using the edit field.

CTX LIBRARY

The Color Text eXtension library module, CTX.trc, attempts to address some of these limitations of the TrueCTRL library and to extend the range of control objects available to the user.

In order to access and use these new features, you must add these library statements to your program:

```
LIBRARY "C:\LIBS\CTX.TRC"  
LIBRARY "C:\LIBS\TrueCTRL.TRC"
```

The pathname for these library modules will depend on where you keep them.

It is important that the LIBRARY statements are in the order shown, otherwise you will get an error message when you run your program. You must use TrueCtrl.trc version dated 19 JUNE 2009 or later. This is because CTX uses PUBLIC variables in this later version of TrueCTRL. In addition, this later version cures some existing bugs.

There are four general rules that you must observe:

- (1) Like TrueCtrl, an object only exists in the window that was TARGET at the time the object was created. A window becomes TARGET the moment you create it, and it remains target unless you create another window or you change the target with:

TC_win_target(wid) or TC_win_switch(wid).

You are advised to create your window then fill it with CTX objects BEFORE creating another window. In this way you can be sure each window contains the relevant objects.

You cannot see the window or its objects until you show the window with:

TC_show(wid). You also need to CT_show each object in order to see that object. This could be a tedious task if your window contains many objects so CTX provides a simple solution that will show all objects in the current window:

CALL CT_showall(wid)

- (2) In CTX all objects are located by rectangular co-ordinates in pixel or user units. CTX can handle -99999 as a default co-ordinate in some cases.
- (3) Only use CT_show, CT_erase and CT_free in the active window. This is because the library uses BOX KEEP to create an image of the area that the object will occupy when it is shown, and BOX SHOW when the object is erased.
- (4) CTX draws its windows objects on the bottom screen layer along with all the other True BASIC graphics. This layer is below the layers occupied by control objects and graphics produced by the TrueCtrl library. As a result

CTX objects do not 'sit-on-top' like TrueCtrl objects. Do not locate CTX objects so that they partially obscure each other because their co-ordinates cannot be disentangled. This is not a problem with drop down lists such as ListBtn and ListEdit which may partially obscure other objects when the list drops. This is because the program knows the list is the current focus so it ignores other objects until the list is retracted.

CT EVENT

CT_event(event\$,window,z1,z2,type,id)

The variable event\$ and the locations mx and my will be tested against the list of objects created by CTX. If a match is found, then (id) will contain the object ID and will be reported back to the main program unit after CTX has taken any appropriate action, such as depressing a push button. The variable (type) is a number from 1 to 25 that denotes what type of object the ID refers to. For example, if type=6 then then object is a push button. A full list of types can be found in the CTX manual.

Note: When a list button is pressed to reveal the drop down list, CT_event returns the ID. When the button is pressed again, the ID is returned as a negative number.

Note that the routine CT_event differs from TC_event in that there are two extra parameters. CT_event monitors events that involve TrueCtrl objects as well as objects created by CTX, and is used in exactly the same as TC_event. When id=0 the event must refer to TrueCTRL objects, but when id<>0 the event must refer to CTX objects. CT_event is used as follows:

DO

```
CALL CT_event(event$,window,z1,z2,type,id)
```

```

IF id=0 then
! Write your TrueCTRL code here
ELSE
! Write your CTX code here
END IF

LOOP

```

If the mouse has been clicked on any CTX object, then CT_event will send back the ID of that object in the variable (id).

All TrueCtrl object event\$ are as described in the True BASIC manuals.

All CTX objects report event\$ as "SINGLE" except for:

Edit fields which report "END EDIT"

Edit fields that are being auto checked report "KEYPRESS" and the object ID in (z2) is negative.

ListEdit fields which report back "END EDIT"

ListBox, ListBtn, ListEdit all report "LIST SELECTED" when an item is selected from the list.

Right Click Menus report back "SINGLE RIGHT"

CAUTION:

The only objects that you cannot produce with the CTX library module are:

- (i) Windows and their attached scroll bars
- (ii) Window menus

All the other original features of the TrueCtrl library remain intact and can be used alongside CTX. Similarly, you may also use the TrueDial library to generate dialog boxes in your program, or you can use the TDX extended dialog box library.

GENERAL PURPOSE ROUTINES

CT_setunitstopixels

CT_setunitstousers

At the start of your program, you must decide whether you are working in pixel units or user units, and you cannot change your mind. The default units are pixels.

CT_erase(id)

This routine can only be used if the object(id) is in the current active window (the window in front). The object will be blanked out on screen but it will still exist and can be seen by calling CT_show(id). (See also CT_clear).

In practice, when an object is erased a BOX KEEP image is displayed, showing the original background.

CT_free(id)

This routine can only be used if the object(id) is in the current active window (the window in front). The object will be blanked out on screen and all internal records removed. It can no longer be seen by calling CT_show(id).

CT_getrect(tid,l,r,b,t)

This routine applies to all CTX routines.

It reports back the current rectangular co-ordinates of the object in pixels or user units.

CT_setrect(tid,l,r,b,t)

This routine applies to all CTX routines.

It sets the current rectangular co-ordinates in pixels or user units.

CT_show(id)

This routine can only be used for objects in the active window.

This routine applies to all CTX routines.

It shows the specified object in the location defined by the co-ordinates.

Prior to showing an object, the program saves the current background as a BOX KEEP image. This image is displayed when the object is erased.

CT_get(id,attribute,value\$,value)

This routine allows you to access the data contained in the internal arrays.

When the attribute number is greater than 0 then the current value of that element is returned. When the attribute number is zero then the current string in the text array is returned (see reference list of attributes in the CTX Manual).

CT_set(id,attribute,value\$,value)

This routine allows you to change the data contained in the internal arrays.

When the attribute number is greater than 0 then value is inserted in the internal array in the element defined by attribute. When the attribute number is zero then value\$ is inserted in the internal text array in the position defined by the ID (see reference list of attributes in the CTX Manual).

CT_clear

This routine sets the visibility flag to zero for ALL objects. A simple CLEAR instruction will then remove all objects from the screen. This is a very quick way to erase large numbers of objects without the need to use CT_erase repeatedly.

CT_showall(window)

This routine sets the visibility flag to one for ALL objects in the current window. The object is then shown. This a quick way to restore window objects after a CLEAR instruction without the need to use CT_show repeatedly.

CT_sensitize(id,flag)

Unlike TrueCtrl, in CTX all objects are mouse sensitive. In the case of Static text, Check boxes, Radio buttons, Push buttons the sensitised object shows a dotted outline when clicked. Another click removes the outline. This is achieved with:
When the flag=1 the text is made sensitive to mouse clicks.

When the flag=0 the text is not sensitive to mouse clicks
Barscales can also be enabled or disabled with
CT_sensitize(id,flag)

CT_select(id,flag)

Normally an edit field becomes active when the user clicks on the field. The current text is then shown as white on blue. The edit field can also be made active with CT_select(id). This feature can also apply to List Edit controls.

STATIC TEXT

In the CTX version the create routine is very similar to TrueCTRL:

CT_Stext_create(id,text\$,l,r,b,t)

Rectangular co-ordinates can be in pixel or user units.

t or b can be default value -99999

text\$ must fit between l and r otherwise it will be truncated on the RIGHT.

Default justification is LEFT.

Justification can be changed.

Default text is 10 point SYSTEM bold.

Default ink color is black.

Default background is the current background color.

If text\$ is the name of an image file (e.g., xyz.bmp) then the static text object will be filled with the image. This feature is not available with the TrueCTRL version.

Static text objects in CTX are all sensitive to mouse clicks, unlike TrueCTRL static text objects.

In order to change the color defaults use:

CT_SetColor(id,ink,paper)

Where ink is the color of the pen and paper is the color of the brush or background.

In order to set the text font, use:

CT_SetFont(id,fontname\$,point,style\$)

Where fontname\$ is the name of the font, e.g., Times

Where point is the size of the font measured in points, e.g., 10 point

Where style\$ describes the style of the font, e.g., BOLD, PLAIN or ITALIC

You may use as many different fonts as you wish in any window.

Unlike TrueCtrl, in CTX static text objects are mouse sensitive. One click shows a dotted outline. Another click removes the outline. This is achieved with:

CT_sensitize(id,flag)

When the flag=1 the text is made sensitive to mouse clicks.

When the flag=0 the text is not sensitive to mouse clicks

At any time you can change the text with:

CT_SetText(id,text\$)

You can also justify the text with:

CT_SetTextJustify(id,justify\$)

Where justify\$ can be LEFT, CENTER or RIGHT

Provided you have allowed enough vertical space, the program will format text into multiple lines. In all other cases, if the text is too long for the space allowed, then it will be truncated on the right. You can avoid this problem by using -99999 as a default value for either top (t) or bottom (b) coordinates.

There are three presentation styles for static text which the user can select with:

CT_Stext_SetRecess(tid,recess)

When recess=1 the text is printed on a recessed area of the screen.

When recess=-1 the text is printed on a raised area of the screen.

When recess=0 the text is surrounded by a single line box

When recess=-99999 (default) the text is printed directly onto the background.

Sub-strings within the text can be shown in a different color to the main text pen color. Only one sub-string can be specified, and the text can only occupy one line.

This feature is available using:

CT_Stext_SetCharColor(tid,color,start,finish)

Where color is the color number of the sub-string.

Where start is the character number of the start of the sub-string within the text.

Where finish is the last character number of the sub-string within the text.

For example, if the main text is "Press the RED button" and the sub-string is "RED", then

CALL CT_Stext_SetCharColor(tid,12,11,13)

Will print the sub-string "RED" in the color red regardless of the color specified for the remaining text.

The demo program is called CTX_Stext_demo.tru.

CHECKBOX

In the CTX version, the create routine is very similar to that in TrueCTRL:

CT_CheckBox_create(id,text\$,l,r,b,t)

Rectangular co-ordinates can be in pixel or user units.

t or b can be the default value -99999

text\$ must fit between l and r otherwise it will be truncated.

Justification is always LEFT.

Justification cannot be changed.

Default tick is black on a white recessed background.

Default text is 10 point SYSTEM bold.

Default ink color is black.

Default background is the current background color.

In order to change the color defaults use:

CT_SetColor(id,ink,paper)

Where ink is the color of the pen and paper is the color of the brush or background.

In order to set the text font use:

CT_SetFont(id,fontname\$,point,style\$)

Where fontname\$ is the name of the font, e.g., Times

Where point is the size of the font measured in points, e.g., 10 point

Where style\$ describes the style of the font, e.g., BOLD, PLAIN or ITALIC.

You may use as many different fonts as you wish in any window.

The checkbox is already sensitive to mouse clicks, and shows a tick mark when clicked with the mouse. You can also make the text alongside the checkbox mouse-sensitive, i.e., a dotted outline is shown around the text when it is clicked with the mouse. This is achieved with:

CT_sensitize(id,flag)

When the flag=1 the text is made sensitive to mouse clicks.

When the flag=0 the text is not sensitive to mouse clicks

At any time you can change the checkbox text with:

CT_SetText(id,text\$)

To find out if the checkbox has been clicked with the mouse use:

CT_CheckBox_Get(id,status)

When status=1 the checkbox has a tick mark

When status=0 the checkbox is not ticked

To set the status of a checkbox use:

CT_CheckBox_Set(id,status)

When status=1 the checkbox has a tick mark

When status=0 the checkbox is not ticked

The demo program is called CTX_CheckBox_demo.tru

RADIO GROUP

In the CTX version the create routine is practically the same as the TrueCTRL version:

CT_RadioGroup_create(id,radio\$(),l,r,b,t)

Rectangular coordinates can be in pixel or user units (t or b can be -99999)

The number of array elements in text\$() determines how many buttons are shown and the pitch of those buttons.

text\$ will be truncated if too long to fit between l and r.

Justification is always LEFT.

Justification cannot be changed.

Default text is 10 point SYSTEM bold.

Default ink color is black.

Default background is the current background color.

The radio button is a recessed white disc with a black central spot.

The legend is situated to the right of the button.

In order to change the color defaults use:

CT_SetColor(id,ink,paper)

Where ink is the color of the pen and paper is the color of the brush or background.

In order to set the text font use:

CT_SetFont(id,fontname\$,point,style\$)

Where fontname\$ is the name of the font, e.g., Times

Where point is the size of the font measured in points, e.g., 10 point

Where style\$ describes the style of the font, e.g., BOLD, PLAIN or ITALIC.

You may use as many different fonts as you wish in any window.

The radio buttons are already sensitive to mouse clicks and show a black spot when clicked with the mouse. You can also make the text alongside the radio button mouse-sensitive, i.e., a dotted outline is shown around the text when it is clicked with the mouse. This is achieved with:

CT_sensitize(id,flag)

When the flag=1 the text is made sensitive to mouse clicks.

When the flag=0 the text is not sensitive to mouse clicks

At any time you can change the text associated with a particular radio button with:

CT_RadioGroup_SetText(id,button,newtext\$)

Where id is the radio group identity

Where button is the button number starting with one at the top

Where newtext\$ is the new text you want to insert.

You can check which radio button is active with:

CT_RadioGroup_On(id,button)

The variable (button) is the button number starting with one at the top of the radio group.

You can set any button in the radio group to be ON with:

CT_RadioGroup_Set(id,button)

The variable (button) is the button number starting with one at the top of the radio group. If button=0 then all radio buttons are set to OFF

The demo program is called CTX_RadioGroup_demo.tru.

EDIT FIELD

In the TrueCtrl library this object is created by:

TC_Edit_create(id,text\$,l,r,b,t)

In the CTX version the routine is very similar:

CT_Edit_create(id,text\$,l,r,b,t)

Rectangular co-ordinates can be in pixel or user units.

t or b can be the default value -99999

text\$ will scroll sideways if it is too long to fit between l and r.

Justification is always LEFT.

Justification cannot be changed.

Default text is 10 point SYSTEM bold.

Default ink color is black.

Default background is the current background color.

CT_select(id)

Normally an edit field becomes active when the user clicks on the field. The current text is then shown as white on blue. As soon as you begin to type, the text and background revert to normal. The edit field can also be made active CT_select(id)

When you press the TAB or RETURN keys, the current edit field is frozen and the focus moves to the next edit field in the list (in the order they were created). The TrueCtrl library works the same way.

If you are working with a large number of data entry edit fields, the input process can be speeded up significantly by making the edit field focus progress automatically, without pressing the

TAB or RETURN keys, when the number of characters in the field reaches a pre-determined number. This is achieved with:

CT_Edit_SetCharacters(id,char)

Where char is the maximum number of characters in the field.

When the focus switches automatically from one edit field to the next or if the user randomly selects a particular edit field, it is sometimes necessary to know the identity of the current active edit field. This can be done with:

CT_Edit_AskCurrent(tid)

The TrueCtrl library offers the user the opportunity to check the format of the data in the edit field with TC_Edit_SetFormat and TC_Edit_Checkfield. These routines are limited in the range of checks that can be performed, so the CTX library module incorporates a feature that intercepts the program flow at the point where the focus changes when a field is completed, and allows the user to carry out external checks on the data before proceeding to the next edit field. This feature can be switched on or off with:

CT_Edit_SetCheck(id,flag)

When flag=1 the program exits from CT_event to allow external checking.

When flag=0 the focus changes from one field to the next without external checking.

The programmer is free to subject the data to any number of checks and can even modify the data before returning to the next edit field. There is also a special feature that returns the user to the previous edit field in order to correct data that has failed the error check.

The external error check intercept is handled immediately after CT_event as follows:

CALL CT_event(event\$,window,z1,z2,id,type)

```
IF z1<0 and event$="KEYPRESS" then
    LET id=(-1)*z1
    REM: CALL external_check_routine(id,ok)
    IF ok=1 then
        LET z1=31 ! or 27 to stop the focus change
    ELSE
        LET z1=307
    END IF
    CALL CT_event(event$,window,z1,z2,id,type)
END IF
```

The important points to note are that CTX sends back a “KEYPRESS” event and makes z1 the negative value of the identity of the current edit field. After the error check has been completed, if the data is accepted then the value of z1 must be set to 31. This will cause the focus to change to the next edit field. If the data fails the error check then the value of z1 must be set to 307. This will cause the focus to stay with the current edit field. If the data is correct but you want to stop the focus change, then set z1 to 27.

The user is responsible for writing the error checking routine. Note: You must follow the check routine with CALL CT_event in order to deal with the results of the check. With the check enabled you do not get an END EDIT event.

In order to change the color defaults use:

CT_SetColor(id,ink,paper)

Where ink is the color of the pen and paper is the color of the brush or background.

In order to set the edit field font use:

CT_SetFont(id,fontname\$,point,style\$)

Where fontname\$ is the name of the font, e.g., Times
Where point is the size of the font measured in points, e.g., 10 point
Where style\$ describes the style of the font, e.g., BOLD, PLAIN or ITALIC.
You may use as many different fonts as you wish in any window.

At any time you can change the text for edit fields with:

CT_SetText(id,text\$)

Or for consistency with TrueCtrl you can use:

CT_Edit_SetText(id,text\$)

When the user has finished using the edit field, event\$ reports "END EDIT".

You can find out what text is currently in the field with:

CT_GetText(id,text\$)

or

CT_Edit_GetText(id,text\$)

For even greater flexibility, CTX allows you to select from three different display styles with:

CT_Edit_SetRecess(id,recess)

When recess=1 the edit field appears to be a recessed area of the screen.

When recess=-1 the edit field appears to be a raised area of the screen.

When recess=0 (default) the edit field is surrounded by a single line border.

The demo program is called CTX_Edit_demo.tru.

PUSH BUTTON

In the CTX version the routine is created in same way as TrueCTRL with:

CT_PushBtn_create(id,text\$,l,r,b,t)

The default font is again 10 point SYSTEM bold, and the default ink color is black printed on gray. The legend is automatically center justified, horizontally and vertically. These push buttons can also have images and graphic symbols instead of text.

CT_SetColor(id,ink,paper)

Where ink is the color of the pen and paper is the color of the brush or background.

In order to set the text font use:

CT_SetFont(id,fontname\$,point,style\$)

Where fontname\$ is the name of the font, e.g., Times

Where point is the size of the font measured in points, e.g., 10 point

Where style\$ describes the style of the font, e.g., BOLD, PLAIN, or ITALIC.

You may use as many different fonts as you wish in any window.

A push button can be made sensitive to keystrokes such as <RETURN> using:

CT_sensitize(id,flag)

When the flag=1 the push button is made sensitive to keystrokes.

When the flag=0 the push button is not sensitive to keystrokes.

When a push button is sensitive a dotted outline is shown around the text on the face of the button.

At any time you can change the text for push buttons with:

CT_SetText(id,text\$)

Instead of a legend on the face of the button you can place a graphics image. Instead of text, just give the file name, e.g.,

CT_SetText(id,"myfile.bmp")

The file format must be MS BMP, JPG, or TBX (True BASIC boxstrings) and the image must be smaller than the button size by a margin of 2 pixels all around. For example, a standard program icon image is 32x32 pixels, so the button size for this image must be 36x36 pixels.

You can also create push buttons with arrow heads on the face. This is done by defining the text as LEFT, RIGHT, UP, or DOWN surrounded by asterisks, e.g.,

CT_SetText(id,"*LEFT*")

An additional feature is the ability to give push buttons a toggle action, i.e., click once to depress the button - and it stays down. Click again to release the button. This effect is achieved with:

CT_PushBtn_SetToggle(id,status)

When status=0 the push button action is normal

When status=-1 the push button has a toggle action.

To find out whether the push button is depressed or normal:

CT_PushBtn_GetStatus(id,status)

When status=1 the button is depressed

When status=0 the button is normal

When using toggle-action buttons, it is sometimes a good idea to indicate the status of the button in addition to the normal shadows. The library provides the option to dim the button in the normal position and to return the button to normal illumination when depressed. The effect is to light up the button when clicked to the ON position.

CT_PushBtn_SetDim(id,status)

When status=0 the push button action is normal

When status=1 the push button has is dimmed when OFF.

The demo program is called CTX_PushBtn_demo.tru.

SCROLL BARS

In the CTX version, scroll bars are created in much the same way as TrueCTRL with:

CT_Sbar_create(id,type\$,l,r,b,t)

Type\$ can be VSCROLL for Win98 style or XPVSCROLL for WinXP style vertical scroll bars, or HSCROLL and XPHSCROLL for the two styles of horizontal scroll bars.

Any system of units can be assigned to the scroll bar. The lowest value of these units (srange) is when the slider is at the top or left end of the scroll bar. The highest value (erange) of these units is when the slider is at the bottom or right end of the scroll bar. The slider and the end buttons are all square so the variable prop is not used by the system. The units and range of the scroll bar is defined by:

CT_Sbar_SetRange(id,srange,erange,prop)

You can find out what the range and units are with:

CT_Sbar_GetRange(id,srange,erange,prop)

You can set how far the slider moves with the end buttons or by mouse clicks either side of the slider with:

CT_Sbar_SetIncrements(id,single,page)

Similarly you can find out what these values are for any scroll bar with:

CT_Sbar_GetIncrements(id,single,page)

The current position of the slider can be found with:

CT_Sbar_GetPosition(id,position)

Where position is measured in the same units defined by CT_Sbar_SetRange.

You can also determine the position of the slider with:

CT_Sbar_SetPosition(id,position)

The color of the scroll “thumb” can be set independently with:

CALL CT_setcolor(scrollID+2,-1,color)

Note: CT_event only recognizes “SINGLE” mouse clicks so the CTX module works out which part of the scroll bar has been clicked and adjusts the slider movements accordingly. CT_event returns which part of the scroll has been clicked using the same conventions as TrueCtrl, i.e., UP, DOWN, PAGEUP, PAGEDOWN, VSCROLL, and LEFT, RIGHT, PAGELEFT, PAGERIGHT, and HSCROLL. In your own program you only need to SET or GET the slider position. Everything else is fully automatic.

The demo program is called CTX_Sbar_demo.tru.

LISTS

In the CTX version, the listbox object is created in exactly the same way as TrueCTRL with:

CT_ListBox_create(id,mode\$,l,r,b,t)

and the list is set with:

CT_SetList(id,list\$())

Where list\$ is an array with a zero element.

In the CTX library, the variable *mode\$* is always “SINGLE”. As a result we can simplify the call to create a listbox and make it more consistent with the two other list objects, e.g.,

CT_List_create(id,list\$(),l,r,b,t)

Rectangular co-ordinates can be in pixel or user units.

There is ALWAYS a vertical scroll bar. This is fitted inside the overall width of the list.

The heading label is defined by list\$(0).

Default justification is LEFT.

Justification can be changed with:

CT_Table_DataFormat(id,column,format\$).

Default text is 10 point SYSTEM bold.

Default ink color is black.

Default background is the current background color.

In the CTX version, the coordinates refer to the outer dimensions of the whole object. The program calculates the space required by the edit field, the push button, and the list itself, and fits them all inside these dimensions.

The font for the list is specified by:

CT_List_SetFont(id,name\$,size,style\$,1)

The colors are defined by:

CT_List_SetColor(id,ink,paper,hrule,heads,hilite,pen,brush)

Where ink is the text color in the list.

Where paper is the background color of the list.

Where hrule is the color of the ruling between items in the list.

Where heading is the background color of the heading row.

Where hilite is the color of the row that has been highlighted.

Where pen is set to zero for a listbox.

Where brush is set to zero for a listbox.

A range of new colors are available with the following numbers:

(16) silver gray

(17) sky blue

(18) pale green

(19) pale cyan

(20) pink

(21) pale lilac

(22) sand

CT_List_GetColor(id,ink,paper,vrule,hrule,heads,hilite)

The current colors can be obtained with this routine:

Default ink color is black.

Default background paper is (22) sand color

Default heading background is gray

Default ruling across the page (hrule) is gray

Default ruling down columns (vrule) is ochre

When the user has selected an item from the list, event\$ reports "LIST SELECTED"

To determine which item in the list has been selected use:

CT_List_Get(id,selection)

and any item can be set as the highlight with:

CT_List_Set(id,selection)

NOTE: the selected item can also be found and set with the same call structure used by TrueCtrl, e.g.,

CT_ListBox_Get(tid,select())

This routine returns the number of the item that has been clicked with the mouse. In the case of "SINGLE" mode the item is returned in select(1).

(see also CT_List_Get)

CT_ListBox_Set(tid,selection)

This routine sets the highlight on the item number specified.

(see also CT_List_Set)

Note: The following list of routines apply to ALL types of LIST

CT_List_Get(id,selection)

CT_List_GetColor(id,ink,paper,hrule,heads,hilite,pen,brush)

CT_List_GetFont(id,name\$,size,style\$,flag)

CT_List_Set(id,selection)

CT_List_SetColor(id,ink,paper,hrule,heads,hilite,pen,brush)

CT_List_SetFont(id,name\$,size,style\$,flag)

All lists have vertical scroll bars. The default style is Win98, but this can be changed to WinXP with:

CT_List_SetOptions(id,option\$)

Where the default options are "HRULE|VSCROLL|RECESS"

For XP style scroll bars define option\$ as

"HRULE|XPVSCROLL|RECESS"

In the CTX version the listbutton is created in exactly the same way:

CT_ListBtn_create(id,list\$(),l,r,b,t)

Rectangular co-ordinates can be in pixel or user units.

There is ALWAYS a vertical scroll bar. This is fitted inside the overall width of the list.

The heading label is defined by list\$(0).

Default justification is LEFT.

Justification can be changed with:

CT_Table_DataFormat(id,column,format\$).

Default text is 10 point SYSTEM bold.

Default ink color is black.

Default background is the current background color.

In the CTX version the co-ordinates refer to the outer dimensions of the whole object. The program calculates the space required by the text field, the push button and the list itself and fits them all inside these dimensions.

The original text in the text field is also provided by list\$(0).

CAUTION: This is not the same as TrueCtrl, which uses list\$(1)

The button shows a black down arrow on a gray button. These default colors can be changed with:

CT_SetColor(id,pen,brush)

Where id is the identity of the ListBtn.

Where pen is the color of the DOWN ARROW.

Where brush is the background of the drop down push button.

The font for the text field is specified by:

CT_List_SetFont(id,name\$,size,style\$,0)

The font for the list itself is defined by:

CT_List_SetFont(id,name\$,size,style\$,1)

The colors are defined by:

CT_List_SetColor(id,ink,paper,hrule,heads,hilite,pen,brush)

Where ink is the text color in the list.

Where paper is the background color of the list.

Where hrule is the color of the ruling between items in the list.

Where heading is the background color of the heading row.

Where hilite is the color of the row that has been highlighted.

Where pen is text color in the text field.

Where brush is the background color of the text field.

A range of new colors are available with the following numbers:

(16) silver gray

(17) sky blue

(18) pale green

(19) pale cyan

(20) pink

(21) pale lilac

(22) sand

CT_List_GetColor(id,ink,paper,vrule,hrule,heads,hilite)

The current colors can be obtained with this routine:

Default ink color is black.

Default background paper is (22) sand color.

Default heading background is gray

Default ruling across the page (hrule) is gray

Default ruling down columns (vrule) is ochre

When the user has selected an item from the list, event\$ reports "LIST SELECTED"

To determine which item in the list has been selected use:

CT_List_Get(id,selection)

and any item can be set as the highlight with:

CT_List_Set(id,selection)

NOTE: the selected item can also be found and set with the same call structure used by TrueCtrl, e.g.,

CT_ListBtn_Get(id,selection)

This routine returns the number of the item that has been clicked with the mouse.

(see also CT_List_Get)

CT_ListBtn_Set(id,selection)

This routine sets the highlight on the item number specified.

(see also CT_List_Set)

CT_SetList(id,list\$())

This routine can be used to change the contents of the list.

Where list\$ is an array with a zero element.

Note: The following list of routines apply to ALL types of LIST

CT_List_Get(id,selection)

CT_List_GetColor(id,ink,paper,hrule,heads,hilite,pen,brush)

CT_List_GetFont(id,name\$,size,style\$,flag)

CT_List_Set(id,selection)

CT_List_SetColor(id,ink,paper,hrule,heads,hilite,pen,brush)

CT_List_SetFont(id,name\$,size,style\$,flag)

All lists have vertical scroll bars. The default style is Win98, but this can be changed to WinXP with:

CT_List_SetOptions(id,option\$)

Where the default options are “HRULE|VSCROLL|RECESS”
For XP style scroll bars define option\$ as
“HRULE|XPVSCROLL|RECESS”

In the CTX version the list is created by:

CT_ListEdit_create(id,list\$(),l,r,b,t)

Rectangular coordinates can be in pixel or user units.

The data list must begin at element zero, i.e., MAT REDIM
list\$(0:n)

The item list\$(0) appears as the list heading and the default
initial text.

Default justification is LEFT.

Justification can be changed with:

CT_Table_DataFormat(id,column,format\$).

Default text is 10 point SYSTEM bold.

There is ALWAYS a vertical scroll bar. This is fitted inside the
overall width of the list.

When the user has finished using the edit field, event\$ reports
“END EDIT” with the ListEdit object ID. When the user has
selected an item from the list, event\$ reports “LIST
SELECTED”

The button shows a black down arrow on a gray button. These
default colors can be changed with:

CT_SetColor(id,pen,brush)

Where id is the identity of the ListBtn.

Where pen is the color of the DOWN ARROW.

Where brush is the background of the drop down push button.

In the CTX version, the coordinates can be in user or pixel units
and refer to the outer dimensions of the whole object. The
program calculates the space required by the edit field, the push
button, and the list itself, and fits them all inside these
dimensions.

The font for the edit field is specified by:

CT_List_SetFont(id,name\$,size,style\$,0)

The font for the list itself is defined by:

CT_List_SetFont(id,name\$,size,style\$,1)

The colors are defined by:

CT_List_SetColor(id,ink,paper,hrule,heads,hilite,pen,brush)

Where ink is the text color in the list.

Where paper is the background color of the list.

Where hrule is the color of the ruling between items in the list.

Where heading is the background color of the heading row.

Where hilite is the color of the row that has been highlighted.

Where pen is the color of the edit field text.

Where brush is the color of the background in the edit field.

The normal list of extended colors is also available to the ListEdit control.

CT_List_GetColor(id,ink,paper,vrule,hrule,heads,hilite)

The current colors can be obtained with this routine:

Default ink color is black.

Default background paper is (22) sand color.

Default heading background is gray

Default ruling across the page (hrule) is gray

Default ruling down columns (vrule) is ochre

To determine which item in the list has been selected use:

CT_List_Get(id,selection)

and any item can be set as the highlight with:

CT_List_Set(id,selection)

NOTE: the selected item can also be found and set with the same call structure used by TrueCtrl, e.g.,

CT_ListEdit_Get(id,text\$)

This routine returns the text that has been clicked with the mouse.

(see also CT_List_Get)

CT_ListEdit_Set(id,text\$)

This routine sets the highlight on the text specified.

(see also CT_List_Set)

CT_SetList(id,list\$())

This routine can be used to change the contents of the list.

Where list\$ is an array with a zero element.

Note: The following list of routines apply to ALL types of LIST

CT_List_Get(id,selection)

CT_List_GetColor(id,ink,paper,hrule,heads,hilite,pen,brush)

CT_List_GetFont(id,name\$,size,style\$,flag)

CT_List_Set(id,selection)

CT_List_SetColor(id,ink,paper,hrule,heads,hilite,pen,brush)

CT_List_SetFont(id,name\$,size,style\$,flag)

All lists have vertical scroll bars. The default style is Win98, but this can be changed to WinXP with:

CT_List_SetOptions(id,option\$)

Where the default options are "HRULE|VSCROLL|RECESS"

For XP style scroll bars define option\$ as

"HRULE|XPVSCROLL|RECESS"

The demo program is called CTX_List_demo.tru.

RIGHT CLICK MENU

This object acts in the same way as the normal menu structure generated by the TrueCtrl library, but with only one menu list. You can have as many sub-menus as you like in the

hierarchical structure. You also have a greater choice of fonts and colors.

In TrueCtrl there is no equivalent to the right click menu object.

The CTX version is very similar to a TrueCTRL menu:

CT_RMenu_Set(id,menu\$(,))

Default justification of menu items is LEFT.

Justification cannot be changed.

Default text is 10 point SYSTEM bold.

Default ink color is black.

Default background is the current background color.

Default hilite color is white lettering on a blue background.

The right-click menu object you create only exists in the current TARGET window. For obvious reasons, you can only have one right-click menu in a window.

The main difference is that the right-click menu is an object in its own right and therefore has an ID, whereas normal menus are attached to windows and therefore use the window ID.

The rectangular coordinates used to define the area on screen occupied by a right-click menu are created when the user clicks the right mouse button. The CTX library calculates the position and the overall dimensions. This process is automatic, so the user is not involved in specifying the menu dimensions.

In normal operation, a right click will show the basic menu at the mouse position. The orientation of the menu depends on how close it is to the edges of the window. Mouse movements across the face of the menu will produce highlights at each item (except those that are disabled). If an item points to a sub-menu, then that will be displayed. To make a selection, just click the left mouse button. To erase the menu, click the mouse anywhere outside the menu.

Here is a typical example of a menu structure:

```
dim menu$(5,0:7)
MAT Read menu$
DATA
Doors@D,Tudor@T,Georgian@G,Victorian@V,@,Modern@
@,@,Fire@F
DATA @Modern,Wood@@,Plain primed@P,"","","","",""
DATA @Wood,External@@,@,Internal@@,"","","",""
DATA @External,Solid mahogany@M,Solid
oak@O,"","","","",""
DATA
@Internal,Pine@P,Maple@M,Cherry@C,Teak@T,"","",""
```

NOTE: There is only one main menu.

@ on its own produces a dividing line across the menu box.

A following pair of @@ symbols indicates that a sub menu is required.

The sub menu heading must begin with @ followed by the same item wording.

The @ symbol followed by a letter indicates which item character will be underlined.

These features are exactly the same as the conventions used by TrueCtrl.

To find out which selection the user has made, use:

CT_RMenu_Get(id,text\$)

This returns the item text that the user has selected (with a left click).

To change the menu text for any item, use:

CT_RMenu_SetText(id,menu,item,text\$)

CAUTION: When the menu is first created, the longest item text determines the width of the menu box. If you subsequently

introduce a longer item of text, this may not fit inside the box. If you think you might run into this problem, then pad out the current longest item with spaces to make room for future expansion.

To find out the text for any item use:

CT_RMenu_GetText(id,menu,item,text\$)

To tick a particular menu item use:

CT_RMenu_SetCheck(id,menu,item,flag)

When flag=1 the item is ticked

When flag=0 there is no tick

To find out whether an item has been ticked use:

CT_RMenu_GetCheck(id,menu,item,flag)

When flag=1 the item is ticked

When flag=0 there is no tick

To disable a particular menu item use:

CT_RMenu_SetEnable(id,menu,item,flag)

When flag=0 the item text is printed in gray not black

When flag=1 the item text is printed in black

To find out whether an item has been disabled use:

CT_RMenu_GetEnable(id,menu,item,flag)

When flag=0 the item text is disabled

When flag=1 the item text is enabled

To add color to the right click menu use:

CT_RMenu_SetColor(id,ink,paper,hilite)

Where ink is the text color.

Where paper is the background in the menu itself.

Where hilite is the color of the menu item that has been selected by left clicking with the mouse.

The current colors can be found with:

CT_RMenu_GetColor(id,ink,paper,hilite)

Where ink is the text color.

Where paper is the background in the menu itself.

Where hilite is the color of the menu item that has been selected by left clicking with the mouse.

A range of new colors are available with the following numbers:

- (16) silver gray
- (17) sky blue
- (18) pale green
- (19) pale cyan
- (20) pink
- (21) pale lilac
- (22) sand

To set the font for the right click menu use:

CT_RMenu_SetFont(id,name\$,size,style\$)

Where name\$ is the name of the font, e.g., Times

Where size is the point size, e.g., 12 point

Where style\$ can be BOLD, PLAIN, or ITALIC.

To find out the current font for this object use:

CT_RMenu_GetFont(id,name\$,size,style\$)

Where name\$ is the name of the font, e.g., Times

Where size is the point size, e.g., 12 point

Where style\$ can be BOLD, PLAIN, or ITALIC.

The appearance, location, and size of the right-click menu is determined by the user making a right mouse click. This will be detected by:

CT_event(timer,event\$,window,mx,my)

If event\$="SINGLE RIGHT" then a right-click menu will appear at the mouse position.

The process is automatic, so DO NOT use:

CT_setrect(tid,txl,txr,tyb,tyt)

CT_show(id)
CT_erase(id)

The demo program is called CTX_RMenu_demo.tru.

DATA TABLE

The new Data Table object is equivalent to a ListBox object with multiple columns. The purpose of this new object is to display data in a simple but professional way in a two-dimensional scrollable tabular format. The source of the original data can be a two-dimensional array, a CSV file, or the output from an SQL search of a database. Each array element (or each line in the CSV file) consists of several "fields" of data. These fields are automatically separated into columns for easy viewing. The zero element in the array (or the first line in the CSV file) provides the column headings. Vertical and horizontal scrolls allow access to all columns and all rows. Columns can be expanded or contracted by dragging the dividers with the mouse. Columns or rows can be highlighted with one click of the mouse. Built-in routines can sort columns alphabetically and numerically.

There is no equivalent to this object in TrueCtrl.trc.

In the CTX library the DataTable object is created with:

CT_Table_create(id,option\$,l,r,b,t)

where option\$ can be:

VSCROLL if you want a vertical scroll bar

HSCROLL if you want a horizontal scroll bar

XPVSCROLL if you want a WinXP style vertical scroll bar

XPHSCROLL if you want a WinXP style horizontal scroll bar

RECESS if you want the table recessed into the background

PROUD if you want the table standing proud of the screen

PLAIN if you just want a simple box around the table

VRULE if you want vertical rulings
HRULE if you want horizontal rulings

For example:

LET

option\$="VSCROLL|HSCROLL|RECESS|VRULE|HRULE"

Rectangular coordinates can be in pixel or user units.
Data will be truncated right if too wide for the column space.
Default column justification is LEFT.
Justification and formatting can be changed.
Default text is 8 point ARIAL plain.
Default ink color is black.
Default background paper is (22) sand color.
Default heading background is gray
Default ruling across the page is gray
Default ruling down columns is ochre

The rectangular coordinates used to define the area on screen occupied by the DataTable includes the scroll bars and margins.

This creates an "empty" table with black print on a sand colored paper with faint gray ruling across the page and ochre ruling down the columns. The heading row is colored gray. These are default settings and can be individually customized by the user.

Before you show the DataTable on screen you must fill it with data. It is essential to understand that the Data Table object only manipulates two dimensional arrays. It can also handle rows of data in which the row consists of a list of comma separated items. This is the same way that data is stored in a CSV file.

Fill the table from a two dimensional array in your own program with:

```
CT_Table_ReadArray(id,array$(,))
```

The zero element in the array provides the titles for the column headings.

The table can also be populated by data imported from an external CSV file with:

```
CT_Table_ReadFile(id,csvname$)
```

Now you can show the DataTable with:

```
CT_show(id)
```

To hide the table use:

```
CT_erase(id)
```

To destroy the table use:

```
CT_free(id)
```

IF CT_event is any event associated with vertical or horizontal scroll bars attached to the DataTable then the CTX library will automatically update the status of the scroll bars and will make appropriate changes to the data on display in the DataTable.

Headings are always center justified.

If the column space is too small, then data will be truncated from the right. You can expand the column width to recover the full data.

A mouse click in the heading area will make that column the highlight focus.

Dragging the mouse in the heading area close to a column ruling will produce a dotted rule that the user can position anywhere. When you release the mouse button the dotted rule will become the new column rule so the whole column will expand or contract accordingly.

A mouse click on any row will move the highlight to that row.

To customize the colors use:

CT_Table_SetColor(id,ink,paper,vrule,hrule,heading,hilite)

This allows the user to define all the DataTable colors.

Any color set as zero will be substituted with the current or default color.

Default ink color is black.

Default background paper is (22) sand color.

Default heading background is gray.

Default ruling across the page (hrule) is gray.

Default ruling down columns (vrule) is ochre.

A range of new colors are available with the following numbers:

(16) silver gray

(17) sky blue

(18) pale green

(19) pale cyan

(20) pink

(21) pale lilac

(22) sand

CT_Table_GetColor(id,ink,paper,vrule,hrule,heading,hilite)

The current colors can be obtained with this routine:

Default ink color is black.

Default background paper is (22) sand color.

Default heading background is gray.

Default ruling across the page (hrule) is gray.

Default ruling down columns (vrule) is ochre.

To set the font for the DataTable use:

CT_SetFont(fontname\$,point,style\$)

To find out the current font for this object use:

CT_GetFont(fontname\$,point,style\$)

The default font is 8 point ARIAL plain.

There are four routines for processing the data.

CT_Table_DataFormat(id,column,format\$)

This routine will format numerical data in the column specified. If column=0 then the current focus column is used.

format\$ follows the same rules as format\$ in the USING\$ and PRINT USING\$ statement in True BASIC, e.g., "#####.###" format\$ can also be used to justify string\$ provided "L", "C" or "R" are used for LEFT, CENTER and RIGHT respectively.

CT_Table_DataOrder(id,column,method)

This routine will "bubble sort" numerical or string data in the column indicated or the current highlighted column if column=0.

The method variable is defined as follows:

Method 1=alphabetic A-Z

Method 2=reverse alphabetic Z-A

Method 3=ascii & numeric ascending

Method 4=reverse ascii & numeric descending

CT_Table_InsertData(id,row,datarow\$)

This routine will insert a set of data fields immediately BEFORE the row indicated. If the value of "row" is zero then the data will be added to the end of the data array.

CT_Table_RemoveData(id,row,datarow\$)

This routine extracts a row of data fields and returns this as the variable datarow\$ containing all the fields separated by commas. In the DataTable, the gap left by the extracted data is closed up.

CT_Table_GetData(id,row,datarow\$)

This routine gets the row of data (fields separated by commas) from the current highlighted row if "row" is zero, or from the row specified. It does not remove any data, it just tells you what is there.

CT_Table_GetRow(id,row)

This routine returns the number of the current highlighted row.

CT_Table_SetData(id,row,datarow\$)

This routine changes the current row of data to the data fields contained in the variable datarow\$.

CT_Table_SetRow(id,row)

This routine sets the highlight row to "row" and puts it at the top of the display page.

A row can also be made the highlight row by clicking the mouse on the row.

There are two methods of reading and writing data to the table:

CT_Table_ReadArray(id,array\$(,))

This routine takes a two dimensional array and inserts the data into the DataTable array. The two dimensional array must be dimensioned as follows:

DIM array\$(0:rows,columns)

the zero row must contain the heading titles, e.g.

array\$(0,1)=name

array\$(0,2)=street

array\$(0,3)=town/city

array\$(0,4)=state

array\$(0,5)=zip

CT_Table_ReadFile(id,csvname\$)

This routine takes the data from an external CSV (comma separated variable) file and inserts the data into the DataTable array.

The first line of data in the file must contain the heading names separated by commas.

CT_Table_WriteArray(id,array\$(,))

This routine disassembles the current internal array into a two dimensional array and returns it to the user. The lowest boundary of this array is zero. The zero elements contain the heading names.

CT_Table_WriteFile(id,csvname\$)

This routine saves the current internal array to an external CSV file. The first line in the file contains the heading names separated by commas.

The demo program is called DataTable_demo.tru.

GROUP BOX

In the TrueCtrl library this object is created by:

TC_GroupBox_create(id,text\$,l,r,b,t)

A black rectangular box is drawn with text\$ printed at the top as a title in the SYSTEM font in black on the current background.

The groupbox performs no other function. It is merely screen decoration.

In the CTX version the routine is similar but has certain very useful properties:

CT_GroupBox_create(id,style,l,r,b,t)

Rectangular co-ordinates can be in pixel or user units.

Default co-ordinates cannot be used.

When style=1 the groupbox is shown on a recessed area of the screen.

When style=-1 the groupbox is shown on a raised area of the screen.

When style=0 the groupbox is a single line box

Default ink color is black.
Default background is gray.

In order to change the color defaults use:

CT_SetColor(id,ink,paper)

Where ink is the color of the groupbox and paper is the color of the background inside the groupbox.

The CTX groupbox can work in two ways:

- (1) If you create the groupbox first, and then fill it with objects, it will work the same way as the TrueCTRL groupbox.
- (2) If you create the objects first, and then create the groupbox, it will work like a child window. Any object within the boundaries of the box will be shown or erased when the groupbox is shown or erased. Similarly if the groupbox is moved, then all the objects inside will move with the groupbox. Individual objects can still be shown or erased separately.

It is important to observe the rules regarding the new GroupBox object:

- (a) You cannot use default co-ordinates for the groupbox.
- (b) You must create the objects first BEFORE creating the groupbox.
- (c) All the objects inside the groupbox must be created consecutively ending with the creation of the groupbox itself.
- (d) Showing, erasing and moving a groupbox affects ALL the objects within the groupbox.
- (e) Individual objects can be erased and shown independently once the groupbox is visible

The CTX groupbox has another important property; it can be moved by clicking on the groupbox provided the groupbox has been made “sensitive” with:

CT_sensitize(id,flag)

When the flag=1 the groupbox is made sensitive to mouse clicks.

When the flag=0 the groupbox is not sensitive to mouse clicks

The demo program is called GroupBox_demo.tru.

ELEVATOR BUTTON

This object is not available in the TrueCtrl library.

In the CTX library this object is created by:

CT_ElevatorBtn_create(id,text\$,style,l,r,b,t)

where text\$ determines the direction with:

UP or DOWN or UP/DOWN or LEFT or RIGHT or
LEFT/RIGHT

Note: initial letters can also be used e.g., U,D,UD,L,R or LR

when STYLE=1 the arrow buttons are mounted on a recessed plate

when STYLE=0 the arrow buttons are surrounded by a roundrect line with a default radius of 10 pixels and a line width of 2 pixels

when STYLE=-1 the arrow buttons are mounted on a raised plate

Rectangular co-ordinates can be in pixel or user units.

the co-ordinates refer to the outer dimensions of the back plate or bezel.

Default button color is green.

Default background is the current background color.

A mouse click on any button will automatically cancel any other illuminated button on the same back plate or bezel before illuminating the target button.

The rectangular co-ordinates used to define the area on screen occupied by a pair of elevator button object includes the back plate or bezel. The program calculates how to draw the buttons to fit inside these dimensions.

Do not obscure one CTX object with another object. CT_event will not be able to tell the difference.

The buttons are shaped like arrows so their direction is clear and labels are not required.

When the button is active it is illuminated in a color specified by the user. The colors are defined by:

CT_ElevatorBtn_SetColor(id,bezel,face,button)

Any color set as zero will be substituted with the current or default color.

Default bezel color is black.

Default face color is the current background color.

Default button color is green.

A range of new background colors are available with the following numbers:

(16) silver gray

(17) sky blue

(18) pale green

(19) pale cyan

(20) pink

(21) pale lilac

(22) sand

CT_ElevatorBtn_SetBezel(tid,radius,line)

This allows the user to define the geometry of the bezel.

The units are PIXELS in both cases.

radius is the corner radius of the roundrect bezel (default 10).

line is the thickness of the line of the bezel (default 2).

The current active button can be found with:

CT_ElevatorBtn_Get(id,button\$)

This routine returns the name of the button that has been clicked, e.g., U for UP, D for DOWN, L for LEFT, or R for RIGHT

Note: the arrow button can be activated by clicking the mouse button anywhere close to the arrow as long as it is inside the boundary of the back plate or bezel.

CT_ElevatorBtn_Set(tid,button\$)

This routine allows the user to activate a button from within the program as if it had been clicked with the mouse.

The variable buttons\$ can be:

U for UP, D for DOWN, L for LEFT or R for RIGHT

The demo program is called Elevator_button_demo.tru.

BAR SCALE

This object is not available in the TrueCtrl library.

In the CTX library this object is created by:

CT_BarScale_create(id,text\$,style,l,r,b,t)

text\$ can be UP or DOWN

or text\$ can be LEFT or RIGHT

Bi-polar versions are UP/DOWN and LEFT/RIGHT

when STYLE=2 the bar scale is surrounded with a single line border

when STYLE=1 the bar scale is mounted on a recessed plate

when STYLE=0 the bar scale has no border

when STYLE=-1 the bar scale is mounted on a raised plate

Default bar scale color is blue.

Default background is the current background color.

There two modes of operation:

- (1) If the mouse click is inside a bar scale then that bar scale becomes active. Merely passing the mouse over the bar will drag the bar to a new position. The mouse buttons do not need to be depressed. Note that the mouse must be close to the leading edge of the bar in order to drag it along. If the mouse leaves the bar area then the bar is deactivated.
- (2) The alternative method of operation is to use mouse clicks to increment the bar position. You must use a control object or a keypress to initiate the process, then call the routine:

CT_BarScale_click(leftID,rightID)

where leftID is the identity of the bar scale operated by the left hand mouse button. Set this to zero if the left button is not used.

where rightID is the identity of the bar scale operated by the right hand button. Set this to zero if the right button is not used.

CT_BarScale_SetIncrement(id,left,right)

This routine sets the increments of movement for each click of a mouse button.

LEFT=increments moved by the left button

RIGHT=increments moved by the right button

Only specify one button. The other button must be set to zero.

One mouse can be used to control two separate bar scales.

To modify the color settings use:

CT_BarScale_SetColor(id,pen,brush,bar)

This allows the user to define all the bar scale colors.

Any color set as zero will be substituted with the current or default color.

Default pen color is black.

Default brush color is the current background color.

Default bar color is blue.

A range of new background colors are available with the following numbers:

- (16) silver gray
- (17) sky blue
- (18) pale green
- (19) pale cyan
- (20) pink
- (21) pale lilac
- (22) sand

In the case of bi-polar scales you must use:

CT_BipolarBarScale_SetColor(id,pen,brush,bar1,bar2)

This allows the user to define all the colors for bipolar bar scales.

Any color set as zero will be substituted with the current or default color.

Default pen color is black

Default brush color is the current background color.

Default bar1 color is green (positive).

Default bar2 color is red (negative).

The bar scale can be calibrated in your own units with:

CT_BarScale_SetRange(id,bar,low,high)

The space on the screen occupied by the bar scale is measured in pixel units, but the range of the bar can be specified in any units by the user. The scales do not have to start at zero. In the case of bi-polar scales, the low value is negative and the high value is positive.

To find out the current value of the bar scale use:

CT_BarScale_Get(id,value)

This routine returns the current value of the bar scale position in user units, i.e., the units defined by

CT_BarScale_SetRange

To set the bar scale at a particular value use:

CT_BarScale_Set(tid,value)

This routine allows the user to set any bar scale to any value between the highest and lowest values defined by CT_BarScale_SetRange

To reset the barscale to its lowest value use:

CT_BarScale_Reset(tid)

The lowest position may not necessarily be zero. In the case of bi-polar scales the reset value is zero.

To display a read-out of the current scale value use:

CT_BarScale_LinkText(id,textID,format\$)

where id is the identity of the bar scale

where textID is the identity of the text field

where format\$ is in the form #####.##

A text field must be created by the user before creating the bar scale, then this can be linked to the bar scale in order to automatically report back the current bar position in terms of user units. The user can define the format of the numeric display using the same conventions as the USING\$ function.

When button click operation is required you must use:

CT_BarScale_click(leftID,rightID)

where leftID is the identity of the bar scale operated by the left hand mouse button. Set this to zero if the left button is not used.

where rightID is the identity of the bar scale operated by the right hand button. Set this to zero if the right button is not used.

If both buttons are set to zero then the mouse cannot be used to activate the bar.

This routine must be called from your main program when a user defined event has happened, such as a keypress or a push button has been clicked. This routine will activate the specified bar scale or a pair of bar scales to operate from left

or right hand mouse button clicks. Each click will advance the bar scale by one increment unit. Holding the button down will repeat these increments to produce an animated effect. A keypress must be used to terminate the process.

The demo programs are: CTX_BarScale_demo1.tru to CTX_BarScale_demo4.tru.

IMAGES

In the TrueCtrl library this object is created by:

`TC_graph_create(id,"IMAGE",l,r,b,t)`

The image itself is set by one of a series of separate sub-routines.

In the CTX version the routine is simpler:

CT_Image_create(id,image\$,locate,l,r,b,t)

Rectangular co-ordinates can be in pixel or user units. Image\$ can be a box string (TBX format files) or a BMP or JPG image file name.

Currently the BMP image format must be 24 bit.

The image is displayed using l and b as the bottom corner co-ordinates.

The other coordinates are adjusted to suit the image size.

The image is sensitive to mouse clicks when shown.

At any time you can change the image with:

CT_SetImage(id,image\$)

If the original image is showing then it is erased and the new image is shown. The coordinates are recalculated depending on the location type.

To change the position of the image with respect to the current coordinates use:

CT_LocateImage(id,location)

When location=0 the image is displayed from the bottom left corner

When location=1 the image is displayed from the top left corner

When location=2 the image is displayed from the bottom right corner

When location=3 the image is displayed from the top right corner

When location=4 the image is displayed about the center of the co-ordinates

Note: Always check the current coordinates with CT_GetRect and change them as necessary with CT_SetRect before locating the image.

Images can also be used in place of text or symbols on push-buttons. The CTX library module recognizes images by their extension, e.g., BMP or TBX.

Images can also be used in place of text in the Static Text object. In this way Chinese, Japanese, and Korean characters can be displayed with a Static Text object.

Chapter 27

Windows Program structure

All your Windows programs can follow the same simple structure. The particular example here is used by the FORMS.EXE program, which generates automatic program code and inserts the generated code into this common standard, called Tbstandard.TRU, which is included in the files that can be downloaded from the True BASIC website related to this book.

Of course you don't have to use automatic code. You are free to write whatever you like, but I would advise you to follow this structure or something similar. You will find that it is so much easier to trace which parts of the program control various TrueCTRL objects and what actions you propose taking when any of these objects are selected by the user. With this type of structure, debugging is a breeze.

```
!Filename: WT27_001.TRU
!Author: J.R.Arscott
!Purpose: TBstandard program

! *** AUTO GENERATED CODE BEGINS HERE ***
! *** SOURCE REF FILE ***XX12345678
! *** PROGRAM AUTHOR ***
LET author$="John Doe"
! *** PROGRAM NAME ***
LET caption$="My Program"
! *** VERSION HISTORY ***
LET version$="1.0 dated 29 August 2007"

! *** LIBRARIES ***

!These library modules must be in the TBlibs !folder
otherwise you must use full pathnames

LIBRARY "CTX7.trc" ! erase if not needed
```

```

LIBRARY "TDX.trc" ! erase if not needed
LIBRARY "TrueCTRL2.trc" ! updated version
LIBRARY "TrueDial.trc"
LIBRARY "WTLIB.tru" ! erase if not needed

DECLARE DEF valstr ! takes care of VAL function

DIM zero$(1,0:9)
MAT READ zero$

DIM menu$(1,0:1) ! main window menu
DIM rmenu$(1,0:1) ! right click menu

! *** ASSIGN VARIABLES ***
LET delim$=chr$(44)
LET eol$=chr$(13) & chr$(10)

! *** INITIALIZATION ***

CALL TC_Init          ! Initialize TrueControls
CALL TC_SetUnitsToPixels
CALL TDX_init ! erase if not needed

CALL splash(caption$,author$,version$,zero$(,)) !Make a
splash window using default window #0

CALL TC_Show (0)      ! So we have something to see

CALL TC_win_switch(0) ! Make default ACTIVE

! *** MAIN EVENT LOOP ***

DO

  WHEN exception in
    CALL TC_Event (delay,event$, window, x1, x2)

    IF event$="KEYPRESS" and x1 = 27 THEN
      EXIT DO      ! Escape pressed so EXIT
    END IF

    IF window=0 then
      IF event$="HIDE" then

```

```

        EXIT DO
ELSEIF event$="MENU" then

        SELECT CASE x2          ! selected menu
! insert suitable cases and actions
! CASE 1
! CASE 2
! CASE 3
! CASE etc

        CASE ELSE              ! safety net
        END SELECT
    END IF
END IF

! *** general purpose error handling ***

USE
    REM: ERROR HANDLER

    REM: insert your own message here

    CALL TD_message("PROGRAM ERROR","The program
has detected an error.", "Continue|Quit",1,result)
    LET anytext$=date$ & "/" & time$ & "/" &
extext$ & "/" & exline$ & eol$

    CALL append_file("ERRLOG.TXT",anytext$)

    IF result>1 then
        EXIT DO
    END IF
END WHEN

LOOP

CALL TC_Cleanup

CALL TC_Win_Switch(0)

! *** FORM ROUTINES ***

! *** AUTO GENERATED CODE ENDS HERE ***

```

```

! *** USER SUB-ROUTINES ***
! This is the area used for actions taken when
! the user clicks on an object

! *** UTILITIES ***
! You could use the WTLIB library to provide these
!utilities.

SUB splash(caption$,author$,version$,zero$(,))

    REM: makes a decorative window announcing author
    and version

    REM: allows user to show all windows created by
    DO_FORM

    !determines the size of the users screen
    !calculates the mid point of the screen
    !creates a window to fill the screen
    !adds menu to window
    !adds color and geometric decoration
    !adds program name, author
    !adds version number

    CALL TC_getscreensize(lscr,rscr,bscr,tscr)
    LET midxscr=int((lscr+rscr)/2)      ! mid x axis
    LET midyscr=int((tscr+bscr)/2)     ! mid y axis
    CALL TC_setRect(0,lscr+4,rscr-4,bscr-36,tscr+44)
    CALL TC_Win_SetTitle (0,caption$)
    CALL TC_Menu_Set (0, zero$(,))

    REM: set up logical window
    ASK PIXELS xpix,ypix
    SET WINDOW 0,xpix-1,ypix-1,0
    REM: get window internal dimensions
    CALL TC_getrect(0,wxl,wxr,wyb,wyt)

    LET xs=wxr-wxl
    LET x0=round(xs/2)
    LET ys=wyb-wyt
    LET y0=round(ys/2)
    LET yq=round(y0/2)

```

```

REM: add main color blocks
SET COLOR 13
BOX AREA 1,xs,y0,0
SET COLOR -1
BOX AREA 1,xs,ys,y0

REM: add geometric decoration
SET COLOR 5
LET yp=int(yq/15)
LET m=0
FOR n=yp TO 1 STEP -1
    LET xn=n*10
    LET m=m+1
    BOX AREA 1,xs,y0+xn,y0+xn-m
    BOX AREA 1,xs,y0-xn,y0-xn+m
NEXT n

REM: Main program title in Yellow

CALL TC_win_setfont(0,"TIMES",24,"BOLD")
SET COLOR 14
LET half=(STRWIDTH(0,caption$))/2
PLOT TEXT, AT x0-half,y0+10: caption$

REM: small print in white against black background
CALL TC_win_setfont(0,"SYSTEM",10,"BOLD")
SET COLOR 15
LET half=(STRWIDTH(0,version$))/2
PLOT TEXT, AT x0-half,(2*y0)-40: version$
CALL TC_win_setfont(0,"HELVETICA",8,"PLAIN")
SET COLOR 15
LET copy$="Copyright " & author$ & " " & date$[1:4]
LET half=(STRWIDTH(0,copy$))/2
PLOT TEXT, AT x0-half,(2*y0)-20: copy$

END SUB

SUB append_file(errorname$,anytext$)
REM: adds one line to an existing text file
CLOSE #11
OPEN #11: name errorname$, create newold, org text,
access outin
RESET #11: end

```

```

        SET #11: MARGIN maxnum
        PRINT #11: anytext$
        CLOSE #11
    END SUB

END

! *** END PROGRAM ***

```

USING FORMS TO CREATE CODE

FORMS is a stand alone program that uses a drag-and-drop process to create a window and to place any CTX object within that window. Forms uses the standard program structure and adds all the necessary code. The following example was automatically generated with FORMS.exe and shows the standard structure with two extra windows.

The first window contains a push button with the label “Press me,” an edit field containing the word “name,” and a list button object containing the days of the week.

The second window contains a static text field with red italic text, an image object (a red stop button with a white cross) and a check box with the label “name.”

To close a window, just click on the red button (close window) in the top right.

Both windows have a menu. Notice that the only part that requires any code from the programmer is the section that deals with what action you want the computer to take in response to the object that the user has clicked on. All the rest was automatically generated: not a line has been added, changed or removed.

```

!Filename: WT27_002.TRU
!Author: J.R.Arscott

```

```

!Purpose: TB FORMS_DEMO program

! *** AUTO GENERATED CODE BEGINS HERE ***
! *** SOURCE REF FILE ***ZG20140609
! *** PROGRAM AUTHOR ***
LET author$="John Doe"
! *** PROGRAM NAME ***
LET caption$="Untitled3 Program"
! *** VERSION HISTORY ***
LET version$="2 dated 09 JUN 2014"

! *** LIBRARIES ***

!These library modules must be in the TBlibs folder
!otherwise you must specify their full pathnames

LIBRARY "c:\TBSilver\tblibs\CTX7.trc"
LIBRARY "c:\TBSilver\tblibs\TDX.trc"
LIBRARY "c:\TBSilver\tblibs\TrueCTRL2.trc"
LIBRARY "c:\TBSilver\tblibs\WTlib.tru"

DECLARE DEF valstr

DIM zero$(1,0:9)
MAT READ zero$
DATA Forms,New form two,New form
one,"","","","","","",""

DIM menu$(1,0:1)
DIM rmenu$(1,0:1)

! *** ASSIGN VARIABLES ***
LET delim$=chr$(44)
LET eol$=chr$(13) & chr$(10)

! *** INITIALIZATION ***
CALL TC_Init                      ! Initialize
TrueControls
CALL TC_SetUnitsToPixels          ! Temporary...
CALL CT_SetUnitsToPixels
CALL TDX_init

CALL splash(caption$,author$,version$,zero$(,))
! Make a splash window using window #0

```



```

CALL TC_Show (0)
! So we have something to see

CALL Form_1("New form one")
CALL Form_2("New form two")
! *** EVENT LOOP ***
CALL TC_win_switch(0)

DO
    WHEN exception in

        CALL CT_Event (event$, window, x1, x2, CTid,
CTtype)
        IF event$="KEYPRESS" and x1 = 27 THEN
            EXIT DO
! Exit the event loop when Escape pressed
        END IF
        IF window=0 then
            IF event$="HIDE" then
                EXIT DO
            ELSEIF event$="MENU" then      ! splash menu
                SELECT CASE x2            ! selected menu item

                    CASE 1
                        CALL TC_show(form2)
                        CALL TC_win_switch(form2)

                    CASE 2
                        CALL TC_show(form1)
                        CALL TC_win_switch(form1)

                    CASE ELSE                ! safety net
                END SELECT
            END IF
        ELSEIF window=form1 then
            CALL Form_1_events
        ELSEIF window=form2 then
            CALL Form_2_events
        END IF

! *** general purpose error handling routine ***
USE

    REM: insert your own message here

```

```

        CALL TDX_message("PROGRAM ERROR","The program
has detected an error.", "Continue|Quit",1,result)
        LET anytext$=date$ & "/" & time$ & "/" &
extext$ & "/" & exline$ & eol$
        CALL append_file("ERRLOG.TXT",anytext$)

        IF result>1 then
            EXIT DO
        END IF

    END WHEN

LOOP
CALL TC_Cleanup
CALL TC_Win_Switch(0)

! *** FORM ROUTINES ***
! *** FORM_1 ***
DIM id1043$(0)

SUB form_1(formname$)
    CALL form_1_window
    CALL form_1_menu
    CALL form_1_objects
    CALL form_1_showall
END SUB
! *** FORM_1 CREATE WINDOW ***
SUB form_1_window
    LET option$="TITLE|SIZE|CLOSE|ICONIZABLE"
    CALL TC_win_create(form1,option$,102,920,689,96)
    LET wintitle$="New form one"
    CALL TC_Win_SetTitle(form1,wintitle$)
    CALL TC_win_setbrush(form1,7,7,"SOLID")
    CALL TC_win_setfont(form1,"SYSTEM",10,"BOLD")
    CLEAR
END SUB
! *** FORM_1 CREATE MENU ***
SUB form_1_menu
    MAT REDIM menu$(1,0:2)
    MAT READ menu$
    DATA File,Save as,Exit
    CALL TC_menu_set(form1,menu$(,))
END SUB
! *** FORM_1 CREATE OBJECTS ***

```

```

SUB form_1_objects
    LET option$="Name"
    CALL CT_Edit_create(idl039,option$,290,390,142,117)
    CALL CT_Edit_SetRecess(idl039,0)
    CALL CT_setcolor(idl039,-1,7)
    CALL CT_setfont(idl039,"SYSTEM",10,"BOLD")
    LET option$="Press me"
    CALL
CT_PushBtn_create(idl040,option$,325,425,203,178)
    CALL CT_setcolor(idl040,-1,7)
    CALL CT_PushBtn_SetToggle(idl040,0)
    CALL CT_setfont(idl040,"SYSTEM",10,"BOLD")
    CALL CT_show(idl040)
    MAT REDIM idl043$(0:5)
    MAT READ idl043$
    DATA Days,Monday,Tuesday,Wednesday,Thursday,Friday
    LET option$="HRULE|VSCROLL|RECESS"
    CALL CT_List_SetOptions(option$)
    CALL
CT_ListBtn_create(idl043,idl043$(),270,442,342,244)
    CALL CT_List_setcolor(idl043,-1,22,7,7,17,-1,7)
    CALL CT_List_setfont(idl043,"SYSTEM",10,"BOLD",1)
END SUB
! *** FORM_1 SHOW ***
SUB form_1_showall
    CALL CT_show(idl039)
    CALL CT_show(idl040)
    CALL CT_show(idl043)
END SUB
! *** FORM_1 PROCESSING ***
SUB form_1_events
    IF event$="MENU" then
        CALL form_1_menuevent
    ELSE
        CALL form_1_objectevent
    END IF
END SUB
! *** FORM_1 MENU PROCESSING ***
SUB form_1_menuevent
    SELECT CASE x1
    CASE 1
        SELECT CASE x2
        CASE 1
            CALL action_menu1_Save_as

```

```

        CASE 2
            CALL action_menu1_Exit
        CASE ELSE
            END SELECT
        CASE ELSE
            END SELECT
END SUB
! *** FORM_1 OBJECT PROCESSING ***
SUB form_1_objectevent
    SELECT CASE event$

        CASE "RMENU SELECTED"
        CASE "SINGLE"
            IF CTid=id1040 then CALL action_id1040
        CASE "END EDIT"
            IF CTid=id1039 then CALL action_id1039
        CASE "LIST SELECTED"
            IF CTid=id1043 then CALL action_id1043
        CASE "KEYPRESS"
        CASE ELSE
            END SELECT
END SUB
! *** FORM_1 OBJECT ACTIONS (user code) ***
SUB action_menu1_Save_as
    REM: What do you want to happen when menu 1 item 1
    is clicked?
END SUB
SUB action_menu1_Exit
    REM: What do you want to happen when menu 1 item 2
    is clicked?
END SUB
SUB action_id1039
    REM: What do you want to happen when the user
    completes the edit field id1039?
    CALL CT_Edit_GetText(id1039,text$)          ! current
    edit field text
END SUB
SUB action_id1040
    REM: What do you want to happen when the push
    button id1040 is clicked?
    CALL CT_PushBtn_GetStatus(id1040,status)    !
    current button status
END SUB
SUB action_id1043

```

```

        REM: What do you want to happen when the list
button id1043 item is selected?
        CALL CT_List_Get(id1043,selection)          ! list item
selected
END SUB

! *** END FORM_1 ***

! *** FORM_2 ***

SUB form_2(formname$)
    CALL form_2_window
    CALL form_2_menu
    CALL form_2_objects
    CALL form_2_showall
END SUB
! *** FORM_2 CREATE WINDOW ***
SUB form_2_window

    LET option$="TITLE|SIZE|CLOSE|ICONIZABLE"
    CALL TC_win_create(form2,option$,102,920,689,96)
    LET wintitle$="New form two"
    CALL TC_Win_SetTitle(form2,wintitle$)
    CALL TC_win_setbrush(form2,7,7,"SOLID")
    CALL TC_win_setfont(form2,"SYSTEM",10,"BOLD")
    CLEAR

END SUB
! *** FORM_2 CREATE MENU ***
SUB form_2_menu

    MAT REDIM menu$(1,0:3)
    MAT READ menu$
    DATA File,Save,Save as,Exit
    CALL TC_menu_set(form2,menu$(,))
END SUB
! *** FORM_2 CREATE OBJECTS ***
SUB form_2_objects

    LET option$="Just a test"
    CALL
CT_Stext_create(id2039,option$,321,421,163,138)
    CALL CT_SetTextJustify(id2039,"LEFT")

```

```

        CALL CT_Text_SetRecess(id2039,-99999)
        CALL CT_setcolor(id2039,12,7)
        CALL CT_setfont(id2039,"Roman",12,"ITALIC")
        LET option$="C:\TBeditor6007\formlib\formclose.bmp"
        CALL
CT_Image_create(id2040,option$,0,328,350,202,180)
        LET option$="My name"
        CALL
CT_CheckBox_create(id2041,option$,318,428,261,236)
        CALL CT_setcolor(id2041,-1,7)
        CALL CT_setfont(id2041,"SYSTEM",10,"BOLD")

END SUB

! *** FORM_2 SHOW ***

SUB form_2_showall

        CALL CT_show(id2039)
        CALL CT_show(id2040)
        CALL CT_show(id2041)

END SUB

! *** FORM_2 PROCESSING ***

SUB form_2_events

        IF event$="MENU" then
                CALL form_2_menuevent
        ELSE
                CALL form_2_objectevent
        END IF

END SUB

! *** FORM_2 MENU PROCESSING ***

SUB form_2_menuevent

        SELECT CASE x1
        CASE 1
                SELECT CASE x2
                CASE 1

```

```

        CALL action_menu2_Save
CASE 2
        CALL action_menu2_Save_as
CASE 3
        CALL action_menu2_Exit
CASE ELSE
END SELECT
CASE ELSE
END SELECT

END SUB

! *** FORM_2 OBJECT PROCESSING ***

SUB form_2_objectevent

    SELECT CASE event$

        CASE "RMENU SELECTED"

        CASE "SINGLE"

            IF CTid=id2041 then CALL action_id2041
            IF CTid=id2040 then CALL action_id2040
            IF CTid=id2039 then CALL action_id2039

        CASE "END EDIT"

        CASE "LIST SELECTED"

        CASE "KEYPRESS"

        CASE ELSE
        END SELECT

END SUB

! *** FORM_2 OBJECT ACTIONS (user code) ***

SUB action_menu2_Save
    REM: What do you want to happen when menu 1 item 1
    is clicked?
END SUB

```

```

SUB action_menu2_Save_as
    REM: What do you want to happen when menu 1 item 2
    is clicked?
END SUB
SUB action_menu2_Exit
    REM: What do you want to happen when menu 1 item 3
    is clicked?
END SUB

SUB action_id2039
    REM: What do you want to happen when the text field
    id2039 is clicked?
END SUB

SUB action_id2040
    REM: What do you want to happen when the image
    id2040 is clicked?
END SUB

SUB action_id2041
    REM: What do you want to happen when the checkbox
    id2041 is clicked?
    CALL CT_CheckBox_Get(id2041,status)      ! current
    status
END SUB

! *** END FORM_2 ***

! *** AUTO GENERATED CODE ENDS HERE ***

! *** USER SUB-ROUTINES ***

! *** UTILITIES ***

SUB splash(caption$,author$,version$,zero$(,))

    REM: makes a decorative window announcing author
    and version

    REM: allows user to show all windows created by
    DO_FORM

!*****
    !*determines the size of the users screen

```



```

!*calculates the mid point of the screen
!*creates a window to fill the screen
!*adds menu to window
!*adds color and geometric decoration
!*adds programme name, author and version number

!*****

CALL TC_getscreensize(lscr,rscr,bscr,tscr)
LET midxscr=int((lscr+rscr)/2)      ! mid x axis
LET midyscr=int((tscr+bscr)/2)     ! mid y axis
CALL TC_setRect(0,lscr+4,rscr-4,bscr-36,tscr+44)
CALL TC_Win_SetTitle (0,caption$)
CALL TC_Menu_Set (0, zero$(,))

REM: set up logical window
ASK PIXELS xpix,ypix
SET WINDOW 0,xpix-1,ypix-1,0

REM: get window internal dimensions
CALL TC_getrect(0,wxl,wxr,wyb,wyt)
LET xs=wxr-wxl
LET x0=round(xs/2)
LET ys=wyb-wyt
LET y0=round(ys/2)
LET yq=round(y0/2)

REM: add main color blocks
SET COLOR 13
BOX AREA 1,xs,y0,0
SET COLOR -1
BOX AREA 1,xs,ys,y0

REM: add geometric decoration
SET COLOR 5
LET yp=int(yq/15)
LET m=0
FOR n=yp to 1 step-1
    LET xn=n*10
    LET m=m+1
    BOX AREA 1,xs,y0+xn,y0+xn-m
    BOX AREA 1,xs,y0-xn,y0-xn+m
NEXT n

```

```

CALL TC_win_setfont(0,"TIMES",24,"BOLD")
SET COLOR 14
LET half=(STRWIDTH(0,caption$))/2
REM: Main program title in Yellow
PLOT TEXT, AT x0-half,y0+10: caption$

REM: small print in white against black background
CALL TC_win_setfont(0,"SYSTEM",10,"BOLD")
SET COLOR 15
LET half=(STRWIDTH(0,version$))/2
PLOT TEXT, AT x0-half,(2*y0)-40: version$

CALL TC_win_setfont(0,"HELVETICA",8,"PLAIN")
SET COLOR 15
LET copy$="Copyright " & author$ & " " & date$[1:4]
LET half=(STRWIDTH(0,copy$))/2
PLOT TEXT, AT x0-half,(2*y0)-20: copy$

END SUB

SUB append_file(errorname$,anytext$)

REM: adds one line to an existing text file
CLOSE #11
OPEN #11: name errorname$, create newold, org text,
access outin

RESET #11: end
SET #11: MARGIN maxnum
PRINT #11: anytext$
CLOSE #11

END SUB

END

```

Clearly you need to substitute your own ID variable names in place of the generated numbers, which can be done with ease, using the FIND and REPLACE feature in the editor.

This is a clear demonstration of how you can create numerous windows containing multiple objects in just seconds, leaving

you so much more time to write the stuff that requires thought and ingenuity.

Chapter 28

Dialog Boxes

OVERVIEW

True BASIC already provides a standard library called TrueDial.trc (Bronze_TD.trc in the case of the Bronze edition), that produces four types of dialog boxes based on the built-in subroutines called TBD and TBDX. These built-in routines are difficult to use normally, so the TrueDIAL library just makes things a whole lot easier.

The program WT_dialog_demo demonstrates how to use ALL the dialog boxes and what they look like.

In order to use any of the dialog boxes in this library, you must reference the library with:

```
LIBRARY "{library} TrueDIAL.trc"
```

Or with the Bronze edition:

```
LIBRARY "{library} Bronze_TD.trc"
```

There is no requirement to initialize the library.

In the case of the Gold edition, there is an extended third-party version of this library called TDX.trc, which offers a greater range of dialog boxes with improved control over colors and fonts. The extra objects include passwords, announcements, and color and font selector boxes. This extended library is written in True BASIC and generates all these objects from True BASIC graphics.

GENERAL PURPOSE ROUTINES

```
TD_SetLocation (xloc,yloc)
```

The purpose of this routine is to change the location of the top left pixel coordinates of the dialog box window.

This routine does not apply to the GET FILE and SAVE FILE dialog boxes.

To reset the central position of dialog boxes use:

```
CALL TDX_SetLocation(-1,-1)
```

FILE HANDLING

```
TD_GetFile (extension$, filename$, changedir)
```

If changedir = 0, the current directory will not be changed and the full pathname of the selected file will be returned in filename\$.

If changedir = 1, the current directory may be changed and only the name of the file will be returned in filename\$.

Extension\$ contains the search extension. If an extension is defined, e.g., "tru", then only those files that fall within this definition will be displayed. If the null string is used, e.g., "", then ALL files within the search directory will be displayed.

Upon exit, filename\$ contains the file name selected
If the user selected the CANCEL button then filename\$ = "".

```
TD_SaveFile (extension$, filename$)
```

Extension\$ contains the search extension. If an extension is defined, e.g., "tru", then only those files that fall within this definition will be displayed. If the null string is used, e.g., "", then ALL files within the search directory will be displayed.

The initial name given to `filename$` will be the suggested file name that appears in the dialog box.

Upon exit when the OK button has been clicked, `filename$` contains the file name selected.

If the user selected the CANCEL button, then `filename$ = ""`.

MESSAGES

```
TD_message(title$,message$,button$,default,result)
```

This routine forms the basis of ALL similar message routines. Title\$ is a string that will appear in the blue title bar of the window.

Message\$ is a string that will be center justified in the dialog box.

Long strings can be broken into lines of text using the character “|” as a separator. Each line of text will be center justified.

Up to 10 lines of text are supported.

The default colors are black text on a gray background.

Button\$ is a string of push button captions separated by the character “|” e.g.

“OK|Back|Forward|Quit”

Up to 4 buttons are supported.

Default defines which of the buttons will be the default button linked to the <RETURN> key. The buttons are numbered from left to right.

Result contains the number of the button that was clicked by the user.

The size of the dialog box depends on the width of the widest text message or the number of buttons, whichever is greater. The height depends on the number of lines of text.

```
TD_Warn(message$,button$,default,result)
```

This routine is the same as the standard message routine except that the title\$ option is pre-defined as the word WARNING. All other details are identical.

```
TD_YN(message$,default,result)
```

This routine is the same as the standard message routine except that the title\$ option is pre-defined as the word CONFIRMATION and the two buttons are defined as YES and NO. All other details are identical.

```
TD_YNC(message$,default,result)
```

This routine is the same as the standard message routine except that the title\$ option is pre-defined as the word CONFIRMATION and the two buttons are defined as YES, NO and CANCEL. All other details are identical.

USER INPUT

```
TD_InputM(title$,message$,button$,label$(),text$(),highlight,default,result)
```

This routine forms the basis of ALL similar input routines. Title\$ is a string that will appear in the blue title bar of the window.

Message\$ is a string that will be center justified in the dialog box.

Long strings can be broken into lines of text using the character “|” as a separator. Each line of text will be center justified.

Up to 10 lines of text are supported.

The default colors are black text on a gray background.

Button\$ is a string of push button captions separated by the character “|” e.g.

“OK|Back|Forward|Quit”

Up to 4 buttons are supported.

Label\$() is a text array. Each item in this array is printed alongside an input edit field and acts like a label that describes what the user must enter in the edit field.

Up to 10 input edit fields are supported.

Text\$() is an array that contains the input from the user in each edit field.

Hilite determines which of the labels will be highlighted initially. The labels are numbered from the top down.

Default defines which of the buttons will be the default button linked to the <RETURN> key. The buttons are numbered from left to right.

Result contains the number of the button that was clicked by the user.

The size of the dialog box depends on the width of the widest text message or the number of buttons, whichever is greater. The height depends on the number of lines of text together with the number of input edit fields.

CAUTION:

The total height of the dialog box is determined by the total lines of text and the total number of edit fields. It is up to the programmer to ensure that this total will fit inside the screen.

```
TD_Input(message$,button$,txt$,default,result)
```

This routine is the same as the standard INPUTM routine except that the title\$ option is pre-defined as the words USER INPUT and there is only one input edit field. The label and the message text are combined. As there is only one edit field, a highlight is not necessary. The user input appears in the variable txt\$. All other details are identical to TD_InputM.

LISTS

```
TD_List(message$,button$,lister$(),choice,default,result)
```

This routine forms the basis of ALL similar list routines.

Message\$ is a string that will be center justified in the dialog box.

Long strings can be broken into lines of text using the character “|” as a separator. Each line of text will be center justified.

Up to 10 lines of text are supported.

The default colors are black text on a gray background.

Button\$ is a string of push button captions separated by the character “|” e.g.

“OK|Back|Forward|Quit”

Up to 4 buttons are supported.

Lister\$() is a text array. Each item in this array is displayed in a scrollable list for the user to make a selection from. The

array **MUST** have a zero element, which is used as the title of the list.

Choice determines which item in the list will be highlighted initially and is the default choice if the user makes no choice. The items are numbered from the top down. When the user makes a selection, the item chosen is indicated by the variable choice

Default defines which of the buttons will be the default button linked to the <RETURN> key. The buttons are numbered from left to right.

Result contains the number of the button that was clicked by the user.

TDX LIBRARY MODULE

For Gold edition users, the following routines are available in the third-party library module called TDX.trc.

The purpose of the TDX.trc library is to increase the range of dialog boxes, and to give the programmer control over fonts and colors.

With the exception of the GET FILE and SAVE FILE dialog boxes, all the TDX dialog boxes are created from a single modal window generated by the TrueCTRL library. A simulated modal window is achieved by creating a standard window in which there are no CLOSE or RESIZE or ICONIZE buttons. There is no opportunity for the user to escape this window without clicking on one of the choice push buttons.

This single window contains all the necessary interior objects such as lists, push buttons, edit fields, etc. required by the whole range of dialog boxes. Some of these objects are shown

and some are erased depending on what each dialog box requires. The size of the dialog box is determined by the objects it contains and the quantity of text that has to be displayed.

All the existing TrueDial dialog boxes are available in the TDX library and can be accessed with virtually identical subroutine calls.

The program WT_TDXdemo.TRU demonstrates how to use ALL the new TDX dialog boxes and what they look like.

In order to access and use these new features you must add these library statements to your program:

```
LIBRARY "{library}TDX.TRC"  
LIBRARY "{library}TrueCTRL.TRC"
```

The pathname for these library modules will depend on where you keep them.

It is important that the LIBRARY statements are in the order shown, otherwise you will get an error message when you run your program. You must use TrueCtrl.trc version dated 19 JUNE 2009 or later. This is because TDX uses a PUBLIC variable in this later version of TrueCTRL. In addition this later version cures some existing bugs. You must also initialize both libraries at the beginning of your program with:

```
CALL TC_init  
CALL TDX_init
```

Initialization is necessary with the TDX library but not with TD because it creates the dialog box window and all the objects within it, and locates this window around the center point of the screen. The dialog boxes are now ready to use.

GENERAL PURPOSE ROUTINES

`TDX_SetLocation (xloc,yloc)`

This routine is identical in all respects to the same routine contained in TrueDial.trc. The purpose of this routine is to change the location of the top left pixel coordinates of the dialog box window.

This routine does not apply to the GET FILE and SAVE FILE dialog boxes.

To reset the central position of dialog boxes use:

`CALL TDX_SetLocation(-1,-1)`

To set the font in dialog boxes use:

`TDX_setfont(fontname$,point,style$)`

There is no equivalent of this routine in the TrueDial library.

This routine sets the font for any text that appears in the dialog box window. All dialog boxes will use this font until it is reset by another call to this routine.

Fontname\$ must be a legitimate font available on this computer.

Point is the size of the font (approximately in pixel units).

Style\$ can be BOLD, ITALIC, PLAIN, BOLD ITALIC.

The default font is SYSTEM 10 point BOLD.

The default color of any text is black (color number -1).

`TDX_setpen(pen)`

There is no equivalent of this routine in the TrueDial library.

This routine sets the text color (pen) used in the dialog box window. All dialog boxes will use this text color until it is reset by another call to this routine. Pen is usually any integer 0-255.

The default text color is black (color number -1).

`TDX_setbrush(brush)`

This routine sets the background color (brush) of the dialog box window. All dialog boxes will use this background color until it is reset by another call to this routine.

Brush is usually any integer 0-255

The default background color is gray (color number 7)

CAUTION:

There is no timeout for TDX dialog boxes. The timeout cannot be set. Once a dialog box is shown, it will remain until the user clicks on a push button or makes some other acceptable input. The only exception to this rule is the ANNOUNCE dialog box, which can be programmed to remain on screen for a fixed period of time.

FILE HANDLING

`TDX_GetFile (extension$, filename$, changedir)`

This routine is identical to the same routine in the TrueDial library, therefore the name and the parameter list have been preserved intact and unchanged.

If `changedir = 0`, the current directory will not be changed and the full pathname of the selected file will be returned in `filename$`.

If `changedir = 1`, the current directory may be changed and only the name of the file will be returned in `filename$`.

`Extension$` contains the search extension. If an extension is defined, e.g., "tru", then only those files that fall within this definition will be displayed. If the null string is used, e.g., "", then ALL files within the search directory will be displayed.

Upon exit, filename\$ contains the file name selected
If the user selected the CANCEL button then filename\$ = "".

```
TDX_GetFile (title$,extension$, filename$, changedir)
```

This routine is similar to the routine in theTrueDial library, except that the user can specify the title.

If changedir = 0, the current directory will not be changed and the full pathname of the selected file will be returned in filename\$.

If changedir = 1, the current directory may be changed and only the name of the file will be returned in filename\$.

Extension\$ contains the search extension. If an extension is defined, e.g., "tru", then only those files that fall within this definition will be displayed. If the null string is used, e.g., "", then ALL files within the search directory will be displayed.

Upon exit, filename\$ contains the file name selected
If the user selected the CANCEL button then filename\$ = "".

```
TDX_SaveFile (extension$, filename$)
```

This routine is identical to the same routine in theTrueDial library, therefore the name and the parameter list have been preserved intact and unchanged.

Extension\$ contains the search extension. If an extension is defined, e.g., "tru", then only those files that fall within this definition will be displayed. If the null string is used, e.g., "", then ALL files within the search directory will be displayed.

The initial name given to filename\$ will be the suggested file name that appears in the dialog box.

Upon exit when the OK button has been clicked, filename\$ contains the file name selected.

If the user selected the CANCEL button then filename\$ = "".

```
TDX_SaveFile (title$,extension$, filename$)
```

This routine is similar to the routine in theTrueDial library, except that the user can specify the title.

Extension\$ contains the search extension. If an extension is defined, e.g., "tru", then only those files that fall within this definition will be displayed. If the null string is used, e.g., "", then ALL files within the search directory will be displayed.

The initial name given to filename\$ will be the suggested file name that appears in the dialog box.

Upon exit when the OK button has been clicked, filename\$ contains the file name selected.

If the user selected the CANCEL button then filename\$ = "".

MESSAGES

```
TDX_message (title$,message$,button$,default,result)
```

This routine forms the basis of ALL similar message routines. Title\$ is a string that will appear in the blue title bar of the window.

Message\$ is a string that will be center justified in the dialog box.

Long strings can be broken into lines of text using the character "|" as a separator. Each line of text will be center justified.

Up to 10 lines of text are supported.

The default colors are black text on a gray background.

Button\$ is a string of push button captions separated by the character “|” e.g.

“OK|Back|Forward|Quit”

Up to 4 buttons are supported.

Default defines which of the buttons will be the default button linked to the <RETURN> key. The buttons are numbered from left to right.

Result contains the number of the button that was clicked by the user.

The size of the dialog box depends on the width of the widest text message or the number of buttons, whichever is greater. The height depends on the number of lines of text.

```
TDX_Warn(message$,button$,default,result)
```

This routine is the same as the standard message routine except that the title\$ option is pre-defined as the word WARNING. All other details are identical.

```
TDX_YN(message$,default,result)
```

This routine is the same as the standard message routine except that the title\$ option is pre-defined as the word CONFIRMATION and the two buttons are defined as YES and NO. All other details are identical.

```
TDX_YNC(message$,default,result)
```

This routine is the same as the standard message routine except that the title\$ option is pre-defined as the word CONFIRMATION and the two buttons are defined as YES, NO and CANCEL. All other details are identical.

`TDX_Announce (message$)`

This dialog box displays *message\$* only. It has no buttons because program control returns to the main program immediately the dialog box is shown. The program can continue functioning while the message is on display. It is up to the programmer to decide when to erase the announcement box using `TDX_CloseAnnounce`:

`TDX_CloseAnnounce`

This routine removes an announcement dialog box from the screen. There are no parameters.

USER INPUT

`TDX_InputM(title$,message$,button$,label$(),text$(),highlight,default,result)`

This routine forms the basis of ALL similar input routines. Title\$ is a string that will appear in the blue title bar of the window.

Message\$ is a string that will be center justified in the dialog box.

Long strings can be broken into lines of text using the character “|” as a separator. Each line of text will be center justified.

Up to 10 lines of text are supported.

The default colors are black text on a gray background.

Button\$ is a string of push button captions separated by the character “|” e.g.

“OK|Back|Forward|Quit”

Up to 4 buttons are supported.

Label\$() is a text array. Each item in this array is printed alongside an input edit field and acts like a label that describes what the user must enter in the edit field.

Up to 10 input edit fields are supported.

Text\$() is an array that contains the input from the user in each edit field.

Hilite determines which of the labels will be highlighted initially. The labels are numbered from the top down.

Default defines which of the buttons will be the default button linked to the <RETURN> key. The buttons are numbered from left to right.

Result contains the number of the button that was clicked by the user.

The size of the dialog box depends on the width of the widest text message or the number of buttons, whichever is greater. The height depends on the number of lines of text together with the number of input edit fields.

CAUTION:

The total height of the dialog box is determined by the total lines of text and the total number of edit fields. It is up to the programmer to ensure that this total will fit inside the screen.

```
TDX_Input(message$,button$,txt$,default,result)
```

This routine is the same as the standard INPUTM routine except that the title\$ option is pre-defined as the words USER INPUT and there is only one input edit field. The label and the message text are combined. As there is only one edit field a highlight is not necessary. The user input appears in the variable *txt\$*. All other details are identical to TDX_InputM.

```
TDX_LineInput (lab$,txt$)
```

This routine is the same as the standard INPUTM routine except that title\$ and message\$ are set to null and there is only one OK button. Lab\$ describes the input required and txt\$ holds the input string from the user.

LISTS

```
TDX_List (message$,button$,lister$(),choice,default,result)
```

This routine forms the basis of ALL similar list routines.

Message\$ is a string that will be center justified in the dialog box.

Long strings can be broken into lines of text using the character “|” as a separator. Each line of text will be center justified.

Up to 10 lines of text are supported.

The default colors are black text on a gray background.

Button\$ is a string of push button captions separated by the character “|” e.g.

“OK|Back|Forward|Quit”

Up to 4 buttons are supported.

Lister\$() is a text array. Each item in this array is displayed in a scrollable list for the user to make a selection from. The array MUST have a zero element, which is used as the title of the list.

Choice determines which item in the list will be highlighted initially and is the default choice if the user makes no choice.

The items are numbered from the top down. When the user makes a selection, the item chosen is indicated by the variable choice

Default defines which of the buttons will be the default button linked to the <RETURN> key. The buttons are numbered from left to right.

Result contains the number of the button that was clicked by the user.

```
TDX_ListM(lister$(),selection$(),flag)
```

There is no equivalent of this routine in the TrueDial library.

This routine is based on the standard TDX_LIST routine but allows the user to make multiple selections.

The title bar of the window is labelled MULTIPLE SELECTION LIST

There are three buttons; SELECT, FINISH and CANCEL.

The standard message reads:

"SELECT picks a list item and adds it to the selection list.
FINISH completes the selection. CANCEL returns an empty selection list."

Liste\$() is a text array. Each item in this array is displayed in a scrollable list for the user to make a selection from. The array MUST have a zero element, which is used as the title of the list.

Selection\$() is an array that contains a list of the selected items.

The variable flag=1 when the selection is finished.
The variable flag=0 when the selection is cancelled.

```
TDX_ListOrder(liste$())
```

There is no equivalent of this routine in the TrueDial library.

This routine is based on the standard TDX_LIST routine but allows the user to select items from the list and to insert them in a different order.

The title bar of the window is labelled ORDERED LIST.

There are three buttons: SELECT, UNDO, and QUIT.

The SELECT button changes to INSERT when each selection is made.

The standard message reads:

"SELECT picks a list item. INSERT puts the selected item BEFORE the highlight."

Lister\$() is a text array. Each item in this array is displayed in a scrollable list for the user to make a selection from. The array MUST have a zero element, which is used as the title of the list.

When the user has finished reordering the list, the QUIT button is pressed.

Lister\$() now contains the list in its reordered condition.

The UNDO button returns the entire list to its original order.

PASSWORDS

`TDX_password(pass$)`

There is no equivalent of this routine in the TrueDial library.

The title bar in the dialog window is labelled PASSWORD.

The standard message reads:

"Enter your password here:"

The default colors are black text on a gray background.

There is one button labelled CONFIRM.

Pass\$ contains the password entered by the user.

As the user types characters into the input edit field, only asterisks are displayed for each character. It is up to the programmer to limit the number of attempts a user has at entering a password. It is also the programmer's responsibility to keep track of passwords. This routine only collects the password and conceals it from onlookers.

FONTS

```
TDX_Font_select(fontname$,point,style$,kolor)
```

There is no equivalent of this routine in the TrueDial library.

The title bar in the dialog window is labelled FONT SELECTION.

The standard message reads:

"Select a font."

The default colors are black text on a gray background.

There are two buttons labelled OK and CANCEL.

There are three selection lists across the screen:

Font name

Font size

Font style

To make a selection the user must click on one item in each of the three lists.

In the bottom right of the dialog box is a 16 color selection palette. To make a selection the user must click the mouse on

one of the color squares, or enter a color number (0-255) in the small edit field below the color palette.

To the left there is a sample box where a printed sample of text will appear in the font, size, style and color selected.

It is the responsibility of the programmer to take this information and apply it to the application to put the selected font specification into effect.

COLOR

```
TDX_Color_select(kolor)
```

This is a simplified version of the font_select dialog box. It only allows the user to select a color from a 16 color palette.

The title of the box is:
"Select a color"

To make a selection the user must click the mouse on one of the color squares, or enter a color number (0-255) in the small edit field below the color palette. Whatever method is used, the edit field will show the selected color number that the routine will return in the variable *kolor*

If the initial *kolor* > 1000 then the range of 16 colors are pastel shades. The color number shown in the edit box will be (*kolor*-1000).

Refer to the demonstration program WT TDX_demo.TRU for more details of how to use all the TDX dialog boxes.

PART V

Application Studies

Chapter 29 Worked examples

ICONEX (icon exchange)

When you write, test, and debug a program, you might consider converting the program to a BOUND version, i.e., a standalone executable program that does not need any library modules because they are already bound within the program unit. The first thing you will notice is that the program has the True BASIC toolbox logo. In order to make your program look more professional, you might want to change this icon to your own private logo.

This how you do it. Firstly you need to know that bound executable programs have a built-in icon. Secondly you need to know where that built-in icon is located within the executable program file, and lastly you need to substitute your own logo for this built-in icon.

All icons, including built-in icons, follow the same format, i.e., 32 pixel by 32 pixel bitmaps. The ICO file consists of 22 bytes of header, followed by 744 bytes of logo (766 bytes in total). The name of the True BASIC icon is WINICON.ICO. I have already converted this into a data file called WINICON.TRC.

As you know, all files are BYTE files, so all we have to do is to read any executable file byte by byte until we find a match for the WINICON data. To make the task run quicker we don't just

look at one byte at a time, we look at chunks of bytes: 32768 bytes to be exact. There is a possibility that the icon data might bridge between two of these chunks, so we do a second pass with half chunks just to be sure. In this way we can locate the start of the embedded icon data in the program file.

Now that we know where the data is located, we exchange the True BASIC icon with one of your own making because they are both exactly the same size. Finally we re-write the file, which now includes your personal icon. I have included a few ICO files in the examples related to this book.

The sub-routine MAKE_MAIN creates an edit field with a label asking the user for the name of the source icon file (the default is winicon.ico). The next edit field asks the user for the name of the EXE file. It is assumed that this file is a bound executable file created with True BASIC and therefore contains the True BASIC icon. A browser push button allows the user to select this source file by clicking a filename browse dialog box.

There is one more edit field, for the name of the new icon file as well as another browse button.

You could easily modify this program to allow the user to specify the original source icon file. In this way it would be possible to UNDO the changes you have made to the internal icon. Originally this program was written so that it only replaced the True BASIC icon file, so you could not backtrack with this program. However, for the purposes of this book, I have modified the program so that you can specify the name of the icon file associated with the executable source, in order to reverse the change to the new icon. In effect, this means that you can change the icon as many times as you want as long as you have access to the current icon file (*.ICO).

```

REM: ICONEX program
REM: substitutes True BASIC program icon with custom
icon
REM: Originally developed by Joe Geluso
<jgeluso@iserv.net>
REM: Revised by J.R.Arscott
REM: Dated 23 FEB 2001
REM: Version 1.00

REM: To convert BMP images to icons use

!*****Program Design*****
!icon file is 766 bytes long
!only 744 bytes stored in exe file
!22 bytes in the header are removed
! True BASIC icon file is embedded in this program
! in the external file WINICON.TRU
! User selects target EXE file
! User selects the new icon file
! This program locates the icon string in the EXE file
! Then substitutes the new icon in its place
! REVISED 25 APRIL 2016
! Originalicon added
! HELP removed
!*****Program begins here*****

LIBRARY "C:\libs\TrueCtrl.trc"
LIBRARY "c:\TBsilver\winicon.trc"
LIBRARY "c:\libs\TrueTDX.trc"

REM: insert your own location for winicon.ico
LET original$="c:\TBbook\winicon.ico"

CALL Read_BoxStringData1(icon$)
LET icon$= icon$(23:maxnum)
CALL Read_BoxStringData2(headbox$)
CALL Read_BoxStringData3(logobox$)

DIM menus$(1,0:1)

LET blank$=repeat$(" ",50)

CALL TC_init

```

```

CALL TDX_init

CALL TC_setunitstopixels
CALL TC_win_target(0)
CALL makemain
CALL TC_show(0)
CALL refresh_fields
CALL TC_win_switch(0)

REM: At this point the program window displays 3
REM: labelled edit fields with browse buttons and a
REM: begin button and a quit button

DO
    WHEN error in
        CALL TC_event(0,event$,window,z1,z2)

        IF event$ = "HIDE" then
            EXIT DO
        ELSEIF event$="KEYPRESS" then
            IF z1=27 then
                EXIT DO
            END IF
        ELSEIF event$="CONTROL DESELECTED" then
            IF z2=begin then
                CALL TC_edit_gettext(user,prog$)
                CALL check_file(prog$,status1)
                CALL
TC_edit_gettext(iconuser,new_icon_file$)
                CALL check_file(new_icon_file$,status2)
                CALL
TC_edit_gettext(oldicon,old_icon_file$)
                CALL check_file(old_icon_file$,status3)

                IF status1=1 and status2=1 and status3=1
then
REM: All necessary files are present and correct

                CALL read_icon_bytes(oldicon$,
old_icon_file$)

                LET icon$= oldicon$(23:maxnum)

```

```

CALL get_icon_location(icon$, prog$,
location)

CALL read_icon_bytes(newicon$,
new_icon_file$)

LET newicon$= newicon$(23:maxnum)

CALL set_icon_location(newicon$,
prog$, location)

LET report$="New icon installed in
executable file|" & prog$

CALL
TDX_message("INFORMATION",report$,"MORE|QUIT",1,result)

IF result>1 then
EXIT DO
ELSE
CALL refresh_fields
END IF
ELSE

CALL TDX_message("ERROR","File name
not valid","Continue",1,result)

END IF
ELSEIF z2=browse1 then
CALL browser(user,"EXE")
ELSEIF z2=browse2 then
CALL browser(iconuser,"ICO")
ELSEIF z2=browse3 then
CALL browser(oldicon,"ICO")
ELSEIF z2=quit then
EXIT DO
END IF
ELSEIF event$="MENU" then
SELECT CASE z1
CASE 1 ! file
SELECT CASE z2

CASE 1 ! quit
EXIT DO

```

```

        CASE else
            REM: no more menu items
        END SELECT
    CASE else
        REM: no more menu items
    END SELECT
END IF

USE

        CALL TDX_message("ERROR","Unspecified program
error.|You may be able to
continue.", "Continue|Quit",1,result)
        IF result>1 then
            EXIT DO
        END IF
    END WHEN

LOOP

CALL TC_cleanup

SUB browser(editid,extn$)
    CALL TD_getfile(extn$,filename$,0)
    CALL check_file(filename$,status)
    IF status=0 then
        CALL TDX_message("ERROR","File name not
valid", "Continue",1,result)
    ELSE
        CALL TC_edit_SetText(editid,filename$)
    END IF
END SUB

SUB check_file(filename$,status)
    LET status=0
    IF filename$>"" then
        WHEN ERROR IN
            OPEN #12: name filename$,access input,org
byte,create old
            ASK #12: FILESIZE bytes
            CLOSE #12
        USE
            LET bytes=-1                ! file doesn't exist
            CLOSE #12
    END IF
END SUB

```

```

        END WHEN
        IF bytes>0 then
            LET status=1
        END IF
    END IF
END SUB

SUB makemain
    MAT READ menus$
    DATA File,quit
    CALL TC_getscreensize(scl,scr,scb,sct)
    LET midx=(scl+scr)/2
    CALL TC_Menu_Set (0, menus$(,))
    CALL TC_Win_SetTitle (0,"Icon Exchange Program")
    ! CALL TC_setRect(0,midx-250,midx+250,350,100)
    CALL TC_setRect(0,midx-250,midx+250,400,50)

    CALL TC_graph_create(heading,"IMAGE",10,330,85,10)
    CALL TC_graph_create(logo,"IMAGE",340,480,80,20)
    CALL TC_graph_SetImageFromBOX(heading,headbox$)
    CALL TC_graph_SetImageFromBOX(logo,logobox$)

    LET y0=100

    CALL TC_Stext_create(oldtex,"Old icon
filename:",10,160,-99999,y0)
    CALL TC_Edit_create(oldicon,blank$,170,390,-
99999,y0)
    CALL
TC_PushBtn_create(browse3,"Browse",400,480,y0+24,y0+4)
    LET y0=y0+40

    CALL TC_Stext_create(tex,"EXE program
filename:",10,160,-99999,y0)
    CALL TC_Edit_create(user,blank$,170,390,-99999,y0)
    CALL
TC_PushBtn_create(browse1,"Browse",400,480,y0+20,y0)

    LET y0=y0+40
    CALL TC_Stext_create(iconctx,"New icon
filename:",10,160,-99999,y0)
    CALL TC_Edit_create(iconuser,original$,170,390,-
99999,y0)

```

```

CALL
TC_PushBtn_create(browse2,"Browse",400,480,y0+24,y0+4)

    LET y0=y0+40
    CALL TC_Stext_create(message1,"This program allows
the user to change the default icon",10,390,-99999,y0)
    CALL TC_Stext_create(message2,"(the True BASIC
toolkit) to any other 32x32 pixel icon.",10,390,-
99999,y0+20)
    CALL TC_Stext_create(message3,"The *.ico file must
be 16 color and must contain 766 bytes.",10,390,-
99999,y0+40)
    CALL
TC_PushBtn_create(begin,"BEGIN",400,480,y0+20,y0)
    LET y0=y0+40

    CALL
TC_PushBtn_create(quit,"QUIT",400,480,y0+20,y0)
END SUB

SUB refresh_fields
    CALL TC_edit_settext(user,blank$)
    CALL TC_edit_settext(iconuser,blank$)
    CALL TC_edit_settext(oldicon,original$)

END SUB

END

SUB get_icon_location(icon$, prog$, loc)
    REM: The original version was not foolproof
    REM: the icon data could span two adjacent resize
chunks
    REM: therefore if first pass fails then repeat at
half chunks

    LET loc= 0
    LET here= 0
    LET chunk=32768

    FOR pass=1 to 2

```

```

        IF pass=2 then
            LET chunk=16384          ! read half chunk
        END IF
        OPEN #1: name prog$, access outin, create
newold, org byte

        DO while more #1
            SET #1: RECSIZE chunk
            READ #1: chunk$
            LET position = pos(chunk$, icon$)
            IF position>0 then
                IF loc>0 then
                    CALL TDX_message("ERROR","Byte
sequence is not unique","Abort",1,result)
                    LET loc=-1
                    LET pass=2
                    CLOSE #1
                    EXIT DO

                END IF
                LET loc= here + position
                LET pass=2          ! no need for second
pass
            END IF
            LET here=here+chunk
            LET chunk=32768        ! reset chunk size to
normal
        LOOP
        CLOSE #1
    NEXT pass

    IF loc=0 then
        CALL TDX_message("ERROR","Icon string not found
","Abort",1,result)
    END IF

END SUB

SUB set_icon_location(icon$, prog$, loc)
    REM: puts new icon in EXE file at loc

    LET iconsize=len(icon$)

```



```

        OPEN #1: name prog$, access outin, create newold,
org byte
        SET #1: RECORD loc
        SET #1: RECSIZE iconsize
        WRITE #1: icon$
        CLOSE #1

END SUB

SUB read_icon_bytes(icon$, infile$)
    LET icon$=""

    OPEN #11: name infile$, access outin, create
newold, org byte

        ASK #11: FILESIZE total
        IF total>766 then
            CALL TDX_message("ERROR","Icon string too
long","Abort",1,result)
            CLOSE #11
            EXIT SUB
        END IF
        SET #11: RECSIZE total
        READ #11: icon$
        CLOSE #11
END SUB

```

BOXSTRINGS Library Module

Overview

Before True BASIC can display a graphics image it must first be converted to an internal format used by True BASIC that is specific to the operating system, and specific to the language series. In other words you cannot interchange internal images across platforms or from Windows to DOS.

This conversion can be carried out with the built-in routine:

```
CALL read_image(type$, boxstring$, filename$)
```

e.g., CALL read_image("MS BMP", boxstring\$, "myfile.bmp")
or CALL read_image("JPEG", boxstring\$, "myfile.jpg")

Displaying boxstring images is done with other built-in statements, e.g.,

```
ASK PIXELS xpix,ypix  
SET WINDOW 0,xpix-1,ypix-1,0  
BOX SHOW boxstring$ AT left,bottom
```

Where left=pixel position across the screen from the left edge
Where bottom=pixel position up from the bottom edge of the screen.

Boxstring images display much faster than reading other formats such as BMP or JPG simply because they don't need to be converted. For animated sequences, smoother results can be obtained by reading BOX string files direct from a hard drive. It is worth doing the preliminary conversion from say JPG images to BOX string format, in order to achieve high speed smooth "frame by frame" animation. There is no data compression so boxstring files tend to be large.

This library module allows you to manipulate boxstrings without displaying them on screen. You can find out what color any pixel is just by specifying the left and bottom coordinates. Similarly, you can change the color of any pixel to any RGB value. Working on the boxstring directly is much quicker than displaying the image and asking what color the screen pixel is.

The library module also allows you to magnify or reduce the image size. You need to exercise great care when doing this when using large magnifications or reductions, because the image quality can suffer badly. Magnifications or reductions by a factor of less than 2 work reasonably well. If you intend to do any significant size changes, I would recommend that you pre-process your image using Photoshop or a similar package.

The library also has a routine that will write a boxstring to a file, and another that will read boxstrings from a file. Many other library modules use the convention of giving boxstring filenames the extension .TBX.

True BASIC internal image format

The format begins with a 64 byte header:

Bytes 1-8 are all zeros

Bytes 9-10 indicate the number of pixels across the image.

$\text{Pixels} = 256 * [10] + [9]$

Bytes 11-12 are zero

Bytes 13-14 indicate the number of pixels down the image.

$\text{Pixels} = 256 * [14] + [13]$

Bytes 15-20 are zero

Bytes 21-22 indicate the total bytes containing image data.

$\text{Total} = 256 * [22] + [21]$

Bytes 23-24 are zero

Byte 25 is always 40

Bytes 26-28 are zero
Bytes 29-30 indicate the number of pixels across the image.
 $\text{Pixels} = 256 \times [30] + [29]$
Bytes 31-32 are zero
Bytes 33-34 indicate the number of pixels down the image.
 $\text{Pixels} = 256 \times [34] + [33]$
Bytes 35-36 are zero
Byte 37 is always 1
Byte 38 is zero
Byte 39 is always 24
Bytes 40-44 are zero
Bytes 45-46 indicate the total bytes containing image data.
 $\text{Total} = 256 \times [46] + [45]$
Bytes 47-64 are zero

The total bytes below the header can also be calculated from the pixels across the image times the pixels down the image times 3. This is because each pixel is defined by their RGB values in 3 bytes. Each RGB byte can be any number from 0-255.

$\text{Total bytes} = 64 + (3 \times 4 \times \text{INT}(\text{width}+1)/4) \times \text{height} + 4$

This calculation allows for 'padding' each line to an even number of bytes.

The file format begins with the bottom left hand pixel in the image, and works from left to right and bottom to top. This is very similar to BMP format. Reading from left to right, the RGB values are entered in reverse order, i.e., BGR so bright red is shown as $[0][0][255]$ and dark blue is shown as $[170][0][0]$ (the brackets are used for clarity).

CAUTION: This format only applies to 24 bit images. Earlier versions of True BASIC used 8 bit images (16 color) so the boxstring format is NOT THE SAME.

Library routines

Here is a list of the routines in the module that we will develop knowing the image format.

SUB readBOXpixel(boxstring\$,x,y,r,g,b)

This routine returns the RGB values (integers 0-255) of the pixel at the coordinates x,y. The coordinates are measured from the left hand edge of the image and from the top edge of the image.

This is very similar to the built-in routine **CALL readCpixel** except that the image doesn't have to be displayed on screen.

```
SUB readBOXpixel(string$,x,y,r,g,b)

    REM: reads the r,g,b color values (integers 0-
255)
    REM: at the image co-ordinates x,y
    REM: x,y are virtual pixel co-ordinates (origin
top left)
    REM: as if the image had been displayed on
screen
    REM: equivalent to CALL readCpixel
    REM: first 64 bytes are the header

    LET width$=string$[9:10]
    LET height$=string$[13:14]
    CALL bytepair(width$,width)
    CALL bytepair(height$,height)
    REM: calculate nearest even number for width
    LET even=4*INT((width*3+3)/4)
    LET r=0
    LET g=0
    LET b=0
```

```

        REM: convert vertical co-ordinate to TB image
convention
        LET row=height+1-y ! image is upside down
        REM: convert horizontal co-ordinate to 3 byte
locations
        IF x>0 then

                LET column=3*(x-1)+64+(row-1)*even
                LET b=ord(string$[column+1:column+1])
                LET g=ord(string$[column+2:column+2])
                LET r=ord(string$[column+3:column+3])
        END IF
END SUB

```

SUB writeBOXpixel(boxstring\$,x,y,r,g,b)

This routine inserts the RGB integer values of a pixel color for the pixel located at the co-ordinates x,y. The coordinates are measured from the left hand edge and from the top of the image. The new RGB values are inserted directly into the boxstring and not into an image on screen.

```

SUB writeBOXpixel(string$,x,y,r,g,b)
        REM: inserts the r,g,b color values (integers
0-255)
        REM: at the image co-ordinates x,y
        REM: x,y are virtual pixel co-ordinates (origin
top left)
        REM: as if the image had been displayed on
screen
        LET width$=string$[9:10]
        LET height$=string$[13:14]
        CALL bytepair(width$,width)
        CALL bytepair(height$,height)
        REM: calculate nearest even number for width
        LET even=4*INT((width*3+3)/4)
        !           LET even=2*INT((width*3+1)/2)
        REM: convert vertical co-ordinate to TB image
convention
        LET row=height+1-y
        REM: convert horizontal co-ordinate to 3 byte
locations
        IF x>0 then
                LET column=3*(x-1)+64+(row-1)*even

```

```

        LET string$[column+1:column+1]=chr$(b)
        LET string$[column+2:column+2]=chr$(g)
        LET string$[column+3:column+3]=chr$(r)
    END IF
END SUB

```

SUB boxstring_size(boxstring\$,width,height)

This routine returns the image width and height in pixel units. This is derived from the header.

```

SUB boxstring_size(string$,width,height)
    LET width$=string$[9:10]
    LET height$=string$[13:14]
    CALL bytepair(width$,width)
    CALL bytepair(height$,height)
END SUB

```

SUB magnifyBOX(boxstring\$,factor)

This routine magnifies the image size by a *factor*, where the multiplying factor is greater than 1. Image quality is generally too poor when the multiplying factor is 2 or greater. The method duplicates some of the horizontal rows of pixels to match the new height. In a photograph, these extra rows are hardly detectable. The same method is employed for vertical columns of pixels. The alternative is to “blur” two lines together which is a lengthy process. For better results use Photoshop or similar.

```

SUB magnifyBOX(string$,magnify)

    REM: extracts first 64 bytes
    REM: modifies width and height data by
magnification
    REM: modifies total bytes for magnifies image
    REM: takes 3 byte blocks and duplicates
(magnify) times
    REM: re-assembles string

    REM: BOX string must be in 24 bit format

    LET temp$=string$[1:64]
    LET length=len(string$)

```

```

LET end$=string$(length-3:length)
LET width$=string$(9:10)
LET height$=string$(13:14)
CALL bytepair(width$,width)
CALL bytepair(height$,height)
REM: calculate nearest even number for width
LET even=4*INT((width*3+3)/4)
CALL makepair(pair$,magnify*width)
LET temp$(9:10)=pair$
LET temp$(29:30)=pair$
CALL makepair(pair$,magnify*height)
LET temp$(13:14)=pair$
LET temp$(33:34)=pair$
REM: calculate new total bytes
LET neweven=4*INT((magnify*width*3+3)/4)
LET total=INT(neweven*magnify*height)
LET newheight=INT(magnify*height)
CALL makepair(pair$,total)
LET temp$(21:22)=pair$
LET temp$(45:46)=pair$
LET rcount=0

FOR row=1 to height
  REM: convert vertical co-ordinate to TB
image convention
  LET row$=""
  LET count=0
  FOR c=1 to 3*width step 3 !width
    REM: convert horizontal co-ordinate to
3 byte locations
    LET column=c+63+(row-1)*even
    LET color$=string$(column+1:column+3)
    LET count=count+1

    LET duplicate=INT(count*magnify)-
INT((count-1)*magnify)

    LET row$=row$ &
repeat$(color$,duplicate)

  NEXT c

  LET factor=neweven-len(row$)
  IF factor>0 then

```



```

        LET row$=row$ & repeat$(chr$(0),factor)
    END IF

    LET repeater=INT(row*magnify)-INT((row-
1)*magnify)
    IF row<height then

        FOR n=1 to repeater
            LET rcount=rcount+1
            LET temp$=temp$ & row$
        NEXT n
    ELSE
        FOR n=rcount+1 to newheight
            LET temp$=temp$ & row$
        NEXT n

    END IF

    NEXT row
    LET temp$=temp$ & end$
    LET string$=temp$

END SUB

```

SUB reduceBOX(boxstring\$,factor)

This routine reduces the size of the image by a *factor*, where the multiplying factor is less than 1. Image quality is generally too poor when the multiplying factor is 0.5 or less. The method involves deleting some rows of pixels until the height matches the new height. Again, in photographs the difference is undetectable. The same method is employed for vertical columns of pixels. The alternative is to “blur” two lines together, which is a lengthy process. For better results use Photoshop or similar.

```

SUB reduceBOX(string$,reduce)
    REM: extracts first 64 bytes
    REM: modifies width and height data by
magnification
    REM: modifies total bytes for magnifies image
    REM: takes 3 byte blocks and eliminates columns
and rows to fit reduction

```

```

REM: re-assembles string
REM: BOX string must be in 24 bit format

LET temp$=string$[1:64]
LET length=len(string$)
LET end$=string$[length-3:length]
LET width$=string$[9:10]
LET height$=string$[13:14]
CALL bytepair(width$,width)
CALL bytepair(height$,height)
REM: calculate nearest even number for width
LET even=4*INT((width*3+3)/4)
CALL makepair(pair$,INT(reduce*width))
LET temp$[9:10]=pair$
LET temp$[29:30]=pair$
CALL makepair(pair$,INT(reduce*height))
LET temp$[13:14]=pair$
LET temp$[33:34]=pair$
REM: calculate new total bytes
LET neweven=4*INT((reduce*width*3+3)/4)
LET total=INT(neweven*reduce*height)
LET newheight=INT(reduce*height)
CALL makepair(pair$,total)
LET temp$[21:22]=pair$
LET temp$[45:46]=pair$
LET rcount=0

FOR row=1 to height
    REM: convert vertical co-ordinate to TB
image convention
    LET row$=""
    LET count=0
    LET reduction=0
    FOR c=1 to 3*width step 3 !width
        REM: convert horizontal co-ordinate to
3 byte locations
        LET column=c+63+(row-1)*even
        LET color$=string$[column+1:column+3]
        LET count=count+1
        LET reduction=INT(reduce*count)
        IF reduction>lastreduction then
            LET row$=row$ & color$
        END IF
    LET lastreduction=reduction

```

```

NEXT c
LET factor=neweven-len(row$)
IF factor>0 then
    LET row$=row$ & repeat$(chr$[0],factor)
END IF

LET rcount=rcount+1
LET reducerow=INT(reduce*rcount)
IF reducerow>lastreducerow then
    LET temp$=temp$ & row$
END IF
LET lastreducerow=reducerow

NEXT row

LET temp$=temp$ & end$
LET string$=temp$

END SUB

```

SUB rotateBOX(boxstring\$,rotate\$,direction)

This routine will rotate the image by 90 degrees. If *direction* is any positive number, e.g., +1, then the image will be rotated clockwise. If *direction* is any negative number, e.g., -1, then the image will be rotated counterclockwise.

```

SUB rotateBOX(string$,rotate$,direction)
    REM: extracts first 64 bytes
    REM: exchanges width and height data
    REM: modifies total bytes for new format
    REM: takes 3 byte blocks and duplicates data in
new format
    REM: re-assembles string
    REM: BOX string must be in 24 bit format
    REM: rotates image ANTICLOCKWISE when
direction<0 CLOCKWISE when direction>0

    LET temp$=string$[1:64]
    LET length=len(string$)
    LET end$=string$[length-3:length]
    LET width$=string$[9:10]
    LET height$=string$[13:14]

```

```

CALL bytepair(width$,width)
CALL bytepair(height$,height)
REM: calculate nearest even number for
newwidth=height
LET oldeven=4*INT((width*3+3)/4)
CALL makepair(pair$,height)
LET temp$[9:10]=pair$
LET temp$[29:30]=pair$
CALL makepair(pair$,width)
LET temp$[13:14]=pair$
LET temp$[33:34]=pair$
REM: calculate new total bytes
LET neweven=4*INT((height*3+3)/4)
LET total=INT(neweven*width)
LET newheight=width
LET newwidth=height
CALL makepair(pair$,total)
LET temp$[21:22]=pair$
LET temp$[45:46]=pair$
LET rcount=0
REM: now transpose columns into rows
LET revtemp$=""

FOR c=1 to 3*width step 3      !width
  REM: convert vertical co-ordinate to TB
image convention
  LET row$=""
  LET count=0
  REM: set up template
  LET row$=row$ & repeat$(chr$[0],neweven)
  IF direction<0 then

    FOR row=height to 1 step-1
      REM: convert horizontal co-ordinate
to 3 byte locations
      LET column=c+63+(row-1)*oldeven
      LET
color$=string$(column+1:column+3)
      LET row$[count+1:count+3]=color$
      LET count=count+3
    NEXT row

    LET temp$=temp$ & row$
  ELSE

```

```

        LET revrow$=""
        FOR row=height to 1 step -1 !height
            REM: convert horizontal co-ordinate
to 3 byte locations
            LET column=c+63+(row-1)*oldeven
            LET
color$=string$[column+1:column+3]
            LET revrow$=color$ & revrow$
            LET count=count+3
        NEXT row

        LET lenrev=len(revrow$)
        LET row$[1:lenrev]=revrow$
        LET revtemp$=row$ & revtemp$
    END IF
NEXT c

    LET temp$=temp$ & revtemp$ & end$
    LET rotate$=temp$
END SUB

```

SUB readBOXfile(filename\$,boxstring\$)

This routine allows the user to read an image file containing a boxstring. It is a common convention to use the filename extension .TBX for boxstring image files. This routine will only read True BASIC boxtring image files.

```

SUB readBOXfile(filename$,string$)

    REM: reads the specified file as a BYTE file
    REM: returns the image in string$

    CALL check_file_status(filename$,status)

    IF status>0 then
        OPEN #12: name filename$,access outin,org
byte,create newold

        ASK #12: FILESIZE bytes
        SET #12: RECSIZE bytes
        READ #12: string$
    
```

```

        CLOSE #12

    END IF

END SUB

```

It is important to remember that this routine only reads the bytes in the file. It does not display the bytes on screen, which can only be achieved with a BOX SHOW statement.

The whole purpose of the BOXSTRINGS library module is not to show the image on screen. Any changes to the size, shape, or pixel colors in the image is done invisibly.

SUB writeBOXfile(filename\$,boxstring\$)

This routine allows the user to write a boxstring directly to a file. It is a common convention to use the filename extension .TBX for boxstring image files. This routine will only write True BASIC boxtring image files.

```

    SUB writeBOXfile(filename$,string$)
        REM: sends the image string (string$) to the
        specified file

        REM: the file format is BYTE

        CLOSE #12
        OPEN #12: name filename$,access outin,org
byte,create newold
        ERASE #12
        LET length=len(string$)
        SET #12: RECSIZE length
        WRITE #12: string$
        CLOSE #12

    END SUB

```

SUB check_file_status(filename\$,status)

This is a general purpose routine that checks if a file exists and how big it is. The parameter *status* returns zero if the file does not exist. If the file exists then the parameter *status* returns the size of the file in bytes.

```
SUB check_file_status(filename$,status)
  LET status=0
  IF filename$>"" then
    WHEN ERROR IN
      OPEN #12: name filename$,access
input,org byte,create old
      ASK #12: FILESIZE status
      CLOSE #12
    USE
      CLOSE #12
    END WHEN
  END IF
END SUB
```

These last three routines unpack and pack the color bytes into decimal values. If you use:

```
ASK MAX COLOR colormax
```

Your computer will tell you how many colors it can handle. Mine says 255.

24 bit BMP images allow for many more colors, but we only need to convert the first two bytes of the color number into a decimal value because True BASIC can only handle 256 maximum colors, i.e., 16 bits.

These last three routines are only used internally by the module so they could be DECLARED PRIVATE.

The first routine calculates the decimal value of any pair of byte\$.

```

SUB bytepair(pair$,value)
! in theory we should convert all 24 bits not just 16
  LET byte1$=pair$(1:1)
  LET value1=unpackb(byte1$,1,8)
  LET byte2$=pair$(2:2)
  LET value2=unpackb(byte2$,1,8)
  LET value=value1+value2*256
END SUB

```

The second routine does exactly the reverse, i.e., it takes any decimal value and converts it to two 8 bit bytes (pair\$)

```

SUB makepair(pair$,value)
! in theory we should convert all 24 bits not just 16
  LET modbit=int(value/256)
  LET rembit=value-256*modbit
  CALL valuetobyte(byte2$,modbit)
  CALL valuetobyte(byte1$,rembit)
  LET pair$=byte1$ & byte2$
END SUB

```

Finally the last routine packs any value into an bit byte:

```

SUB valuetobyte(byte$,value)
  CALL packb(byte$,1,8,value)
END SUB

```

A demonstration program called BOXSTRINGS_DEMO.TRU is available from the True BASIC website. This program demonstrates some of the features in the BOXSTRING library module.

PART VI

INDEX

Note: Only the text in the printed book is indexed. The code examples are not included in the INDEX.

24 bit images, 451
2nd speech center, 307
3D views, 294
8 bit images, 451
About True BASIC, 41
Abs, 120-121
Accessing files, 192
Acos, 126
Active windows, 337
Add new, 43
Administrator rights, 18
Adobe reader, 41
Algebra, 51-52
Algorithm, 310
Alias command, 49
Alias menu, 39
Alias pathnames, 39
Aliases, 39, 48, 187
Alt key, 27
Ampersand sign, 22, 54
And, 60
Angle, 126
Animate, 295
Animated sequences, 448
Animation, 303, 448
Announce dialog box, 418, 427
Ansi basic, 269
Append data to files, 189, 195
Application studies, 438
Areas, 279
Arguments and parameters, 255
Arithmetic addition, 54
BMP, 398, 448
Arithmetic examination, 318
Arithmetic operators, 58, 59
Arrays, 56, 94, 219, 223
Ascending order, 78, 224, 226
 Ascii character set, 60, 211
Ascii key code, 66
Ascii table, 98
Asin, 126
Ask max color, 461
Ask max cursor, 136, 147, 275
Ask pixels, 274, 297, 332
Ask, 448
Asterisk, 58, 436
Atn, 126
Autolinenumber, 47
Automatic program code, 400
Back plate, 392
Back-up copy, 37
Backup on save, 37
Bad outline, 312
Bar scale, 394
Bar scales, 327
Base, 225, 226
Before, 435
Begin, 341
Bezel, 392
Binary search, 222
Binary trial, 225
Bind, 34
Binder, 38
Binding process, 35
Block comments, 45
Blue prints, 296
Blur, 298
BMP format, 273, 450
Call readcpixel, 451

Bold italic, 343	Call read_image, 448
Bold, 343	Call statement, 249, 252
Bomb-proof, 109	Call tc_getscreensize, 332
Boolean expressions, 59	Call tc_init, 425
Bound program, 33	Call tc_setunitstopixels, 331
Bound version, 438	Call tc_win_switch, 338
Box area, 284, 330	Call tdx_init, 425
Box circle, 284	Calling a subroutine, 249
Box clear, 284	Can't undo, 30
Box ellipse, 284	Cancel, 341
Box keep, 276, 285	Caret, 58
Box lines, 284	Case else, 74
Box show boxstring\$ at, 448	Cd-rom disk, 17
Box show statement, 134	Ceil, 120, 121
Box show, 135, 284, 329	Cent, 318
Boxdata, 134	Center, 344
Boxlib, 276, 289	Chain statement, 18, 29, 264
Boxstrings library, 448	Chain windows app, 30
Brackets, 59	Chain, 18, 264, 307
Breakpoint, 33, 37	Change, 31
Brief outline, 21	Changedir, 428
British, 318, 325	Channel numbers, 195
Bronze_tc, 326	Channel zero, 187
Bronze_td, 418	Channel, 187-190
Browser push button, 439	Chapter number, 63
Btrees version, 215	Character positions, 55
Bubble sort, 219, 220	Checkbox, 344, 360
Built-in functions, 70	Child window, 391
Built-in icon, 438	Chr\$, 99
Built-in routine, 36	Classroom schedule, 236
Built-in tools, 91	Clear, 63
Byte file, 198, 204	Clipboard, 41
Byte, 188	Close all windows, 30
Bytepair, 263, 461	Close, 27, 42
Cad applications, 296	Closing files, 192
CAD systems, 294-295	Coins, 317
CAD/CAM drawings, 273, 295	Colon, 65
Calculators, 317	Color text extension, 352
Call ct_showall, 353	Color, 279, 437
Call getsize, 216	Colortext, 39
Call name, 249	
Column ruling, 386	Ctrl-x, 30

Comma separated variables, 189	Ctx draws, 353
Comma, 140, 163, 166, 189, 216, 77, 79	Ctx library, 147, 286, 327, 352
Command line, 20, 47	Ctx object, 405
Comments, 21	Ctx, 274
Commercial applications, 25	Ctx_barscale_demo, 397
Compile errors, 269	Ctx_checkbox_demo, 362
Compile, 18, 23, 34	Ctx_edit_demo, 367
Compiling process, 34	Ctx_elevator_demo, 394
Complex algorithm, 314	Ctx_list_demo, 379
Compounding words, 25	Ctx_pushbtn_demo, 369
Computer aided design, 294	Ctx_radiogroup_demo, 363
Computer controlled-machining, 295	Ctx_rmenu_demo, 383
Con, 183	Ctx_sbar_demo, 371
Concatenation, 54, 218	Ctx_stext_demo, 360
Conditional statement, 59	Ct_barscale_click, 395, 397
Confirm quit, 37	Ct_barscale_create, 394
Confirm, 436	Ct_barscale_get, 396
Contents list, 45	Ct_barscale_linktext, 397
Contents menu, 40	Ct_barscale_reset, 396
Contents, 44	Ct_barscale_set, 396
Control deselected, 329	Ct_barscale_setcolor, 395
Controls and objects, 335	Ct_barscale_setincrement, 395
Conversion functions, 98	Ct_barscale_setrange, 396
Convert strings to-numbers, 55	Ct_bipolarbarscale-setcolor, 396
Copy, 30, 42	Ct_checkbox_create, 360
Copy_file, 262	Ct_checkbox_get, 361
Core instructions, 85	Ct_checkbox_set, 361
Cos, 126	Ct_clear, 357
Cosh, 126	Ct_edit_askcurrent, 364
Cot, 126	Ct_edit_create, 364
Counting backwards, 78	Ct_edit_gettext, 367
Courier 10 point, 29	Ct_edit_setcharacters, 364
Cpos, 109-111	Ct_edit_setcheck, 365
Cposr, 111	Ct_edit_setrecess, 367
Creating files, 192	Ct_edit_settext, 366
Csc, 126	Ct_elevatorbtn_create, 392
Csv files, 189, 220, 383	Ct_elevatorbtn_get, 393
Ctrl-c, 31	Ct_elevatorbtn_set, 394
Ctrl-v, 31	Ct_elevatorbtn_setbezel, 393
Ct_elevatorbtn_setcolor,	Ct_rmenu_setcheck, 381

393
 Ct_erase, 353, 355
 Ct_event, 353, 365
 Ct_free, 353, 355
 Ct_get, 356
 Ct_getfont, 387
 Ct_getrect, 356
 Ct_gettext, 367
 Ct_groupbox_create, 390
 Ct_image_create, 398
 Ct_listbox_create, 371
 Ct_listbox_get, 373
 Ct_listbox_set, 373
 Ct_listbtn_create, 373
 Ct_listbtn_get, 375
 Ct_listbtn_set, 376
 Ct_listedit_create, 376
 Ct_listedit_get, 378
 Ct_listedit_set, 378
 Ct_list_create, 371
 Ct_list_get, 372, 373-378
 Ct_list_getcolor, 372, 373-379
 Ct_list_getfont, 373-379
 Ct_list_set, 372-379
 Ct_list_setcolor, 372-379
 Ct_list_setfont, 372-379
 Ct_locateimage, 398
 Ct_pushbtn_create, 367
 Ct_pushbtn_getstatus, 369
 Ct_pushbtn_setdim, 369
 Ct_pushbtn_settoggle, 369
 Ct_radiogroup_create, 362
 Ct_radiogroup_on, 363
 Ct_radiogroup_set, 363
 Ct_radiogroup_settext, 363
 Ct_rmenu_get, 381
 Ct_rmenu_getcheck, 382
 Ct_rmenu_getcolor, 382
 Ct_rmenu_getenable, 382
 Ct_rmenu_getfont, 383
 Ct_rmenu_gettext, 381
 Ct_rmenu_set, 379
 Ct_table_writearray, 389
 Ct_rmenu_setcolor, 382
 Ct_rmenu_setenable, 382
 Ct_rmenu_setfont, 383
 Ct_rmenu_settext, 381
 Ct_sbar_create, 369
 Ct_sbar_getincrements, 370
 Ct_sbar_getposition, 370
 Ct_sbar_getrange, 370
 Ct_sbar_setincrements, 370
 Ct_sbar_setposition, 370
 Ct_sbar_setrange, 370
 Ct_select, 357, 364
 Ct_sensitize, 357-363, 368, 391
 Ct_set, 356
 Ct_setcolor, 358-390
 Ct_setfont, 358-387
 Ct_setimage, 398
 Ct_setlist, 371, 376, 378
 Ct_setrect, 356
 Ct_settext, 359-361, 366-368
 Ct_settextjustify, 359
 Ct_setunitstopixels, 355
 Ct_setunitstousers, 355
 Ct_show, 353, 356
 Ct_showall, 357
 Ct_stext_create, 358
 Ct_stext_setcharcolor, 360
 Ct_stext_setrecess, 359
 Ct_table_create, 384
 Ct_table_dataformat, 377, 387
 Ct_table_dataorder, 388
 Ct_table_getcolor, 387
 Ct_table_getdata, 388
 Ct_table_getrow, 388
 Ct_table_insertdata, 388
 Ct_table_readarray, 385, 389
 Ct_table_readfile, 385, 389
 Ct_table_removedata, 388
 Ct_table_setcolor, 386
 Ct_table_setdata, 388
 Ct_table_setrow, 389
 Demonstration programs, 17

Ct_table_writefile, 389	Descending, 78, 224
Cumulative sum, 320	Det, 183
Currency system, 317	Dialog box, 33, 418
Current button position, 67	Dialog_test, 418
Current cursor position, 31	Dim statement, 149
Current version number, 20	Dimension, 57
Cut, 30, 42	Dinarius, 318
Data compression, 448	Direction of rotation, 289
Data statement, 134	Directory, 187
Data statements, 132, 155	Discard, 43
Data table object, 385	Disk drive, 62
Data table, 147, 148, 327, 383	Divide, 120, 121
Data, 51	Do format command, 315
Database programs, 214	Do format option, 72
Database, 215, 231	Do format, 24, 36, 79
Date\$, 128	Do loop, 36, 80, 119
Date, 21, 22, 129	Do lower, 36
Dates & time, 128	Do upper, 36
Debug mode, 33, 35, 37	Do, 36
Debug, 24, 311	Docs folder, 41
Decimal currency, 318, 325	Dollar sign, 54, 140
Decimal notation, 319	Dos command level, 266
Decimal numbers, 319	Dos convention, 187
Decimal value, 461	Dos type system command-
Decisions, 68	window, 264
Declare def valstr, 104	DOS, 18
Declare def, 93, 243	Dot matrix printer, 137
Declare public, 261, 262	Dot, 183
Declare statement, 246	Dotted outline, 345
Declare sub-routines, 248	Double precision-
Declare private, 461	floating point, 51
Def valstr, 101	Down, 368
Def, 103, 240, 246, 258, 262	Download, 63
Default condition, 37, 38,	Drag-and-drop interface, 41
330	Draw, 276, 292
Default directory, 33	Drawing with box-
Default font, 38	statements, 283
Default location, 17	Drop down index, 40
Default page color, 38	Drop-down list, 44
Default value, 77	Dynamically re-dimension-
Default, 18, 19	arrays, 57
Deg, 125	Dynamically redimension,
Delete key, 63	150
Edit , 42	Event\$=extend, 336

Edit field, 329, 339, 363	Event\$=hide, 336
Edit menu, 30	Event\$=hscroll, 347
Edit menu, 63	Event\$=keypress, 351
Editor window, 63	Event\$=left, 347
Elapsed time, 129	Event\$=menu, 339
Elevator button, 392, 327	Event\$=mouse, 336
Else, 68	Event\$=pagedown, 348
Elseif, 70, 71	Event\$=pageleft, 347
Embed an image, 134	Event\$=pageright, 348
Embedded icon data, 439	Event\$=pagup, 348
End def, 103, 242	Event\$=refresh, 336
End edit, 355, 377	Event\$=right, 347
End function, 242	Event\$=select, 336
End if, 71	Event\$=single, 335
End module, 256, 261	Event\$=size, 336
End of a line, 22	Event\$=up, 348
End picture, 291	Event\$=vscroll, 348
End statement, 92, 103, 256	Excel spreadsheet, 30
End sub, 248	Exception to the rule, 57
End when, 269, 270	Exchange rate, 319
End, 23, 63, 292	Exclamation mark, 21
Eps, 120, 121	Execlib library, 152
Epson printers, 211	Execlib, 213
Erange, 348	Executable application, 34
Erase a text file, 189	Executable program, 134, 438
Erase, 63, 64	Exec_readdir, 153
Error detection, 23	Exit char, 351
Error handler block, 270	Exit def, 96, 242
Error handling, 269	Exit do, 80
Error message, 66	Exit for, 79
Error trapping, 269	Exit picture, 291
Error, 269	Exit sub, 96, 249
Error-free programs, 312	Exit the editor, 21
Error-free, 311	Exit, 30
Esc, 351	Exline\$, 272
Escape conditions, 81	Exline, 272
European union, 325	Exp, 121
Event handling, 327	Exponential notation, 142
Event\$=control, 339-342, 345-351	Exponentiation, 58
Event\$=double, 335	Expression, 253, 255
Event\$=down, 348	Extension speakers, 306
Event\$=end, 347, 348	Extension txt, 189
Extension\$, 428	Full path, 187

External functions, 238
 External is optional, 246
 External libraries, 256
 External procedure, 92
 External program, 23, 36, 134
 External subroutines, 247, 248
 External, 33, 34, 256, 261
 Extent\$, 270, 272
 Extra line, 63
 Extra lines of code, 32
 Extreme conditions, 312
 Exttype, 270, 72
 Factorial value, 243
 Fields and arrays, 200
 File channel numbers, 94
 File handling, 186, 419, 427
 File menu, 27
 File, 27, 42, 188, 221, 229
 Find again, 31
 Find operation, 31
 Find, 31
 Fixed pitch fonts, 137
 Flashing cursor, 19
 Floating Point, 51
 Flood statement, 287
 Flood, 286
 Flow of logic, 68
 Fnmlib, 260
 Fonts, 436
 Fontsize, 343
 For next step, 76
 for your own benefit, 27
 Format string, 139, 144
 Format, 23
 Formatting data, 137
 Forms program, 328, 400
 Forms, 40
 Forms, 45
 Fp, 122
 Fractions, 317, 319
 Frame by frame, 448
 Full alphabetical index, 40
 Hotstart, 21, 38
 Function keys, 39, 47
 Function parameters, 240
 Function statements, 241
 Function unit heading, 239
 Function, 44, 92, 246
 Fundamentals of drawing, 276
 General purpose, 355, 419, 426
 General reading, 197
 General writing, 203
 Generates the program-code, 41
 Get key, 66, 72, 85, 209
 Get mouse, 67
 Get, 339
 Getting started, 17
 Get_all_files, 263
 Get_size, 262
 Gigabyte, 186
 Golden rule, 24
 Good defensive-programming, 271
 Good habits, 25
 Good practice, 21
 Graphics screen, 274
 Graphics, 273
 Graphlib, 294
 Green arrow buttons, 20
 Group box, 390
 Handling strings, 319
 Hard drive serial number, 266
 Headings, 386
 Help file, 40, 96
 Help for True BASIC, 40
 Help window menu, 42
 Help, 40, 341
 Hides the window, 336
 Hierarchical structure, 379
 Highlight focus, 386
 Highly structured-programming, 328
 graphics, 298

Hruler, 384
 Hruler|vscroll|recess, 373
 Hruler|xpscroll|recess, 373
 Hscroll, 347
 Ico file, 438, 439
 Iconex icon exchange, 438
 Identities, 333
 Identity zero, 337
 Idn, 181
 IEEE standard 64 bit-format, 51
 If key input, 72
 If then else, 68
 If then structure, 72
 Image height, 452
 Image width, 452
 Images, 327, 397
 Import, 43
 Inadequate testing, 312
 Include, 32
 Increment the bar-position, 394
 Indent keywords, 23
 Indentation, 312, 313
 Indented, 72
 Information boxes, 19
 Initial value, 84
 Initialization, 425
 Initializing, 84
 Ink jet printer, 137, 296
 Input and output, 62
 Input instruction, 65
 Input prompt, 65, 116
 Input, 64, 192, 339
 Insert, 42, 63, 435
 Inserted, 32
 Installation process, 18
 Installer, 17-19
 Int, 121, 126
 Integers, 51, 319
 Interaction between-variables, 263
 Interactive computer-library modules, 23, 25, 96
 Internal format, 448
 Internal functions and-subroutines, 263
 Internal image format, 134, 449
 Internal motherboard-speaker, 306
 Internal procedure, 92, 263
 Internet browser, 20
 Initial letters, 392
 Initialize, 418
 Inv, 184
 Ip, 122
 Isolation, 237, 250
 Is_file_there, 262
 Italic, 343
 JPG format, 273, 368, 398, 448
 Justify the text, 344
 Keep, 31
 Key input, 85
 Key, 232
 Keyboard input, 64
 Keypress, 355, 366
 Keystroke, 87
 Keyword do, 81
 Keyword loop, 81
 Keywords, 315
 Kilobyte, 186
 Latest edition, 17
 Latest version, 22
 Lbound, 184
 Lcase\$, 60, 106
 Learning the basics, 50
 Left mouse button, 67
 Left\$, 107
 Left, 344, 368
 Legacy code, 137, 146
 Legacy programs, 39
 Len, 105, 107, 108
 Let, 25
 Libra, 318
 Libraries, 237, 255
 Manipulating images, 276

Library routines,451	Manufacturing drawings,
Library statement,260	295
Library, 91, 97-100,259	Mask,304
License,17,18,19	Mat input prompt,160
Line feed,188	Mat input,156,157
Line input,131	Mat line input,161,168
Line number,19	Mat print,162,166,167
Lines,277	Mat read,133,153,155,164,
Linking process,34	169
List box,349	Mat redim,150
List button,350	Mat redim,151
List edit,351	Mat statements,149
List of instructions,21	Mat write,169
List selected,355,372,375,	Matrices,149
377	Matrix addition and-
Listbtn,340	subtraction,175
Listed errors,32	Matrix and array-
Listing,29	functions,181
Lists,371,423,433	Matrix arithmetic,171
Literal strings,54	Matrix assignments,172
Local parameter variables,	Matrix multiplication-
246	and division,177
Local variable,241,261	Matrix redimensioning,174
Locating the do program,36	Maximum of 8 charcters,18
Location coordinates,331	Maxnum,123
Log,121	Megabyte,186
Log10,121	Menu bar,20,27,331
Logic flow control,76	Menu headings,27
Logical errors,269	Menu items,27
Logical expressions,60	Menu option,20,36
Logical operators,60	Menus,338
Logical windows,329	Messages,420,429
Long filenames,187,212	Metric system,325
Loop until,81	Mid\$,107
Lower case,23,24	Midway,225,227
Ltrim\$,106	Milling,295
Magnify,449	Min,123
Main program,235	Mistakes,23
Maintenance,311	Mod,122
Makepair,263,461	Modal window,424
Make_main,439	Modify code lines,46
Manipulate boxstrings,449	Modify,42,43
Manuals,41	Module header,260
Module initialization,260,	Num\$,98

262
 Module name, 260
 Module, 33, 34, 256, 260
 Monetary addition, 318
 Monetary value, 320
 Monitor, 62
 More input, 131
 More output, 136
 Mouse sensitive, 346
 Mouse, 89, 90
 Move to, 32
 Movies, 303
 Mp3, 306
 MS bmp, 368
 MS excel, 189, 264
 MS paint, 298
 MS word, 189, 264
 MSinstaller, 18
 Multiple dimensions, 57
 Multiple nested-loops, 313
 Multiplication symbol, 58
 Music, 307
 Name, 21, 187
 National test, 318
 Ncpos, 112
 Ncposr, 113
 Negative number, 354
 Nested for next loops, 79, 80, 313
 New colors, 372
 New, 27, 192
 Newold, 192
 Next, 76
 Nickel, 318
 No spaces, 18
 No, 341
 Not wrapped, 22
 Not, 60
 Notation, partial string, 107
 Notebook, 189
 Notepad, 67
 Nul\$, 184
 Page setup, 29
 Num, 98
 Number of characters, 38
 Number of lines, 38
 Numbers to strings, 55
 Numbers, 51
 Numeric array, 56
 Numeric constants, 53
 Numeric functions, 120, 121
 Numeric variables, 53
 Numerical addition, 319
 Objects, 331, 333
 Off by one bug, 312
 Old fashioned currency, 318
 Old, 192
 One instruction per line, 22
 One line word processor, 339
 Open file, 39, 198, 204
 Open, 27
 Opening files, 186
 Operators, 58
 Optical illusions, 299
 Option angle degrees, 125
 Option angle radians, 125
 Option statements, 262
 Or, 60
 Ord, 98
 Order of priority, 59
 Org random, 192
 Org, 188
 Organization, 188, 190
 Origin, 331-333
 Outer dimensions, 371
 Outin, 192
 Outline, 310
 Output, 192
 Overview, 418
 Overwrite the existing, 205
 Pack, 203, 461
 Pack_array, 263
 Pack_data, 216
 Padding, 450
 Print instruction, 64

Pair of byte\$, 461
 Palindromes, 245, 260
 Parameters, 92
 Parentheses, 253
 Partial strings notation, 107
 Partial strings, 55
 Passwords, 418, 435
 Paste, 31, 42
 Pdf document format, 42
 Period, 139
 Persistence of vision, 303
 Personal computers, 317
 Photographic type image, 299
 Photoshop, 298, 299, 499
 Physical window, 329, 331, 332
 Pi, 124
 Picture libraries, 294
 Picture procedures, 290
 Picture, 276, 290
 Pixel units, 274, 330-332
 Plain, 343
 Plan first program later, 310
 Plan first, 319
 Play, 307
 Plot area, 279
 Plot lines, 278
 Plot points, 277
 Plot statement, 275
 Plot text at, 283, 342
 Plot, 329
 Plus sign, 54, 320
 Point size, 332
 Points, 276, 343
 Pos function, 113, 114, 227
 Posr, 114
 Preferences, 38
 Preliminary conversion, 448
 Presentation, 319
 Principles, 326
 Read statement, 132
 Print using, 137-139, 142, 145
 Print, 29, 42, 62, 329, 342,
 Printer as text file, 210
 Printer driver, 211
 Printer, 62
 Private statement, 261
 Problem solving, 317
 Procedures, 91
 Profound difference, 326
 Program files, 17, 18
 Program statement, 265
 Program structure, 309, 311
 Program style, 312
 Program unit, 23, 235
 Program, 18, 19
 Programming for windows, 326
 Prompt, 65
 Prop, 349
 Proportional fonts, 137
 Protected block of-
 statements, 269
 Proud, 384
 Public statement, 261
 Public variables, 352
 Public, 261
 Punching, 295
 Purpose of the program, 21
 Push button, 367, 341
 Pwlib, 211, 273, 296
 Question mark, 65
 Quit, 341
 Rad, 125
 Radio group, 345, 362
 Random access files, 206, 214, 215
 Random, 191
 Randomize, 123
 Re-dimension the array, 57
 Re-initialized, 255
 Re-order, 218
 Read & data, 131
 Run, 23, 32, 63, 64

Read/write protected, 18
 Readme, 18
 Read_anyfile, 262
 Recent files, 36
 Recess, 384
 Record files, 214
 Record will be-
 overwritten, 208
 Record, 190
 Rectangular co-ordinates,
 353
 Redimension an array, 157
 Redo, 30
 Reduce, 449
 Reference manual, 25, 91, 92,
 130, 178
 Relational database, 231
 Relational operators, 59,
 60, 68
 Rem, 22
 Reminder notes, 21
 Rename, 29
 Repeat\$, 106
 Replace, 31
 Restore, 135
 Return key, 22, 65, 66
 Return, 63, 264, 351
 RGB value, 449
 Right click menu, 47, 327,
 379
 Right\$, 107
 Right, 344, 368
 Rnd, 123
 Root directory, 266
 Rotate, 292
 Rotating machine, 303
 Round, 121
 Row and column position, 57
 Rtrim\$, 106
 Rubber boxes, 276, 289
 Rule of thumb, 343
 Run demos, 42
 Run menu, 32, 62, 72
 Sgn, 122
 Run-time errors, 269, 271
 Rundo, 36
 Runs a program, 18
 Save as, 28
 Save on close, 30, 37
 Save, 28, 43
 Scale, 276, 292
 Screen output, 62
 Screen sizes, 332
 Scroll across, 22
 Scroll bar, 22, 347, 369
 Scroll sideways, 20
 Scroll up and down, 20
 Search box, 40
 Search unsorted arrays, 228
 Search_sorted_array, 262
 Search_sorted_fields, 262
 Search_sorted_file, 262
 Search_unsorted_array, 262
 Search_unsorted_fields, 262
 Search_unsorted_file, 262
 Sec, 126
 Select all, 32, 63
 Select case, 73
 Select the text, 29
 Select, 435
 Semi-colon, 65, 79, 166
 Sensitive to mouse clicks,
 358
 Serial number, 63
 Set back, 280
 Set color background, 280
 Set color mix, 281
 Set color, 279, 280
 Set cursor, 136, 275
 Set mode, 275
 Set window, 275, 332
 Set zonewidth, 163
 Set, 448
 Setfilerecord, 263
 Setting the cursor-
 position, 275
 Settings menu, 21, 33, 35, 37
 Stop button, 208

Share statement, 261
 Shear, 276, 292
 Shift, 276, 292
 Short cut icon, 18
 Short cuts, 39
 Show_help, 96
 Shut_down, 30
 Shutter, 303
 Side arrow keys, 27
 Simple script, 18
 Sin, 126
 Single character, 66
 Single right, 355, 383
 Single, 354, 371
 Sinh, 126
 Sit-on-top, 353
 Size changes, 449
 Size of the dialog box, 422
 Size of, 20
 Size, 185
 Skeleton, 327
 Smudge, 298
 Solidus, 318
 Sort data, 218
 Sorting, 219
 Sortlib, 219
 Sort, 262
 Sort_file, 262
 Sound and music, 306
 Sound, 306
 Spaces in a file path, 265
 Spaces, 265
 Speak, 307
 Special editor features, 45
 Specify the size, 206
 Spreadsheets, 57
 Sprites, 303
 Sqr, 124
 Strange, 348
 Statements, 44, 97
 Static text, 344, 358
 Static variables, 261
 Step, 77
 Target windows, 337
 Stop your program, 211
 Stopdata, 135
 Str\$, 100
 String array, 57
 String constants, 54
 String functions, 105
 String manipulation, 325
 String searching, 109
 String variables, 53
 Strings, 53
 Sub boxstring_size, 452
 Sub check_file_status, 459
 Sub exec_askdir, 213
 Sub exec_chdir, 213
 Sub exec_climkdir, 213
 Sub exec_diskspace, 213
 Sub exec_mkdir, 213
 Sub exec_readdir, 213
 Sub exec_rename, 213
 Sub exec_rmdir, 213
 Sub exec_setdate, 214
 Sub exec_settime, 214
 Sub magnifybox, 452
 Sub readboxfile, 459
 Sub readboxpixel, 451
 Sub reducebox, 455
 Sub rotatebox, 457
 Sub writeboxfile, 460
 Sub writeboxpixel, 452
 Sub-directory, 187
 Sub-menus, 379
 Sub-routines, 94, 320
 Sub-strings, 55, 255, 319, 360
 Subroutine parameters, 250
 Subroutine unit heading, 248
 Syntax errors, 269
 Syntax, 92
 Tab space, 79
 Tab, 146, 351
 Tables, 57
 Tan, 126
 Tanh, 127
 Td_message, 420

TB6007,17	Td_savefile,419
TB6007setup,17	Td_warn,421
Tbd,418	Td_yn,421
Tbdx,418	Td_ync,421
TBeditor files,189	Telephone ring tone,306
TBexitroutine,35	Temporary alias,39,47
Tbhelp folder,45	Test and debug carefully,
Tbilt,41	311
Tbstandard,400	Test drive,228
TBsystem531,18,187,212	Text and fonts,342
TBX,368,398	Text files,214
Tc sensitize,345	Text label,331,344
Tc_event responses,335	Text processors,19
Tc_event,90,327,328	Text to speech,307
Tdx extended dialog box,	Text,188,283
355	The printer,208
Tdx library module,424,425	Then,68
Tdx_announce,431	Three-column accounting,
Tdx_closeannounce,431	318
Tdx_color_select,437	Time\$,129
Tdx_font_select,436	Time,129
Tdx_getfile,427	Timeout,427
Tdx_getfile,428	Title,43
Tdx_input,432	Toggle action feature,33
Tdx_inputm,431	Toggle action switch,37
Tdx_lineinput,432	Toolbar,20
Tdx_list,433	Toolbox logo,438
Tdx_listm,434	Tools,91
Tdx_listorder,434	Top-down,326
Tdx_message,429	Topics title,40
Tdx_password,435	Total height of the-
Tdx_savefile,428,429	dialog box,423
Tdx_setbrush,427	Trace is normally-
Tdx_setfont,426	disabled,35
Tdx_setlocation,426	Trace,35,37
Tdx_setpen,426	Trigonometric functions,
Tdx_warn,430	125
Tdx_yn,430	Trim\$,106,118
Tdx_ync,430	Trn,182
Td_getfile,419	True or false,59,68
Td_input,423	True BASIC,97-100
Td_inputm,421	True BASIC editor,19,96
Td_list,423	True BASIC editor menus
True BASIC for windows,17	Using forms,405

True BASIC graphics, 274, 327
 True BASIC icon, 439
 True BASIC keyword, 26
 True BASIC toolbox icon, 134
 True BASIC website, 50, 63, 94, 135
 Truectrl library, 274
 Truectrl, 44, 326, 334
 Truectx, 45
 Truedial, 44, 418
 Truetdx, 45
 Truncate, 124
 Truncated, 386
 Turning, 295
 Two dimensional array, 57, 383
 Two-way parameters, 251
 TXT files, 45
 Type instructions, 20
 Ubound, 153, 184
 Ucase\$, 60, 105
 Underscore, 25, 63
 Understand the problem, 310, 318
 Undo the changes, 439
 Undo, 30, 435
 Unique features, 19
 Unpack, 200, 202, 203, 461
 Unpack_array, 262
 Unpack_data, 216, 262
 Unsave, 28
 Until, 81
 Untitled 1, 19
 Up and down arrow keys, 27
 Up, 368
 Updating your code, 25
 Upper case, 23, 24, 70
 USB memory device, 62
 Use of comments, 314
 Use, 269
 User input, 340, 421, 431
 User units, 274, 330
 User, 263
 Winxp, 373
 Using the editor, 44
 Using transforms with-pictures, 293
 Using\$, 145, 146
 Utility feature, 23
 Val, 93, 101, -103,
 Valstr, 92, 93, 104
 Valuable aid to debugging, 33
 Valuetobyte, 263, 462
 Variable name, 51, 314
 Variables, 51
 Vector data files, 295
 Vector files, 273
 Version, 20, 41
 Vertical bar, 216
 Vertical scroll bar, 371
 Vertical text, 287
 VGA colors, 281
 Vigesimal-duodecimal, 317
 Vol command, 266
 Vrule, 384
 Vscroll, 347
 Vtext, 263, 289
 Wav, 306
 Well-designed programs, 235
 When exception in, 269
 While, 81-83
 Whole numbers, 332
 Whole screen, 330
 Win98, 373
 Window menu, 36
 Window size, 274
 Window, 330
 Windows co-ordinates, 329
 Windows media player, 307
 Windows program structure, 400
 Windows push button, 299
 Windows vista, 17
 Windows, 18, 19, 335
 Winicon data, 439
 Winicon, 438
 programs, 309

Word document file,30	WTdata,228
Word,67	WTlib library module,89,
Worked examples,438	94,104,215
Working window,19	WTlib library,90,193,197,
Wrap,32	271,287
Write simple programs,311,	WTtext,208
319	WT_engine,303
Write_anyfile,262	WT_table_demo,229
Writing and using-	WT_vine,304
functions,243	Yes,341
Writing computer programs,	Zer,185
21	Zero element,384
Writing structured-	Zoom,299

LIST OF EXAMPLE PROGRAMS (Alphabetical)

ANALYSING INPUT, WT10_013
 ANALYSING INPUT,WT10_014
 ARRAYS PROMPT, WT16_008
 BINARY SORTING, WT18_015
 BOXLIB.TRU
 BOXTRINGS.TRU
 CENTER SUB-ROUTINE, WT19_006
 CHR\$, WT8_002
 COLOR GRADIENT, WT21_005
 CPOS SEARCHING, WT10_002
 CPOS START SEARCHING, WT10_004
 CPOSR SEARCHING, WT10_005
 CTX_BARSCALE_DEMO
 CTX_ELEVATOR_BUTTON DEMO
 DATA, WTDATA.csv
 DATA TABLE DEMO
 DECISIONS, WT5_001
 DEF VALSTR, WT8_005
 DIM ARRAY\$, WT16_004
 DO LOOP NESTED, WT6_008
 DO LOOP UNTIL, WT6_009
 DO LOOP WHILE, WT6_010
 DO LOOP, WT6_007
 DO UNTIL LOOP, WT6_011
 DO WHILE LOOP, WT6_012
 DRAWING AREAS, WT21_003
 DRAWING LINES, WT21_002

DRAWING PICTURES, WT21_009
ELAPSED TIME, WT13_002
ENGINEX, WT_ENGINEX
ENGINEX.EXE
ERASING DRAWINGS, WT21_004
ERROR HANDLING, WT20_001
EXEclib.trc
EXIT FOR, WT6_006
EXPONENTIAL NUMBERS, WT15_003
EXTERNAL FUNCTIONS, WT19_001
FACTORIAL FUNCTION, WT19_003
FLOOD, WT21_006
FOR NEXT NESTED, WT6_005
FOR STEP NEXT, WT6_001
FORMATTED STRINGS, WT15_004
FULL LSD PROBLEM, WT24_002
FUNCTIONS, WT19_002
FUNCTIONS, WT19_004
GET KEY, WT4_007
GROUP BOX DEMO
HD SERIAL NUMBER, WT19_012
ICONEX.EXE
IF THEN ELSE BETTER, WT5_004
IF THEN ELSE IMPROVED, WT5_003
IF THEN ELSE NESTED, WT5_005
IF THEN ELSE, WT5_002
INPUT PROMPT, WT4_005
KEY INPUT, WT5_006
KEYBOARD INPUT, WT6_013
LBOUND, WT17_004
LEN, WT9_001
LIBRARY, WT19_010
LINE INPUT, WT16_009
MAT INPUT & MAT PRINT, WT16_011
MAT INPUT, WT16_006
MAT PRINT USING, WT16_013
MAT PRINT, WT16_010
MAT READ & WRITE, WT16_014
MAT READ & WRITE, WT16_015
MAT READ, WT14_001
MAT READ, WT16_005
MAT READ, WT16_012
MAT REDIM WITH FILES, WT16_002
MAT REDIM, WT16_001

MAT REDIM, WT16_003
MATRIX ADDITION & SUBTRACTION, WT16_020
MATRIX ADDITION, WT16_019
MATRIX ASSIGNMENT, WT16_016
MATRIX ASSIGNMENT, WT16_017
MATRIX ASSIGNMENT, WT16_018
MATRIX DETERMINANT, WT17_003
MATRIX IDENTITY, WT17_001
MATRIX MULTIPLICATION, WT16_021
MATRIX TRANSPOSE, WT17_002
MODIFIED INPUT, WT4_006
MODIFIED SUB-STRINGS, WT4_003
MONETARY POBLEM, WT24_001
MORE CPOS SEARCHING, WT10_003
MORE FOR STEP NEXT, WT6_002
MOUSE INPUT, WT6_014
MULTIPLE INPUT, WT10_012
NCPOS SEARCHING, WT10_006
NCPOSR SEARCHING, WT10_007
NEGATIVE STEP, WT6_004
NESTED LOOPS, WT23_001
NUMSREC, WT_NUMS.rec
ORD, WT8_001
PACK ARRAYS, WT18_006
PACK SUB-STRINGS, WT18_012
PALINDROME, WT19_011
PARTIAL STRINGS, WT9_003
PHONE RING TONE, WT22_001
PICTURE TRANSFORMS, WT21_010
PLOTING POINTS, WT21_001
POS SEARCHING, WT10_008
POS START SEARCHING, WT10_009
POSR SEARCHING, WT10_010
PRINT USING, WT15_002
PUSH BUTTON DRAWING, WT21_012
PWLIB PRINTING, WT21_011
READ ANYFILE, WT18_001
READ BYTE FILES, WT18_002
READ RECORD/RANDOM FILE, WT18_004
READ TEXT FILE, WT18_003
REPEAT\$, WT9_002
ROUND INTEGERS, WT11_002
ROUND WITH PLACES, WT11_001
RUBBER BOXES, WT21_008

SELECT CASE ELSE, WT5_008
SELECT CASE, WT5_007
SIMPLE HELLO, WT4_001
SIMPLE INPUT, WT4_004
SINGLE DIMENSION ARRAYS, WT16_007
SIZE, WT17_006
SKELETON, WT_skeleton
SORTING ARRAYS, WT18_014
STEP, WT6_003
STR\$, WT8_003
STRING SEARCHING, WT10_001
STRINGS & NUMBERS, WT15_005
SUB BOXSTRING_SIZE
SUB BYTEPAIR
SUB CHECK_FILE_STATUS
SUB MAGNIFYBOX
SUB MAKEPAIR
SUB READBOXFILE
SUB READBOXPIXEL
SUB REDUCEBOX
SUB ROTATEBOX
SUB USER, WT6_015
SUB VALUETOBYTE
SUB WRITEBOXFILE
SUB WRITEBOXPIXEL
SUB-STRINGS, WT4_002
SUM FUNCTION, WT19_005
TBFORMS DEMO, WT25_002
TBSTANDARD, WT25_001
TC EVENT, WT25_001
TDX DIALOG BOXES, WT_TDXDEMO
TEXT TO SPEECH, WT22_002
TEXT WINDOW SIZE, WT15_001
TIME, WT13_001
UBOUND, WT17_005
UNPACK ARRAYS, WT18_005
UNPACK SUB-STRINGS, WT18_013
UNSORTED ARRAYS, WT18_016
USING VALSTR, WT8_006
VAL, WT8_004
VARIABLE PARAMETERS, WT19_007
VARIABLE PARAMETERS, WT19_008
VARIABLE PARAMETERS, WT19_009
VERTICAL TEXT, WT21_007

```
VINE, WT_VINE
VINE.EXE
WRITE BYTE FILES, WT18_007
WRITE RANDOM OR RECORD, WT18_009
WRITE TEXT FILES, WT18_008
WRITE TO PRINTER, WT18_010
WRITE TO PRINTER, WT18_011
WTlib
WTtext.txt
```

ABOUT THE AUTHOR

John R. Arscott, a retired Chemical Engineer and father of five children, lives in the heart of rural England. A prolific inventor and innovator, the author is also an artist and a Fellow of the Institution of Analysts and Programmers. He acquired his initial skills in computer programming in 1962 on a Ferranti Mercury computer with barely 16K memory, while working for Imperial Chemical Industries. He has maintained and improved his skills since that time, and has a wide range of commercial programs to his credit, including the latest True BASIC editor.

In his spare time he has written and published many novels. In his own words, "As an artist I paint what I see, and in my books I paint in words what I see in my mind." During his career he has traveled to most countries in the world, and this is reflected in the scope and subject matter of his writing. His novels cover a wide range of genres from thrillers to period romance and science fiction, and he admits that background research often takes much longer than writing the novel itself. In most of his novels he takes a simple moral issue to extreme limits and provokes his characters, and the reader, into confronting the issue in unconventional ways.

He admits that writing textbooks about computer programming is a far more exacting and painful departure from writing novels. However, he feels justified in writing this book in order to pass on some of his experience, so that the novices of today don't make the same mistakes that he made many times yesterday.

COMPUTER PROGRAMMING MADE EASY FOR BEGINNERS AND BEYOND

An easy to read introduction to programming with TrueBASIC that covers everything you need to know about programming for Windows. A common sense book written in a friendly style by an expert, and laced with pearls of wisdom, experience and good advice. The latest editor (v6.007) is featured, but the general concepts, statements, functions, and code examples (over 150) continue to work with all previous windows versions of TrueBASIC (5 and 6). The book also includes several library modules of useful routines. If it is not in this book then it is probably not worth knowing. This is a 'must have' comprehensive companion for anyone using the TrueBASIC language system.